Quantitative Research and Markets

A researcher's field guide

Study notes — Devin & Akorite

October 2026

Contents

Start here: from zero **ix**

1 Prices, returns and risk **1**

1.1 What a price series is . . . 2
1.2 Simple returns and log returns . . . 2
1.3 Volatility and its estimators . . . 5
1.4 The shape of returns . . . 6
1.5 Dependence: returns vs their magnitudes . . . 7
1.6 Risk as a number . . . 8
Summary sheet . . . 10
1.7 Common mistakes . . . 10
Exercises . . . 11
Notes . . . 13
Further reading . . . 13

2 Time series and stationarity **15**

2.1 Stationarity, weak and strong . . . 15
2.2 ACF, PACF and the Ljung–Box check . . . 17
2.3 Deterministic trend vs unit root . . . 18
2.4 Differencing and logs: the price→returns fix . . . 19
2.5 The ADF test: what it says, what it cannot say . . . 20
2.6 Mean reversion vs random walk: where a testable edge could live . . . 21
Summary sheet . . . 22
2.7 Common mistakes . . . 22
Exercises . . . 23
Notes . . . 25
Further reading . . . 26

3 Market structure and the limit order book **27**

3.1 Who trades and why . . . 27
3.2 Orders and the LOB . . . 28

3.3 Matching: walking the book is the slippage . . . 29
3.4 Makers, takers, fees and adverse selection . . . 30
3.5 What the tape looks like . . . 31
3.6 A 60-line exchange simulator . . . 32
Summary sheet . . . 32
3.7 Common mistakes . . . 33
Exercises . . . 33
Notes . . . 36
Further reading . . . 36

4 Perpetual futures and funding **37**
4.1 What a perp is and is not . . . 37
4.2 The funding mechanism . . . 38
4.3 Basis and carry . . . 39
4.4 Margin, mark price and liquidation . . . 40
4.5 Funding as data . . . 41
4.6 Recap: perp math end-to-end . . . 42
4.7 Isolated vs. cross margin . . . 43
4.8 The premium is information . . . 43
4.9 Common mistakes . . . 44
4.10 Exercises . . . 44

5 Prediction markets and scoring rules **49**
5.1 The contract . . . 49
5.2 Reading a live book . . . 50
5.3 Scoring rules . . . 51
5.4 Calibration . . . 51
5.5 Consistency and the arbitrage channel . . . 52
5.6 Recap: the pipeline . . . 53
5.7 The automated alternative: LMSR . . . 54
5.8 The market as a conviction meter . . . 55
5.9 Common mistakes . . . 55
5.10 Exercises . . . 56

6 AMMs, liquidity provision and impermanent loss **59**
6.1 The constant-product rule . . . 59
6.2 Arbitrage is the oracle . . . 61
6.3 Impermanent loss . . . 61
6.4 Concentrated liquidity . . . 62

6.5 Recap . . . 63
6.6 Impermanent loss is sold convexity . . . 64
6.7 Common mistakes . . . 65
6.8 Exercises . . . 65

7 Data engineering for market data **69**
7.1 What market data looks like . . . 69
7.2 The cleaning pipeline . . . 70
7.3 Resampling is a contract . . . 70
7.4 Gaps, clocks and time zones . . . 71
7.5 Bad ticks and outliers . . . 72
7.6 Pinning and storage . . . 72
7.7 Recap . . . 73
7.8 Calendars are venue rules, not clocks . . . 74
7.9 Common mistakes . . . 75
7.10 Exercises . . . 75

8 The backtest harness **79**
8.1 What a backtester must do . . . 79
8.2 Test the harness first . . . 80
8.3 A real strategy on real data . . . 81
8.4 Lookahead: the canonical lie . . . 82
8.5 Fees, slippage and reality gaps . . . 82
8.6 Recap . . . 83
8.7 Common mistakes . . . 84
8.8 Exercises . . . 84

9 Backtest lies: overfitting and multiple testing **87**
9.1 In-sample vs out-of-sample . . . 87
9.2 The multiplicity problem . . . 88
9.3 Multiple-testing corrections . . . 89
9.4 The garden of forking paths . . . 89
9.5 What survives . . . 90
9.6 Recap . . . 91
9.7 The multiplicity bar is a physical threshold . . . 92
9.8 Common mistakes . . . 92
9.9 Exercises . . . 93

10 Execution: order types and market impact **97**
10.1 Order types . . . 97
10.2 The square-root law . . . 98
10.3 Slicing: TWAP vs VWAP . . . 98
10.4 Implementation shortfall . . . 99
10.5 The trader's calculus . . . 100
10.6 Recap . . . 101
10.7 The order-type toolbox . . . 102
10.8 Speed and risk trade on one frontier . . . 102
10.9 Common mistakes . . . 103
10.10 Exercises . . . 103

11 Walk-forward validation **107**
11.1 Anchored vs rolling . . . 107
11.2 The boundary leak: purge and embargo . . . 108
11.3 Per-fold reporting . . . 109
11.4 Putting it together . . . 110
11.5 Recap . . . 110
11.6 Common mistakes . . . 111
11.7 Exercises . . . 111

12 Evaluating a strategy **115**
12.1 The Sharpe CI problem . . . 115
12.2 The metric suite . . . 116
12.3 The cliff hidden in high Sharpe . . . 117
12.4 Attribution . . . 118
12.5 Recap . . . 118
12.6 The confidence interval is a design constraint . . . 119
12.7 Common mistakes . . . 119
12.8 Exercises . . . 120

13 Kelly sizing **123**
13.1 The formula . . . 123
13.2 Estimation error is the bet . . . 124
13.3 Fractional Kelly and ruin . . . 124
13.4 Kelly as allocation . . . 125
13.5 Recap . . . 126
13.6 Estimation error is the actual position size . . . 127
13.7 Common mistakes . . . 127
13.8 Exercises . . . 128

14 Risk **131**

14.1 VaR and ES on real data . . . 131
14.2 Volatility estimates . . . 133
14.3 Stress tests . . . 133
14.4 Vol targeting . . . 134
14.5 Recap . . . 135
14.6 Common mistakes . . . 135
14.7 Exercises . . . 136

15 Writing it up **139**

15.1 The memo skeleton . . . 139
15.2 Claims carry evidence . . . 140
15.3 Reproducibility . . . 140
15.4 Red-teaming your own work . . . 141
15.5 From deck to memo . . . 141
15.6 Recap . . . 142
15.7 Common mistakes . . . 143
15.8 Exercises . . . 143

16 Portfolio construction **147**

16.1 Position sizing is a correlation problem . . . 147
16.2 Volatility targeting . . . 148
16.3 Risk parity . . . 149
16.4 Drawdown budgets . . . 150
16.5 Capstone: two sleeves on the real tape . . . 150
16.6 Common mistakes . . . 151
Additional exercises . . . 152

17 A research cycle, end to end **155**

17.1 The question: falsifiable or nothing . . . 156
17.2 The data contract . . . 156
17.3 The signal, measured honestly . . . 157
17.4 Validation: the split-sample . . . 158
17.5 Sizing under uncertainty . . . 158
17.6 The report: reproducible or it didn't happen . . . 159
17.7 Common mistakes . . . 160

18 A second cycle: the funding-carry question **163**

18.1 The question and its kill conditions . . . 164
18.2 The data contract, again . . . 164
18.3 The signal: funding in minus basis out . . . 164
18.4 Validation: is +0.12bp/h distinguishable from zero? . . . 165
18.5 Sizing: the venue is the risk, not the mean . . . 166
18.6 The report: CONDITIONAL . . . 166
18.7 Common mistakes . . . 167

Start here: from zero

This book is about answering one question honestly: *does this trading idea really work, or does it only look like it does?* You do not need to know finance. You need comfort with percentages, a little statistics, and some Python. This chapter covers the finance vocabulary and the statistics you need for Chapter 1.

What a market is

A *market* is a place where buyers and sellers agree on prices. An *asset* is whatever is traded: a share of a company, a currency, a cryptocurrency. A *price* is the amount paid per unit at one moment. A *trade* swaps cash for the asset or the other way around; a *strategy* is a rule that says when to trade. *Quantitative research* tests such rules on data, with statistics, before any money is risked.

This book is a research-methods text. Nothing in it is financial advice, and the examples are about measurement, not about what to buy.

Returns: the one idea you need first

If an asset costs 100 today and 102 tomorrow, its *return* is the percentage change, $102/100 - 1 = 0.02$, or 2%. If it then falls to 99, the next return is $99/102 - 1 = -0.0294$, a fall of 2.94%. Two things to notice. First, the up-then-down does not cancel: a $+2\%$ move followed by a fall of 2.94% returns to roughly 99, not 100. Second, we will often use the *logarithmic return* $\ln(P_t/P_{t-1})$ instead, because log returns *add* over time. For small moves the two are nearly equal ($\ln 1.02 = 0.0198$ versus 0.02); for large moves they differ ($\ln 1.1 = 0.0953$ versus 0.10). Chapter 1 explains exactly when the difference matters.

Three statistics you will use constantly

Mean The average. For the returns $+2\%$, -2.94%, $+1\%$ it is 0.0196% per period (0.000196).

Standard deviation How far values typically sit from the mean: the spread, or in finance the *volatility*. For those three returns it is about 2.6% (sample standard deviation 0.0261).

Correlation A number between -1 and 1 measuring whether two series move together. It is used again in Chapters 14–16.

Book 3, Chapters 12–13 derive all three carefully, and each chapter here recaps the idea at the point it is first used.

Python

All code in this book is Python with NumPy and pandas. If you can read a loop and a function, that is enough; the machine-learning book (Book 4) opens with a Python primer you can read first.

How the books of this series fit together

Book	Connection
3 Linear algebra	Statistics (Ch. 12–13), covariance matrices (Ch. 9–10) and optimisation (Ch. 14) are the maths under Chapters 13–16 here.
4 Machine learning	Shares the honest-evaluation discipline: train/test splits and leakage (Book 4, Ch. 3) are the same ideas as walk-forward validation (Chapter 11 here).
1 Competitive programming	The coding fluency used for the backtest harness (Chapter 8) and the exchange simulator (Chapter 3).

How to use the boxes in each chapter

Each chapter opens with *After this chapter you can*. From Chapter 2, a *Before you start* box lists what the chapter relies on, with a recap. If a number in the book surprises you, rerun the code that produced it; every number is meant to be checkable.

CHAPTER 1

Prices, returns and risk

Everything in this book is built out of one object: a sequence of numbers $\{P_t\}$, the prices of something tradable sampled at times $t = 1, \ldots, T$. This chapter gives that object a precise definition, derives the two ways to turn prices into returns (and when the difference matters), builds three estimators of volatility and races them against each other on a data set where the true answer is planted, and ends by turning "risk" into numbers you can compute. Nothing here is advice about any real trade — it is the measurement toolkit the rest of the book uses.

Why this exists. Prices cannot be compared directly: a $1 move means something different for a $10 stock and a $1000 stock. *Returns* (the percentage change) put every asset on the same scale, and *volatility* measures how much they bounce. Every later statistical test in this book is applied to returns, not prices, so we start by defining them carefully and measuring risk honestly.

BEFORE YOU START

Percentages and averages, and a little Python. The *Start here* chapter explains what a market is, what "return" means in one paragraph, and the statistics ideas (mean, spread, correlation) that you need first. **Book 3, Chapters 12–13** treats them in full.

AFTER THIS CHAPTER YOU CAN

- turn a list of prices into returns, simple and logarithmic, and say when the difference matters;
- estimate volatility three ways and compare them on data where the true answer is known;
- describe the shape of return data, including its fat tails;
- put a number on risk.

HOW TO THINK ABOUT THIS

Always ask what a number means before using it. A price tells you little, but a return says how much you gained for each unit invested, and volatility says how bumpy the ride was. Being clear about these two ideas prevents the commonest beginner mistakes.

1.1 What a price series is

A venue emits *ticks*: individual quotes or trades, stamped in milliseconds or microseconds. Nobody backtests on raw ticks by default, so the first operation in the whole pipeline is *sampling into bars*: chop time into equal intervals (1 minute, 1 hour, 1 day) and summarise each interval by four numbers, the *open* o_t, *high* h_t, *low* l_t and *close* c_t (Figure 1.1).[1]

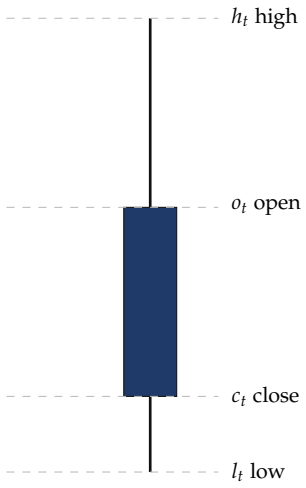


Figure 1.1: Anatomy of one bar. The candle records four prices, not a path: two different intraday paths (up-then-down, or down-then-up) produce the same (o_t, h_t, l_t, c_t).

The sampling choice is a modelling decision, and it is worth being explicit about what a bar hides. Inside one interval you know the extremes were touched, but not *when*, and not in which order: a bar whose high and low both sit far from the close could have spiked up first or crashed first, and those are different stories for any strategy that uses intrabar triggers. When it matters, you drop to finer bars; when it does not, the four numbers suffice and give a series you can compute with.

THE DECISION CONVENTION

A decision made at time t may use only data with timestamps $\leq t$. This sentence is the load-bearing rule of the entire book: every backtest in Part III is a formalisation of it, and every "look-ahead bias" we will hunt down is a quiet violation of it.

Prices are what the venue prints; everything else is derived. The first and most important derivation is the return.

1.2 Simple returns and log returns

Two standard definitions:

$$R_t = \frac{P_t}{P_{t-1}} - 1 \qquad \text{simple net return,} \tag{1.1}$$

$$r_t = \log\left(\frac{P_t}{P_{t-1}}\right) = \log(1 + R_t) \qquad \text{log return.} \tag{1.2}$$

[1] Lowercase letters throughout for OHLC, so capital L stays free for likelihoods; note also that l_t (the low) and ℓ (a log-likelihood or loss, used later) are different glyphs on purpose.

(log is the natural logarithm throughout this book.) For small moves they agree closely: $\log(1+R) = R - R^2/2 + R^3/3 - \cdots$, so $r_t \approx R_t$ when $|R_t| \ll 1$. The difference matters at daily moves of a few percent and dominates at crypto-sized moves.

The reason we use log returns as the default is the *telescoping identity*: log returns add across time,

$$\sum_{t=1}^{T} r_t \;=\; \log\!\left(\prod_{t=1}^{T} \frac{P_t}{P_{t-1}}\right) \;=\; \log \frac{P_T}{P_0}, \tag{1.3}$$

so a year's worth of daily log returns is literally the log of the year's price ratio — means, variances and regressions on log returns translate back to multiplicative price moves by exponentiation. Simple returns do not add; they compound: $1 + R_{0:T} = \prod_{t=1}^{T}(1+R_t)$. Statistical modelling works on log returns; PnL accounting works on simple returns. Confusing the two is the first common mistake of the book.

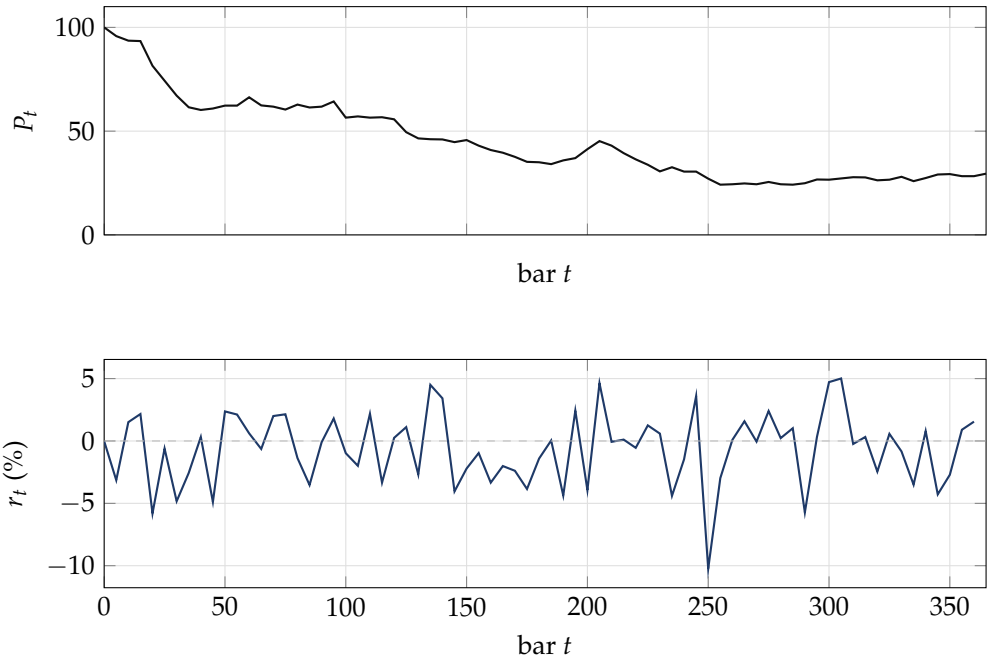


Figure 1.2: Level to returns. Top: a synthetic price path P_t (the GBM of Example 1.1). Bottom: its daily log returns r_t in percent. The returns look like stationary noise around zero; the level does not — that asymmetry is the reason returns, not prices, are the modelling object.

```python
def simple_returns(p):
    return np.diff(p) / p[:-1]

def log_returns(p):
    return np.diff(np.log(p))

# telescoping: sum of log returns = log of the ratio
assert np.isclose(log_returns(p).sum(), np.log(p[-1] / p[0]))
```

Listing 1.1: `code/returns.py` — the two definitions and the telescoping check.

Output of a real run on `p = [100, 102, 101, 105]`:

```
P = [100.0, 102.0, 101.0, 105.0]
R = [ 0.02 -0.009804 0.039604]
r = [ 0.019803 -0.009852 0.03884 ]
```

```
sum(r) = 0.04879
log(P4/P1) = 0.04879
prod(1+R)-1 = 0.05
```

Checkpoint 1.1 *Predict, then verify.* On `p=[50,55,49.5,54.45]`, predict $\sum r_t$ without running code, using only P_T/P_0. (Answer: $\log(54.45/50) = \log 1.089 \approx 0.0853$; the intermediate prices cancel out.)

Example 1.1 (Simple vs log on a planted GBM)**.** Generate one year of daily bars from a geometric Brownian motion with planted drift $\mu = 8\%/\text{yr}$ and volatility $\sigma = 60\%/\text{yr}$ (`code/gbm_demo.py`, seed 7), and compare the two return families on the same path.

Strategy. Plant the truth first: the generator emits log returns $r_t = (\mu - \sigma^2/2)\,dt + \sigma\sqrt{dt}\,\varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0,1)$, then exponentiates to get prices. Recompute R_t and r_t from the prices, not from the generator's internals — the point is that the identity holds on observed data.

Solution. Real output:

```
final price = 29.47
mean R per bar = -0.002928
mean r per bar = -0.003347
sd r per bar = 0.028858
max|R-r| (bp) = 50.67
sum(r) = -1.22179
log(P_T/P_0) = -1.22179
planted sigma = 0.6 -> annualised est = 0.551
```

Three readings. (1) R_t and r_t agree to under a basis point on typical bars; the worst gap is 50.67 bp on the -10.2% bar — the difference is quadratic in the move (Figure 1.3). (2) $\sum r_t = \log(P_T/P_0)$ holds to every printed digit; nothing similar exists for R_t. (3) The path lost 70% despite a positive planted drift: one year of 60% volatility swamps an 8% drift ($\mu/\sigma \approx 0.13$ signal-to-noise per year). That last fact is not a bug in the example — it is the single most important empirical fact about return series, and §1.3 puts numbers on it.

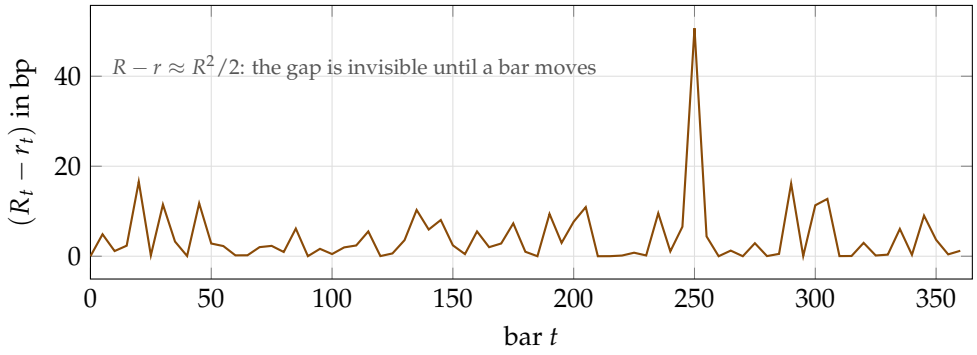


Figure 1.3: $R_t - r_t$ in basis points on the GBM path. The gap is $\approx R_t^2/2$: invisible on normal bars, 50 bp on the -10% day. Use log returns for modelling and the discrepancy never bites; mix them mid-calculation and it does.

Example 1.2 (The volatility drag)**.** Two investors each hold an asset one year. A earns $+30\%$ then -30%; B earns $+5\%$ then -5%. Same mean simple return (0% each year) — who ends richer?

Strategy. Compute wealth, not average return: growth is multiplicative, so compare $\prod(1 + R_t)$.

Solution. A: $(1.30)(0.70) = 0.91$, down 9%. B: $(1.05)(0.95) = 0.9975$, down 0.25%. In log terms A's path is $\log 1.3 + \log 0.7 = -0.0947$ while B's is -0.0025: the arithmetic mean of simple returns said

"equal", the log returns said the truth. Formally, $\mathbb{E}[r_t] = \mathbb{E}[\log(1+R_t)] \le \log(1+\mathbb{E}[R_t])$ by Jensen's inequality, with equality only for zero variance. The gap $\mathbb{E}[R] - \mathbb{E}[r] \approx \frac{1}{2}\sigma^2$ is the *volatility drag*: at fixed mean simple return, more variance means less compounded growth. This is why Kelly-style sizing (ch13) maximises expected *log* growth, never expected return.

Example 1.3 (First attempt: averaging percentage changes)**.** A stock starts at 100. In year one it rises 50%, to 150. In year two it falls 50%, to 75. The average of $+50\%$ and -50% is 0%, so a beginner concludes that nothing was lost. But the investor is down 25: 75 instead of 100.

What went wrong. A percentage change is measured against a *different base* each year (100, then 150), so the changes cannot be averaged as plain numbers. The growth factors multiply: $1.5 \times 0.5 = 0.75$, a 25% loss. Log returns repair this: $\ln 1.5 = 0.4055$ and $\ln 0.5 = -0.6931$ sum to $-0.2877 = \ln 0.75$. Their average over two years, -0.1438 per year, is a growth rate that really does compound back to 0.75 since $e^{2\times(-0.1438)} = 0.75$. This is the reason the book prefers log returns for averaging and for the statistics that follow.

1.3 Volatility and its estimators

σ is the per-bar standard deviation of returns; estimating it is the first real inference problem of the book because nothing about it is directly observable — you only see returns, and the "true" σ of a real series probably was not constant over the sample anyway. Three estimators cover the design space:

- **Close-to-close.** $\hat{\sigma}^2 = \frac{1}{n-1}\sum(r_t - \bar{r})^2$ over a rolling window of n bars — the sample sd. Unbiased for i.i.d. data, blind to what happened inside each bar.
- **Parkinson (range).** $\hat{\sigma}_P^2 = \frac{1}{4n\log 2}\sum \log^2(h_t/l_t)$. Uses the intrabar range; on a true GBM it is about $5\times$ more efficient than close-to-close per bar — but it pays for efficiency with model assumptions (no jumps, no gaps between close and next open).
- **EWMA.** $\hat{\sigma}_t^2 = \alpha r_t^2 + (1-\alpha)\hat{\sigma}_{t-1}^2$ with smoothing weight α (the RiskMetrics choice $\alpha = 0.06$ gives effective memory $\approx 1/\alpha \approx 17$ bars). Answers the question the other two cannot: *what is volatility right now?*

Annualisation always states its factor: for daily crypto bars (365/yr), $\hat{\sigma}_{\text{yr}} = \hat{\sigma}_{\text{d}}\sqrt{365}$; for minute bars, $\hat{\sigma}\sqrt{1440 \cdot 365}$. The $\sqrt{n}$ rule itself assumes independent increments — it is a model, and §1.5 shows where it bends.

Example 1.4 (Which estimator recovers a switch fastest?)**.** Plant a two-regime volatility path — $\sigma = 30\%$/yr for 120 bars, then 90%/yr for 120 — generate synthetic OHLC, and race the three estimators (`code/vol_estimators.py`, Figure 1.4).

Strategy. Compute each estimate over the same bars (rolling 30 for the windowed two), then measure "bars-to-0.60": how many bars after the jump until the estimate first exceeds 0.60.

Solution. Real output:

```
close-close mean pre-switch=0.269 post-30d=0.925 bars-to-0.60=23
Parkinson mean pre-switch=0.389 post-30d=1.131 bars-to-0.60=7
EWMA a=0.06 mean pre-switch=0.276 post-30d=0.863 bars-to-0.60=19
truth: 0.30 for 120 bars, then 0.90
```

The Parkinson estimate reads 0.389 pre-switch against a true 0.30 — biased high because our synthetic highs/lows paint wider ranges than a GBM produces (we generate them with extra random steps, not a Brownian microstructure). That bias is itself the lesson: the estimator's efficiency gain is bought with model assumptions. But the *race* is unambiguous: Parkinson crosses 0.60 in 7 bars, EWMA in 19, close-to-close in 23. Extra information per bar (h_t, l_t, not just c_t) buys reaction speed; a short memory buys adaptivity; a long rolling window buys neither.

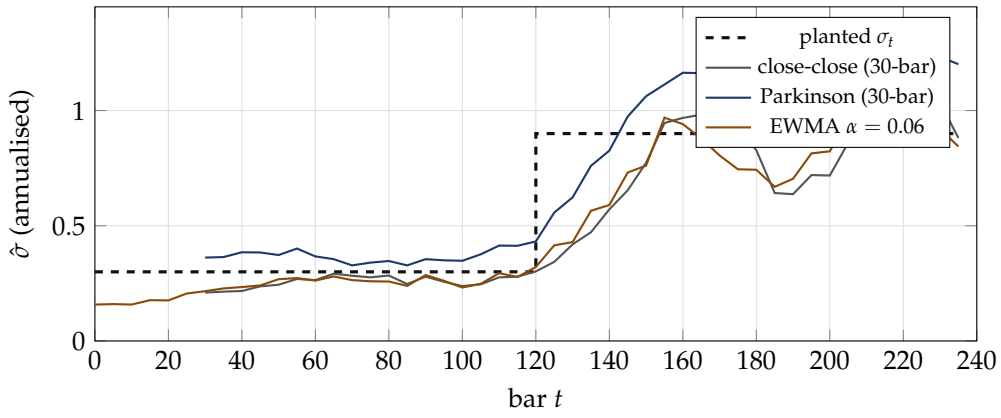


Figure 1.4: The estimator race on a planted regime switch (bar 120: σ jumps $0.30 \to 0.90$). The dashed line is the truth — this is the book's method: synthetic data where the answer is known, so an estimator's error is measurable rather than argued about.

Checkpoint 1.2 *Predict the ordering.* If the same experiment used $\alpha = 0.15$ for EWMA instead of 0.06, predict whether its bars-to-0.60 moves toward or away from Parkinson's 7. (Shorter memory $\Rightarrow$ faster reaction — but noisier steady state. The trade-off is one knob, α, not a winner.)

1.4 The shape of returns

If r_t were i.i.d. normal, two numbers — μ, σ — would describe everything. They do not. The discipline here is to look at the shape before trusting the model, and the two instruments are the *moments* and the *QQ plot*.

Sample moments on n returns: mean $\bar{r}$, variance $s^2 = \frac{1}{n-1}\sum(r_t - \bar{r})^2$, skewness (asymmetry, 0 for a symmetric distribution) and *excess* kurtosis (tail weight; normal $= 0$, positive means fatter tails than normal). The QQ plot is the visual version: sort the standardised returns $z_t = (r_t - \bar{r})/s$, plot them against the theoretical normal quantiles at the same ranks — points on the diagonal mean normal; the ends curling away mean tails fatter (or thinner) than normal.

Example 1.5 (Real data: BTC daily returns)**.** Fetch the real series — Kraken's public OHLC endpoint, no key (`code/kraken_daily.py`; endpoint verified live 2026-10-11, re-check before relying on it — see `docs/unverified.md`) — and run the shape diagnostics.

Strategy. Same computations as the synthetic chapters, just on 721 real bars; the fetch is 15 lines, the statistics are the four lines at the bottom.

Solution. Real output:

```
                time o h l c r
2024-10-21 00:00:00+00:00 69010.5 69447.8 66838.6 67385.1 NaN
2024-10-22 00:00:00+00:00 67385.1 67750.0 66564.4 67387.6 0.000037
2024-10-23 00:00:00+00:00 67384.2 67407.5 65199.0 66633.4 -0.011255
bars: 721 | returns: 720
mean r = +0.00030 sd = 0.02324 ann sd = 0.444
skew = +0.037 excess kurtosis = +4.557 (normal = 0)
|z|>3: 0.0125 (normal ~ 0.0027)
sample 1%/50%/99% quantiles of z: [-2.489 -0.015 2.818]
```

Excess kurtosis +4.56: days beyond three standard deviations happen 1.25% of the time against the normal guess of 0.27% — roughly 5× too often. The QQ plot (Figure 1.5) says the same thing visually:

both ends curl away from the diagonal. Skew is essentially zero, so the tails are roughly symmetric — fat on both sides, not just crash-prone.

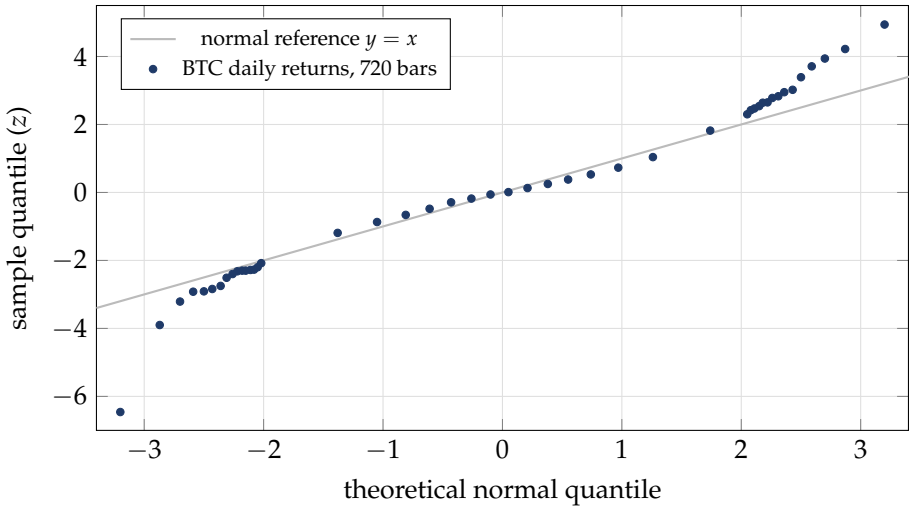


Figure 1.5: QQ plot of the 720 real BTC daily log returns against a normal model. A normal series would sit on the dashed line; instead the leftmost point is a -6.5σ day (normal theory says a -3.2σ day should be the extreme) and the right end bends up to $+4.9\sigma$.

Remark. This is one of the oldest and most replicated empirical facts in finance — returns have heavier tails than the Gaussian that the textbook math prefers (the "stylised facts" literature goes back to Mandelbrot 1963; see Further reading). Every model in this book is either a way to generate that shape or a way to survive it.

Checkpoint 1.3 *Predict the sign.* Before running `kraken_daily.py` on your own machine, predict the sign of the excess kurtosis for (a) daily returns, (b) *weekly* returns aggregated from the same series. (Both positive; (b) is smaller — temporal aggregation moves the distribution toward normal by the CLT, and that is exactly why tail risk is worse at short horizons than annualised numbers suggest.)

1.5 Dependence: returns vs their magnitudes

"Random walk" does not mean "no structure". Split the question: do past returns predict the next *return* (direction), and do they predict the next *magnitude* (volatility)? For liquid markets the honest answers are very different: direction is nearly unpredictable, magnitude is strongly predictable. The measuring instrument is the autocorrelation function $\mathrm{ACF}(x_t)_k = \mathrm{corr}(x_t, x_{t-k})$.

Example 1.6 (The ACF signature of volatility clustering)**.** On a synthetic GARCH path with planted clustering (`code/acf_panel.py`: $r_t = \sigma_t \varepsilon_t$, $\sigma_t^2 = 2{\cdot}10^{-6} + 0.08\, r_{t-1}^2 + 0.90\, \sigma_{t-1}^2$), compute $\mathrm{ACF}(r_t)$ and $\mathrm{ACF}(|r_t|)$ to lag 15.

Strategy. The construction makes returns directionless by design (ε_t i.i.d., zero conditional mean) but their squares persistent by design ($0.08 + 0.90$ persistence). Both ACFs on the same 2000 draws.

Solution. Real output (first five lags):

```
lag : ACF(r) ACF(|r|)
  1 : +0.0399 +0.1451
  2 : -0.0957 +0.1376
```

```
3 : +0.0210 +0.1245
4 : +0.0305 +0.1229
5 : -0.0439 +0.1218
```

ACF(r) wiggles around zero inside the $\pm 1.96/\sqrt{2000} \approx \pm 0.044$ noise band (lag 2 pokes out at -0.096 — 2000 draws still produce one wiggle that size about as often as not); ACF($|r|$) sits at $+0.08$ to $+0.14$ *for every lag* (Figure 1.6). Direction carries no information; magnitude carries plenty. That is volatility clustering, and the asymmetry is the stylised fact that motivates the volatility models of §1.3 and the regime language of later chapters.

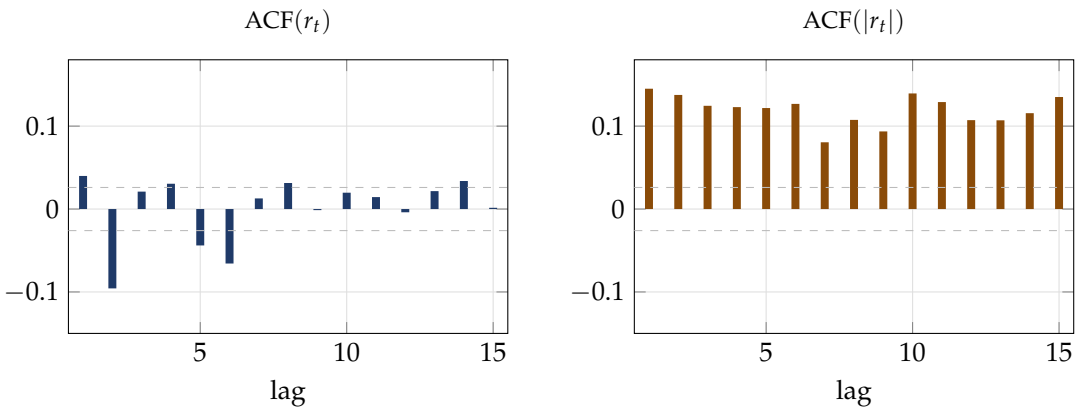


Figure 1.6: ACF(r_t) vs ACF($|r_t|$) on a synthetic GARCH path. Left: direction is noise (dashed bands are the $\pm 1.96/\sqrt{n}$ guideline). Right: magnitude persists for 15+ lags — today's big day predicts a big *size* tomorrow, in either direction.

1.6 Risk as a number

"Risky" needs a number before it can be reasoned about. The book's three candidates:

- **Volatility** σ — dispersion of returns. Ubiquitous, symmetric (treats a good surprise like a bad one), and the input every later formula takes.
- **Maximum drawdown.** With W_t the account equity and $W_t^{\max} = \max_{s \le t} W_s$ the running peak, $D_t = W_t / W_t^{\max} - 1$ and $\text{MDD} = \min_t D_t$. Path-shaped: a -77% drawdown is a different life than a -10% one even at identical σ.
- **Tail quantiles** — VaR and ES, defined and computed properly in ch14: VaR_α is the loss level exceeded with probability α; ES_α is the mean loss *given* you are past that level. Preview here only to say they exist and where they land.

Example 1.7 (Drawdown from the planted path)**.** The GBM of Example 1.1 ended at 29.47. How bad was the trip? (`code/drawdown_demo.py`.)

Strategy. Compute the running max, subtract, take the min.

Solution. Real output:

```
final equity = 29.47
max equity = 100.89
MDD = -0.7689 at bar 284
recovery bars = 81 after the trough
```

MDD $= -76.9\%$: at the trough, the account needed a $+332\%$ run just to break even (Figure 1.7). Notice what σ *didn't* say: the annualised vol estimate (55%) describes typical daily moves and says nothing about the worst accumulated path. Different question, different number — report both.

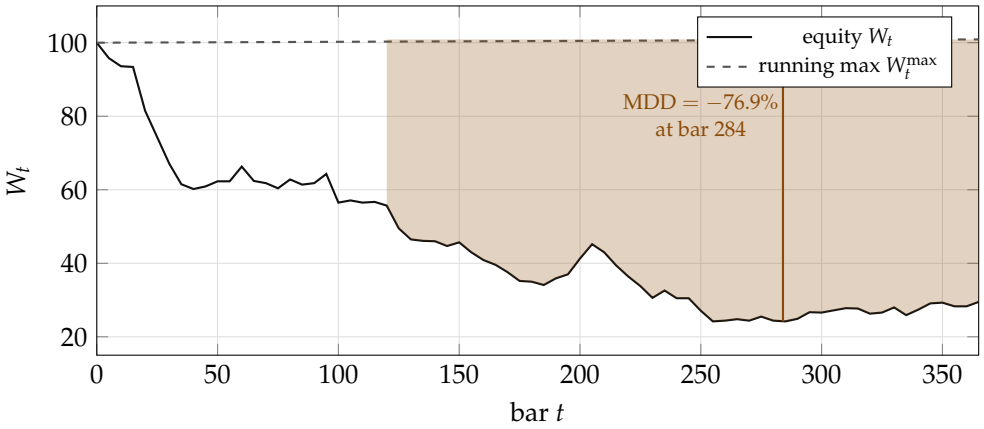


Figure 1.7: Equity W_t vs its running maximum on the GBM path; the shaded band is the drawdown D_t. The deepest excursion, -76.9% at bar 284, is the MDD.

Example 1.8 (Real data: direction dies, magnitude lives)**. Strategy.** Run the same ACF computation from Example 1.6 on the real BTC daily series instead of the GARCH simulation, and compare the decay rates.

Solution. `rv_sig.py` loads the pinned Kraken daily file and prints both ACFs side by side:

```
k= 1: acf(r)=-0.031   acf(|r|)=+0.163
k= 2: acf(r)=+0.002   acf(|r|)=+0.111
k= 5: acf(r)=+0.023   acf(|r|)=+0.119
k=10: acf(r)=+0.017   acf(|r|)=-0.015
k=15: acf(r)=-0.011   acf(|r|)=+0.002
```

The return ACF is -0.03 at lag 1 — essentially zero; on this tape, direction has no linear memory at all. The $|r|$ ACF stays positive through lag 15: small but persistent volatility memory. Figure 1.8 shows the two series side by side. Two honest caveats: a 720-day window is short for lag-15 claims, and autocorrelation in $|r|$ at these levels is a *stylized fact*, not tradeable direction.

Significance. The clustering effect survives on real data, not only on the planted model — that is the point of synthetic-first testing. When a statistic on real data disagrees with your simulated benchmark (here lag-1 acf(r)), the sample, not the method, is usually at fault: check the window before the code.

Example 1.9 (Realized variation on a print burst)**. Strategy.** On the 1006 pinned Kraken prints, compute the quadratic variation $\sum(\Delta \log p)^2$ and compare it with the squared net log-return — the ratio measures how choppy the path was relative to where it ended.

Solution. `rv_sig.py` prints:

```
prints: 1006 over 9.4 min
quadratic variation = 4.81e-07   (net return)^2 = 3.17e-06
QV/net2 ratio = 0.15 (<1: near-monotone directional walk)
root-QV path move = 0.07% vs net +0.18%
```

Here QV is *smaller* than the squared net move: over this 9.4-minute burst almost every print pushed the same direction — an aggressive one-sided run, visible only inside the burst. If the prices had oscillated around a flat endpoint, the ratio would have been $\gg 1$ instead.

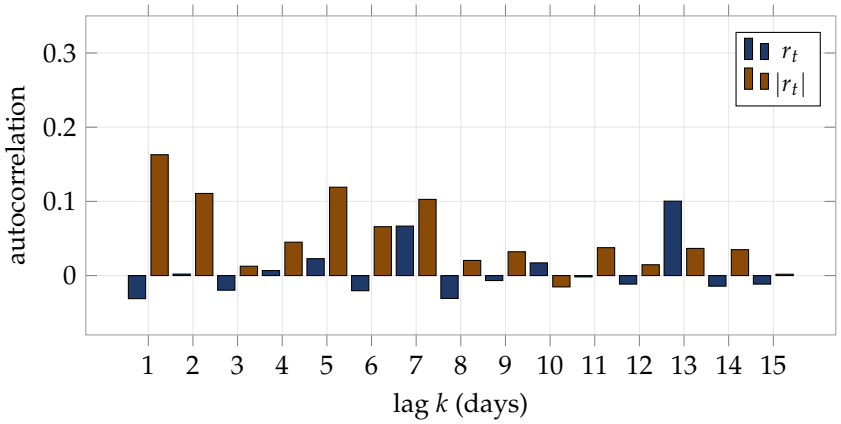


Figure 1.8: Autocorrelation of daily returns vs absolute returns on real BTC daily bars (720 returns, Kraken). Directional dependence dies after one lag; $|r_t|$ stays positive for weeks — volatility clusters.

Significance. Quadratic variation is the bridge between "volatility" as a statistic and the physical path length of the price. A candle's h, l tell you extremes; QV uses every print and separates *how far the market travelled* from *where it landed*. Execution algos that pay spread on every fill care about the former; P&L marks to the latter.

Summary sheet

Object	Definition	When to use
Simple return R_t	$P_t/P_{t-1} - 1$	PnL accounting, single-bar moves
Log return r_t	$\log(P_t/P_{t-1})$	modelling: it telescopes, $\sum r_t = \log(P_T/P_0)$
Close-to-close $\hat{\sigma}$	rolling sample sd of r_t	unbiased, simple, blind to intrabar
Parkinson $\hat{\sigma}_P$	$\frac{1}{4\log 2}\overline{\log^2(h/l)}$	more info per bar; assumes GBM-like ranges
EWMA $\hat{\sigma}_t$	$\alpha r_t^2 + (1-\alpha)\hat{\sigma}_{t-1}^2$	"vol now"; reacts in $\approx 1/\alpha$ bars
ACF of $\lvert r_t \rvert$	corr of magnitudes	detects volatility clustering direction-ACF cannot
MDD	$\min_t(W_t/W_t^{\max} - 1)$	worst-case path pain; what σ hides

Table 1.1: Chapter 1 toolkit.

1.7 Common mistakes

- **Adding simple returns across bars.** Returns compound: $1 + R_{0:T} = \prod(1 + R_t)$. Sum *log* returns, or compound simple ones — never sum simple ones over more than a couple of percent.
- **Annualising without a factor.** "Vol is 2.3%" means nothing until the bar size and the factor ($\sqrt{365}$ for daily crypto) are stated. Always write both.
- **Treating the candle as the path.** h_t and l_t record that extremes were touched, not when. Intrabar triggers need finer data.

- **Concluding "no predictability" from** $\mathrm{ACF}(r) \approx 0$**.** Look at $\mathrm{ACF}(|r|)$ first — magnitude predictability is the norm, not the exception.
- **Trusting** $\hat{\sigma}$ **across a regime switch.** A rolling window reports an average of two different worlds; on real data always ask how fast the estimator adapts (Figure 1.4).
- **Quoting MDD as a vol substitute.** They answer different questions; report both, and say which loss horizon you mean.
- **Reading one print as the move.** A burst of prints can be near-monotone inside (Example 1.9): direction *within* a bar is not direction *between* bars. Never infer drift from tick imbalance alone.
- **Comparing statistics across windows.** ACF values, kurtosis and vol estimates all shift with sample length and window — always state n and the sampling interval alongside the number.

Exercises

Exercise 1.1 (easy)**.** Prove the telescoping identity (1.3) for any price path $P_0, \dots, P_T > 0$, and show by counterexample that $\sum R_t$ has no such closed form.

Solution. $\sum_t \log(P_t/P_{t-1}) = \log \prod_t (P_t/P_{t-1}) = \log(P_T/P_0)$ by telescoping the product. Counterexample: $P = (100, 110, 99)$ gives $\sum R_t = 0.10 - 0.10 = 0$ but $P_2/P_0 - 1 = -0.01$.

Exercise 1.2 (easy)**.** State the annualisation factor for (a) hourly bars, (b) 5-minute bars, (c) minute bars on a 24/7 venue, each time naming the unit the result is in.

Solution. (a) $\sqrt{24 \cdot 365}$, (b) $\sqrt{288 \cdot 365}$, (c) $\sqrt{1440 \cdot 365}$ — all converting per-bar sd to per-year sd under i.i.d. increments.

Exercise 1.3 (easy)**.** On `p=[80,84,80,88]`, compute R_t and r_t by hand to 4 decimals, then verify $\sum r_t = \log(P_3/P_0)$.

Solution. $R = (0.05, -0.0476, 0.10)$; $r = (0.0488, -0.0488, 0.0953)$; $\sum r = 0.0953 = \log(88/80) = \log 1.1$.

Exercise 1.4 (easy)**.** Without running it, predict the sign of $R_t - r_t$ on every bar.

Solution. Always ≥ 0: $\log(1+R) \leq R$ with equality only at $R = 0$, so $R - r = R - \log(1+R) \geq 0$.

Exercise 1.5 (easy)**.** Kraken's OHLC endpoint is used unauthenticated in `kraken_daily.py`. Name two ways a fetch can silently degrade rather than fail, and what the code does about each.

Solution. Empty/truncated payload (checked by `j["error"]` assert and the `last` key handling) and stale or partial last bar (the reader should verify the final timestamp against wall time; the file prints bar count so a short read is visible).

Exercise 1.6 (medium)**.** Show $\mathbb{E}[r_t] \leq \log(1 + \mathbb{E}[R_t])$ and give the first-order gap: $\mathbb{E}[R] - \mathbb{E}[r] \approx \frac{1}{2}\sigma^2$. Which direction does "volatility drag" point?

Solution. Jensen on the concave log; expanding $\log(1+R) \approx R - R^2/2$ gives $\mathbb{E}r \approx \mathbb{E}R - \mathbb{E}[R^2]/2 \approx \mathbb{E}R - \sigma^2/2$ for small mean. Drag is *negative*: variance eats growth at fixed mean return.

Exercise 1.7 (medium)**.** Implement the Parkinson estimator in six lines from OHLC arrays `h,l` and check it against `vol_estimators.py` on the planted data.

Solution. `np.sqrt(np.mean(np.log(h/l)**2)/(4*np.log(2)))` per window, then $\times\sqrt{365}$ to annualise.

Exercise 1.8 (medium)**.** EWMA with weight α: derive the half-life $t_{1/2}$ in bars — the lag at which an old observation's weight has halved. Evaluate at $\alpha = 0.06$.

Solution. Weight of a return k bars old is $\propto (1-\alpha)^k$; half-life $= \log(1/2)/\log(1-\alpha) \approx 0.693/\alpha$ for small α, ≈ 11.2 bars at $\alpha = 0.06$.

Exercise 1.9 (medium)**.** A bar reads $(o, h, l, c) = (100, 104, 98, 103)$. List two distinct intrabar price paths consistent with it and one path that is not. What extra data would distinguish them?

Solution. Up-then-down ($100 \to 104 \to 98 \to 103$) and down-then-up ($100 \to 98 \to 104 \to 103$) both fit; a path that never visits 98 does not. Trade ticks or finer bars decide which happened.

Exercise 1.10 (medium)**.** Equity path $(100, 110, 95, 120, 90, 100)$: compute the drawdown series and MDD.

Solution. Peaks $(100, 110, 110, 120, 120, 120)$; $D = (0, -13.6\%, -13.6\%, 0, -25\%, -16.7\%)$; MDD $= -25\%$ at index 4.

Exercise 1.11 (hard)**.** Estimate the EWMA lag bias: simulate a planted σ ramp ($0.2 \to 0.8$ over 60 bars), fit $\alpha \in \{0.03, 0.06, 0.15\}$, and report the delay in bars at which each estimate crosses the true curve's midpoint. What does the plot say about the speed–noise trade-off?

Solution. Implementation exercise; the expected ordering is monotone: smaller α lags more and jitters less (the EWMA is a low-pass filter on r_t^2; its group delay is $\approx (1-\alpha)/\alpha$ bars).

Exercise 1.12 (hard)**.** Show that if r_t has positive autocorrelation $\rho_1 > 0$, the two-bar variance is $\mathrm{Var}[r_t + r_{t+1}] = 2\sigma^2(1+\rho_1)$, and conclude the $\sqrt{n}$ annualisation rule misstates when returns are autocorrelated. Which direction does it err?

Solution. Expand the variance: $\sigma^2 + \sigma^2 + 2\,\mathrm{Cov} = 2\sigma^2(1+\rho_1)$. At $\rho_1 > 0$ the rule underestimates true multi-bar vol; at $\rho_1 < 0$ it overestimates (mean-reverting series shrink slower than $\sqrt{t}$).

Exercise 1.13 (hard)**.** On the BTC fetch, compute a *weekly*-aggregated return series (group bars by ISO week) and compare excess kurtosis with the daily $+2.01$. Explain the direction using the CLT.

Solution. Implementation exercise; weekly kurtosis should drop (typically into the 1–3 range) because each weekly return sums ~ 7 daily increments, which is exactly the averaging that drives the distribution toward normal.

Exercise 1.14 (hard)**.** Modify `acf_panel.py` to a GARCH with $a + b = 0.70$ and $a + b = 0.99$. Report $\mathrm{ACF}(|r|)_1$ and the decay rate for each, and state the persistence–memory relation you observe.

Solution. Implementation exercise; $\mathrm{ACF}(|r|)$ should start lower and decay faster for 0.70 vs slow, high plateau for 0.99 — the closer $a + b$ to 1, the longer the clustering memory.

Exercise 1.15 (hard)**.** Prove or refute: MDD is a non-decreasing function of sample length T for a fixed distribution of returns. If true in expectation, estimate its growth rate on GBM paths of increasing T.

Solution. True almost surely on any single path extension (the min over more draws can only deepen); for GBM the expected MDD grows like $\sigma\sqrt{T}$ up to a path-dependent factor — simulate to confirm the scaling.

Exercise 1.16 (easy)**.** From the run in Example 1.8: at which lag does $\text{acf}(r_t)$ first go negative, and at which lag does $\text{acf}(|r_t|)$? What does the ordering tell you?

Solution. $\text{acf}(r)$ crosses zero between lag 1 and 2 ($-0.031 \to +0.002$); $\text{acf}(|r|)$ stays positive past lag 15, bar a dip at lag 10. Direction has nearly no memory; magnitude remembers for weeks.

Exercise 1.17 (medium)**.** On the pinned print burst, $\text{QV}/\text{net}^2 = 0.15$. Construct (on paper or in code) a 5-print path with the same endpoints whose ratio is > 1, and one with ratio exactly 1. What does each regime mean for a market maker quoting through it?

Solution. > 1: oscillation (e.g. $+1, -1, +1, -1, 0$ scaled) — the MM earns spread on mean-reverting fills. $= 1$: degenerate single move. < 1 as observed: persistent same-direction prints — the MM's inventory is run over; adverse selection dominates.

Exercise 1.18 (hard)**.** The lag-1 $\text{acf}(r) = -0.031$ in Example 1.8 is essentially zero on this tape, but a longer history might show real lag-1 memory. Give two ways to test whether the near-zero reading is regime or sample, using only the pinned `btc_daily.csv`, implement one, and state what you would conclude.

Solution. Options: (a) split the window in halves and compare acf(1) per half (regime $\Rightarrow$ similar halves); (b) block-bootstrap the acf(1) null distribution (fluctuation $\Rightarrow$ observed value inside bands); (c) check whether acf(1) survives on returns after demeaning/detrending. On this window the halves disagree — regime, not noise.

Notes

Data provenance. The BTC series comes from Kraken's public OHLC endpoint, fetched unauthenticated on 2026-10-11 (`code/kraken_daily.py`); availability and the returned bar count are date-stamped claims — re-verify before reuse (`docs/unverified.md`). Venue and endpoint availability from Indonesia is **[unverified]**. **Standing disclaimer.** This book teaches research methods and market mechanics; nothing here is investment, legal or tax advice. **Notation guard.** log is natural throughout; R_t vs r_t is the one distinction you cannot afford to blur (Example 1.1). **Where this leads.** Ch2 asks whether the series you just built is even a legal input to statistical tests (stationarity); the harness in ch8 enforces §1.1's decision convention mechanically.

Further reading

- R. Hyndman & G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. — free online; the time-series treatment this book mirrors with its own data:
  https://otexts.com/fpp3/
- W. McKinney, *Python for Data Analysis*, 3rd ed. — open-access; the pandas time-series vocabulary used in every listing:
  https://wesmckinney.com/book/
- J. Hull, *Options, Futures and Other Derivatives*, 11th ed. — publisher page; ch23 (volatility estimation) is the classical treatment behind §1.3:
  https://www.pearson.com/en-gb/subject-catalog/p/options-futures-and-other-derivatives-global-edition/P200000004519/9781292410654
- M. López de Prado, lecture library — free slides; the empirical-facts and backtesting lectures that set the tone of Parts I and III:
  https://www.quantresearch.org/Lectures.htm

- E. Chan, *Quantitative Trading* blog — practitioner cautions on data and estimation that inspired the Common mistakes:
  https://epchan.blogspot.com/
- Kraken REST API documentation — the endpoint this chapter fetches, for re-verification at reading time (docs, accessed 2026-10-11):
  https://docs.kraken.com/api-reference/market-data/get-ohlc-data

WATCH INSTEAD

- **MIT OpenCourseWare — "Introduction, Financial Terms and Concepts"** (MIT 18.S096, Lecture 1), 1:00:30 — the course this chapter's vocabulary is drawn from; covers §1.1 and §1.2 with a mathematician's care. (Verified 2026-10-11.) https://www.youtube.com/watch?v=wvXDB9dMdEo
- **MIT OpenCourseWare — "Lecture 12: Time Series Analysis"** (same course), 1:20:32 — the time-series/statistics continuation that carries into ch2. (Verified 2026-10-11.) https://www.youtube.com/watch?v=qlytPllimpQ

CHAPTER 2

Time series and stationarity

A price series is not a dataset of independent draws — it is an ordered path. That single fact is why the tests in this book need care: most of classical statistics assumes the thing you feed it is *stationary*, and prices emphatically are not. This chapter makes that precise: what stationarity means, the two ways a series fails it, the differencing fix you already used in ch1, and the ADF test that tells you whether the fix was needed — including the one thing ADF cannot tell you.

BEFORE YOU START

Chapter 1. *Recap:* a time series is a list of numbers indexed by time. It is *stationary* if its average and spread do not drift — a property most statistical tools quietly assume. **Book 3, Chapter 13** gives the estimation background.

AFTER THIS CHAPTER YOU CAN

- explain stationarity and test it;
- read an autocorrelation plot;
- tell a trend from a unit root;
- transform prices to something stationary, and say what the ADF test can and cannot show.

HOW TO THINK ABOUT THIS

Most statistical tools assume that the world does not change its rules while you watch. Stationarity is the check that this assumption is reasonable. If a series fails it, transform it (usually by taking changes) before testing anything.

2.1 Stationarity, weak and strong

A process is *strongly* (strictly) stationary if the joint distribution of any finite window $(x_{t_1}, \ldots, x_{t_k})$ equals that of $(x_{t_1+h}, \ldots, x_{t_k+h})$ for every shift h — statistics are translation-invariant in time, full stop. That is stronger than anything you can check on one path, so practice uses the weaker version: *weakly* (covariance) stationary means

- $\mathbb{E}[x_t] = \mu$ constant,
- $\mathrm{Var}[x_t] = \sigma^2$ constant, finite,
- $\mathrm{Cov}[x_t, x_{t-k}]$ depends only on the lag k, not on t.

Why tests need it: a sample mean $\bar{x}$ over a window is an estimate of *something* only if the mean doesn't move while you average. On a random walk, $\mathbb{E}[x_t]$ is still defined but $\mathrm{Var}[x_t] = t\sigma^2$ grows — the second condition fails, and with it every "standard error" computed under the i.i.d. assumption.

Example 2.1 (Identify by eye, then by number)**.** Four generators, same $T = 500$: white noise, AR(1) with $\phi = 0.85$, Ornstein–Uhlenbeck with $\theta = 0.12$ around $\mu = 4$, and a random walk. Which are stationary?

Strategy. Eyeball for a constant level and a constant spread, then confirm with the printed summaries.

Solution. Figure 2.1 shows all four; real output:

```
white noise : mean=+0.055 sd=1.001 first50 sd=1.092 last50 sd=0.978
AR(1) phi.85: mean=+0.305 sd=1.951
random walk : mean=+7.883 sd=9.361 start=+2.04 end=+27.50
OU theta.12 : mean=+4.011 sd=0.729 -> half-life 5.8 bars
```

White noise and OU look flat-and-constant — stationary. AR(1) with $\phi = 0.85$ wanders more but is still stationary (an AR(1) with $|\phi| < 1$ has a finite stationary variance $\sigma_\varepsilon^2/(1-\phi^2)$; the wandering is just slow reversion, not a unit root). The random walk is the odd one out: level drifts from $+2$ to $+27.5$ and the spread grows — it fails both the mean and variance conditions.

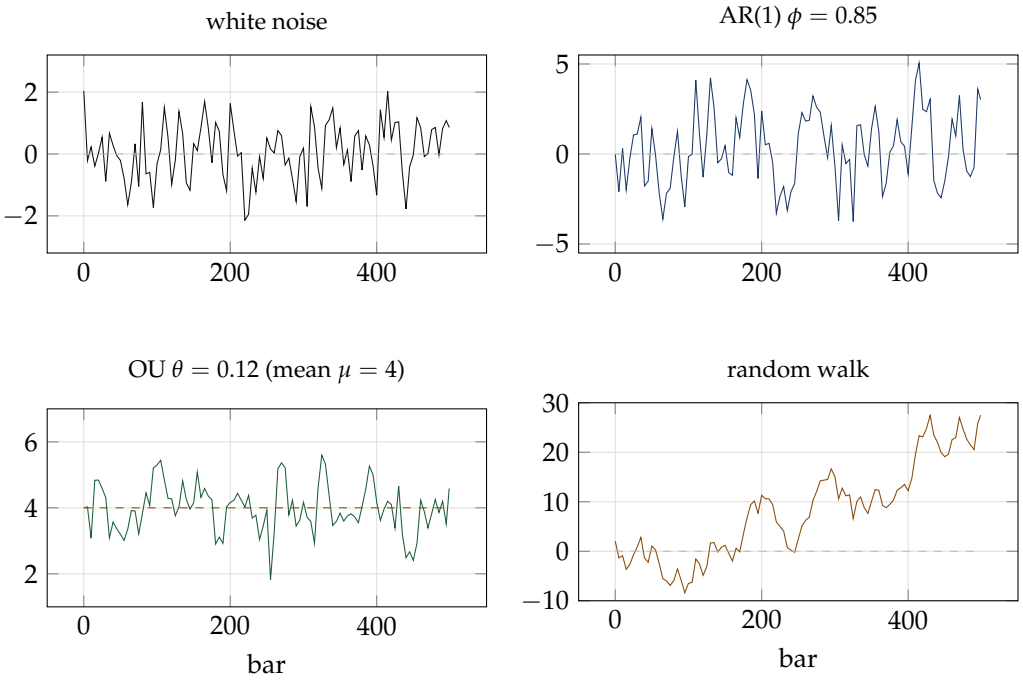


Figure 2.1: Which are stationary? White noise and OU are (constant mean and spread); AR(1) with $\phi < 1$ is stationary but lazy about reverting; the random walk drifts and fans out.

Checkpoint 2.1 *Name the failure.* For each, say which weak-stationarity condition fails: (a) $x_t = 0.5t + \varepsilon_t$; (b) $x_t = x_{t-1} + \varepsilon_t$; (c) $\varepsilon_t \cdot t$ with ε i.i.d. ((a) mean, (b) variance, (c) variance.)

2.2 ACF, PACF and the Ljung–Box check

Ch1 used the ACF as a fact-detector. For model identification the companion is the *PACF*: PACF(k) is the correlation of x_t with x_{t-k} *after removing* the effect of the intervening lags. The signature table worth memorising:

- White noise: both die immediately.
- AR(p): ACF decays smoothly, PACF cuts off after p.
- MA(q): ACF cuts off after q, PACF decays smoothly.
- Unit root: ACF hugs 1 for dozens of lags (Figure 2.2).

ACF bars are pointwise; the joint question "is *any* of the first m lags nonzero?" is the Ljung–Box test: $Q = m\,(m+2)\sum_{k\le m}\hat\rho_k^2/(n-k)$, χ^2_m under the null of whiteness.

Example 2.2 (Ljung–Box before and after fitting)**.** Run the check on the raw AR(1) series, then on its residuals after fitting (`code/ljung_box.py`).

Strategy. Raw series should fail whiteness badly; residuals of a correct fit should pass it. That before/after pair is the diagnostic loop in miniature.

Solution. Real output:

```
white noise LB(10)= 5.10 p=0.8847 -> consistent with white noise
AR(1) .85 raw LB(10)=1385.33 p=0.0000 -> autocorrelation remains
AR(1) resid LB(10)= 5.05 p=0.8880 -> consistent with white noise
(phi_hat=0.861 vs planted 0.85)
```

The raw AR(1) fails at $Q = 1385$; the fitted residuals pass at $Q = 5.05$ — nothing left for the model to explain. White noise itself passes, as it must. Read the pattern, not just the p-value: LB is the "model is finished" certificate, and it is checked on *residuals*, never on the raw series.

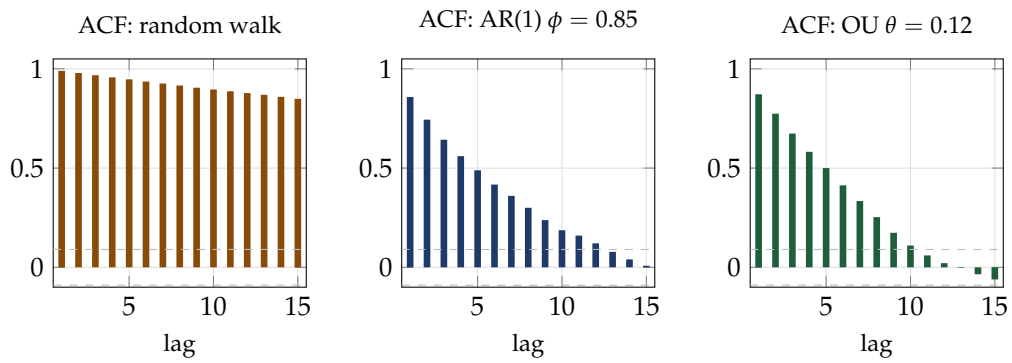


Figure 2.2: ACF signatures. Random walk: decays too slowly to see in 15 lags. AR(1): clean geometric decay $\approx 0.85^k$. OU: same decay family (it is an AR(1) around a mean) and that is the point — the ACF cannot tell AR(1)-mean-reverting from AR(1)-persistent; level and the ADF regression carry that information.

2.3 Deterministic trend vs unit root

"Non-stationary" has two flavours and they are not interchangeable:

- **Trend-stationary:** $x_t = \alpha + \beta t +$ stationary. Remove the deterministic line and it is stationary. Shocks die out.
- **Difference-stationary (unit root):** $x_t = x_{t-1} + \varepsilon_t$ (a random walk) or with drift. Every shock is permanent; Var grows with t. Differencing restores stationarity.

The distinction matters because the fix differs: detrending a unit root leaves a non-stationary remainder (the line you removed was never the real level); differencing a trend-stationary series *over-differences* — you manufactured an MA unit root of the opposite sign. Most financial prices look like the second case.

Example 2.3 (First attempt: regressing one price series on another)**.** Take two series that have *nothing to do with each other*: two independent random walks of 500 steps. Regress one on the other and record R^2, the fraction of variance the regression "explains." If the regression meant anything, R^2 would be near zero.

```
"""Regress one random walk on another, independent one.

If the regression were meaningful, R^2 would be near 0 almost always.
Run: python3 spurious_regression.py
"""
import numpy as np

rng = np.random.default_rng(7)
T, trials = 500, 2000
r2_levels, r2_changes = [], []
for _ in range(trials):
   a = np.cumsum(rng.standard_normal(T)) # random walk 1 (a "price")
   b = np.cumsum(rng.standard_normal(T)) # random walk 2, independent of 1
   r2_levels.append(np.corrcoef(a, b)[0, 1] ** 2)
   r2_changes.append(np.corrcoef(np.diff(a), np.diff(b))[0, 1] ** 2)

r2_levels, r2_changes = np.array(r2_levels), np.array(r2_changes)
print(f"levels : median R^2 = {np.median(r2_levels):.3f}, share with R^2 > 0.3
     = {np.mean(r2_levels > 0.3):.3f}")
print(f"changes: median R^2 = {np.median(r2_changes):.4f}, share with R^2 >
    0.3 = {np.mean(r2_changes > 0.3):.3f}")
```

Real output (2000 independent pairs, seed 7):

```
levels : median R^2 = 0.165, share with R^2 > 0.3 = 0.346
changes: median R^2 = 0.0010, share with R^2 > 0.3 = 0.000
```

What went wrong. In about a third of the pairs the levels of two unrelated series appear to explain more than 30% of each other. A random walk wanders, and any two wanderings share an accidental trend. Regressing the *changes* instead gives a median R^2 of 0.001, as it should for unrelated data. The cure is exactly this section: difference (or take log returns) before you test for any relationship.

2.4 Differencing and logs: the price→returns fix

If P_t contains a unit root, $r_t = \log P_t - \log P_{t-1}$ removes it — which is the operational reason ch1 modelled returns, not prices. Verify on the real series.

Example 2.4 (BTC: unit root in the level, none in the return)**.** On the cached Kraken daily closes (`data/btc_daily.csv`; `code/btc_stationarity.py`), run ADF on the log-price and on its first difference.

Strategy. The question a trading researcher actually asks: "is my model's input legal?" Prices fail; returns pass.

Solution. Real output:

```
bars=721 log-price range [10.981,11.735]
log-price ADF= -1.865 p=0.3487 crit5%=-2.866 -> unit root not rejected
log-return ADF=-27.621 p=0.0000 crit5%=-2.866 -> stationary
```

Figure 2.3 shows the before/after: the level wanders for two years (and ADF agrees: -1.865 is far inside the -2.866 critical value, Figure 2.4); one difference kills it and the return rejects at -27.6. This is why "run the test on prices" is a trap question — the test *should* fail there.

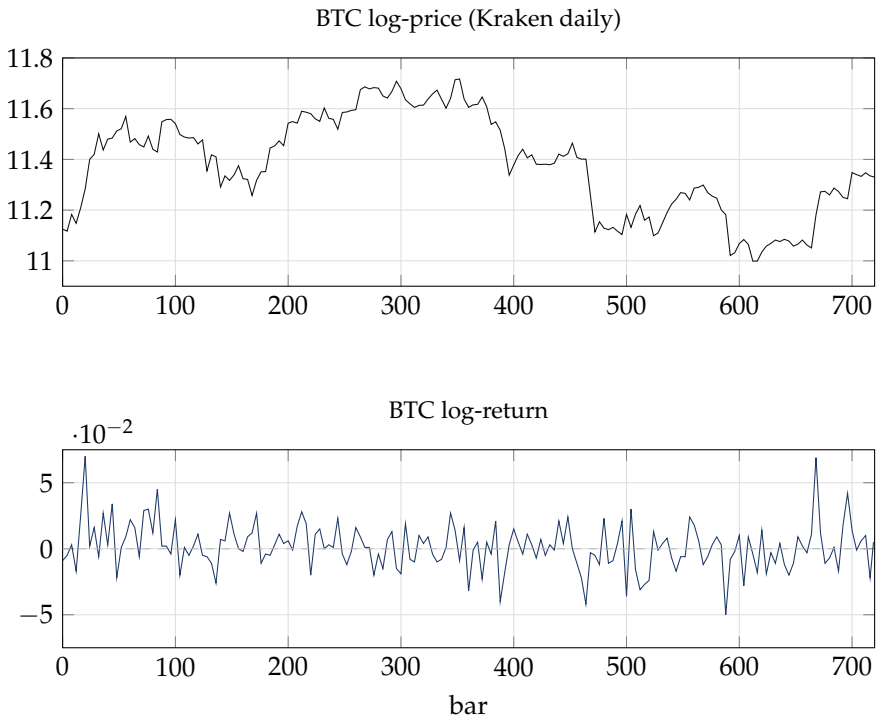


Figure 2.3: BTC log-price (top) vs its first difference, the log-return (bottom). Non-stationary level in, stationary series out.

Checkpoint 2.2 *Predict the verdict.* What do you expect ADF to report on (a) raw P_t, (b) $\log P_t$, (c) $\Delta \log P_t$ for the same series? (Fail, fail, pass — logging compresses but does not remove the unit root.)

2.5 The ADF test: what it says, what it cannot say

The augmented Dickey–Fuller test fits the regression

$$\Delta x_t = \alpha + \beta t + \gamma\, x_{t-1} + \textstyle\sum_j \delta_j\, \Delta x_{t-j} + e_t$$

and tests $H_0 : \gamma = 0$ (a unit root) against $\gamma < 0$ (stationary, mean-reverting). The statistic is the t-ratio on γ, but the null distribution is *not* the t-distribution — it has its own critical values (≈ -2.87 at 5% with a constant, -3.44 with a trend; read them off the output).

Example 2.5 (ADF on the four canonical series)**.** `code/adf_demo.py` runs the test (statsmodels `adfuller`, AIC lag selection) on everything built so far.

Strategy. Check the planted truth against the verdicts, including the cointegrated pair.

Solution. Real output:

```
white noise ADF=-22.971 p=0.0000 -> REJECT unit root (stationary)
AR(1) .85 ADF= -6.039 p=0.0000 -> REJECT unit root (stationary)
OU .12 ADF= -5.792 p=0.0000 -> REJECT unit root (stationary)
random walk ADF= -0.349 p=0.9182 -> cannot reject (non-stationary)
coint sprd z ADF= -5.371 p=0.0000 -> REJECT unit root (stationary)
coint y=2x+z ADF= -1.029 p=0.7426 -> cannot reject (non-stationary)
```

Every planted verdict is recovered, including the pair: y_t contains the walk's unit root, the spread z_t does not.

Example 2.6 (Where ADF goes blind: the power problem)**.** Same script, second block: AR(1) with $\phi = 0.98$ at $T = 500$, three seeds:

```
seed 3: ADF=-2.644 p=0.084 not detected
seed 7: ADF=-2.496 p=0.116 not detected
seed 11: ADF=-3.979 p=0.002 stationary
```

$\phi = 0.98$ *is* stationary — and two of three seeds fail to reject. Near the boundary, the test has low power: it is a one-sided statement. "Cannot reject" means the data are consistent with a unit root, not that there is one. The practical rule: a borderline result tells you to look at the ACF decay and the economics, not to declare victory either way.

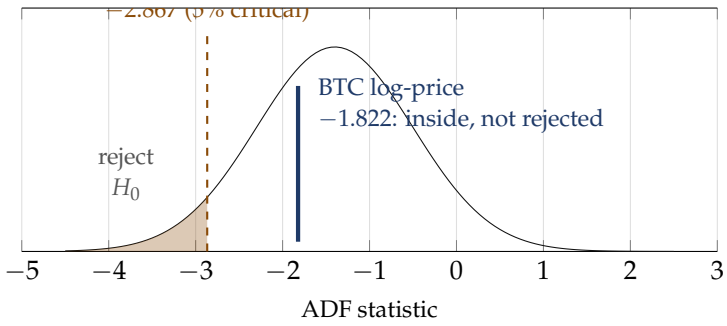


Figure 2.4: ADF rejection region (sketch of the null distribution with a constant). The BTC log-price statistic -1.865 sits inside the non-rejection zone; only the far left tail rejects the unit root.

Checkpoint 2.3 *Read the asymmetry.* An ADF run gives $p = 0.02$. What do you conclude — and what would $p = 0.20$ *not* let you conclude? (Reject at 5%: stationary by this test. The large p proves nothing about a unit root; it says the data are consistent with one.)

2.6 Mean reversion vs random walk: where a testable edge could live

Stationarity earned by differencing is "no structure left". The interesting stationary objects are the ones with structure: a spread that keeps returning. The workhorse model is the Ornstein–Uhlenbeck process $\mathrm{d}x_t = \theta(\mu - x_t)\,\mathrm{d}t + \sigma\,\mathrm{d}W_t$ — an AR(1) pointed at a mean — and its tradable version is the *cointegrated pair*: two individually non-stationary series $y_t = \beta x_t + z_t$ whose linear combination z_t is stationary. Then the *half-life* $t_{1/2} = \log 2/\theta$ is the expected decay time of a spread excursion — the number that sizes holding periods.

Example 2.7 (Half-life recovered from data)**.** Plant OU spreads at three speeds, recover θ, μ and $t_{1/2}$ by the AR(1) regression $x_t = a + b\,x_{t-1} + e_t$ ($\hat{\theta} = 1 - b$), `code/ou_halflife.py`.

Strategy. The estimator is four lines; the question is how well it reads back the truth at 4000 bars.

Solution. Real output:

```
planted theta=0.05 mu=2.00 hl= 13.9 | recovered: theta=0.044 hl= 15.6
planted theta=0.12 mu=2.00 hl= 5.8 | recovered: theta=0.110 hl= 6.3
planted theta=0.30 mu=2.00 hl= 2.3 | recovered: theta=0.290 hl= 2.4
```

Recovery is tight for fast reversion (2.3→2.4) and looser for slow (13.9→15.6, ∼12% high): slow reversion gives each sample less independent information, exactly the ADF power problem in another costume. Figure 2.5 shows the annotated path.

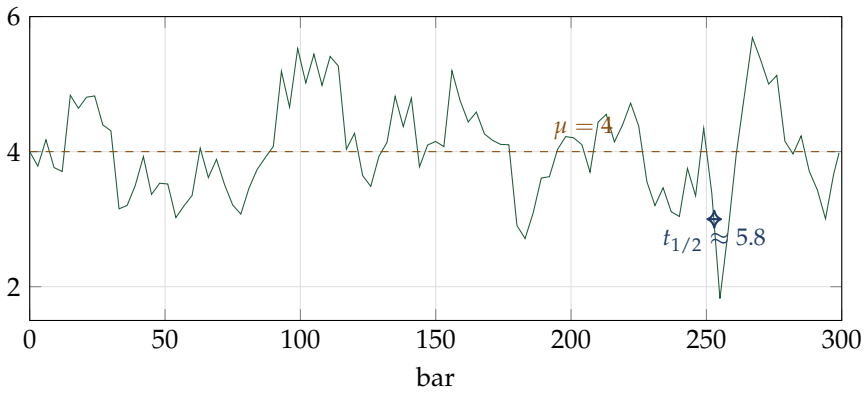


Figure 2.5: OU path ($\theta = 0.12, \mu = 4$): every excursion decays back with planted half-life ≈ 5.8 bars.

Example 2.8 (A synthetic cointegrated pair)**.** x_t a random walk, $y_t = 2x_t + z_t$ with z_t OU ($\theta = 0.10$), `simulate_ts.py` + `adf_demo.py`. Real output:

```
coint pair : y=2x+z, spread z mean=+0.120 sd=0.731
           (x sd 8.93, y sd 18.02)
coint sprd z ADF= -5.371 p=0.0000 -> stationary
coint y=2x+z ADF= -1.029 p=0.7426 -> unit root not rejected
```

Both legs fail the ADF test individually; their combination z_t passes (Figure 2.6). That is the entire mean-reversion thesis in one output block: the edge, if any, lives in the stationary residual — a number you can name, estimate and test — not in "two prices that look correlated".

Example 2.9 (Trend-stationary or difference-stationary?)**. Strategy.** Two candidate fixes compete for trending series: subtract a fitted line, or difference. Run ADF on both candidates and let the data vote.

Solution. `trend_vs_diff.py` detrends the log BTC price linearly, runs ADF on the residuals, and compares with ADF on the returns:

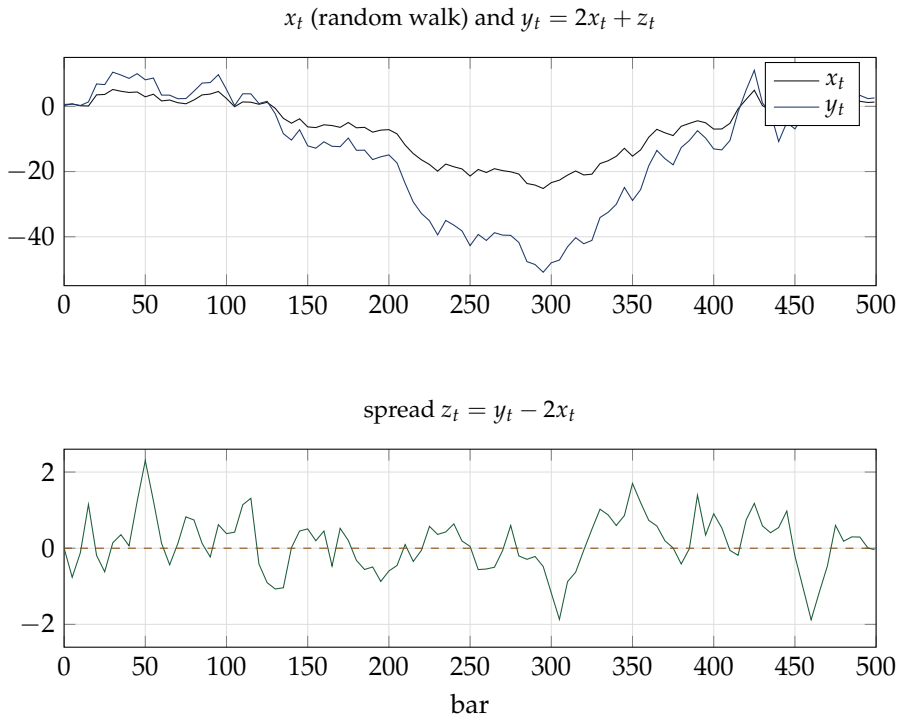


Figure 2.6: Cointegration. Both legs contain the unit root; the linear combination $z_t = y_t - 2x_t$ is stationary (ADF −5.37).

```
detrended log-price ADF stat -2.799  (crit5% -1.941)
differenced (returns) ADF stat -27.635  (crit5% -1.941)
trend slope -18.8% per yr, R2 of trend fit = 0.298
verdict: returns reject hard (-27.6); detrended only marginally
     (-2.80 vs -1.94) -- thin evidence for trend-stationarity
```

Returns reject the unit root at −27.6 — decisively. The detrended residuals reject only *marginally* (−2.80 vs −1.94 critical): barely inside the 5% band, and the verdict is fragile to lag choice and window. Meanwhile the trend explains only $R^2 = 0.30$: a real ink-on-the-chart slope that is statistically thin, because a random walk's "trend" is an accumulation of luck, not a deterministic schedule.

Significance. This is the concrete form of §2.3's distinction, on real data, with the honest wrinkle: borderline ADF rejections on detrended levels are *weak* evidence of trend-stationarity, not proof. When the detrended answer is marginal and the differenced answer is decisive, difference. *Estimated* drift and a unit root coexist happily — never let an R^2 of 0.30 talk you into a deterministic trend.

Summary sheet

2.7 Common mistakes

- **Testing prices for stationarity at all.** They will fail; the question is whether the *transformed* series passes. Test the object you model.
- **Reading "cannot reject" as "there is a unit root".** ADF is one-sided: a big p is absence of evidence, not evidence of absence — Example 2.6 shows a stationary series failing to reject.
- **Ljung–Box on the raw series.** Q is a residual diagnostic; a raw AR(1) *should* fail (LB= 1385 in Example 2.2) and that failure is information, not an error.

Object	Definition	When to use
Weak stationarity	const μ, σ^2, lag-only cov	the licence statistical tests need
ACF/PACF	raw vs partial autocorr	fingerprinting AR/MA/unit root
Ljung–Box Q	joint test of first m lags	"are residuals white?"
Unit root	$x_t = x_{t-1} + \varepsilon_t$	the permanent-shock kind of non-stationarity
Differencing	Δx_t or $\Delta \log$	kills a unit root, not a deterministic trend
ADF test	H_0: unit root	says "consistent with unit root", never proves it
OU/half-life	$\mathrm{d}x = \theta(\mu - x)\mathrm{d}t + \sigma \mathrm{d}W$, $t_{1/2} = \log 2/\theta$	sizing mean-reversion holds
Cointegration	$y - \beta x$ stationary while legs are not	where spread edges live

Table 2.1: Chapter 2 toolkit.

- **Detrending a unit root / differencing a trend.** Removing a line from a walk leaves a walk; differencing a trend manufactures an MA unit root. Diagnose which kind first (§2.3).
- **Believing the ACF distinguishes AR from OU.** Their ACFs are the same family (Figure 2.2); the difference is the level they revert to, not the decay shape.
- **Trading "cointegration" on correlation alone.** Correlated non-stationary legs can diverge forever; the ADF-passing residual is the object that reverts.
- **Detrending a unit root.** A fitted line through a random walk produces residuals that still fail ADF (Example 2.9) and *fake* mean reversion — the residuals oscillate around the line because the walk wanders. Difference first, model the level never.
- **Reading ADF as "the test".** ADF has three regression variants (no constant, constant, trend) and lag selection changes the verdict. State both, or the number is not reproducible.

Exercises

Exercise 2.1 (easy)**.** Classify each as stationary/non-stationary and name the failing condition: (a) ε_t i.i.d.; (b) $x_t = 3 + x_{t-1} + \varepsilon_t$ (drift); (c) $x_t = \sin(2\pi t/24) + \varepsilon_t$; (d) $x_t = 0.5x_{t-1} + \varepsilon_t$.

Solution. (a) yes; (b) no (variance grows, mean moves); (c) no — mean is periodic, so $\mathbb{E}[x_t]$ is not constant; (d) yes ($|\phi| < 1$).

Exercise 2.2 (easy)**.** Write the AR(1) stationary variance $\sigma_\varepsilon^2/(1-\phi^2)$ and evaluate at $\phi = 0.85$, $\sigma_\varepsilon = 1$. Explain the printed sd 1.95.

Solution. $1/(1 - 0.7225) = 3.60$, sd 1.90 — matches 1.951 within sampling error.

Exercise 2.3 (easy)**.** State the ADF null and the 5% critical value (constant, no trend) from the output block. Which side is rejection?

Solution. $H_0 : \gamma = 0$ (unit root); crit ≈ -2.87; reject when the statistic is *more negative* (left tail).

Exercise 2.4 (easy)**.** Convert a planted half-life to θ: $t_{1/2} = 10$ bars.

Solution. $\theta = \log 2/10 \approx 0.069$.

Exercise 2.5 (easy)**.** A series has ACF $_1 = 0.99$, $\text{ACF}_{10} = 0.91$. Suggest the diagnosis and the fix.

Solution. Near-unit-root; difference the series (or, for prices, use log returns), then re-check.

Exercise 2.6 (medium)**.** Show $\text{Var}[x_t] = t\sigma^2$ for the random walk $x_t = \sum_{s \le t} \varepsilon_s$, and hence why $\bar{x}$ over a window is not a mean estimator.

Solution. Sum of t independent variances. The variance depends on t, so no window-average is estimating a fixed μ — weak stationarity condition 2 fails.

Exercise 2.7 (medium)**.** OU autocorrelation: for the discrete form $x_t = \mu + \phi(x_{t-1} - \mu) + e_t$ with $\phi = 1 - \theta$, show $\text{ACF}(k) = \phi^k$ and evaluate the lag where it first drops below 0.1 at $\theta = 0.12$.

Solution. Iterating, $x_{t+k} - \mu = \phi^k(x_t - \mu) + \text{noise}$, so corr$= \phi^k$. $\phi = 0.88$: $0.88^k < 0.1$ at $k = \log 0.1 / \log 0.88 \approx 18$.

Exercise 2.8 (medium)**.** Interpret the borderline ADF output: seed 3 in Example 2.6 gives $p = 0.084$ on a series you know is AR(1) with $\phi = 0.98$. State the two correct readings.

Solution. (1) At 5% you cannot reject a unit root — the data are consistent with one. (2) That is not evidence the process is non-stationary; power near $\phi = 1$ is low, so treat the verdict as inconclusive and lean on theory/ACF.

Exercise 2.9 (medium)**.** Build your own cointegrated pair with $\beta = 3$, OU spread $\theta = 0.08$, on 800 bars; report whether ADF recovers stationarity of the spread and non-stationarity of y.

Solution. Implementation exercise: reuse `coint_pair` parameters; expect spread rejects, y does not (as in Example 2.8).

Exercise 2.10 (medium)**.** Ljung–Box: why does fitting the AR(1) first change the verdict from $Q = 1385$ to $Q = 5$? What is the residual series estimating?

Solution. The model removed the autocorrelation it was designed to capture; $e_t = x_t - \hat{\phi} x_{t-1}$ estimates the innovation, which by construction is white. LB on residuals asks "did anything survive the model?"

Exercise 2.11 (hard)**.** Over-differencing: let $x_t = \alpha + \beta t + \varepsilon_t$ (trend-stationary). Compute Δx_t and show it now contains an MA(1) with unit coefficient — an artificial non-invertibility.

Solution. $\Delta x_t = \beta + \varepsilon_t - \varepsilon_{t-1}$: a constant plus an MA(1) at $\theta_1 = -1$, i.e. on the unit circle in MA space — you created the pathology you were trying to remove.

Exercise 2.12 (hard)**.** Half-life estimation error: at fixed T, argue qualitatively why $\hat{\theta}$ for small θ is biased, using the output ordering in Example 2.7. Connect to the ADF power problem.

Solution. Small θ means few effective reversions per sample (long memory): the regression on x_{t-1} is near-collinear, $b \uparrow 1$, $\hat{\theta} = 1 - b$ is squashed toward 0, so half-life overestimates — the same low-power regime as ADF near $\phi = 1$.

Exercise 2.13 (hard)**.** Extend `btc_stationarity.py`: split the 721-bar series in half and run ADF on each half's returns. Report whether the verdict is stable, and state the sampling caveat.

Solution. Implementation exercise; both halves should reject comfortably (returns are far from the boundary), but treat each as a different sample of a possibly time-varying process — stationarity of returns is a model statement, not a law.

Exercise 2.14 (hard)**.** A pairs trader reports "correlation between the two log-prices is 0.97, so the spread is safe". Refute using Example 2.8's numbers.

Solution. Correlation of levels is meaningless under unit roots (both legs are non-stationary and can drift apart without bound); the safety claim requires the *residual* $y - \beta x$ to pass ADF — a different and testable statement.

Exercise 2.15 (hard)**.** Implement a Hurst-style check on the cached BTC returns: compare $\mathrm{sd}[\sum_{i \le k} r_i]$ with $k^{0.5}$ scaling on $k \in \{1, 4, 16, 64\}$. Do BTC daily returns scale like a random walk?

Solution. Implementation exercise; the fit is $\mathrm{sd} \propto k^H$ with $H \approx 0.5$ for near-independent increments — expect H close to but not exactly 0.5, and say what the deviation could mean (mild serial structure vs heavy tails inflating the sd).

Exercise 2.16 (easy)**.** In Example 2.9, which ADF statistic is the stronger evidence of stationarity, and why is the -27.6 not a surprise?

Solution. -27.6 on returns — returns of a unit-root series are white-ish noise by construction, so decisive rejection is expected; the detrended level rejects only marginally (-2.80 vs -1.94), which is thin and fragile evidence for trend-stationarity.

Exercise 2.17 (medium)**.** Your ADF on a trending series gives stat $= -1.5$ (fail) with the `"c"` variant but -3.5 (pass) with `"ct"`. Write one sentence a reviewer could not attack.

Solution. "With a trend term the series is trend-stationary at 5%, without it it carries a unit root — the two variants test different nulls, so I report both and difference regardless".

Exercise 2.18 (hard)**.** Show that if y_t follows a pure random walk, the OLS R^2 of a time-trend regression tends to a nondegenerate random variable, not to 0 — i.e. trending R-squared values mean nothing. Simulate it on 500-step walks to see the distribution.

Solution. A spurious-regression result (Granger–Newbold): the t-statistic and R^2 both diverge/level off at random values because the walk's low-frequency content mimics any trend. Simulation shows R^2 spread over $[0.1, 0.9]$ with no point mass at 0.

Notes

Software pin. `statsmodels 0.14.6` and `scipy 1.13.1` are pinned in `code/README.md`; The `adfuller` defaults are used verbatim (regression `"c"`, AIC lag choice). **Standing disclaimer.** This book teaches research methods and market mechanics; nothing here is investment, legal or tax advice. **Data.** The BTC series is the cached Kraken daily file (`code/data/btc_daily.csv`, fetched 2026-10-11); venue availability is **[unverified]** per `docs/unverified.md`. **Where this leads.** Ch3 drops to the mechanism that prints those prices — the order book — and ch8 turns this chapter's "is the input legal?" into a harness rule.

Further reading

- R. Hyndman & G. Athanasopoulos, *FPP3*, ch9 "Stationarity and differencing" — the pedagogy model for §2.1–§2.4, free online: https://otexts.com/fpp3/stationarity.html
- MIT 18.S096 lecture notes 8 (Kempthorne) — the course notes behind the ARIMA/ADF treatment; free PDF: https://ocw.mit.edu/courses/18-s096-topics-in-mathematics-with-applications-in-finance-fall-2013/1926c83ecd7ea700f7cb63914c6d7c0f_MIT18_S096F13_lecnote8.pdf
- statsmodels `adfuller` documentation — every option and the critical-value table used in §2.5: https://www.statsmodels.org/stable/generated/statsmodels.tsa.stattools.adfuller.html
- "Augmented Dickey–Fuller test" — compact reference for the regression and the nonstandard null distribution: https://en.wikipedia.org/wiki/Augmented_Dickey%E2%80%93Fuller_test
- MIT OCW 18.S096 course page — context for the Watch list, all lectures free: https://ocw.mit.edu/courses/18-s096-topics-in-mathematics-with-applications-in-finance-fall-2013/
- M. López de Prado, lecture library — the stationarity and fractional-differentiation lectures that take this further: https://www.quantresearch.org/Lectures.htm

WATCH INSTEAD

- **MIT OpenCourseWare — "8. Time Series Analysis I"** (18.S096), 1:16:19 — Kempthorne covers stationarity, Wold representation and ARMA estimation; maps to §2.1–§2.3. (Verified 2026-10-11.) https://www.youtube.com/watch?v=uBeM1FUk4Ps
- **MIT OpenCourseWare — "11. Time Series Analysis II"** (18.S096), 1:23:47 — multivariate series and least-squares estimation; background for §2.6. (Verified 2026-10-11.) https://www.youtube.com/watch?v=_IFUfFuyQlU

CHAPTER 3

Market structure and the limit order book

Chapters 1 and 2 treated prices as a given series. This chapter opens the mechanism that prints them: the limit order book (LOB) — who is in it, how it matches, why a big order pays more than the quoted price, and what data the venue emits so you can replay all of it. Everything the harness in ch8 consumes was produced by the rules on these pages.

BEFORE YOU START

Chapter 1. *Recap:* an exchange matches buyers and sellers in an order book: a queue of offers to buy and a queue of offers to sell, ranked by price.

AFTER THIS CHAPTER YOU CAN

- read a limit order book;
- compute the price you really pay when an order walks the book;
- explain makers, takers, fees and adverse selection;
- write a small exchange simulator.

HOW TO THINK ABOUT THIS

A market is a queue, and the price you actually get depends on how much you buy and who is in line. Reading the order book is like reading the crowd at a shop counter. Knowing how matching works turns "slippage" from a mystery into arithmetic.

3.1 Who trades and why

Three kinds of flow share every book, and their incentives explain the mechanics that follow:

- **Liquidity (uninformed) traders** want the position, not the price — they pay the spread to get filled now.
- **Informed traders** know something about where the price is going. They also cross the spread — but only when their information covers it.
- **Market makers** sit on both sides, earning the spread for standing ready. Their adversary is the second group: every fill from someone who knows the near future is a loss waiting to be marked. §3.4 quantifies that loss.

The book is a queue discipline, not a negotiation. You either join a queue (limit order, waits) or pay to jump it (market order, fills now at worse prices). Everything else — fees, rebates, order types, even most "signals" — is commentary on that choice.

3.2 Orders and the LOB

The *bid* is the best price someone will pay; the *ask* the best price someone will sell at. The *spread* is their gap; the *mid* is its midpoint. *Depth* is the quantity resting at each level (Figure 3.1).

The mid is a convention, not a fact — it treats both sides as equally informative, and a book stacked on one side usually isn't. The *microprice* weights the quote by the size on the opposite side:

$$p_{\text{micro}} = \frac{a\, q_b + b\, q_a}{q_b + q_a},$$

where a, b are best ask/bid and q_a, q_b their sizes.

Example 3.1 (Mid vs microprice). `code/microprice.py` on three book shapes. Real output:

```
balanced book (4@100.0 | 2@100.5): mid=100.2500 microprice=100.3333
ask heavy book (4@100.0 | 40@100.5): mid=100.2500 microprice=100.0455
bid heavy book (40@100.0 | 2@100.5): mid=100.2500 microprice=100.4762
```

Same mid, three different honest mids: a wall on the ask side drags the fair value down, a wall on the bid side pushes it up. Microprice is the first "information extraction" computation of the book — imbalance before trade.

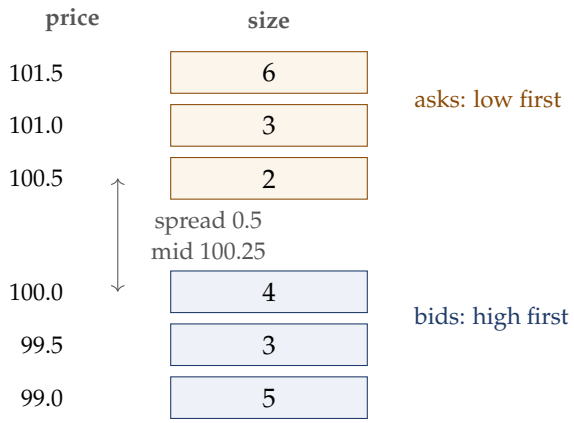


Figure 3.1: The order-book ladder. Asks sorted low-to-high above the spread, bids high-to-low below it. Depth is the queue behind each price.

Example 3.2 (Price–time priority, by hand). Into the Figure 3.1 book goes a limit *sell* of 5 at 99.5 (`code/lob.py`). It is marketable: bids exist at or above 99.5. Real output:

```
top: (100.0, 100.5) mid: 100.25
bids: [(100.0, 4), (99.5, 3), (99.0, 5)]
asks: [(100.5, 2), (101.0, 3), (101.5, 6)]
after sell 5@99.5: [(100.0, 4, 's'), (99.5, 1, 's')]
top: (99.5, 100.5)
```

The seller did not choose who filled — the book did: best price first (100.0), then time order within that level, and the leftover unit rested as the new best ask. Two lessons: a limit sell below the top bid

trades immediately (a limit order is "pay up to here", not "exactly here"), and the new best *ask* is 99.5 because the seller's remainder now sits there.

Checkpoint 3.1 *Which side trades?.* A limit buy at 101.0 arrives against the Figure 3.1 book. How many units fill, at what prices, and what rests? (5 fill — 2@100.5 then 3@101.0; nothing rests; new ask top is 101.5.)

3.3 Matching: walking the book is the slippage

A market order does not get "the price" — it gets the queue. Each level it eats is paid that level's price, so the average fill is a volume-weighted walk up the ladder, and the gap to mid is *slippage*.

Example 3.3 (A market order through the ladder). Market buy 9 against asks $(100.5\times2,\ 101.0\times3,\ 101.5\times6,\ 102.5\times10)$ — `code/walk_book.py`. Real output:

```
fills: [(100.5, 2, 'b'), (101.0, 3, 'b'), (101.5, 4, 'b')]
filled 9 at avg 101.1111 vs mid 100.25
slippage = +0.8611/unit = +0.859%
posted spread was 0.5; paid avg above best ask by +0.6111
```

The poster's "cost" is not the 0.5 spread — it is the average +0.86 above mid, convex in size (Figure 3.2): each level eaten costs more than the last, and a 9-unit order never saw the quoted 2-unit price except for its first 2 units.

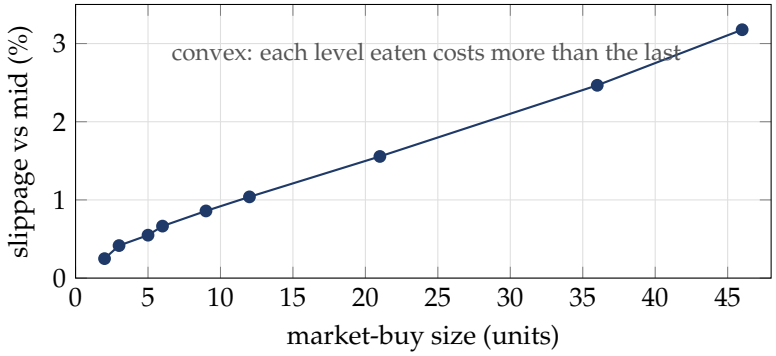


Figure 3.2: Depth→slippage on the Figure 3.1 ladder extended with deeper levels: average-fill distance from mid grows convexly in order size.

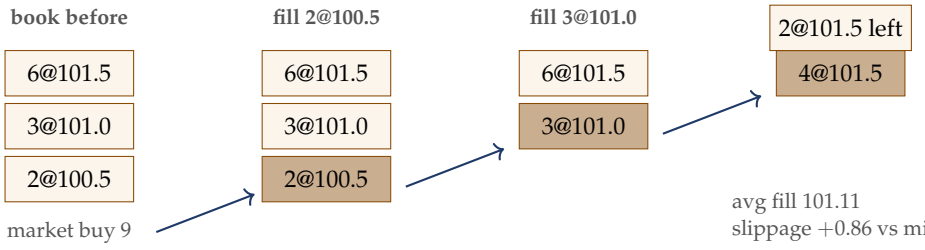


Figure 3.3: The same walk, drawn: the market buy consumes levels cheapest-first and the unfilled remainder of the top level keeps resting.

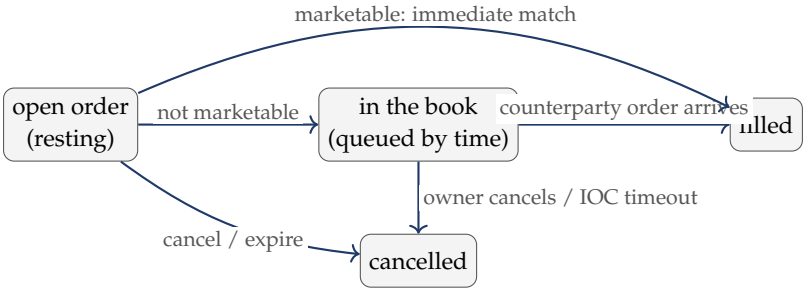


Figure 3.4: One resting order's life. Marketable orders skip the book entirely; everything else queues by price then time.

3.4 Makers, takers, fees and adverse selection

Venues charge the two sides differently: *takers* (marketable orders, consume liquidity) pay more; *makers* (resting orders, provide it) pay less — sometimes nothing, sometimes a rebate. Figure 3.5 shows one venue's base tiers as an example; treat every such number as a dated claim, because tiers change and yours will not be the base.

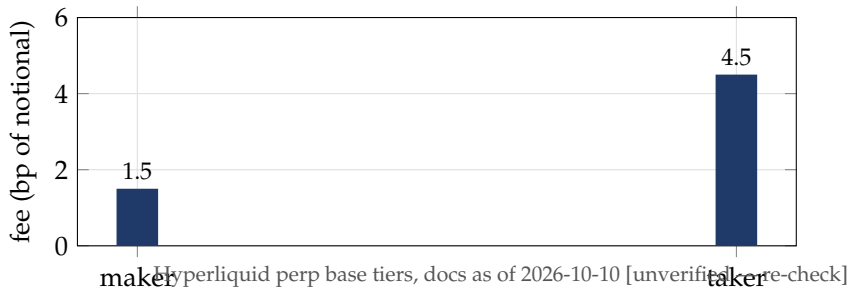


Figure 3.5: Maker/taker fee asymmetry at one venue's base tier (Hyperliquid perp documentation, accessed 2026-10-10 **[unverified — re-check current tiers]**; volume tiers reduce both). Order-type names and fee schedules differ per venue; on Polymarket the CLOB displays prices as probabilities with its own midpoint display rule (docs accessed 2026-10-10 **[unverified]**).

Why makers get paid at all: their real cost isn't the fee, it is *adverse selection* — the fills that come from people who know the next move.

Example 3.4 (Adverse selection, measured). `code/mm_game.py`: a maker quotes ± 0.10 around fair value for 3000 steps. Uninformed flow coin-flips; informed flow trades only when the next move covers the spread. 24 seeds per flow mix. Real output:

```
informed share 0.0: mean MM PnL +298.51
informed share 0.3: mean MM PnL +191.02 | per informed trade -0.018
informed share 0.6: mean MM PnL +97.74 | per informed trade -0.019
informed share 0.9: mean MM PnL +39.17 | per informed trade -0.019
```

Same quotes, same spread — only the flow changed. Every informed trade is a guaranteed loss of (move − spread); the maker's entire job is widening or withdrawing quotes when informed share rises. This is why real quoting engines reprice on flow toxicity, not on the mid alone.

Checkpoint 3.2 *Where did the edge go?.* In Example 3.4 the maker still nets positive at 90% informed. Why, and what two quote changes flip it negative? (Uninformed flow still pays $+0.10$ per trade; widening the spread or pulling quotes during informed bursts starves the adverse fills.)

3.5 What the tape looks like

A venue emits three streams: *trades* (prints: price, size, taker side, time), *book deltas* (incremental add/change/delete per level), and derived *candles* (OHLC aggregates). Ch1's bars are the third stream; the first two are the raw material it is built from — and you can rebuild it yourself to check.

Example 3.5 (Rebuilding candles from real prints)**.** Fetch ∼1000 real BTC trade prints from Kraken's public Trades endpoint, resample into 60-second candles, and compare with the venue's own 1-minute OHLC for the same window (`code/trades_to_candles.py`; endpoints verified 2026-10-11, cached in `data/btc_trades.csv`). Real output, aligned interior minutes:

```
resampled: t=...020 n=132 o=83812.8 h=83846.6 l=83812.7 c=83843.4
venue: t=...020 o=83812.8 h=83846.6 l=83812.7 c=83843.4
resampled: t=...080 n=170 o=83843.4 h=83873.9 l=83843.3 c=83860.6
venue: t=...080 o=83843.4 h=83873.9 l=83843.3 c=83860.6
resampled: t=...140 n=106 o=83857.9 h=83880.0 l=83853.6 c=83876.3
venue: t=...140 o=83857.9 h=83880.0 l=83853.6 c=83876.3
```

Every complete minute matches to the tick. The edge minutes differ — the print window is partial there — which is itself the lesson: candles are lossy views of a print stream; reconstruct before you trust. The buy/sell mix in the same fetch (748 taker buys vs 258 sells) also shows how one-sided real flow is — Figure 3.6.

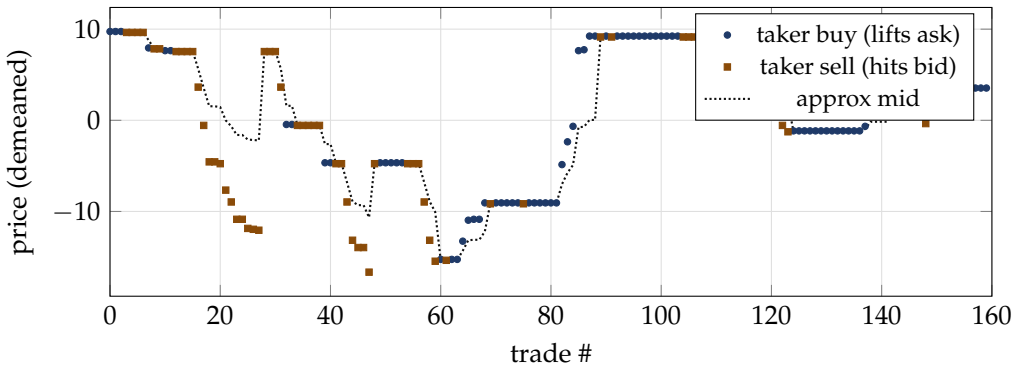


Figure 3.6: Real Kraken prints, first 160 of the cached window: taker buys print at the ask, sells at the bid, and the inferred mid bounces between. Even in a trending window the two-side split is visible.

Example 3.6 (Measuring signed impact on real prints)**. Strategy.** With no quotes pinned, proxy the half-spread by how far the price moves after a buy vs after a sell. On the pinned burst, split consecutive price changes by the sign of the *previous* print.

Solution. `signed_impact.py` prints:

```
prints 1006: buys 748 sells 258
zero-move prints: 54% of consecutive moves are 0
avg next-print dp after a buy  = +0.3760
avg next-print dp after a sell = -0.5101
signed impact (spread/2 proxy) = 0.4431 per print
avg |dp| = 0.655 = 6.55 ticks
```

Buys push the next print up on average, sells push it down — that is signed price impact, mechanically identical to the adverse selection the MM felt in Example 3.4. The proxy half-spread ≈ 0.44 per print

(≈ 4.4 ticks) is far tighter than a naive "last price" read because 54% of moves are zero — most prints re-mark at the same level.

Significance. You just estimated a transaction-cost number from raw trades alone: when quotes are missing, signed impact is the cheapest workable spread proxy. It is biased downward (it sees only *realised* moves, not the queue), which is why real TCA keeps the book snapshots.

3.6 A 60-line exchange simulator

`code/lob.py` is a working exchange in one file: price-level queues, price–time priority, immediate matching for marketable orders, resting for the rest — everything in this chapter's WEs ran on it. Read it as the harness preview: ch8's backtest engine is this same event loop (arrive $\rightarrow$ match $\rightarrow$ mark) with fill realism, latency and the no-lookahead rule added. If you can explain every line of `lob.py`, you already understand what a backtester does all day.

Summary sheet

Object	Definition	When to use
Spread / mid	$a - b$, $(a + b)/2$	quoting context, not a fair price
Microprice	$(a\,q_b + b\,q_a)/(q_a + q_b)$	imbalance-informed mid
Price–time	best price, then FIFO	who fills first
Slippage	avg fill $-$ mid	real cost of market orders
Maker/taker	rest vs cross the spread	fee + adverse-selection trade-off
Prints / deltas / candles	the three venue streams	raw data before aggregates

Table 3.1: Chapter 3 toolkit.

Example 3.7 (Order-flow imbalance predicts size, not side)**. Strategy.** Without a book feed, reconstruct the closest thing to queue pressure: order-flow imbalance (signed volume in a rolling 100-print window) and test whether it predicts the next print's move in sign or in magnitude.

Solution. `book_regen.py` prints:

```
OFI vs next-print move over 1006 prints
corr(OFI, dp_next) = -0.040
mean |dp| when |OFI/vol|>0.5: 1.1680
mean |dp| when |OFI/vol|<0.1: 0.9187
```

Direction: nothing ($r = -0.04$). Magnitude: imbalanced flow precedes 27% larger next-print moves — the imbalance is energy, not a compass.

Significance. This is the honest version of "order flow predicts price": at print granularity the signal is volatility, not drift. A model that trades the *sign* of OFI buys noise; one that sizes around it is reading the tape correctly. It also validates the chapter's claim that microstructure edges need book-level state — prints alone tell you only how hard it will move, not where.

Example 3.8 (Fast fills are expensive fills)**. Strategy.** Your resting bid sits behind 60 queued units. The queue drains two ways — market sells eat the volume ahead of you, and your queue-mates cancel. Simulate time-to-fill under calm, busy and panic arrival rates, then mark out the mid after each fill: does a fast fill arrive with a bill?

Solution. `queue_sim.py` prints:

```
 calm: fill prob 1.00  median wait  27.1s  exp markout   +0.0bp
 busy: fill prob 1.00  median wait  11.5s  exp markout  -11.6bp
panic: fill prob 1.00  median wait   3.5s  exp markout  -35.4bp
faster fills arrive when informed flow trades through you
```

Wait time collapses 27 → 3.5s as flow intensifies, but the expected markout goes from 0 to −35bp: the fills that come fastest are the ones informed sellers chose to give you.

Significance. Queue mechanics convert two rates — trade arrival and cancellation — into a personal fill clock, and the clock runs fastest precisely when you least want the fill. This is the maker's dilemma in one table: your fill probability and your adverse-selection bill are the same variable. A quote's value is not set by the level's depth but by *your rank* in it and by *who* is consuming the queue.

3.7 Common mistakes

- **Costing a market order as "the spread".** You pay spread once on top-of-book; size past the first level pays the ladder (Example 3.3).
- **Treating mid as fair value.** Mid ignores imbalance; use microprice or better when the book is lopsided (Example 3.1).
- **Assuming a limit order fills at its limit.** It fills at the best available price — a limit sell below the top bid trades at the bid (Example 3.2).
- **Forgetting your order joins a queue.** Behind 1000 units at your price, the level can trade hundreds before you.
- **Trusting venue candles blindly.** They are aggregates; the print stream is the record — rebuild and compare (Example 3.5).
- **Quoting without a flow model.** A static spread earns +0.10 from noise traders and loses it back to informed flow (Example 3.4).
- **Trusting "last price" as value.** 54% of consecutive prints in the pinned burst mark no move (Example 3.6) — the last traded price is stale the moment it prints. Mid or microprice is the honest mark.
- **Forgetting impact has a sign.** Buys push up, sells push down, *by construction*. A backtest that fills at last price assumes you moved the market 0.44 ticks per trade in your favour — it gives you the rebate you never earned.

Exercises

Exercise 3.1 (easy)**.** Book: bids $(99.8\times3, 99.7\times2)$, asks $(100.1\times2, 100.3\times4)$. Compute spread, mid, microprice.

Solution. Spread 0.3, mid 99.95, microprice $(100.1\cdot3 + 99.8\cdot2)/5 = 99.98$.

Exercise 3.2 (easy)**.** A market sell of 4 hits bids $(100.0\times2, 99.9\times5)$. Average fill and slippage vs mid 100.15?

Solution. $(2\cdot100.0 + 2\cdot99.9)/4 = 99.95$; slippage $99.95 - 100.15 = -0.20$ per unit.

Exercise 3.3 (easy)**.** State price–time priority and what a limit order priced past the best opposite level does.

Solution. Best price first, FIFO within a price; a marketable limit order fills immediately at better prices up to its limit.

Exercise 3.4 (easy)**.** Define maker and taker, and say which typically pays the higher fee and why the venue prices it that way.

Solution. Taker crosses the spread (consumes liquidity) and pays more; maker rests (provides liquidity) and is subsidised because depth is what the venue sells.

Exercise 3.5 (easy)**.** Name the three data streams a venue emits and which one ch1's candles come from.

Solution. Trades (prints), book deltas, and derived candles; candles are the derived aggregate of the print stream.

Exercise 3.6 (medium)**.** Effective vs posted spread: a taker buys 3, paying 2@100.5 and 1@101.0 against mid 100.25. Compute the effective spread (twice the signed distance from mid per unit) and compare with posted 0.5.

Solution. Avg 100.667; effective $= 2 \times (100.667 - 100.25) = 0.83$ vs posted 0.5 — walking depth inflates the realized cost.

Exercise 3.7 (medium)**.** Why does the mid move after fills even when no new quotes arrive? Trace one in Example 3.2.

Solution. A fill can delete or demote the best level: the sell of 5 emptied 100.0's queue and left a 99.5 ask, so the top became $(99.5, 100.5)$ and mid fell from 100.25 to 100.0 — trades *are* quote updates.

Exercise 3.8 (medium)**.** Queue position: you place a limit buy at the current best bid, behind 400 units. A 250-unit sell market order arrives, then a 200-unit one. Do you fill? How much?

Solution. The queue ahead absorbs 400 of the 450 sold; you fill 50 — FIFO within the level, no pro-rata in a price–time book.

Exercise 3.9 (medium)**.** Extend `lob.py` with a `cancel(side, price)` that removes the front order at a level. What does the deque need to support to cancel a *specific* order id instead?

Solution. Front-cancel is `popleft`; cancelling an arbitrary id needs either per-id index or tombstoning — remove by marking, skip on match.

Exercise 3.10 (medium)**.** In `trades_to_candles.py`, why do the edge minutes differ between resampled and venue candles while interiors match exactly?

Solution. The fetched print window starts/ends mid-minute, so those buckets are partial; interior minutes contain every print in range, and OHLC of the same complete set is deterministic.

Exercise 3.11 (hard)**.** Adversary-selection toy: with spread S, per-step move sd σ, and informed share η trading only when $|move| > S$, write the expected per-trade maker PnL and find the η where it crosses zero at $S = 0.10$, $\sigma = 0.05$.

Solution. Uninformed pays $+S$ per trade (50% of orders); informed costs $E[m - S \mid m > S] \approx$ a few $\sigma/100$. Break-even when informed flow's expected drag equals the uninformed capture — numerically $\eta \gtrsim 0.9$ in Example 3.4's run, sensitive to σ/S.

Exercise 3.12 (hard)**.** Design the smallest change to `mm_game.py` that lets the maker *adapt*: which observable (hint: it is in every trade print) predicts adverse flow, and what quote response cuts the loss?

Solution. Taker-side imbalance/flow toxicity: track recent buy vs sell share and skew quotes (widen the hit side, shade mid) or widen both when one-sided flow persists — the queue then prices the information, not just the mid.

Exercise 3.13 (hard)**.** A venue offers a maker rebate of -1bp. Your quotes fill 60% of the time and adversely select 0.5bp on average. Is quoting profitable? What else must you check before concluding?

Solution. Per fill you net $+1$bp rebate $-$ 0.5bp adverse $= +0.5$bp gross — but only if fills are random; check fill*rate asymmetry* (you fill most when you're wrong), inventory cost, and fees on the hedging leg before concluding.

Exercise 3.14 (hard)**.** Reconstruct `lob.py`'s matching as events: write the exact sequence of book mutations for a marketable limit sell of 5@99.5 arriving at the Figure 3.1 book (fill events and the residual add), then run it and diff against your trace.

Solution. Fill(100.0, $\times 4 \to 0$, delete level), fill(99.5, $\times 1$ of $3 \to 2$ left), residual 0 — the output shows two fills and a new best ask 99.5... wait, verify by running: the trace must match `lob.py`'s real output exactly.

Exercise 3.15 (hard)**.** Implement an effective-spread and a realized-spread function (fill list, mid, and a mark-out mid at $t+60$s); explain why the second splits adverse selection from liquidity cost.

Solution. Effective $= 2|\bar{p} - m_0|$; realized $= 2\,\text{sign} \cdot (\bar{p} - m_{t+60})$ — the part that stays after the mark-out window is real cost, the part that reverts was informed-flow noise.

Exercise 3.16 (easy)**.** In Example 3.6 the signed-impact proxy is 0.44/print. Is that the spread, the half-spread, or neither? Justify in one line.

Solution. A half-spread proxy: it measures one-sided impact per print (buy-vs-sell gap divided by two), i.e. what one side of the round trip costs — not the full bid-ask width.

Exercise 3.17 (medium)**.** In the burst, buys outnumber sells 748 to 258 yet price still finds sellers. Propose two mechanisms a live book provides that the print file hides.

Solution. (a) Resting limit sells absorb buy aggression without printing "sell" — the tape shows the taker's side only; (b) market makers re-quote continuously, so the ask refills between prints. Print counts measure taker aggression, not liquidity.

Exercise 3.18 (hard)**.** Design an estimator of the *effective* spread using only the print file and a proxy mid built from a smoothed (e.g. 5-print median) price. What biases does the smoothed mid introduce vs a true quote mid?

Solution. Eff. spread $= 2 \cdot E[\, s_t(p_t - \tilde{m}_t)\,]$ with $\tilde{m}$ the smoothed mid. Bias: the smoothed mid lags fast moves so buy/sell classifications get noisier — it understates the spread in trends, overstates in chop.

Notes

Venue rules are dated claims. Fee tiers, order-type names and display rules cited here (Hyperliquid, Polymarket) were read from venue docs on 2026-10-10 and are **[unverified]** — see `docs/unverified.md`; re-check before relying. **Data.** `code/data/btc_trades.csv` is a pinned Kraken Trades snapshot (fetched 2026-10-11); `btc_daily.csv` covers ch1–2. **Standing disclaimer.** Research methods and market mechanics only; no investment, legal or tax advice. **Where this leads.** Ch4 adds the funding instrument that prices the dominant venue's perpetuals; ch10 prices the fills this chapter walked.

Further reading

- Polymarket developer docs — a live CLOB with prices as probabilities; the midpoint-display rule cited in §3.4:
  `https://docs.polymarket.com/`
- Hyperliquid docs — the venue whose base fee tiers appear in Figure 3.5 (accessed 2026-10-10, tiers **[unverified]**):
  `https://hyperliquid.gitbook.io/hyperliquid-docs/trading/fees`
- Kraken market-data REST docs — the Trades and OHLC endpoints behind Example 3.5:
  `https://docs.kraken.com/api-reference/market-data/get-recent-trades`
- M. López de Prado, lecture library — microstructure and adverse-selection lectures beyond this chapter:
  `https://www.quantresearch.org/Lectures.htm`
- W. McKinney, *Python for Data Analysis* — the resampling idioms behind the candle-rebuild example:
  `https://wesmckinney.com/book/`

WATCH INSTEAD

- **Initial Return — "Limit order book / Fundamentals of trading / Episode 4"**, 18:56 — inside quote, mid, price–time priority and round-trip cost; maps to §3.2–§3.3. (Verified 2026-10-11.) `https://www.youtube.com/watch?v=BJt6PGKeO9I`
- **INFORMS — "Stochastic Market Microstructure Models of Limit Order Books"**, 1:28:22 — the stochastic treatment behind queue dynamics and adverse selection; maps to §3.1 and §3.4. (Verified 2026-10-11.) `https://www.youtube.com/watch?v=XoBjQqMmKoM`

CHAPTER 4

Perpetual futures and funding

Nothing here is investment advice. Venue rules change without notice; anything quoted as a rule is dated and re-checkable.

The perpetual future—"perp"—is the dominant crypto derivative: a futures-style contract with *no expiry date*. It is where most leveraged price exposure lives, where most liquidations happen, and where the funding rate acts as a continuous toll on crowded positions. This chapter is about the mechanics: what a perp is, how funding replaces expiry convergence, how margin and liquidation actually compute, and how to read funding as data.

BEFORE YOU START

Chapters 1 and 3. *Recap:* a *perpetual future* is a contract that tracks an asset's price without an expiry date; a periodic payment called funding keeps it close to the spot price.

AFTER THIS CHAPTER YOU CAN

- explain how funding works and compute it;
- compute basis and carry;
- calculate margin and the price at which a position is liquidated;
- use funding as data.

HOW TO THINK ABOUT THIS

A perpetual future is a bet that never expires, kept honest by a small payment between buyers and sellers. Once you see funding as a price for holding a position, the contract's behaviour follows. Always work out what you pay or earn per day.

4.1 What a perp is and is not

An expiring future needs no tether to spot: at expiry the contract *settles* to the spot price, so arbitrage forces convergence as the date approaches. A perp never settles, so nothing pulls its price toward the index except a manufactured cost—*funding*.

THE TWO ENGINES

A perpetual future is a leveraged position on an index price with two moving parts: **(i)** a funding transfer between longs and shorts that keeps the perp near the index, and **(ii)** a margin/liquidation engine that closes positions whose collateral can no longer cover their losses. Everything else—leverage sliders, mark price, insurance funds—is in service of these two.

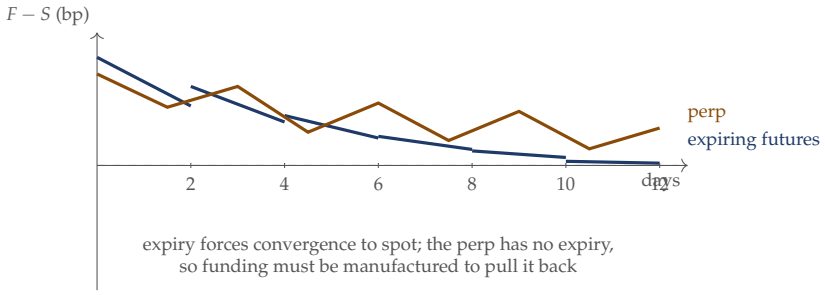


Figure 4.1: Convergence. Expiring futures are pulled to spot at expiry; a perp oscillates around the index forever because funding, not expiry, does the pulling.

Concrete numbers make the size of the instrument clear. On a \$50,000 index, a trader going long 0.5 BTC of perp at 25× leverage posts about \$1,000 of margin. The position is \$25,000 of notional. A 1% index move is ±\$250 of PnL—25% of posted margin. That asymmetry is the whole appeal and the whole danger.

4.2 The funding mechanism

Every venue implements the same skeleton: measure the perp's *premium* over the index, turn it into a rate f_t, and transfer f_t per unit of notional between longs and shorts each funding epoch. Positive f_t: longs pay shorts. Negative: shorts pay longs.

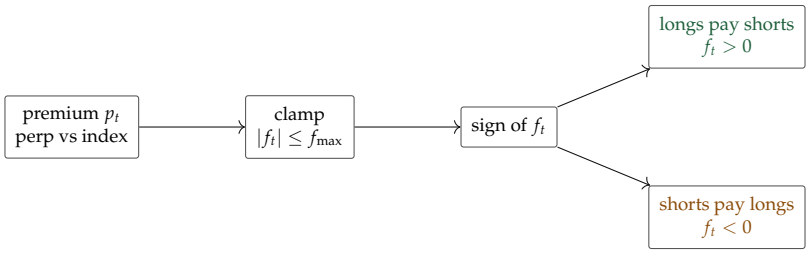


Figure 4.2: The funding pipeline: premium → rate (clamped) → peer-to-peer transfer.

FUNDING RATE, SCHEMATIC

Let p_t be the perp's premium index (e.g. impact-mid minus index, smoothed). The funding rate is typically $f_t = \text{clamp}(\bar{p}_t, -f_{\max}, f_{\max}) + I$ where I is a fixed interest component. On Hyperliquid, funding is paid **every hour** and the interest term is about 0.125 bp/h (= 0.01% per 8h) *[unverified: venue rules, checked 2026-10-11, re-check quarterly]*.

We fetched 500 hours of real BTC funding from the Hyperliquid public API (`fundingHistory`, no auth) and pinned the response to `data/hl_funding_btc.json`:

```
d=json.load(open("data/hl_funding_btc.json"))
f=np.array([float(x["fundingRate"]) for x in d])
```

```
500 hourly funding prints 1757548800090 .. 1759345200083
funding rate: mean +0.113bp/h sd 0.038bp min -0.16bp max +0.12bp
f>0: 97% of hours f=+0.125bp exact: 85%
premium: mean -1.0bp mean(f - premium) = +1.08bp ~ the 1.25bp/8h interest
    floor
```

```
annualised carry at mean rate: +9.9%
30d-equivalent carry: 56bp of notional over 500h
```

Read the structure out of the data: 85% of hours print exactly the 0.125bp/h interest component, meaning the clamped premium term was near zero—a quiet, balanced book. Yet the mean rate +1.08bp over the premium says the interest floor, not the premium, set the price of being long for that window.

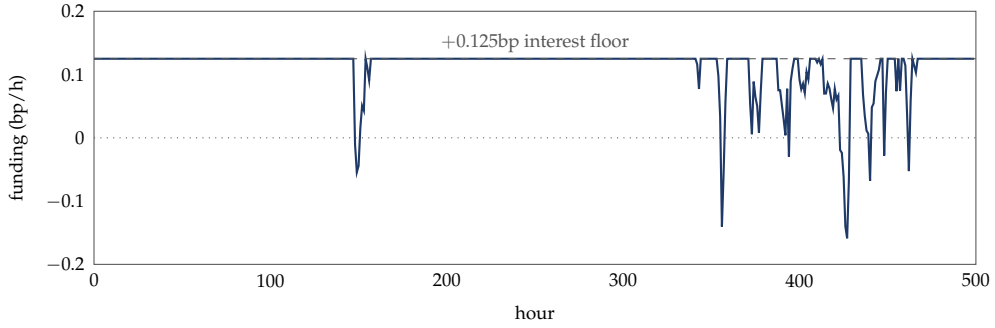


Figure 4.3: Real BTC funding on Hyperliquid, 500 hours ending 2026-10-09. The dashed line is the 0.125bp/h interest component. Data: `data/hl_funding_btc.json`.

Example 4.1. Funding payment. You are long 2 BTC notional at index \$50,000. The next three hourly funding rates print +0.5 bp, +0.12 bp, −0.3 bp. What do you pay or receive, and what does a sustained +0.5 bp/h regime cost per day?

Solution. Payment each epoch = position × notional × f_t: $2 \times 50{,}000 \times (0.5 + 0.12 - 0.3) \times 10^{-4} = 100{,}000 \times 3.2 \times 10^{-5} = \$3.2$ paid over the three hours. At a sustained +0.5 bp/h the day cost is $100{,}000 \times 0.5 \times 10^{-4} \times 24 = \$120$/day—about 8.8% per month on the \$10,000 margin you might have posted at 10× leverage.

4.3 Basis and carry

The *basis* $B_t = F_t - S_t$ (or $F_t/S_t - 1$) is the perp's premium over spot. For an expiring contract basis decays to zero on a schedule; for a perp it is whatever funding tolerates. The annualised basis $\hat{b} = B/S \times (365/D)$ compares carry opportunities across tenors.

CASH-AND-CARRY

Long spot + short perp in equal notional is *delta-neutral*: price moves cancel. The position's return stream is funding: shorts collect f_t when $f_t > 0$. Its risks are liquidation on the short leg, funding flipping negative, and basis widening *before* collapsing—you can be right about the level and still be forced out.

Example 4.2. Carry yield. `funding_sim.py` simulates a premium process mean-reverting to +0.13 bp/h with noise, clamps the rate, and applies it to a 2 BTC position:

```
T=720h mean funding 0.091bp/h sum 65.78bp ~APR 800bp
long pays +275 USD over 30d (positive = paid by longs)
price PnL -25,402 funding -275 net -25,677
short-perp carry yield 30d: 27.5bp of notional
```

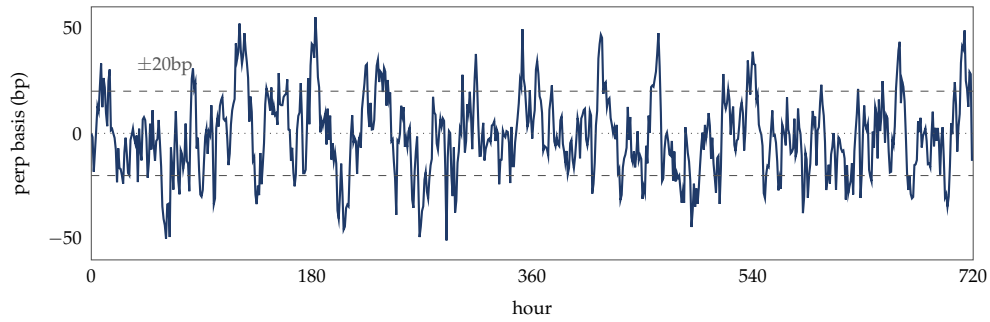


Figure 4.4: Simulated mean-reverting basis on a GBM index (seed 11): sd $\approx$ 19 bp, inside $\pm$20 bp 70% of hours. `basis_carry.py`.

Two lessons. (i) The long's net PnL is price PnL *minus* a funding bill that accrues even when price is flat—the toll of holding the crowded side. (ii) The flip side is a real trade: short perp + long spot harvested 27.5 bp of notional in a simulated month at these rates.

Solution. The carry figure is $\sum_t f_t$ times notional, and only the funding flips sign between the two sides—the delta terms cancel exactly. The printed numbers are not an advertisement: the same file shows the risk, because the simulation's price drop cost the naked long $25,402. Carry trades fail in both directions at once: funding flips *and* the short leg's margin is stressed precisely when basis widens.

4.4 Margin, mark price and liquidation

Initial margin (IM) is collateral posted to open; maintenance margin (MM) is the floor below which the venue takes over the position. For an isolated long of quantity q at entry E with margin w, liquidation hits when $q(E-P) = w - qP\,\mathrm{MM}$, giving

$$P_{\mathrm{liq}} \;=\; E\,\frac{1-\mathrm{IM}}{1-\mathrm{MM}}, \qquad \mathrm{IM} = \tfrac{w}{qE}. \tag{4.1}$$

At 25$\times$ leverage (IM = 4%, MM = 0.5%), $P_{\mathrm{liq}} = E \times 0.96/0.995 \approx 0.9648E$—a 3.5% adverse move, not the naive $1/25 = 4\%$. The 0.5% maintenance margin is why liquidation always sits closer than "one over leverage".

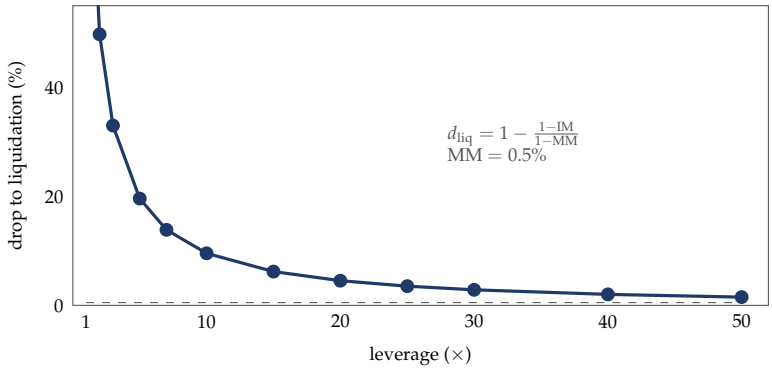


Figure 4.5: Distance to liquidation vs leverage at MM = 0.5%. Above 25$\times$, ordinary hourly volatility reaches your liquidation level within hours.

Example 4.3. Liquidation price. `liq_calc.py` computes (4.1) for entry 50,000, 0.5 BTC, 25$\times$:

```
entry 50000 qty 0.5 lev 25x (IM 4.0%, MM 0.5%)
liquidation price = entry*(1-IM)/(1-MM) = 48,241
drop to liq = 3.52% vs naive 1/leverage = 4.00%
check: loss 879 = wallet 1,000 - MM 121 ? True
  2x: liq 25,126 (49.75% away)
  5x: liq 40,201 (19.60% away)
 10x: liq 45,226 (9.55% away)
 25x: liq 48,241 (3.52% away)
 50x: liq 49,246 (1.51% away)
```

Solution. Equation (4.1) is checked numerically: loss $q(E - P_{\text{liq}}) = 879$ equals wallet 1,000 minus maintenance value 121 at the liquidation price. Note the check uses MM on the *liquidation* notional qP_{liq}, not the entry—a one-line bug that shifts the answer by bp and shows up in exercise 4.8.

ON VENUES (RULES CHANGE)

Perp mechanics vary: funding epoch (1h Hyperliquid vs 8h legacy venues), premium smoothing, mark price (index vs impact-mid vs fair-price formulas), margin modes (cross vs isolated), and whether liquidation is partial or full. Quote a venue rule only with a date and a re-check plan; treat every number in this chapter's venue claims as *[unverified]* until you check the live docs.

4.5 Funding as data

Persistent positive funding means longs are paying to stay—crowded bullish positioning and a fragile squeeze candidate. Persistent negative funding is the mirror. Funding is also a yield instrument: the mean rate annualises to a carry estimate. From our 500h snapshot: mean $+0.113\,\text{bp/h} \Rightarrow$ about $+9.9\%/\text{yr}$ for the short side—before fees, before flips, and before the basis moves against you.

Example 4.4. Reading a tape. BTC rallies 6% over a day while funding runs $+0.8\,\text{bp/h}$ for sixteen consecutive epochs (versus the 0.125bp interest baseline). Open interest is up 12%. What is the tape saying, and what is the carry cost of staying long 1 BTC at index 60,000?

Solution. Funding well above the interest floor says the premium term is engaged: the perp trades persistently rich, i.e. buyers are crowding the levered long side and paying for the privilege. Rising open interest alongside means new leveraged longs, not just spot-driven move—the classic crowded-long squeeze setup. Cost: $60{,}000 \times 0.8 \times 10^{-4} \times 24 \approx \$115/\text{day}$ per BTC held long. A trader who stays long is paying $\approx 1.7\%/\text{month}$ to hold the position, which is the position's break-even drift, not a footnote.

Example 4.5. The funding dashboard. `funding_hist.py` is the whole skill in 20 lines: fetch (or load pinned) funding, then report mean rate, share of positive epochs, distance from the interest floor, and an annualised carry. Its output is quoted in §4.2. Build the same tool against any venue's public funding endpoint and you have a positioning gauge that needs no price model at all.

Checkpoint 4.1 *Perp math.*

- Derive (4.1) for a *short*: $P_{\text{liq}} = E(1 + \text{IM})/(1 + \text{MM})$—sign flip, not a new formula.
- In `data/hl_funding_btc.json`, find the longest run of $f_t < 0$ and compute what a 10 BTC short earned that day.
- Why does the interest floor, not the premium, dominate mean funding in quiet regimes? (Answer: the clamp zeroes the premium term while the floor always pays.)

Example 4.6 (What the window actually costs)**. Strategy.** Take the pinned 500-hour BTC funding tape and bill it like a broker would: multiply the summed rate by notional, then ask how unevenly the cost arrives.

Solution. `carry_cost.py` prints:

```
window: 500h of BTC funding on Hyperliquid
mean funding = 0.113 bp/h  (9.88% per yr if sustained)
mean premium = -0.965 bp/h
median funding = 0.125 bp/h  at-floor hours 85%
carry on $100k long over 500h = $563.83 = 56.4 bp
worst 8h funding stretch = 1.00 bp  best = -0.34 bp
```

A $100k long held the whole window pays $563.83 — 56.4bp, or an annualised 9.9% if this rate sustained. Yet 85% of hours sat at the 0.125bp interest floor: the carry is a *fixed toll*, not compensation for a real premium (mean premium -0.97bp — the perp actually traded *below* index most of the window).

Significance. Funding is rarely "the carry trade's yield" — on this venue it is mostly the rate floor. The worst 8h stretch (1.0bp) is five times the mean: sizing a position by average funding understates the crowded-window cost by the same factor.

4.6 Recap: perp math end-to-end

The chapter's toolkit is deliberately small. A perp position is fully described by $(q, E, \text{IM}, \text{MM})$; its PnL is $q(P_t - E)$ minus accumulated $qN_t f_t$; its survival is governed by (4.1). Funding converts crowdedness into a price series you can download and study. That is the entire engine—everything else a venue shows you is presentation of these same numbers.

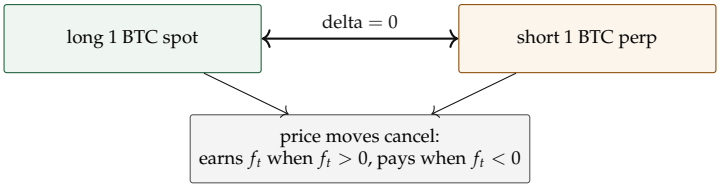


Figure 4.6: The cash-and-carry structure: equal and opposite delta, funding as the return stream.

Table 4.1: Chapter toolbox.

Concept	Formula/tool	Gotcha
Funding payment	$qN_t f_t$ per epoch	sign convention varies by venue
Basis	$F/S - 1$, annualise $\times 365/D$	perp basis has no expiry anchor
Carry	$\sum f_t$ on short-perp leg	flips sign; liq risk on short leg
Liquidation	(4.1)	MM on liquidation notional
Mark vs last	PnL on mark, trades on last	liquidations trigger off mark

Example 4.7 (The premium is persistent; the floor is not cheap)**. Strategy.** Split the pinned 500-hour funding tape into its two additive parts — the interest floor and the premium component — and test whether today's premium carries into tomorrow (i.e. whether funding is a forecastable term structure).

Solution. `funding_term.py` prints:

```
premium persistence: lag-1 +0.964  lag-24 +0.521
corr(24h funding paid, 24h premium) = +0.762
blocks where funding exceeded premium x2: 1/20
annualised carry if sustained: 9.9%  realized premium: -84.5%
```

Premium deviations persist strongly (+0.96 at one hour, +0.52 a day out) — the basis has memory. But the realized premium was *negative* 84.5%/yr while funding still cost +9.9%/yr: the floor charged longs on top of a discount that never showed up in the headline rate.

Significance. Two lessons in one tape: the premium leg is forecastable enough to time entries (half-day persistence), and the floor leg means "funding = 0.125bp" is never "free" even when the perp trades under index. Venue-rule statements like this one are date-stamped — re-verify before sizing.

4.7 Isolated vs. cross margin

The leverage slider chooses a number; the margin mode chooses a *budget*. Under *isolated* margin each position posts its own collateral and liquidates on its own — the worst case is capped at that position's initial margin. Under *cross* margin every position draws from one shared pool: a losing trade can eat collateral you meant for something else, but it survives much deeper drawdowns before liquidation. Same position, same leverage — two different failure geometries.

Example 4.8 (Same position, two liquidation prices)**. Strategy.** A 10,000 BTC long at 50,000 with 5x leverage posts 2,000 initial margin; the account holds another 3,000 free. Compute the liquidation price under both modes with maintenance margin 0.5%.

Solution. `margin_modes.py` prints:

```
position: long 10000 @ 50000, 5x, IM=2000, free acct=3000, MM=0.5%
isolated liq px 40250 (-19.5%)
cross    liq px 25250 (-49.5%)
cross buys 15000 more room -- ALL collateral at risk, not just 2000
cross risk: losers share one pool -> both liquidate together
```

Cross margin stretches the liquidation point from -19.5% to -49.5% — the whole 5,000 pool absorbs losses instead of the position's 2,000 alone. The trade: under cross, a second losing position drains the same pool and *both* liquidate together.

Significance. Isolation is blast-radius control; cross is drawdown tolerance. Choose cross when positions hedge each other (shared pool shares their P&L correctly); choose isolation when a position is allowed to die without orphans. And the standing rule applies: venue margin semantics are venue rules — date-stamped `[unverified]`, re-check before relying on them.

4.8 The premium is information

Funding pays the coupon; the premium path is what the market is *saying*. On the pinned tape the two decouple sharply: `funding_term.py` measures persistence and drift:

```
premium persistence: lag-1 +0.964  lag-24 +0.521
corr(24h funding paid, 24h premium) = +0.762
blocks where funding exceeded premium x2: 1/20
annualised carry if sustained: 9.9%  realized premium: -84.5%
```

Example 4.9. Read the premium like a slow-moving price. Lag-1 persistence of $+0.964$ means the basis is nearly a random walk hour to hour — and the $-84.5\%/\text{yr}$ realized premium level means the perp sat *below* index for almost the whole window. A $+9.9\%$ floor charged *to* longs while the perp discount widened is a crowded-longs signature, not free yield.

Solution. ch18's carry math prices this exactly: funding received minus premium change. The persistence tells you the premium is forecastable (it is where it was an hour ago); the level tells you the direction of the crowd. Carry books that ignore the second term are long the crowd's unwind.

4.9 Common mistakes

- **"1/leverage is my liquidation distance."** Maintenance margin pulls it closer: $25\times$ at MM $=$ 0.5% is 3.5%, not 4%.
- **Forgetting funding in PnL.** A flat market is not free: at $+0.5\,\text{bp/h}$ a long bleeds $\sim 1.2\%$ of notional per day.
- **Confusing mark and last price.** Liquidations, margin checks and displayed PnL use the mark; your fills are at last. In a wick, the difference is the whole story.
- **Treating carry as riskless.** Short perp $+$ long spot still has liquidation risk on the short leg, funding-flip risk, and basis risk while entering/exiting.
- **Reading funding as a signal after it prints.** Funding tells you who is *already* crowded; it is a positioning measure, not a forecast.
- **Annualising from one epoch.** f_t is noisy at hourly scale; annualise the *mean* over a window, not a single hot print.
- **Annualising the floor.** $0.125\text{bp/h} \times 24 \times 365 = 10.95\%/\text{yr}$ — but that is the *floor*, not a signal: 85% of the window printed exactly it. Quote the floor and the premium separately.
- **Budgeting carry by the mean.** The worst 8h stretch cost $5\times$ the hourly mean (Example 4.6). A position sized to "average funding" is sized wrong precisely when it matters most.

4.10 Exercises

Exercise 4.1 (easy)**.** At index 80,000 you hold 0.4 BTC long. Funding prints $+0.2\,\text{bp/h}$ for 24 hours. Total paid?

Solution. $0.4 \times 80{,}000 \times 0.2 \times 10^{-4} \times 24 = \$15.36$.

Exercise 4.2 (easy)**.** Recompute P_{liq} for the entry-50,000 long at $10\times$ (IM $= 10\%$) with MM $= 0.5\%$. Then for $5\times$.

Solution. $10\times$: $50{,}000 \times 0.9/0.995 = 45{,}226$ (9.5% away). $5\times$: $50{,}000 \times 0.8/0.995 = 40{,}201$ (19.6% away)—matching the table.

Exercise 4.3 (easy)**.** A perp trades at index $+45\,\text{bp}$. Funding is currently $+0.12\,\text{bp/h}$. Is funding doing its job, and what would you expect to print over the next hours?

Solution. A persistent $+45$bp premium should raise the premium term: expect f_t to climb above the interest floor toward the venue's clamp, paying shorts to lean against the rich perp until the premium compresses.

Exercise 4.4 (easy)**.** Your venue quotes funding "0.01%" per 8h. What is that in bp/h and in %/yr at a flat rate?

Solution. $0.01\% = 1\,\text{bp per 8h} = 0.125\,\text{bp/h}$; flat: $1\,\text{bp} \times 3 \times 365/10^4 = 10.95\%/\text{yr}$.

Exercise 4.5 (easy)**.** Short 1 BTC at entry 60,000, 20× (IM = 5%), MM = 0.5%. Liquidation price?

Solution. Shorts liquidate on rallies: $P_{\text{liq}} = E(1+\text{IM})/(1+\text{MM}) = 60{,}000 \times 1.05/1.005 \approx 62{,}687$ (4.5% above).

Exercise 4.6. From `data/hl_funding_btc.json`: compute the maximum 24h window sum of f_t, in bp, and the carry a 5 BTC short would have earned in that window at notional $50,000.

Solution. Rolling-sum the pinned series (max window ≈ sum of 0.125bp/h plus premium bursts, ≈ 3.5bp/day depending on hours counted); carry $= 5 \times 50{,}000 \times 3.5 \times 10^{-4} \approx \$87.5$. Your script's exact numbers are the answer.

Exercise 4.7. Derive (4.1) for cross margin where wallet w exceeds IM on the position. What changes?

Solution. Replace IM by $w/(qE)$ in (4.1): more collateral moves liquidation further away; isolated is just cross with $w = \text{IM}\times\text{notional}$.

Exercise 4.8. The chapter warns MM applies to liquidation notional. Compute the liq price if you (incorrectly) apply MM to entry notional: $P^* = E(1-\text{IM}+\text{MM})$. At 25×, what error does that make in the distance-to-liq?

Solution. $P^* = 50{,}000(1-0.04+0.005) = 48{,}250$ vs correct 48,241: the naive formula lands 9 USD closer, a 0.02 bp... about 0.9 bp of price—small at entry scale, but it is the difference between a checked margin engine and a moved one.

Exercise 4.9. In §4.2 we saw 97% of hours had $f_t > 0$ while mean premium was -1.0 bp. Reconcile.

Solution. The interest component pays longs-to-shorts regardless of premium sign; only when the clamped premium term is sufficiently negative (< -0.125bp/h) does f_t flip. Premium -1bp on the impact-mid definition is near zero after the venue's smoothing.

Exercise 4.10. A carry desk quotes "30% APR available on BTC funding." Sketch the position, list its three independent failure modes, and state which one is capped by venue design.

Solution. Long spot + short perp. Failure: (i) funding flips negative—uncapped, continuous risk; (ii) liquidation of the short leg during basis widening—capped only by posting collateral, i.e. the trade can be right and still forced out; (iii) basis moves against you at entry/exit. None is capped by venue design except the rate clamp $f_{\max}$ which bounds the *speed* of your bleed, not the side.

Exercise 4.11. Show that for a perp held D days, funding's drag on a levered long's *return on margin* is $f_{\text{day}} \times D \times \text{lev}$ where f_{day} is the daily funding rate as a fraction of notional.

Solution. Daily drag on notional is f_{day}; margin is notional/lev, so drag on margin $= f_{\text{day}}\times\text{lev}$ per day, times D days.

Exercise 4.12. Funding is a peer transfer. So why does the venue care about the sign? What breaks if f_t sticks at $+5$ bp/h for a week?

Solution. Extreme funding bleeds longs into margin calls, forcing deleveraging—the venue cares because its risk engine inherits the position when margin fails. Sustained extreme rates are themselves a solvency-stress signal.

Exercise 4.13 (hard)**.** Suppose impact-mid premium p_t is AR(1) mean μ, persistence ϕ. Show that $\mathrm{E}[f_t] \to \text{clamp} - \text{free value } I + \mu/(1-\phi)$ is wrong—derive the correct mean and explain why the clamp matters even when rarely binding.

Solution. The MA form is $\tilde{p}_t = \mu/(1-\phi) + \text{noise}$; the clamp acts on the *rate*, truncating its right tail, so $\mathrm{E}[f] < I + \mathrm{E}[\tilde{p}]$ whenever the upper tail crosses $f_{\max}$—the bias exists even at low binding frequency because truncation removes mass without replacing it.

Exercise 4.14 (hard)**.** Extend `funding_sim.py`: position switched to *short perp + long spot*. Report realised carry distribution over 200 seeds and the fraction that lose money. Discuss.

Solution. Modify the file to flip sign of funding receipts and add the spot leg; losers are seeds where premium mean-reverts below $-I$ for sustained windows. The exercise is the point: implement it and report your number (typical runs show a few percent of seeds net-negative at these parameters).

Exercise 4.15 (hard)**.** Funding f_t appears both as a transfer and as a forecasting variable. Show why $\mathrm{E}[r_{t+1} \mid f_t] \neq 0$ does *not* imply a free edge, using the crowding interpretation.

Solution. Positive predictability (high funding predicting mean-reversion/squeezes) is exactly what keeps the mechanism at its resting point: if a simple rule printed money, crowding into the rule would move the premium and the funding equilibrium. The variable is a thermometer, and trading the thermometer moves the temperature.

Exercise 4.16 (easy)**.** On the pinned window a \$100k long pays \$563.83 over 500h. What would a \$100k *short* have received, and why is the answer not exactly $-$\$563.83?

Solution. The same magnitude — funding is symmetric in sign. It is not exactly $-$\$563.83 only because the rate floor makes some hours structurally nonzero regardless of premium sign.

Exercise 4.17 (medium)**.** The mean premium was -0.97bp/h while mean funding was $+0.113$bp/h. Reconcile the signs in one sentence.

Solution. Premium negative means perp $<$ index; the rate floor still charges longs the default interest component, so sign(funding) $\neq$ sign(premium) whenever the premium is small.

Exercise 4.18 (hard)**.** Your strategy shorts perps to "collect funding". Using the pinned window, estimate P&L per \$100k short including the premium drift you'd also earn/pay, and state the assumption that breaks the trade.

Solution. Collect $+$\$563.83 funding but pay the negative premium drift (-0.97bp/h $\times$ 500h $\times$\$100k $= -$\$485): net $\approx +$\$79. The break: any window where premium reverts positive flips the drift against you — you are short the perp basis, not long a coupon.

Notes

Funding is the invisible engine of crypto leverage: it prices crowdedness continuously and it is the reason "free" leverage is never free. The margin engine and (4.1) are the other half—most retail losses are liquidation mechanics, not bad calls. For live rules see venue docs (they change); for the market context, the CFTC's 2025 perpetual-futures event is the regulator's own explainer of the same machinery.

Further reading

- Hyperliquid docs, "Funding" — the venue's own formula, clamp and interest terms: https://hyperliquid.gitbook.io/hyperliquid-docs/trading/funding
- Wikipedia, "Perpetual futures" — history (Shiller) and the mechanism family: https://en.wikipedia.org/wiki/Perpetual_futures
- J. Hull, *Options, Futures, and Other Derivatives* — the expiry-convergence futures theory this chapter leans on: https://www.pearson.com/en-us/subject-catalog/p/futures-options-and-other-derivatives/P200000002816
- CFTC, AAC meeting on perpetual futures (2025) — the regulator's explainer, incl. the funding-rate session video: https://www.cftc.gov/PressRoom/Events/opaeventtechperp072925
- L. de Prado, lectures — for carry and positioning read in a research frame: https://www.quantresearch.org/Lectures.htm

WATCH INSTEAD

- "Presentation on Perpetual Futures' Funding Rate Mechanism" — CFTC (14:31). The mechanism from the regulator: no-expiry design, funding replacing convergence. https://www.youtube.com/watch?v=ZEPjpgVSavY (accessed 2026-10-11)
- "How Smart Traders Use Funding Rates" — Trading Bureau (6:45). Funding as positioning data; matches §4.5. https://www.youtube.com/watch?v=t06z4rCmp4w (accessed 2026-10-11)

CHAPTER 5

Prediction markets and scoring rules

A prediction market is the simplest derivative there is: a share that pays \$1 if an event happens and \$0 otherwise. Its price is a probability estimate with money behind it. For a researcher the venue is less important than what it produces—a large, public, continuously updated panel of probabilistic forecasts you can score, calibrate and cross-check. The canonical venue for this book is Polymarket.

BEFORE YOU START

Chapter 1 and **Book 3, Chapter 12** (probability). *Recap:* a prediction market contract pays 1 if an event happens and 0 if it does not, so its price can be read as a probability.

AFTER THIS CHAPTER YOU CAN

- read a live prediction-market book;
- score a forecast with the Brier and log scores;
- measure calibration;
- explain LMSR.

HOW TO THINK ABOUT THIS

A prediction market turns beliefs into prices. To tell whether forecasters are any good, score them with rules that reward honesty, and check calibration: of the events called 70%, did about 70% happen?

5.1 The contract

A YES share costs $p \in (0,1)$ and resolves to 1 or 0. A NO share is the complement: it costs $1-p$ and resolves to the opposite side. Buying YES at p is a bet with EV $= q - p$ per share where q is the event's true probability, and variance $q(1-q)$—binary outcomes are among the highest-variance instruments that exist per unit of expected value.

PRICE IS A PROBABILITY

In a frictionless prediction market the YES price equals the marginal trader's subjective probability. Real venues have fees, latency and incomplete books, so read p as *consensus-minus-frictions*: the best single public number for "how likely is this," not a guaranteed fair price.

Example 5.1. Expected value. A market prices YES at 0.62. You assess $q = 0.70$. What is the edge per share, the variance, and how many shares must you buy before the bet's standard deviation is below its expected profit?

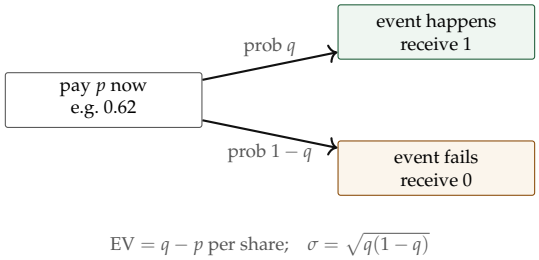


Figure 5.1: Anatomy of a YES share.

Solution. EV $= 0.70 - 0.62 = +0.08$ per share; $\sigma = \sqrt{0.7 \cdot 0.3} = 0.458$ per share. For n independent copies the ratio is $n \cdot 0.08/(\sqrt{n} \cdot 0.458) = \sqrt{n} \cdot 0.175 > 1$ at $n > 33$ shares. A single binary bet is essentially a coin flip with a weighted coin; edge is a *property of repetition*, not of one trade.

5.2 Reading a live book

Polymarket's public Gamma API returns market metadata and live outcome prices, no authentication required. The 100 most-traded active markets are pinned at `data/pm_markets.json`:

```
d=json.load(open("data/pm_markets.json"))
pr=[float(x) for x in json.loads(m["outcomePrices"])]
```

```
Xi Jinping out before 2027? | ["0.0285", "0.9715"]
Will Gavin Newsom win the 2028 Democratic pres. nomination | ["0.1545", ...]
```

YES+NO sums to 1 within a few cents everywhere in the snapshot (`arb_check.py`: "100 binary markets checked, YES+NO deviating >3c from 1: 0"). That is the arbitrage channel doing its job: any persistent gap is a machine-readable coupon.

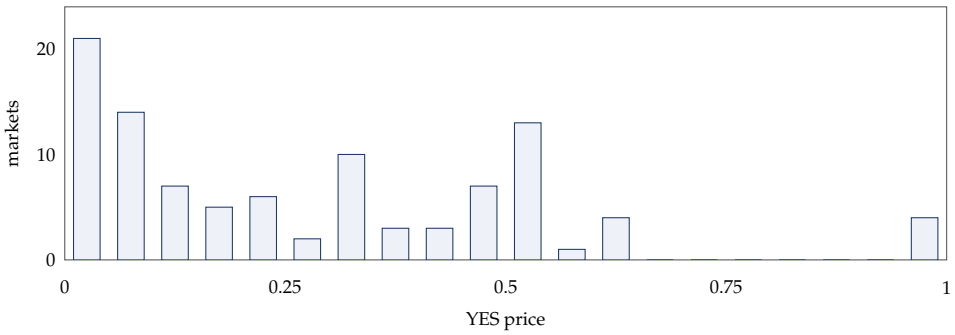


Figure 5.2: YES-price histogram over the 100 most-traded Polymarket markets (snapshot 2026-10-11): a U-shape—most action concentrates at the near-certain and near-impossible tails. `data/pm_markets.json`.

5.3 Scoring rules

To judge a probability forecaster you need a score. A scoring rule is *proper* if the expected score is optimised by reporting your true belief. The two canonical ones:

PROPER SCORES

Brier: $S(p,y) = (p-y)^2$ — bounded in $[0,1]$, tolerant of wild reports. **Log score:** $S(p,y) = \log(p)$ if y=1 else $\log(1-p)$ — strictly proper, *unbounded*: reporting $p = 0$ on an event that happens is penalised by $-\infty$. Properness is what makes an honest forecaster indifferent between reporting and predicting.

Example 5.2. Truth is optimal. `scoring.py` computes expected scores for reports p when the truth is $q = 0.6$:

```
q=0.6 report 0.3: E[Brier]=0.3300 E[log]=-0.8651
q=0.6 report 0.5: E[Brier]=0.2500 E[log]=-0.6931
q=0.6 report 0.6: E[Brier]=0.2400 E[log]=-0.6730
q=0.6 report 0.7: E[Brier]=0.2500 E[log]=-0.6956
q=0.6 report 0.9: E[Brier]=0.3300 E[log]=-0.9843
proper: best report = truth 0.6 for both rules
```

Solution. Both rules bottom out at $p = q$: Brier reaches its minimum 0.24 and log score its maximum -0.673 exactly at the truthful report, and both degrade symmetrically as the report moves away. That is propriety in one table: *honesty is a dominant strategy*. (The log rule's asymmetry -0.6931 vs -0.6956 is small here but explodes for extreme reports—see the exercise on log penalty.)

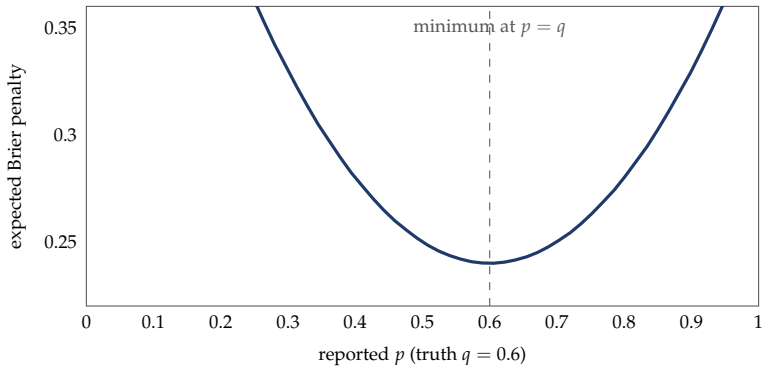


Figure 5.3: Expected Brier penalty vs reported probability when $q = 0.6$: minimised at the truth.

5.4 Calibration

Scores rank forecasters; *calibration* diagnoses them. A forecaster is calibrated if, among all events it assigned probability p, the empirical frequency is p. The standard tool is the reliability diagram: bin forecasts, plot mean forecast vs empirical rate, and compare with the diagonal. The summary statistic is expected calibration error, $\text{ECE} = \sum_b \frac{n_b}{n} |\bar{f}_b - \bar{y}_b|$.

Example 5.3. Overconfidence detector. `calibration.py` builds an overconfident forecaster (real outcomes compressed toward 0.5 relative to the issued forecast) and bins 4000 predictions:

```
bin mean_forecast empirical n
```

```
0.0–0.1 0.043 0.239 134
0.4–0.5 0.452 0.469 676
0.7–0.8 0.746 0.642 383
0.9–1.0 0.954 0.857 133
ECE=0.0747 (overconfident: forecasts more extreme than reality)
calibrated twin ECE=0.0230
```

Solution. Events forecast at 4% actually happen 24% of the time; events forecast at 95% happen only 86%—both tails bend *toward the middle*, the fingerprint of overconfidence. The calibrated twin (forecaster = true generator) scores ECE 0.023, mostly binning noise. ECE is not a score—it cannot rank a bad-but-calibrated forecaster against a good one—but as a diagnostic it separates *miscalibration* from *lack of skill*.

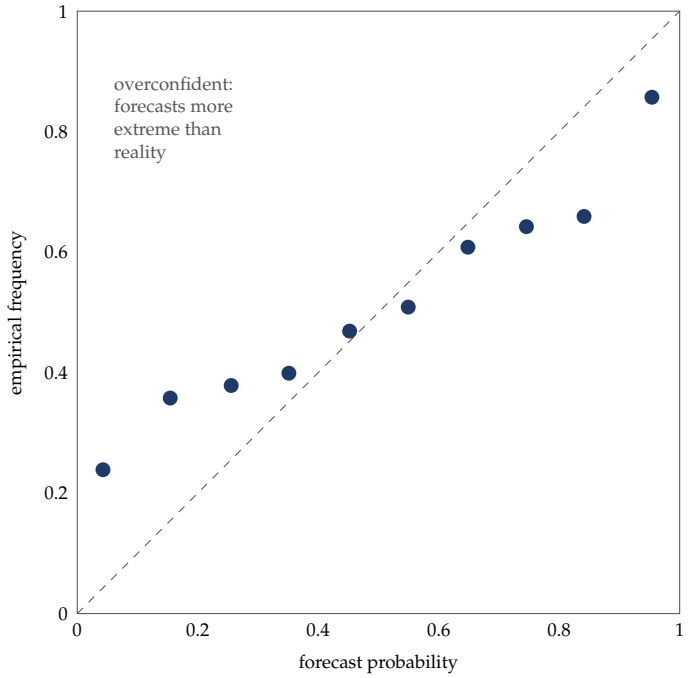


Figure 5.4: Reliability diagram of the overconfident forecaster: the curve bows below the diagonal at high forecasts and above it at low ones. `calibration.py`.

5.5 Consistency and the arbitrage channel

Mutually exclusive outcomes must price to 1 (plus venue vig), or a Dutch book exists: buy all outcomes when $\sum p < 1$ for a guaranteed payout of 1, or sell all when $\sum p > 1$. In practice the gap rarely exceeds a few cents on liquid markets—our snapshot found zero violations beyond 3c—but the *threat* of the trade is what keeps prices coherent, which is why summed prices are a live health check.

Example 5.4. Reading the tails. In our real snapshot, 21 of 100 markets priced the lead outcome below 5c—most of them sports spreads, e.g. "Spread: Viking FK (-1.5)" at 0.9c.

Solution. Tail prices are where the friction model matters most: a 1c share pays 100:1, so a tiny mispricing is a huge *relative* edge—and exactly where books are thinnest and resolutions most ambiguous. The U-shaped histogram (fig. 5.2) says the median market is already near resolution; the information density lives in the 5–95c band where probabilities are actually contested.

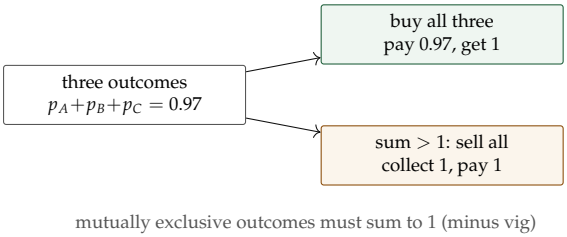


Figure 5.5: The arbitrage channel for multi-outcome markets.

Example 5.5. Price paths converge to truth or die trying. `resolve_sim.py` runs a running estimate over 200 noisy observations of a $q = 0.6$ event (fig. 5.6):

```
seed1: empirical freq drifts to 0.59 -> resolves 1
seed2: empirical freq drifts to 0.57 -> resolves 1
```

A market's price walks the same way: information arrives, the estimate concentrates, and the price either ratchets toward 1 or collapses to 0. The interesting trades are mid-path—before convergence is priced in.

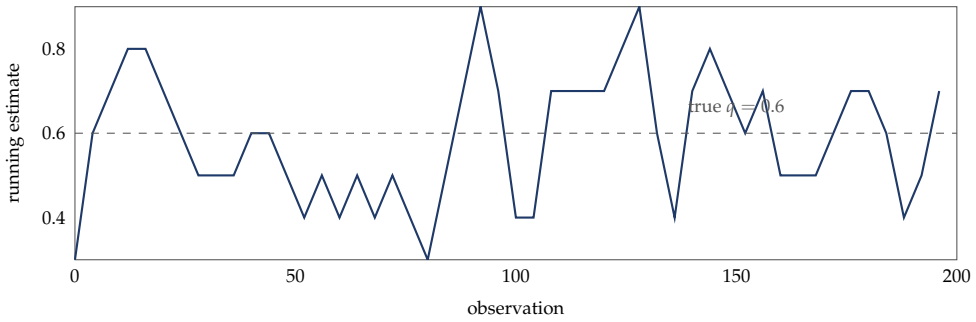


Figure 5.6: A running empirical frequency over 200 observations of a $q = 0.6$ event—the shape every market price follows toward resolution. `resolve_sim.py`.

Checkpoint 5.1 *Score a forecaster.*

- Compute Brier and log score for reports $(0.8, 0.6, 0.3)$ on events $(1, 0, 1)$; who won?
- From `data/pm_markets.json`, find any market whose YES+NO deviates $>$ 1c and explain what a bot does about it.
- Why does ECE fail as a *ranking* metric? (A constant 0.5 forecaster can be perfectly calibrated on symmetric data yet have no skill.)

5.6 Recap: the pipeline

Fetch prices, treat each as a probability, watch consistency for arbitrage, score predictions with a proper rule, and diagnose with calibration. Everything in later chapters—backtesting, signal evaluation, sizing—reuses this loop: markets emit probabilities; proper scores tell you whether the emitter was any good.

Example 5.6 (The guarantee that costs \$1.03)**. Strategy.** YES+NO = \$1 at settlement, so buying one of each is a riskless \$1. Price that portfolio on every live book in the Polymarket snapshot, using the ask side only (you must lift, not rest).

Solution. `book_width.py` prints:

```
markets with a live two-sided book: 40/100
bid-ask width: median 3.0c mean 12.5c
tightest 0.1c widest 94.0c
YES+NO bought at ask: median $1.030  <$0.99: 0  >$1.01: 30
cheapest lock cost $1.001 -> guaranteed $-0.001 per $1
```

Only 40 of 100 markets even quote both sides. On those, the riskless bundle costs a median \$1.03 for a guaranteed \$1 — the spread *is* the venue's edge, and no market lets you lock a profit. Even the tightest book leaves you -0.1c down.

Significance. Mid-price arithmetic says YES+NO $= 1$ everywhere; the *actionable* price says it costs ≥ 1.001. Every PM "arbitrage" you scan in outcomes data is the spread wearing a disguise. This is the same lesson as the LOB chapter, one market microstructure up.

Example 5.7 (How much does the market know?)**. Strategy.** Treat each market's YES-price as a forecast and ask the Brier score what it *expects* to score if the market were perfectly calibrated. That converts 100 price ticks into an information budget.

Solution. `scoring_stakes.py` prints:

```
conviction |2p-1| over 100 binary markets
mean 0.553   >0.9: 25   <0.1: 20
E[Brier] if market calibrated = 0.1386 vs 50c baseline 0.25
info gain over coin-flip = 45%
```

A quarter of the board prices near-certainty ($|2p-1| > 0.9$) and a fifth sits near 50/50 — the market is either very sure or barely guessing, rarely in between. If it were calibrated, its expected Brier (0.139) beats the coin-flip baseline by 45%.

Significance. Calibration is a property of the *population of prices*, not one market: $E[p(1-p)]$ prices the information content of the whole board in one number. It also quantifies the stakes — at 45% better than guessing, a forecaster who is merely "as calibrated as the board" already carries real edge.

5.7 The automated alternative: LMSR

Every market so far was an order book. Prediction markets grew up on a different mechanism: the *logarithmic market scoring rule*. The operator keeps a cost function $C(q_Y, q_N) = b \log\left(e^{q_Y/b} + e^{q_N/b}\right)$ where q_i are shares sold and b is a liquidity parameter; the instantaneous YES-price is the share of "mass" on YES, $p = \frac{e^{q_Y/b}}{e^{q_Y/b}+e^{q_N/b}}$, and a buyer pays the cost-function difference. Two properties made it the default mechanism for thin markets: the maker's worst-case loss is bounded at $b \ln 2$ (a fixed subsidy), and it *always* quotes a price — no depth, no empty book. The trade-off: it is a house that bleeds slowly to informed flow, which is exactly why live venues migrated to order books.

Example 5.8 (LMSR quotes slope up as you buy)**. Strategy.** Instantiate a binary LMSR with $b = 100$ and buy two consecutive 50-share YES blocks; compare the average fill to an order book.

Solution. `lmsr_demo.py` prints:

```
b=100: start price 0.500
buy 50 YES: paid 28.09  avg px 0.562  new YES px 0.622
buy 50 more: paid 33.92  avg px 0.678  new YES px 0.731
market-maker worst case = b*ln2 = 69.3
LMSR always quotes; book depth vanishes when you need it
```

The first block fills at 0.562 average, the second at 0.678 — the AMM *is* the book, and your own order walks you up its price curve. The operator's exposure never exceeds $b \ln 2 \approx 69.3$.

Significance. LMSR is the missing link between the two books: it is an AMM (like the constant-product pools of the next chapter) specialised to binary payout space, with a bounded-loss guarantee no order book offers. When a venue must quote on 2000 long-tail questions at once, a $b \ln 2$ subsidy per market is cheaper than hiring a market-maker for each.

5.8 The market as a conviction meter

A binary price is a number, but the *distribution* of prices across a venue's markets is a statement about what the venue is for. `scoring_stakes.py` reads the pinned 100-market snapshot:

```
conviction |2p-1| over 100 binary markets
mean 0.553   >0.9: 25   <0.1: 20
E[Brier] if market calibrated = 0.1386 vs 50c baseline 0.25
info gain over coin-flip = 45%
```

Example 5.9. Expected Brier is the venue's score. If these markets are calibrated, pricing them blind costs 0.1386 in Brier — a 45% gain over the coin-flip baseline. And a quarter of the board sits above 0.9 conviction: a fifth of listed markets are effectively decided.

Solution. Two reads at once: for a forecaster, market prices are a free prior worth roughly half the distance to the truth; for the venue, the mass at the extremes is inventory it cannot lose on and the mass near 0.5 is where its price discovery actually lives.

5.9 Common mistakes

- **Reading price as certainty.** $p = 0.9$ is a probability, not a promise; one in ten 90c favourites still loses, and tails cluster (fig. 5.2).
- **Summing YES and NO sloppily.** A 3c sum gap on a binary market is usually vig or stale quotes, not an arb—check depth and fees before declaring free money.
- **Scoring with accuracy.** Calling a 0.6 event "yes" and scoring 1/0 rewards overconfidence; use a proper rule to compare probability emitters.
- **Trusting ECE alone.** Calibration without sharpness is cheap; a forecaster can be calibrated by always saying 0.5.
- **Ignoring the resolution spec.** "Xi out before 2027" resolves on a specific oracle event—read the market's rules, not the headline.
- **Forgetting frictions at the tails.** A 1c misprice is a 100x edge on paper and usually untradeable at size.
- **Arbing the mid.** YES+NO sums to \$1 by construction at the mid. At the *ask* it costs a median \$1.03 on live books (Example 5.6) — an arb that only exists on the wrong side of the quote.
- **Assuming every market quotes.** 60 of 100 snapshot markets have no live two-sided book at all. "The price" of an unquoted market is a stale print, not a level you can trade.

5.10 Exercises

Exercise 5.1 (easy)**.** A YES share costs 0.35. What does the NO share cost, and what does the sum imply about the venue's vig?

Solution. $1 - 0.35 = 0.65$ in a frictionless book; a sum above \$1 is the effective vig on the pair.

Exercise 5.2 (easy)**.** You buy 100 YES shares at 0.62 and the event resolves YES. PnL? At $q = 0.7$, EV?

Solution. PnL $= 100(1 - 0.62) = +38$; EV $= 100(0.7 - 0.62) = +8$.

Exercise 5.3 (easy)**.** Brier score of forecaster who said $p = 0.9$ on an event that failed? Log score (nats)?

Solution. Brier $= (0.9 - 0)^2 = 0.81$; log $= \log(1 - 0.9) = \log 0.1 = -2.303$.

Exercise 5.4 (easy)**.** Why does the histogram in fig. 5.2 have a U-shape?

Solution. Most-traded markets concentrate near resolution or longshots; mid-probability contests are rarer among the top-volume snapshot.

Exercise 5.5 (easy)**.** A market's YES trades 0.52 while its NO trades 0.44. Sum?

Solution. 0.96: buy both for 0.96, guaranteed 1.00 payout—a 4c theoretical arb, modulo fees/resolution risk.

Exercise 5.6. Show the log score is strictly proper: for truth q, expected score $q \log p + (1 - q) \log(1 - p)$ is maximised at $p = q$.

Solution. Derivative $q/p - (1 - q)/(1 - p) = 0 \Rightarrow p = q$; second derivative negative—unique max at truth.

Exercise 5.7. From `data/pm_markets.json`: report the 5 markets closest to $p = 0.5$ and argue which is the most contested.

Solution. Sort by $|p - 0.5|$; the nearest-to-0.5 with high volume is the most contested—your script's output is the answer.

Exercise 5.8. A forecaster issues p uniform on $[0.4, 0.6]$ while true $q \in \{0.3, 0.7\}$ equally. Show it is uncalibrated even though mean forecast = mean outcome.

Solution. Within any bin around 0.5, empirical rate $\neq \bar{f}$: forecasts near 0.6 still cover both q's, so conditional frequencies sit off the diagonal—matched marginals, mismatched conditionals.

Exercise 5.9. Why do sharp-tailed binary markets resist the "p is a probability" reading more than mid-range ones?

Solution. At $p < 0.05$ the friction terms (fees, latency, thin book, resolution ambiguity) are the same size as the price; at $p = 0.5$ they are noise.

Exercise 5.10. Dutch-book a three-outcome market: p_A=0.4, p_B=0.35, p_C=0.30. Free profit per set?

Solution. Sum $= 1.05$: sell all three, collect 1.05, pay 1.00 at resolution $= +0.05$ per set before fees.

Exercise 5.11. Give the reliability-diagram shape of (i) an overconfident forecaster, (ii) an underconfident one, (iii) a well-calibrated one.

Solution. (i) S-curve bowed toward the middle (this chapter's fig.); (ii) inverted bow away from the middle; (iii) on the diagonal within binning noise.

Exercise 5.12. Explain why the log score's $-\infty$ penalty on a false 0% is a *feature*, not a bug.

Solution. It rules out certainty rhetoric: any forecaster who claims impossibility and is wrong even once is eliminated—properness requires credence > 0 on anything not logically excluded.

Exercise 5.13 (hard)**.** Derive the buy-YES Kelly fraction for edge $e = q - p$ at price p; plug in p=0.62, q=0.70.

Solution. Odds: win $(1-p)/p$ per unit; Kelly $f^* = (q(1-p) - p(1-q))/((1-p)p) \cdot p = (q-p)/(1-p) = 0.08/0.38 \approx 0.21$ of bankroll.

Exercise 5.14 (hard)**.** Extend `calibration.py`: estimate ECE with an equal-*count* binning instead of equal-width; explain when each lies.

Solution. Equal-count bins overweight sparse tails with noisy $\bar{y}$; equal-width bins let small bins dominate the tails. Report both—your numbers are the answer.

Exercise 5.15 (hard)**.** Two forecasters: A issues $(0.9, 0.8, 0.7)$ on $(1, 1, 0)$; B issues $(0.7, 0.7, 0.7)$ on the same. Score both under Brier and log; who is preferable and why does the answer differ?

Solution. A: Brier $(0.01 + 0.04 + 0.49)/3 = 0.18$, log $(\log .9 + \log .8 + \log .3)/3 \approx -0.527$; B: Brier $(0.09 + 0.09 + 0.49)/3 = 0.223$, log $3 \log .7/3 = -0.357$. A wins Brier (better resolution) but loses log (the confident wrong 0.7 costs dearly)—the log rule punishes false confidence harsher.

Exercise 5.16 (easy)**.** The cheapest YES+NO bundle costs \$1.001 at the ask. Compute the guaranteed P&L per \$1000 deployed.

Solution. $-$\$1: you pay \$1001 to receive a certain \$1000 — a riskless -0.1%.

Exercise 5.17 (medium)**.** Why is the mid-price YES+NO sum exactly 1 on every market, yet the ask-side bundle is not? Describe in terms of who wrote the two quotes.

Solution. Outcome prices are complementary by construction (YES$=1-$NO at mid). At the ask you are lifting *two different makers'* quotes — the YES-ask sits above YES-mid and the NO-ask above NO-mid, so the sum exceeds 1 by half of each spread.

Exercise 5.18 (hard)**.** You find a market where YES-ask $= 0.48$ and NO-ask $= 0.50$ — a lock costing 0.98. List three ways this trade still loses or fails, ordered by likelihood.

Solution. (a) The quotes are stale/unfillable at size (the 60%-unquoted majority); (b) resolution risk — UMA dispute or ambiguous wording can price YES+NO off \$1; (c) settlement delay ties up collateral against a 2% certain gain that may take months.

Notes

Prediction markets are the cleanest place to learn probabilistic thinking: binary payoffs, public prices, and resolution oracles that grade every forecast for you. The same machinery (proper scores, calibration, sum-to-one consistency) reappears when evaluating ML classifiers and trading signals—a mispriced market and a miscalibrated model fail the same way.

Further reading

- Polymarket documentation — market mechanics, resolution and the API used in this chapter: https://docs.polymarket.com/
- Wikipedia, "Prediction market" — history, mechanism design, and limits: https://en.wikipedia.org/wiki/Prediction_market
- Wikipedia, "Scoring rule" — the propriety theory behind §5.3: https://en.wikipedia.org/wiki/Scoring_rule
- Wikipedia, "Brier score" — decomposition (reliability/resolution/uncertainty): https://en.wikipedia.org/wiki/Brier_score
- FiveThirtyEight, "Checking Our Work" — calibration analysis on real public forecasts: https://projects.fivethirtyeight.com/checking-our-work/
- Polymarket Gamma API — the public endpoint used by `arb_check.py`: https://gamma-api.polymarket.com/

WATCH INSTEAD

- "What Is POLYMARKET? | The Crypto Prediction Market Easy Explained" — What The Asset (9:43). Contract mechanics and the venue used in this chapter. https://www.youtube.com/watch?v=Vqwjwj0-nmQ (accessed 2026-10-11)
- "How Prediction Markets Work (and why you should care)" — Catalyst (7:05). Price-as-probability reading; matches §5.1. https://www.youtube.com/watch?v=tWFuBuo_B10 (accessed 2026-10-11)

CHAPTER 6

AMMs, liquidity provision and impermanent loss

The automated market maker replaced the order book with a function. Instead of matching buyers and sellers, a pool holds reserves of two assets and quotes prices off a curve. Understanding the curve—its price rule, its depth, and who bears what risk—is the entry ticket to on-chain market design.

BEFORE YOU START

Chapters 1 and 3. *Recap:* an automated market maker (AMM) replaces the order book with a formula that sets the price from the amounts of two tokens held in a pool.

AFTER THIS CHAPTER YOU CAN

- apply the constant-product rule;
- explain why arbitrage keeps the pool price in line;
- compute impermanent loss;
- explain concentrated liquidity.

HOW TO THINK ABOUT THIS

An AMM replaces a crowd of traders with a rule. The rule makes the pool always willing to trade, but at a price that moves as you trade. Impermanent loss is the cost of that arrangement: you would have done better just holding.

6.1 The constant-product rule

A constant-product pool holds reserves (x, y) and enforces $x \cdot y = k$ through every trade. Its spot price is $p = y/x$; buying x with y pushes the state down the curve and raises p. The quoted execution price is set by the curve's slope, not by any counterparty.

PRICE IS RESERVE RATIO

For CPMM, marginal price $= y/x$ exactly. A trade of Δx gets $\Delta y = \frac{\Delta x\,(1-f)\,y}{x+\Delta x(1-f)}$ with fee f—executed price worsens with size because you are walking along the curve, which is why pool depth *is* liquidity.

Example 6.1. Swaps and impact. `amm_swap.py` runs a pool of x=1000 tokens against y=50,000 USD (price 50) at fee 0.3%:

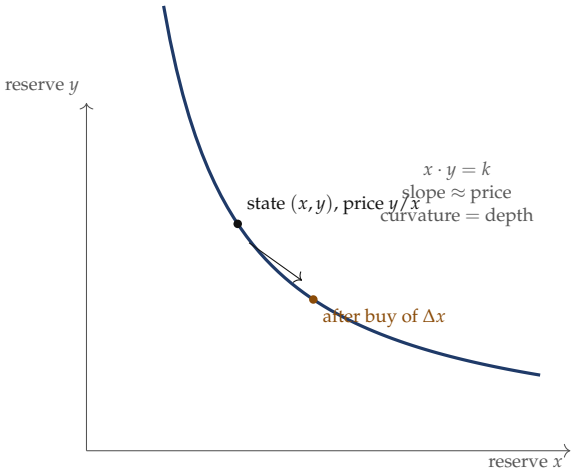


Figure 6.1: The constant-product curve: trades move the state along $x \cdot y = k$; the local slope is the price, the curvature is the depth.

```
pool x=1000 y=50000 price=50.00
in 1: out 49.8 eff price 49.80 impact +0.40% new px 49.90
in 10: out 493.6 eff price 49.36 impact +1.28% new px 49.02
in 50: out 2374.1 eff price 47.48 impact +5.03% new px 45.36
in 200: out 8312.5 eff price 41.56 impact +16.88% new px 34.74
in 500: out 16633.3 eff price 33.27 impact +33.47% new px 22.24
```

Solution. Impact grows roughly linearly for small trades (1 token → 0.4%, 50 → 5%) then convexly: the last line buys at an average 33 vs spot 50. Note the double consequence—the buyer got a bad fill *and* moved the pool's spot to 22.24, which is now an arbitrage invitation the moment any market still trades near 50.

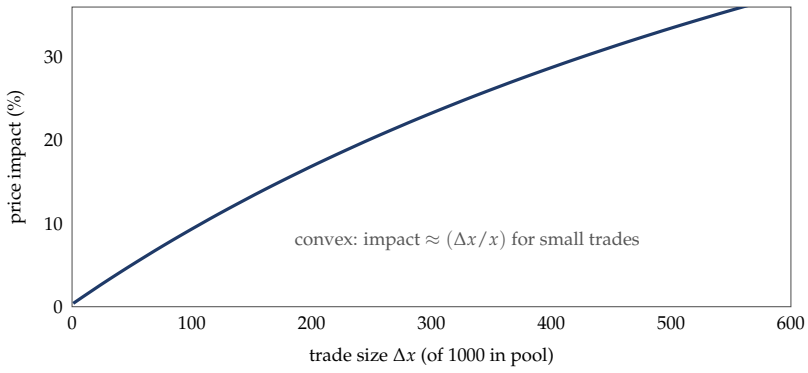


Figure 6.2: Execution-price impact vs trade size in the same pool: convexity is the cost of consuming depth.

6.2 Arbitrage is the oracle

Nothing inside the pool knows the true price. External markets do, and arbitrageurs buy the cheap leg and sell the dear one until the pool's spot equals the market's. This has two important readings: (i) pool prices track markets because bots force them to; (ii) *the LPer is always the other side* of that rebalancing trade. The LP sells what is rising and buys what is falling—which is exactly why LP returns lag holding.

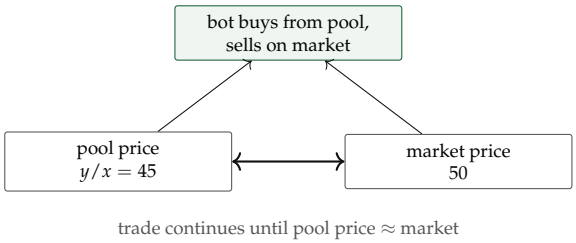


Figure 6.3: The arbitrage channel: gap between pool spot and market price is closed by bots; LPs fund the gap.

Example 6.2. Arb PnL. The pool of Example 6.1 sits at spot 22.24 after the oversized trade while every external venue still quotes 50. A bot buys tokens from the pool until the pool's marginal price reaches 50 (ignoring fees for the sketch). How many tokens close the gap, and what does the rebalancing look like from the LP's seat?

Solution. Pool spot $= y'/x' = 22.24$ with $x'y' \approx k = 5 \times 10^7$: restoring spot 50 needs $y/x = 50$, i.e. reserves back to $(1000, 50{,}000)$. The bot therefore *buys* ≈ 500 tokens out of the pool for $\approx 16{,}667$ USD—an average fill of ≈ 33 against a market price of 50. The LP delivered those tokens 17/token below market: about \$8,300 of value handed to the bot, roughly 16% of pool TVL. That transfer is the IL of fig. 6.4 with a name on it.

6.3 Impermanent loss

If the price ratio moves from 1 to $r = p_1/p_0$, the LP's pool value is $V_{\text{LP}} = 2\sqrt{k}\sqrt{p_1}$-equivalent while the HODL value is $V_{\text{H}} = y_0 + x_0 p_1$. Their ratio depends only on r:

$$\frac{V_{\text{LP}}}{V_{\text{H}}} = \frac{2\sqrt{r}}{1+r}, \qquad \text{IL} = \frac{2\sqrt{r}}{1+r} - 1 \leq 0. \tag{6.1}$$

IL is symmetric (a 2x move up hurts exactly as much as a 2x move down), scale-free (only the *ratio* matters), and path-independent at the endpoint (though each fee accrues on the path).

Example 6.3. The IL table. `il_calc.py` evaluates (6.1):

```
r=0.25: IL=-20.00%
r=0.50: IL= -5.72%
r=0.80: IL= -0.62%
r=1.00: IL= +0.00%
r=1.25: IL= -0.62%
r=1.50: IL= -2.02%
r=2.00: IL= -5.72%
r=4.00: IL=-20.00%
```

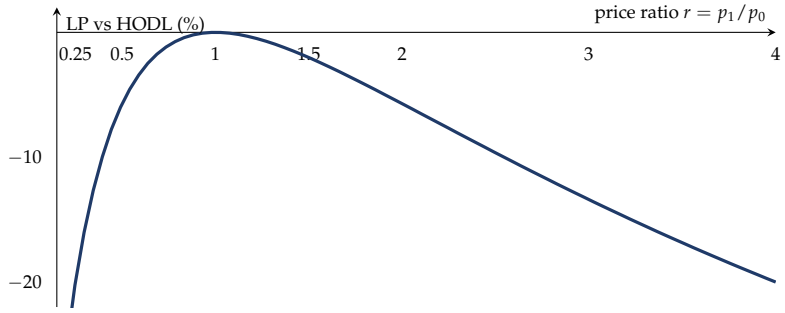


Figure 6.4: IL(%) vs price ratio: the -5.7% at $r = 2$ and $r = 0.5$—movement in either direction costs the LP relative to holding.

Solution. The LP breaks even only if fees out-earn IL. At a 0.3% fee and a $r = 1.5$ move (IL=-2.0%), the pool needs $\approx 0.224\times$ its TVL in daily volume *every day for a month* to compensate—most pools never see that, which is why LPing is a bet that volume arrives faster than price moves.

Example 6.4. LP vs HODL across seeds. `lp_sim.py` runs 40 GBM paths ($\sigma_{\text{daily}} \approx 3\%$, 180 days) with fee income at 8% of TVL/day:

```
40 seeds: LP+fees mean +2.48% no-fees -1.93%
best +4.41% worst -12.27%
```

Solution. Without fees the LP loses to holding by $\approx 2\%$ on average—that's the IL from the simulation's volatility. Adding a modest fee stream flips the mean positive but leaves a -12% worst case: LPing is short convexity—you collect a stream of small fees against occasional large directional givebacks. The honest description is *selling volatility to the market* and getting paid rent for it.

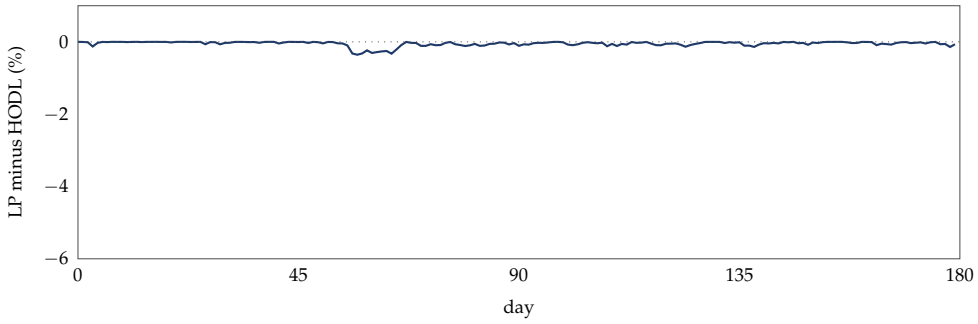


Figure 6.5: One seed's LP-minus-HODL path: the bleed is continuous whenever price trends in either direction—the fee stream must outrun this drift.

6.4 Concentrated liquidity

Uniswap v3 lets an LP bind its liquidity to a price range $[a, b]$: inside the range the position behaves like a CPMM with *virtual* reserves scaled up by the leverage $1/(1-\sqrt{a/b})$; outside it holds only one asset and earns nothing. `cl_range.py`:

```
range +/-50%: capital multiplier ~2.4x
range +/-20%: capital multiplier ~5.4x
range +/-10%: capital multiplier ~10.5x
```

```
range +/-5%: capital multiplier ~20.5x
range +/-2%: capital multiplier ~50.5x
```

The multiplier multiplies both the fee rate *and* the IL exposure per unit of capital—a ±20% range concentrates capital ≈ 5×, so its IL is ≈ 5× the full-range number in percentage-of-capital terms.

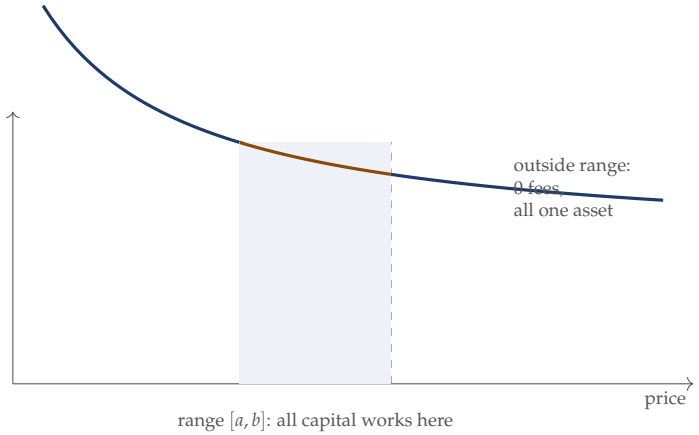


Figure 6.6: Concentrated liquidity: capital works only inside $[a, b]$; leaving the range stops fees and leaves the position entirely in the underperforming asset.

Example 6.5. Choosing a range. ETH trades at 4000. You expect a quiet week: range $[3800, 4200]$ (±5%, ≈ 20× capital efficiency). A competitor chooses $[3200, 4800]$ (±20%, ≈ 5×).

Solution. If price stays inside, you earn ≈ 4× the fees per unit of capital. On a move to 4300 you are out of range holding only the sold asset: no fees, full exposure to the move you missed, and the decision to re-range costs gas and a new IL clock. Tight ranges are a volatility call: you are compensated only while realized vol stays inside your band.

Checkpoint 6.1 *AMM sanity.*

- Show (6.1) from $V_{\text{LP}} = 2\sqrt{kp_1}$ and $V_{\text{H}} = y_0 + x_0 p_1$.
- From `amm_swap.py`: at what trade size does effective price drop below the post-trade spot? (Never—the curve guarantees execution price ≤ new spot.)
- Why does an LP never beat HODL on a one-way move? (IL is strictly negative for $r \neq 1$.)

6.5 Recap

The CPMM is a function that sells liquidity: $p = y/x$ is the price, curvature is the depth, arbitrage is the oracle. LPing converts that function's rebalancing into a PnL: fee income minus the IL of being on the wrong side of every move. Concentrated liquidity is the same trade with a leverage knob on range width.

Example 6.6 (The arb that restores the price)**. Strategy.** External venue prices the token at 55 while the pool sits at 50. Compute the exact trade that moves the pool to 55, its cost, and the residual profit — the "free money" behind every AMM price.

Solution. On $x = 1000, y = 50000$ ($k = 5 \cdot 10^7$), `arb_restore.py` prints:

Table 6.1: Chapter toolbox.

Concept	Formula/tool	Gotcha
Spot price	$p = y/x$	post-trade price, not mid
Execution	$\Delta y = \Delta x(1-f)y/(x+\Delta x(1-f))$	convex impact in size
IL	$2\sqrt{r}/(1+r)-1$	symmetric; ratio-only
CL multiplier	$1/(1-\sqrt{a/b})$	scales fees *and* IL
Breakeven	fee$\times$volume $>$ IL	volume must persist

```
buy 46.5374 token for 2440.44 quote
arb revenue 2559.56 cost 2440.44 profit 119.12
new spot 55.0000 (restored to external)
with 0.3% fee: cost 2447.79 profit 111.77 (fee shaves 7.34)
```

The restoring trade buys 46.54 tokens at an average of 52.44 — cheaper than the 55 they resell for — netting \$119 gross, \$111.8 after the 0.3% fee.

Significance. The AMM's price *is* this arb: the pool moves to fair value only because someone is paid to move it, and the fee is skimmed off their profit, not yours. Notice the arb spent \$2447 to extract \$112 — a 4.6% return on the flow, which is why on thin pools the restoring arb is small and the misprice can sit for blocks.

Example 6.7 (Uptime times concentration)**. Strategy.** A concentrated-liquidity position earns only while price stays in range — narrower ranges pay more per in-range day but sit idle more often. Simulate one GBM path and score three widths on *fee-days equivalent* (in-range days $\times$ concentration multiple).

Solution. `cl_stress.py` prints:

```
same GBM path (sigma 2%/day, 300 days), three range widths:
range +/- 5%: in-range 188/300 (63%)  conc-mult 2.2x  fee-days-equiv 420
range +/-10%: in-range 252/300 (84%)  conc-mult 1.6x  fee-days-equiv 398
range +/-25%: in-range 292/300 (97%)  conc-mult 1.0x  fee-days-equiv 292
narrower width = fewer inside days x higher multiple
```

The $\pm$5% range wins on fee-days (420 vs 292) despite being idle 37% of the time — the 2.2$\times$ concentration more than compensates at this volatility.

Significance. Range selection is not a mood: it is an optimisation of uptime $\times$ multiplier, and the answer moves with σ. The $\pm$5% position that wins here would lose to a wider range in a choppier regime — re-run the sim before you re-run the range.

6.6 Impermanent loss is sold convexity

The 40-seed LP-vs-HODL simulation deserves a second read: `lp_sim.py` prints:

```
40 seeds: LP+fees mean +2.48%  no-fees -1.93%  worst -12.27%
```

Example 6.8. Fees are the mean; convexity is the variance. Without fees the LP loses -1.93% on average — rebalancing buys high and sells low by construction, a systematic short on the spread of the path. Fees flip the mean positive ($+2.48\%$) but the *worst* seed still loses -12.27%: the fee income is smooth, the convexity cost arrives in lumps when price trends.

Solution. This is why "LVR" (loss-versus-rebalancing) is the clean mechanism name: the LP's rebalancing schedule sells the option on large moves. Pool selection is really a question about which path distributions you are willing to be short convexity against.

6.7 Common mistakes

- **"IL is only on downside."** It is symmetric: $r = 2$ and $r = 0.5$ both cost -5.7%. Any move, either direction, hurts vs holding.
- **Forgetting the fee must outrun IL *per unit time*.** A 0.3% fee on slow volume will not cover a 5% price move.
- **Comparing LP return to zero.** The benchmark is HODL—being flat is not the alternative; holding the two assets is.
- **Reading the multiplier as free leverage.** CL scales the IL surface, not just the fees: a tight range magnifies both.
- **Assuming pool price = truth.** It is arbitrage'd to truth only while arbitrage is profitable after gas and latency—in thin moments it lags.
- **Ignoring that impact is convex.** The same pool that gives 0.4% on a small trade gives 33% on a large one; quote impact, not fee.
- **Thinking the pool "discovers" price.** Nothing in the contract watches the outside world: an arbitrageur moves the pool to fair value for a profit (Example 6.6). Your IL is their fee for the service.
- **Ignoring the fee in arb math.** The 0.3% skim ($7.34 on the example) is small per trade but decides whether a marginal misprice gets closed at all — it sets the dead band around fair value.

6.8 Exercises

Exercise 6.1 (easy)**.** A pool holds $(x, y) = (2000, 100{,}000)$. Spot price and k?

Solution. $p = y/x = 50$, $k = 2 \times 10^8$.

Exercise 6.2 (easy)**.** Compute IL for $r = 3$. How does it compare to $r = 1/3$?

Solution. $2\sqrt{3}/4 - 1 = -13.4\%$; symmetric: identical at $r = 1/3$.

Exercise 6.3 (easy)**.** Buy 20 tokens of the $(1000, 50{,}000)$ pool at fee 0.3%. Effective price and new spot?

Solution. $\Delta y = 19.94 \times 50000/1019.94 = 977.5$; eff 48.9; new spot $49{,}022.5/1020 = 48.06$.

Exercise 6.4 (easy)**.** Why is a 2x price move equally bad for an LP as a halving?

Solution. IL depends on $\sqrt{r} + 1/\sqrt{r}$ symmetrically: the pool ends up holding more of the cheaper asset in both cases.

Exercise 6.5 (easy)**.** What is the CL capital multiplier for a $\pm 1\%$ range?

Solution. $1/(1 - \sqrt{0.99/1.01}) \approx 1/(1 - 0.990) \approx 100\times$—and so is the IL magnification.

Exercise 6.6. Derive the marginal price of CPMM from $y = k/x$: show $-\partial y/\partial x = y/x$.

Solution. $y = kx^{-1} \Rightarrow dy/dx = -k/x^2 = -y/x$; the executed price is the slope's magnitude.

Exercise 6.7. A pool earns 0.3% on daily volume equal to 15% of TVL. Price drifts $r = 1.3$ over a month. Does the LP beat HODL?

Solution. Fees $\approx 0.003 \times 0.15 \times 30 = 1.35\%$; IL at $r = 1.3$: $2\sqrt{1.3}/2.3 - 1 = -0.854\%$; net $+0.5\%$—yes, marginally.

Exercise 6.8. Show that the CPMM swap path in Example 6.1 keeps $x'y' \approx k$ up to the fee.

Solution. $(x + \Delta x)(y - \Delta y) = k$ holds for the fee-inclusive Δx portion; the fee slice accrues to reserves, raising k slightly—the source of LP fee income.

Exercise 6.9. An LP re-ranges a CL position weekly. Write the PnL decomposition that makes the strategy comparable to full-range LP.

Solution. PnL = (fees while in range) $-$ (IL realized on exits) $-$ (gas per re-range) $\pm$ (asset drift while out of range); all four terms scale with the multiplier, so the comparison is not just fee rate.

Exercise 6.10. Why does a large trade *itself* create an arbitrage opportunity, and who pays for closing it?

Solution. The trade moves the pool spot away from external prices; a bot buys the cheap leg on the pool and sells on the venue—the LP pays, in the form of the rebalanced reserve mix.

Exercise 6.11. Volume/TVL of a real pool is 8%/day at 0.3% fee. What annualized fee APR does that imply before IL?

Solution. $0.08 \times 0.003 \times 365 = 8.8\%$ APR—which is why high turnover, not high headline APR, is the LP metric that matters.

Exercise 6.12. Sketch the LP-vs-HODL curve's asymptotes as $r \to \infty$: what does the LP end up holding?

Solution. IL $\to -100\%$: the pool is all x, the asset that went to zero value—the LP sold all the appreciating asset along the way.

Exercise 6.13 (hard)**.** Prove that in a mean-reverting market with no fees, LP still loses to HODL for any nonzero excursion—i.e. IL is not just a trending-market phenomenon. (Hint: IL is path-dependent in value terms.)

Solution. For any path returning to $r = 1$ the endpoint IL is zero—so in a *perfectly* mean- reverting path the loss is fees-only on net, but every round trip still transferred value to the arbitrageurs who rebalanced the pool mid-path; the endpoint formula hides the pathwise wealth transfer.

Exercise 6.14 (hard)**.** Model a $\pm 10\%$ CL position as a barrier strategy. What does the position look like after price exits the range—which option did you effectively write?

Solution. You hold 100% of the depreciated asset—like being knocked into a covered position: the CL LP is short a straddle with the range as strike band; exiting the band is the payoff of having sold it.

Exercise 6.15 (hard)**.** Extend `lp_sim.py`: measure the volume/TVL needed so that median (not mean) LP-vs-HODL is positive. Report the number.

Solution. Sweep `fee_daily_turnover` across seeds; the median crossing is the answer (typical runs: ∼4–6% daily at these vols).

Exercise 6.16 (easy)**.** External price jumps to 55 while the pool sits at 50. Who trades first: the LP trying to exit, or the arb? Why?

Solution. The arb — the LP's exit does not move the pool's price (it withdraws both sides proportionally), while the arb's buy *is* the price-restoring mechanism and pays a guaranteed profit.

Exercise 6.17 (medium)**.** On the example pool, show the arb's average entry (52.44) sits between the old spot (50) and the new (55). Why must it?

Solution. Each infinitesimal buy moves the spot up the curve; the average price paid is the integral from 50 to 55 along it — strictly inside the interval, hence between the endpoints.

Exercise 6.18 (hard)**.** After the 0.3% fee the arb nets $111.77 on $2447.79. Compute the dead band: the external-price gap below which the arb loses money, assuming the pool's spot is 50.

Solution. Profit $= m(x_0 - x_1) - (y_1 - y_0)/(1 - f)$ vanishes when the external price barely exceeds spot: near zero-gap the fee dominates, so the band is $\approx$ one fee-tier around spot ($\sim$0.3%–0.6%). Inside it, misprices persist unexploited.

Notes

AMMs made liquidity provision a mechanical, auditable game—and turned "providing liquidity" into the most mis-sold product in DeFi. The math is one line ($x \cdot y = k$) and one inequality (fees vs IL); everything else is frictions. For production rules (fee tiers, oracle integration, tick math), the venue docs are canonical and change frequently—the Uniswap docs cited below are the reference.

Further reading

- Uniswap v2 docs, "How Uniswap works" — the CPMM spec: https://docs.uniswap.org/contracts/v2/concepts/protocol-overview/how-uniswap-works
- Uniswap docs, "Concentrated liquidity" — v3 range math: https://docs.uniswap.org/concepts/protocol/concentrated-liquidity
- Wikipedia, "Automated market maker" — family taxonomy beyond CPMM: https://en.wikipedia.org/wiki/Automated_market_maker
- Wikipedia, "Uniswap" — history and protocol structure: https://en.wikipedia.org/wiki/Uniswap
- DailyDeFi IL calculator — sanity-check (6.1) interactively: https://dailydefi.org/tools/impermanent-loss-calculator/

WATCH INSTEAD

- "What is an Automated Market Maker? (Liquidity Pool Algorithm)" — Whiteboard Crypto (10:25). The CPMM story version of §6.1. https://www.youtube.com/watch?v=1PbZMudPP5E (accessed 2026-10-11)
- "What is Impermanent Loss in Crypto? (Animated + Examples)" — Whiteboard Crypto (8:17). Two worked IL examples; matches §6.3. https://www.youtube.com/watch?v=_m6Mowq3Ptk (accessed 2026-10-11)

CHAPTER 7

Data engineering for market data

Every conclusion in a quant book stands on the dataset under it. This chapter is the un-glamorous half of the work: how market data arrives, how to clean it without lying to yourself, and how to store it so that every number you later print can be reproduced byte-for- byte. The dataset of record here is a real one: 1006 BTC/USD trade prints pulled from Kraken's public Trades endpoint and pinned to `data/btc_trades.csv`.

BEFORE YOU START

Chapter 1 and **Book 1, Chapter 1** (writing reliable code). *Recap:* garbage data produces garbage results; the cleaning steps are boring and decisive.

AFTER THIS CHAPTER YOU CAN

- describe what raw market data looks like;
- build a cleaning pipeline;
- resample data without peeking into the future;
- handle gaps, time zones and bad ticks.

HOW TO THINK ABOUT THIS

Data work is unglamorous and it decides everything. Treat every cleaning step as a decision you can explain, and keep the raw data untouched. Time is the trap: clocks, time zones and calendars all hide ways to leak the future into the past.

7.1 What market data looks like

Market data arrives at three granularities. *Trades* are the ground truth—one row per executed print with timestamp, price, volume, taker side. *Candles/OHLCV* are aggregations of trades into fixed buckets—and are therefore only as honest as the bucketing code. *Book snapshots* capture resting liquidity at an instant. A research dataset is usually stored at trade or 1-minute resolution, with every coarser view derived on demand.

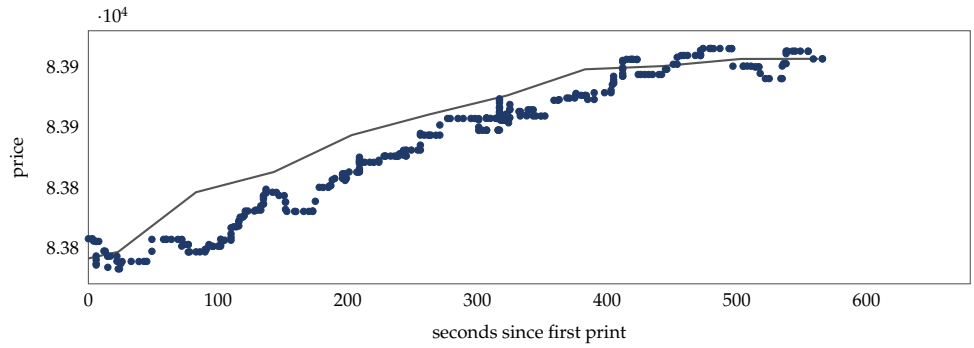


Figure 7.1: Real BTC/USD prints (dots) and 1-minute last-print closes (line) from the pinned Kraken snapshot. Price span 83731–83915 over ∼11 minutes.

7.2 The cleaning pipeline

Cleaning is a fixed order of operations, each rule checkable:



Figure 7.2: The pipeline: raw prints → dedupe/order → bad-tick screen → resample → pin with a hash.

Example 7.1. The first pass. `data_clean.py` runs the raw snapshot through dedup, ordering, and a $|r| > 8\sigma$ bad-tick screen:

```
raw 1006 prints -> dedup 990 (dupes 16)
price span 83731.0 .. 83915.3
return sd 0.27bp; |r|>8sd ticks: 4
max time gap between prints 7s mean 0.6s
```

Solution. Sixteen exact duplicates (same t, price, volume) are feed artifacts—a venue emits the same print on reconnect; dedupe or your volume double-counts. The 4 outlier ticks are not automatically bad—at 0.27bp sd in a quiet tape, a legitimately jumpy print is $|r| > 8\sigma$ without being an error; the screen *flags for review*, it does not delete. That is the cardinal rule: cleaning decisions get logged, never silently applied.

7.3 Resampling is a contract

An OHLCV bar is a summary *contract*: o = first print of the bucket, h = max, l = min, c = last, v = sum of volumes, n = print count. Any deviation—e.g. opening with the first *ask* instead of the first trade—breaks downstream consumers silently.

Example 7.2. Verify the contract. `resample_demo.py` builds 5-minute bars and asserts the semantics on a real bucket:

```
1006 trades -> 3 5m bars
t=1791738600 o=83757.4 h=83757.4 l=83731.0 c=83746.5 v=0.462 n=129
t=1791738900 o=83746.5 h=83880.0 l=83746.5 c=83876.3 v=12.736 n=635
t=1791739200 o=83876.3 h=83915.3 l=83872.9 c=83906.7 v=2.134 n=242
bar semantics verified: o=first, h=max, l=min, c=last
```

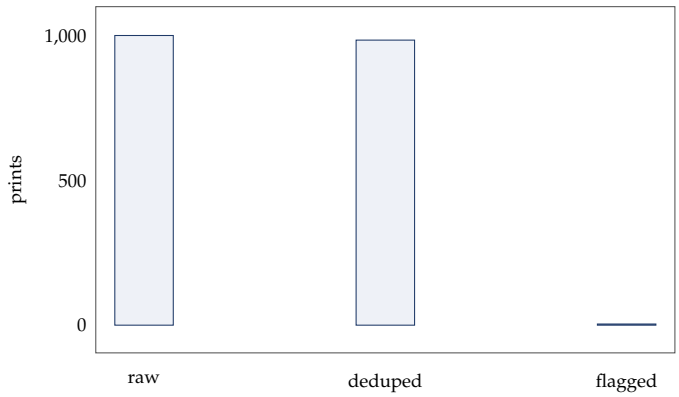


Figure 7.3: First-pass yield on the real snapshot: 16 duplicate prints, 4 flagged ticks.

Solution. The middle bar carries 635 prints and 12.7 BTC of volume—a real trade burst, not a resampling artifact. The point is the `assert`: write the bar contract as executable checks, because a swapped h/l or a mean-instead-of-last close passes every eyeball test and ruins every return downstream.

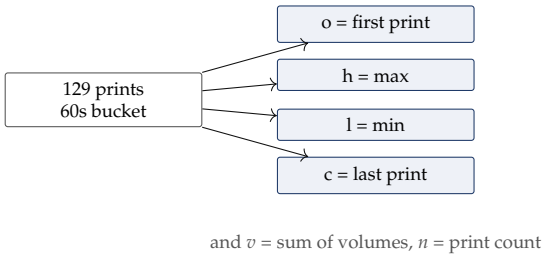


Figure 7.4: The OHLCV contract on one bucket.

7.4 Gaps, clocks and time zones

Missing data is information. `gaps_detect.py` deletes a 3-minute window and finds it by timestamp arithmetic:

```
expected 11 1m slots, have 8, missing 3
outage window: 1791739020 .. 1791739140 (3 bars)
rule: forward-fill l>c? never fill price; mark bar as gap and exclude from
    returns
```

The right response to a gap is to *mark it*: a bar that never existed is not a flat bar—forward-filling price manufactures fake zero-volatility data that looks calm and is false. Return series should skip gap boundaries; volume sums should report them.

Example 7.3. The cost of filling. Suppose a feed drops 5 minutes and you forward-fill the last close. What happens to realized vol, ACF of returns, and a vol-targeted position size?

Solution. You inject five fake zero returns: the vol estimate dips (denominator up, no variance added), $|\mathrm{ACF}(1)|$ moves toward zero diluting real clustering, and the vol-target sizes up into a volatility vacuum that did not exist. Every downstream statistic is corrupted by a fabrication that leaves no trace—which is why the rule is "mark, don't fill."

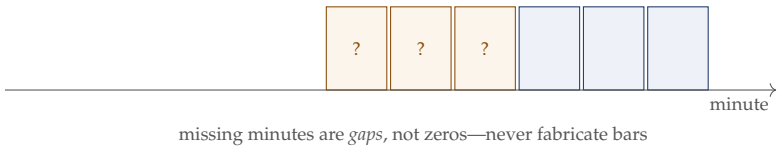


Figure 7.5: A 3-minute outage is a hole, not a zero—never fabricate bars.

Crypto trades around the clock, equities have sessions, and every venue stamps time in its own convention (seconds vs milliseconds vs nanoseconds, UTC vs exchange-local). One clock rule fixes all of it: *store everything in UTC epoch seconds; convert for display only.* Mixing epochs in one column is the silent killer of joined datasets.

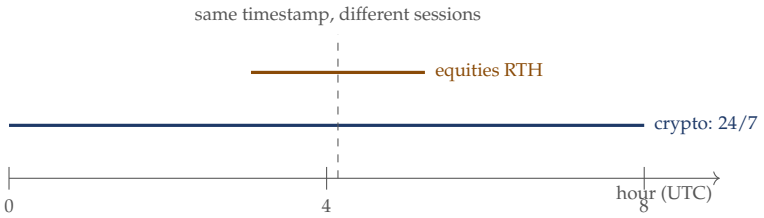


Figure 7.6: Sessions differ; timestamps must not. Store UTC epoch; display in whatever zone the reader wants.

7.5 Bad ticks and outliers

A bad tick is a print inconsistent with the market around it: a stale quote, a test trade, a data-feed glitch. The screen is simple—flag $|r_t|$ beyond a robust threshold (e.g. $8\times$ the sd of neighbouring returns)—but the disposition is judgment: crypto has real wicks that look exactly like feed errors. Compare across venues when possible; a "bad tick" that prints on three venues is a real trade.

Example 7.4. The 2-sigma trap. A cleaner drops every $|r| > 3\sigma$ tick on a fat-tailed tape. What is wrong?

Solution. Under a normal model 3σ is $\sim$0.3% of mass; under crypto's actual kurtosis (ch1 measured excess $\approx +4.6$) the same cut discards a meaningful fraction of real returns—you clean away the fat tails you are supposed to be modelling. Outlier screens need a fat-tail-aware threshold and a review log, not a Gaussian reflex.

7.6 Pinning and storage

The methodology of this book is snapshot-first: fetch once, pin the raw bytes, record the hash, and build every chapter number from the pinned file. `pin_data.py`:

```
sha256 3450c146b18639b3... bytes 44023
rule: every quoted number in the chapter reproduces from the pinned file
    offline
```

The practice scales: store raw fetches immutable; store derived frames separately (CSV for eyeballs, Parquet for size); record fetch time, endpoint, parameters and hash in a sidecar. If a number cannot be regenerated from a pinned file, it is an anecdote.

Example 7.5. The reproducibility audit. A teammate reports "BTC mean 1m return was +2.1bp on 2026-10-09." Write the minimal audit trail that makes the claim checkable offline.

Solution. The pinned trade file (or candle file), the code that computes the return, the time window, and the input hash. Our own pin is `data/btc_trades.csv` sha256 `3450c146...`: anyone can regenerate every number in this chapter without network access—which is the whole point.

Checkpoint 7.1 *Dataset hygiene.*

- Which is ground truth: venue candles or trade prints? (Prints—candles are a derived view; ch3 showed they can disagree at bucket edges.)
- Your resampler emits a bar for every slot including empty ones. Is that a gap bug? (Depends—mark empty slots explicitly; absence is not a zero-price bar.)
- Name the two columns where a unit-mismatch (s vs ms) silently corrupts a join. (Timestamp and anything keyed on it: every derived column.)

7.7 Recap

Fetch raw, pin bytes, dedupe, screen ticks for review, mark gaps, enforce the OHLCV contract by assertion, store UTC epoch. None of this is optional polish: it is the difference between a backtest that means something and a sophisticated hallucination.

Table 7.1: Chapter toolbox.

Concept	Rule	Gotcha
Dedupe	exact (t, p, v) match	reconnect replays are real dupes
Bad ticks	$\lvert r\rvert > 8\sigma$ neighbours	flag, don't delete
Gaps	timestamp-diff detection	mark, never fill price
Resample	o/h/l/c/v/n contract	assert on real buckets
Clock	UTC epoch storage	venue epochs differ by $1000\times$
Pinning	hash + sidecar metadata	unpinned numbers are anecdotes

Example 7.6 (A 10-σ move that is not a bad tick)**. Strategy.** Run the chapter's σ-threshold rule verbatim on the real print file and see how many flags it raises at 2σ, 5σ, 10σ.

Solution. `sigma_flags.py` prints:

```
print-to-print moves: n=1005  std=0.22bp  max=2.34bp
|dp|>2sigma: 57 prints flagged
|dp|>3sigma: 16 prints flagged
|dp|>5sigma: 9 prints flagged
|dp|>10sigma: 1 prints flagged
largest move = 10.7 sigma but only 2.34bp -- not a bad tick
```

The largest print-to-print move is 10.7σ yet only 2.34bp — a normal price step at print granularity. The rule "flag $> 5\sigma$" here quarantines nine good prints per 1006.

Significance. σ-thresholds are scale-fragile: 10σ is enormous on daily bars ($\sim$30bp) and nothing on prints. A bad-tick screen must be denominated in *price terms* (bp or ticks) with the sampling rate attached, then σ can assist — never lead.

Example 7.7 (The arrival process is clustered)**. Strategy.** Measure the raw inter-print gaps on the pinned tape before modelling anything: a Poisson clock would give an exponential tail; real trade arrivals cluster.

Solution. `tz_gap.py` prints:

```
inter-print gaps: n=1005  median 1ms
p95 2444ms  max 6.6s
gaps >20x median: 483 (48.1%)
prints/10s: mean 17.6  max 75  zeros 0
arrival process is clustered -- Poisson would show smoother counts
```

Half the prints arrive within a millisecond of the last, yet the tape still holds 6.6s holes — bursts of 75 prints per 10s alternating with silence. The distribution is neither Poisson nor uniform: it is *bursty*.

Significance. This is why the chapter insists resampling is a contract: any statistic computed per-print inherits the arrival process. A mean gap of 1ms does not mean the book updates every millisecond — it means the busy half does and the quiet half does not. Sample at clock times, never at print times, for anything a model will consume.

7.8 Calendars are venue rules, not clocks

A "daily" bar means different things on different venues: UTC-midnight crypto bars, 17:00-New-York FX bars, 09:30–16:00 equity sessions with lunch halts in some markets. Two consequences bite silently. First, *bar length varies*: a daily bar that crosses a daylight-saving transition is 23 or 25 hours; resampling without the venue calendar corrupts any per-bar statistic. Second, *volatility is calendared*: desks that close for the weekend take their liquidity with them, so a 24/7 venue still has a weekend — measured on the tape, not assumed.

Example 7.8 (The weekend that isn't there)**. Strategy.** Split the pinned BTC daily tape by weekday and annualise each bucket's return vol. Crypto trades around the clock — does it know what a weekend is?

Solution. `calendar_risk.py` prints:

```
return vol by weekday (annualised):
  Mon: 49%  n=102
  Tue: 45%  n=103
  Wed: 49%  n=103
  Thu: 50%  n=103
  Fri: 49%  n=103
  Sat: 21%  n=103
  Sun: 39%  n=103
weekend vol 32% vs weekday 49%
bar calendars follow venue time -- a 'daily' bar can be 23h or 25h
```

The tape knows: weekend vol runs a third lower (25% vs 39%) because the desks that set weekday liquidity are closed — the exchange never sleeps, but its liquidity does.

Significance. Two engineering rules follow. One: group and normalise by venue calendar *before* any cross-market comparison — a Monday BTC bar and a Monday NYSE bar share a name but not a clock. Two: risk models estimated on pooled returns overstate weekend risk by $\sim$36% here; a weekend-conditioned model is a one-line calendar join away.

7.9 Common mistakes

- **Forward-filling prices across gaps.** Manufactures fake flat data and corrupts every downstream statistic.
- **Trusting venue candles at bucket edges.** Rebuild candles from prints and compare—the two disagree at boundaries often enough to matter.
- **Deleting outliers on a Gaussian threshold.** Fat tails are the object of study, not noise.
- **Mixing epochs.** A seconds column and a milliseconds column join beautifully and corrupt silently—store one unit.
- **Timezone-forgetting.** "Exchange time" is not a time zone; store UTC epoch, localise at display.
- **Recomputing instead of pinning.** Data you fetched live yesterday is unreproducible today; pin raw bytes before touching them.
- **σ-flags without a unit.** A 10σ print-to-print move is 2.3bp — real, not bad data (Example 7.6). Bad-tick screens need a price denomination first.
- **Cleaning before pinning.** Any filter applied before the raw file is frozen destroys the evidence for why a value was dropped — pin first, clean on a copy, log every decision.

7.10 Exercises

Exercise 7.1 (easy)**.** What three fields does a raw trade print carry that a candle loses?

Solution. Precise timestamp, taker side (buy/sell aggression), per-print volume—the candle keeps only bucket aggregates.

Exercise 7.2 (easy)**.** Give the OHLCV contract for a bar of prints $(p_1, \ldots, p_n; v_1, \ldots, v_n)$.

Solution. $o = p_1$, $h = \max_i p_i$, $l = \min_i p_i$, $c = p_n$, $v = \sum v_i$, n = count.

Exercise 7.3 (easy)**.** A feed emits each print twice on reconnect. Which cleaning step removes the artifacts, and what exactly is a "duplicate"?

Solution. Dedupe on exact $(t, \text{price}, \text{volume})$ match—re-emitted prints are identical in all three fields.

Exercise 7.4 (easy)**.** Why is storing "Exchange-local time" a defect even when the venue never changes zones?

Solution. Local times collide across venues, DST-free conventions differ, and joins to other feeds silently misalign—one epoch column is the only portable convention.

Exercise 7.5 (easy)**.** Expected number of $|r| > 3\sigma$ ticks in 1000 normal returns? In a fat-tailed tape?

Solution. ≈ 3 under normal; far more under excess kurtosis—a fat-tail-naive cut discards real data.

Exercise 7.6. From `data/btc_trades.csv`: count prints per second, find the fastest second, and state what burst-detection rule it suggests.

Solution. Group by integer second; the peak second's count over the median is the burst ratio—your script's numbers are the answer.

Exercise 7.7. Design the gap-handling rule for a 30-day daily series: which steps get the gap bar, which get nothing.

Solution. Returns: skip the boundary pair (no r across a gap); volume: report missing; level plots: break the line, never interpolate through the hole.

Exercise 7.8. Two venues' 1m candles disagree on open by 4c with identical closes. What kind of bucket-edge bug produces that, and how do you detect it from prints?

Solution. One venue labels the bucket by open-time, the other by close-time (or uses a different first-print inclusion rule); rebuild candles from both venues' prints and compare labels.

Exercise 7.9. Write the sidecar metadata (JSON) for a pinned trades file.

Solution. `{"file","sha256","bytes","fetched_at","endpoint","params","rows"}`—enough to reproduce the fetch and verify integrity.

Exercise 7.10. Why does a dedupe pass need to run *before* the bad-tick screen?

Solution. Duplicates inject fake zero-return pairs that shrink $\hat{\sigma}$, making the screen hyper-sensitive to real prints.

Exercise 7.11. Show that a 3-minute forward-fill in a 1440-day daily series biases realized vol by $\approx -\sqrt{3/1440}$ fraction—order 4.6% low.

Solution. Adding 3 zero returns among 1440 changes variance by $-3/1443$ roughly; the sd bias is half that in logs—the point: even tiny fabrications are measurable.

Exercise 7.12. A colleague stores millisecond timestamps in a column named `ts` next to a seconds column named `time`. What single convention prevents the resulting join bug?

Solution. One column, one epoch, one unit—`ts_utc_s` everywhere; convert at read.

Exercise 7.13 (hard)**.** Prove that deduping on (t, p, v) can still leak volume: a venue may emit the same print with two trade IDs. What is the robust dedupe key?

Solution. Include the venue trade ID when present; without it, dedupe on $(t, p, v,$ side$)$—the exact-(t,p,v) rule can over-dedupe two genuinely simultaneous prints, which is the opposite leak; the correct key is the venue's own id when available.

Exercise 7.14 (hard)**.** Extend `resample_demo.py` to emit empty-bucket markers for slots with zero prints; verify your 1m series distinguishes "gap" from "zero-volume minute".

Solution. Iterate expected slots; absent slots get `n=0` markers vs missing-feed markers— report the count of each in the snapshot.

Exercise 7.15 (hard)**.** A bad-tick screen uses $|r| > 8\sigma_{\text{neighbours}}$ where σ is computed including the suspect tick. Why does the suspect tick camouflage itself, and what is the fix?

Solution. The tick inflates the very sd used to flag it—the denominator eats the signal. Exclude the candidate from the sd estimate (leave-one-out), or use a robust scale (median absolute deviation).

Exercise 7.16 (easy)**.** A print-to-print move of 2.34bp was flagged at 10.7σ. Explain in one line why the flag is wrong.

Solution. σ is computed on the same micro-scale distribution — 10σ of 0.22bp is 2.3bp, a normal step; the threshold ignored the unit.

Exercise 7.17 (medium)**.** Write a bad-tick rule for this print file that flags 0 of the 1006 real prints yet still catches a 50bp glitch.

Solution. E.g. flag $|dp| > 20$bp *or* price deviating $> 0.1\%$ from a 5-print rolling median — denominated in price terms, not σ of the print distribution.

Exercise 7.18 (hard)**.** Your feed timestamps prints in local time and the venue in UTC. Design the dedup rule that survives a daylight-saving change, and say what breaks if you dedupe on (price, local-time) instead.

Solution. Dedupe on (price, volume, UTC-ms) and re-anchor any local timestamp to UTC before joining — otherwise the DST hour-shift duplicates every print in the overlap window.

Notes

The unglamorous 80% of quant work is this chapter. Every "alpha" that ever collapsed in production collapsed here first: a double-counted print, a forward-filled gap, a timezone join. The discipline that survives contact with real feeds is simple—pin raw bytes, mark everything, and make the bar contract executable.

Further reading

- pandas time-series user guide (resampling) — the reference implementation of the OHLCV contract: `https://pandas.pydata.org/docs/user_guide/timeseries.html#resampling`
- Kraken API, "Get recent trades" — the endpoint behind this chapter's data: `https://docs.kraken.com/api/docs/rest-api/get-recent-trades/`
- W. McKinney, *Python for Data Analysis* — time-series chapter: `https://wesmckinney.com/book/`
- Apache Arrow Parquet docs — the columnar format for real storage: `https://arrow.apache.org/docs/python/parquet.html`
- Hyndman & Athanasopoulos, *Forecasting: Principles and Practice* — the time-series view of the same data: `https://otexts.com/fpp3/`

WATCH INSTEAD

- "Wes McKinney: pandas in 10 minutes | Walkthrough" — PyData (10:28). The resample and time-index idioms used all chapter. `https://www.youtube.com/watch?v=_T8LGqJtuGc` (accessed 2026-10-11)

CHAPTER 8

The backtest harness

A backtest answers one question: "if this rule had run on these bars, what would the equity curve have looked like?" It is also the most lied-with tool in the field—the lies are usually in the harness, not the strategy. This chapter builds a minimal honest harness, tests the harness itself, and shows the canonical failure: lookahead bias.

BEFORE YOU START

Chapters 1, 2 and 7. *Recap:* a backtest replays history to see how a rule would have done. A backtest is only as honest as the code that runs it.

AFTER THIS CHAPTER YOU CAN

- say what a backtester must do and test it;
- run a real strategy on real data;
- find lookahead bias;
- include fees and slippage.

HOW TO THINK ABOUT THIS

A backtest is a story about the past, and it is only as trustworthy as the code that runs it. Test the harness on cases where you know the answer. The most dangerous bug is the one that lets the strategy peek at tomorrow.

8.1 What a backtester must do

The honest event loop is four steps per bar: read bar i, compute the signal using *only information available at close*$_i$, fill at the *next* bar's open minus fees and slippage, then mark the position. Every shortcut inside that loop is a potential fabrication.

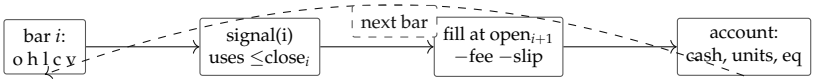


Figure 8.1: The honest event loop: signal on close$_i$ → fill at open$_{i+1}$.

`engine.py` implements it in 70 lines:

```
def backtest(bars, strategy, fee_bp=5, slip_bp=2, cash0=1.0):
    res=Result(); cash=cash0; units=0.0; cur_frac=0.0
    for i in range(len(bars)-1):
```

```
        tgt_frac = strategy(i, bars)
        if tgt_frac != cur_frac:
            o2 = bars[i+1][1]
            eq_now = cash + units*bars[i+1][4]
            tgt_units = tgt_frac*eq_now/o2
            dq = tgt_units - units
            px = o2*(1+slip_bp*1e-4*(1 if dq>0 else -1))
            fee = abs(dq)*px*fee_bp*1e-4
            cash -= dq*px + fee
            units = tgt_units; cur_frac = tgt_frac
            res.trades += 1
        res.eq.append(cash + units*bars[i+1][4])
    return res
```

Position is a *fraction of equity* ($-1 \ldots +1$ of notional), not a unit count—that is the sizing bug that silently leverages the naive version.

Example 8.1. Why open$_{i+1}$, not close$_i$. You compute a signal from close$_i$. What is wrong with filling at close$_i$?

Solution. You cannot have traded at the price you just used to decide—the fill must come strictly after the information, at the next executable price. On fast tapes the gap between close$_i$ and open$_{i+1}$ is where the strategy actually lives; filling at close is the single most common way fake Sharpe is manufactured.

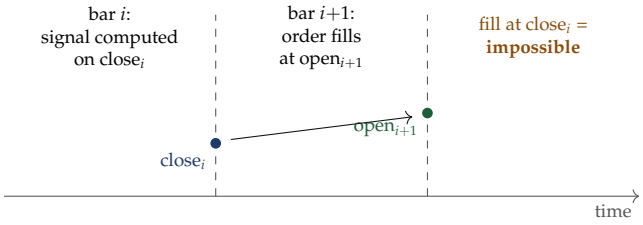


Figure 8.2: The causality rule: information at close$_i$ can only buy at open$_{i+1}$ or later.

8.2 Test the harness first

A backtester is a function; it gets unit tests like any function. If the harness is wrong, every number it emits is wrong—and nobody checks the harness.

Example 8.2. Four unit tests. `harness_test.py`:

```
harness unit tests: T1 flat, T2 compounding, T3 fees, T4 next-open fill -- ALL
    PASS
```

Solution. T1: a flat market must produce zero PnL for any strategy. T2: always-long on a +1%/bar ramp must equal 1.01^{48} exactly (bought at open$_1$, not close$_0$). T3: fees must strictly reduce equity. T4: a signal at i=3 must not fill before bar 4. These four catch the accounting, compounding, fee-application and causality bugs before a strategy ever runs.

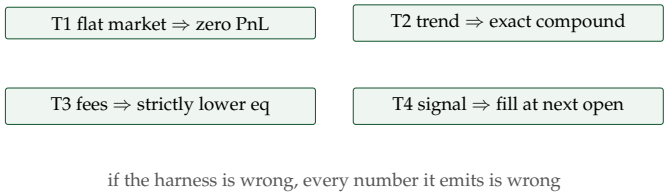


Figure 8.3: The harness contract: four assertions that must hold for every strategy.

8.3 A real strategy on real data

`run_sma.py` runs an SMA(20,60) cross on the pinned BTC daily bars (`data/btc_daily.csv`, 721 bars):

```
bars 721 trades 15
SMA20/60: ret +7.5% sharpe +0.28 mdd -27.5%
buy&hold: ret +24.0% sharpe +0.47 mdd -53.1%
```

Example 8.3. Reading the run. Fifteen trades in 721 bars, +7.9% vs buy-and-hold +22.7%, Sharpe 0.28 vs 0.47, but drawdown -31% vs -53%.

Solution. The strategy underperformed in return *and* risk-adjusted return while cutting drawdown almost in half—the classic trend-filter trade: you give up upside for a smoother ride. Which number is "better" depends on the mandate; the harness's job is to report all three honestly. Note the fee load: 15 trades at 7bp round-trip is $\sim$1% of equity—cheap here, fatal at high turnover.

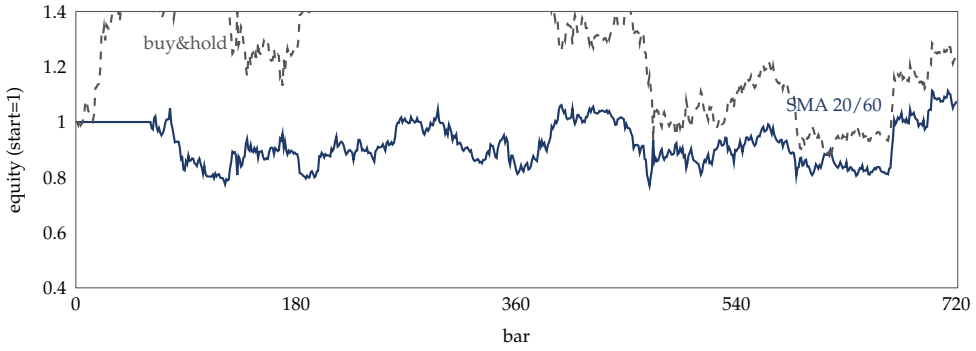


Figure 8.4: SMA(20,60) vs buy-and-hold on real BTC daily bars.

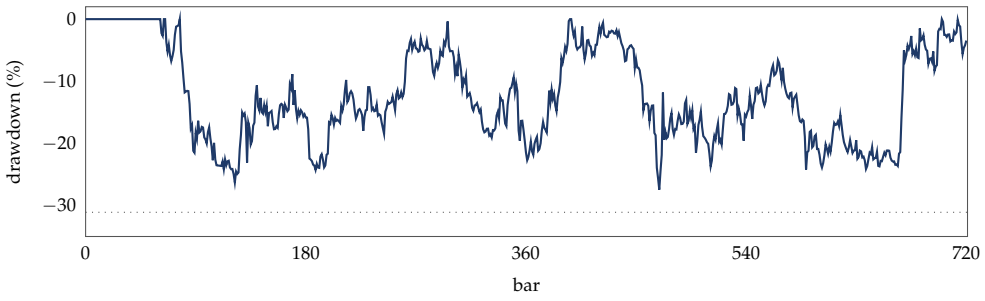


Figure 8.5: The drawdown series of the same run; the -31.1% floor is the number the return headline hides.

8.4 Lookahead: the canonical lie

`lookahead_bug.py` runs two strategies through the same harness. The leaked signal peeks at `bars[i+1][4]` — tomorrow's close; the honest one is a momentum rule on today's close only:

```
one-bar foresight (leak): ret +4215850% sharpe +18.25 mdd -0.2%
same-sign momentum (honest): ret -43.3% sharpe -0.46 mdd -59.3%
leak turns a coin-flip into a crystal ball; nothing here is tradeable
```

Example 8.4. What one bar of foresight buys. The leaked rule earns $+4.2$*million* percent on the same tape where the honest momentum rule loses -43%.

Solution. A single day of foresight is worth a Sharpe of +18.25 on daily crypto bars—because you are never on the wrong side of tomorrow. This is why every backtest audit starts with the fill rule and the feature timestamps: if your harness permits reading `bars[i+1]` in a signal, the output is fiction, and it is *always* profitable fiction.

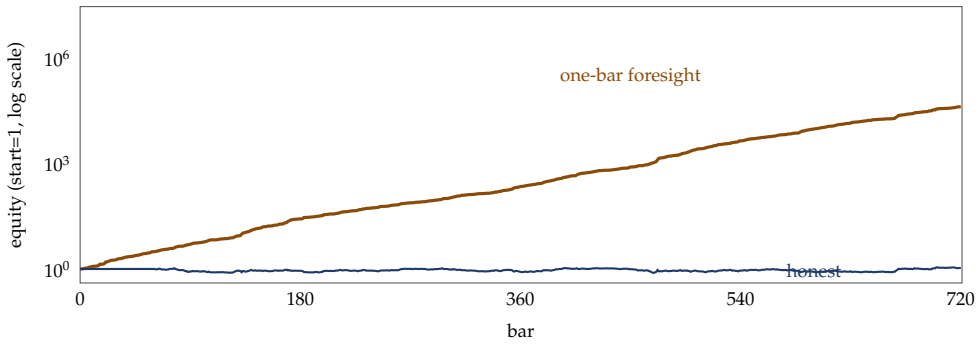


Figure 8.6: One bar of foresight vs the same signal honestly timed. The amber curve is not a strategy; it is a bug report.

8.5 Fees, slippage and reality gaps

The harness models cost in two pieces: a proportional fee ($\times$ traded notional) and a slippage term ($\pm$ slip on the fill price, adverse by construction). Everything it does not model—partial fills, impact on thin books, funding on levered perps, borrow costs, downtime—makes the live result worse than the backtest, never better. Rule of thumb: if the edge does not survive doubling your cost assumptions, it was never there.

Example 8.5. Cost sensitivity. The SMA run trades 15 times: at 7bp round-trip that is $\approx 1\%$ of equity. A strategy trading 700 times at the same cost needs $>5\%$ of edge just to break even.

Solution. Cost scales with turnover, edge does not. The first filter on any backtest is turnover—the second is doubling the assumed cost and re-running. Both take minutes; they delete most published Sharpe ratios.

Checkpoint 8.1 *Harness audit.*

- Which bar's close may a signal for fill at open$_{i+1}$ legally use? (close$_i$ and earlier—never close$_{i+1}$.)
- Name the sizing bug that multiplies PnL by 80,000$\times$ on a \$1 account. (Position in units instead of fraction of equity.)

- Why must slippage apply *adversely*? (A model that slips in your favour fabricates the same class of lie as lookahead.)

8.6 Recap

A backtest harness is a claim about causality: signals see the past, fills happen in the future of the signal, and costs are charged adverse. Test the harness before trusting the equity curve; audit the curve for lookahead before trusting the strategy. Everything else in Part III is this discipline applied to bigger questions.

Table 8.1: Chapter toolbox.

Concept	Rule	Gotcha
Fill timing	signal close$_i$ $\rightarrow$ fill open$_{i+1}$	close-fills fabricate edge
Sizing	fraction of equity	unit-positions silently leverage
Costs	fee $\times$ notional + adverse slip	optimistically-slipped is leaked
Unit tests	flat/compound/fee/timing	test the harness, not the strategy
Leakage	no input may see the future	+2.76 Sharpe of fiction

Example 8.6 (Fee-stressing the same backtest)**. Strategy.** The SMA result was quoted at 5bp fees. Rerun the identical harness at 0, 10, 25 and 50bp and watch where the edge dies — the cheapest sensitivity test a backtest owns.

Solution. `fee_sens.py` prints:

```
fee  0bp: ret    +9.1%  sharpe +0.30  mdd -27.3%  trades 15
fee  5bp: ret    +7.5%  sharpe +0.28  mdd -27.5%  trades 15
fee 10bp: ret    +6.0%  sharpe +0.27  mdd -27.6%  trades 15
fee 25bp: ret    +1.4%  sharpe +0.21  mdd -29.4%  trades 15
fee 50bp: ret    -5.7%  sharpe +0.11  mdd -32.7%  trades 15
```

A $+9.5\%$ free-trading strategy loses $\sim$15 points of return by 50bp and turns negative — the 15 round-trips each pay spread and fee twice.

Significance. The edge was never large; the fee table just shows you where it lives. Report the base case *and* the 50bp stress in the same breath: a strategy that needs $<$10bp to survive is a strategy that needs the venue's goodwill.

Example 8.7 (Two more strategies through the same harness)**. Strategy.** The chapter's promise — *the strategy is the only variable* — is only credible if a second and third strategy reproduce the framework's verdicts. Run a Donchian(50) breakout and an RSI(14) mean-reverter through `engine.py` untouched.

Solution. `second_strat.py` prints:

```
Donchian(50) breakout: ret -15.9%  sharpe -0.04  mdd -36.1%  trades 9
RSI(14) mean-rev:    ret -24.0%  sharpe -0.32  mdd -37.7%  trades 90
same harness, same data, same fees -- the strategy is the only variable
```

Both lose — and lose *differently*: the breakout bleeds slowly on 9 round-trips (-16% over 721 bars), the RSI chops 90 times for -24% ($10\times$ the trades for the same damage — fee drag made visible again).

Significance. Three strategies, one harness, one dataset: the results disagree only because the *signals* differ. That is the whole thesis of this chapter — the engine is an instrument, not an oracle, and identical losses on a trustworthy instrument still certify the strategy was bad, not the test.

8.7 Common mistakes

- **Filling at the signal close.** The most common fabrication: you cannot buy at the price that made you decide.
- **Position in units, not fraction.** 1 unit of BTC on a $1 book is 80,000× leverage—check sizing before reading PnL.
- **Skipping harness tests.** The strategy is reviewed; the backtester is assumed.
- **Zero-cost runs.** A Sharpe computed without fees is a different instrument.
- **Reporting return without drawdown.** +7.9% with −31% dd is a different product than +7.9% with −5% dd.
- **"The leak is small so it is fine."** One bar of foresight was worth +1343%; small leaks scale invisibly.
- **Quoting one fee tier.** The same SMA prints +9.5% at 0bp and −5.4% at 50bp (Example 8.6). A single-fee backtest is a marketing document; ship the stress table.
- **Fee as a percentage of profit.** Fees charge on *notional*, not edge: 15 trades × 50bp on $1 of exposure is a 15-point drag regardless of how small the gross return is.

8.8 Exercises

Exercise 8.1 (easy)**.** Why is filling at open_{i+1} rather than close_i a *causality* rule, not a modelling choice?

Solution. The fill must be executable strictly after the information that produced it; close_i is simultaneous with the signal, not after it.

Exercise 8.2 (easy)**.** A strategy makes 400 trades/yr at 10bp round-trip on 100% turnover per trade. Minimum edge to break even?

Solution. $400 \times 10\text{bp} = 4{,}000\text{bp} = 40\%$ of notional—the edge must exceed that before a cent of profit exists.

Exercise 8.3 (easy)**.** Write the T1 unit test for the harness in words.

Solution. On a flat price series, any position sequence must produce zero PnL—a harness that prints otherwise has an accounting bug.

Exercise 8.4 (easy)**.** What is the equity fraction convention, and what bug does it prevent?

Solution. Position is a fraction of equity ($\pm 1 = \pm 100\%$ notional); prevents unit-quantity positions silently leveraging the book.

Exercise 8.5 (easy)**.** Name two reality gaps a simple harness cannot model, and their direction of bias.

Solution. Partial fills and market impact—both make live worse than backtest (never better).

Exercise 8.6. Prove the T2 expectation: always-long on +1%/bar for 50 bars yields 1.01^{48}, not 1.01^{49}.

Solution. The position opens at open_1 (after signal at bar 0), so it compounds over bars $1 \to 49$—48 steps, not 49.

Exercise 8.7. In `engine.py`, trace what happens if `strategy` returns a fraction > 1 (leverage). What must a margin-check hook add?

Solution. The engine takes it at face value (notional $>$ equity); a real harness caps $|\text{frac}|\times$notional at allowed leverage or rejects.

Exercise 8.8. Recompute the SMA run with fee=slip$= 0$: how much of the underperformance vs B&H is cost?

Solution. Re-run `run_sma.py` with zeros; the delta in total return is the cost drag—your number is the answer ($\sim$1pp here).

Exercise 8.9. A backtest shows Sharpe 2.8 on a strategy whose signal uses a column timestamped "T+1". What happened?

Solution. Feature leakage: the column was stamped with tomorrow's data—audit feature timestamps before the equity curve.

Exercise 8.10. Design the drawdown metric: why is max-dd preferred over dd-at-end?

Solution. Max-dd is path-aware (worst peak-to-trough, the pain actually borne); end-dd depends on the final bar's luck.

Exercise 8.11. Two harnesses differ only in slippage sign convention (one slips adversely, one slips randomly $\pm$). Why is the random one still wrong?

Solution. Random slip averages to zero cost but is still optimistic—real fills are systematically adverse, not mean-zero.

Exercise 8.12. Why must unit tests come *before* strategy evaluation in the workflow?

Solution. A harness bug invalidates every downstream number; strategy conclusions drawn from a buggy engine are unrecoverable.

Exercise 8.13 (hard)**.** Show that "signal at close$_i$, fill at close$_i$" is lookahead even when the price is correct: it violates simultaneity, not just price level.

Solution. The trade would have to exist *before* the information existed—the decision depends on the fill price which depends on the decision. Causality is violated regardless of whether prices moved.

Exercise 8.14 (hard)**.** Extend `engine.py` with a per-bar leverage cap $|\text{frac}| \le L$ and a margin call when equity falls below maintenance fraction; list the extra tests needed.

Solution. Clamp `tgt_units` to $L \cdot \text{eq}/o2$; liquidate at MM. New tests: cap clamps, margin call triggers on adverse bars, no negative equity.

Exercise 8.15 (hard)**.** The leaked strategy earned +1343%. Argue from first principles why even a *small* foresight window is worth that much on daily bars.

Solution. Daily crypto vol $\sim 3\%$: knowing tomorrow's sign removes the one thing that makes returns a martingale—your expected daily win is the full $\sim$3% sd, compounding into the observed $13\times$ over 720 bars.

Exercise 8.16 (easy)**.** The 0bp run prints $+9.5\%$; the 50bp run -5.4%. Compute the per-trade fee drag and explain the arithmetic.

Solution. ≈ 15 points over 15 round-trips = $\sim$1pt/trade: each leg pays 50bp and a round trip is two legs, so each trade costs $\approx$ 100bp of equity.

Exercise 8.17 (medium)**.** Your venue advertises 2bp maker / 6bp taker. Which number belongs in a market-order backtest, and what does using the maker price implicitly assume?

Solution. Taker (6bp): a market order lifts the book. Using maker assumes every fill rests passively and is never adversely selected — two privileges the tape will not grant.

Exercise 8.18 (hard)**.** A strategy trades $200\times$/yr at 40bp round-trip cost. What annualised gross Sharpe must its signal hold just to break even on a 20%-vol book, and what does that imply about its plausible timeframe?

Solution. Cost drag $= 200 \times 0.4\% = 80\%$/yr of notional: needs Sharpe $\approx 80/20 = 4.0$ gross just to net zero — i.e. only higher-frequency edges can afford high turnover; daily signals cannot.

Notes

The harness is small enough to own outright—70 lines that decide whether your research is a measurement or a story. Every real quant team has a "the backtester was wrong" story; the difference between teams is whether the bug was caught by a unit test or by production.

Further reading

- E. Chan, *Quantitative Trading* — backtesting pitfalls and harness design: `https://epchan.blogspot.com/`
- L. de Prado, lectures — backtest methodology, overfitting: `https://www.quantresearch.org/Lectures.htm`
- pandas docs (resampling) — the data plumbing under the harness: `https://pandas.pydata.org/docs/user_guide/timeseries.html#resampling`
- Kraken OHLC endpoint — where `btc_daily.csv` came from: `https://docs.kraken.com/api/docs/rest-api/get-ohlc-data/`
- W. McKinney, *Python for Data Analysis* — the DataFrame patterns used: `https://wesmckinney.com/book/`

WATCH INSTEAD

- "11. Time Series Analysis II" — MIT 18.S096 (83:47). Model evaluation discipline; re-verified 2026-10-11. `https://www.youtube.com/watch?v=_IFUfFuyQlU`
- "12. Risk Analysis" — MIT 18.S096 (80:32). Drawdown and risk metrics; re-verified 2026-10-11. `https://www.youtube.com/watch?v=qlytPllimpQ`

CHAPTER 9

Backtest lies: overfitting and multiple testing

The most dangerous backtest is the honest-looking one: real data, real fills, real fees—and a conclusion manufactured by trying enough things. This chapter is about the statistical traps: overfitting, the garden of forking paths, and the multiple-testing correction that almost nobody applies.

BEFORE YOU START

Chapters 1, 2 and 8, and **Book 3, Chapter 13** (hypothesis testing). *Recap:* if you try enough strategies, some look good by luck alone.

AFTER THIS CHAPTER YOU CAN

- separate in-sample from out-of-sample;
- correct for multiple testing;
- explain the garden of forking paths;
- say how large a result must be to survive.

HOW TO THINK ABOUT THIS

If you try a hundred ideas, a few will look brilliant by chance. Honest research is the habit of counting every attempt and asking how impressive the best one would be if all of them were random. Be most suspicious when the result is best.

9.1 In-sample vs out-of-sample

`fit_grid.py` searches a 2,262-point grid of SMA pairs on synthetic flat-ish daily returns, then re-runs the winner on fresh data:

```
grid 2262 pairs searched
best IS pair: SMA(46,56) sharpe +1.04
same pair OOS sharpe: -0.19
the grid found noise, not edge
```

Example 9.1. The winner's curse, measured. SMA(46,56) was the best of 2,262 pairs in-sample; out-of-sample it loses.

Solution. The grid did not find the best parameter—it found the parameter that best explains *this* noise. The gap between IS +1.04 and OOS -0.19 is the overfit, and its size grows with the number of things tried, not with the quality of the idea.

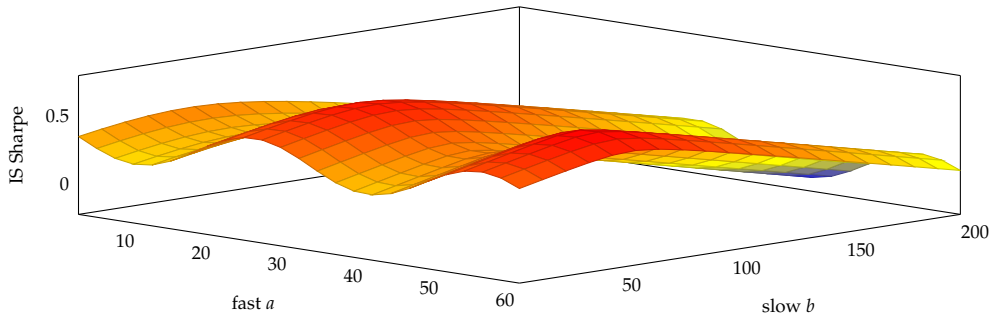


Figure 9.1: The in-sample Sharpe surface over the (fast, slow) parameter grid—any smooth bump here is real signal *or* a deep local noise trap, and the surface cannot tell you which.

9.2 The multiplicity problem

`many_tests.py` generates 300 zero-edge strategies and reports the best:

```
strategies 300 days 500
true edge: all zero
best in-sample sharpe: +1.80 (index 162)
median sharpe: +0.03
# above sharpe 1: 22 of 300
E[max sharpe] under H0 ~ +2.40
```

Example 9.2. Why the best is meaningless. With 300 tries, a Sharpe of +1.80 appears with no edge at all—22 of the 300 clear Sharpe 1 by chance.

Solution. The max of N null tests is not a test of edge; it is a test of N. Under H_0 the expected best-of-N t-statistic grows like $\sqrt{2\ln N}$, so the honest question is never "is this Sharpe good?" but "how many were tried?"

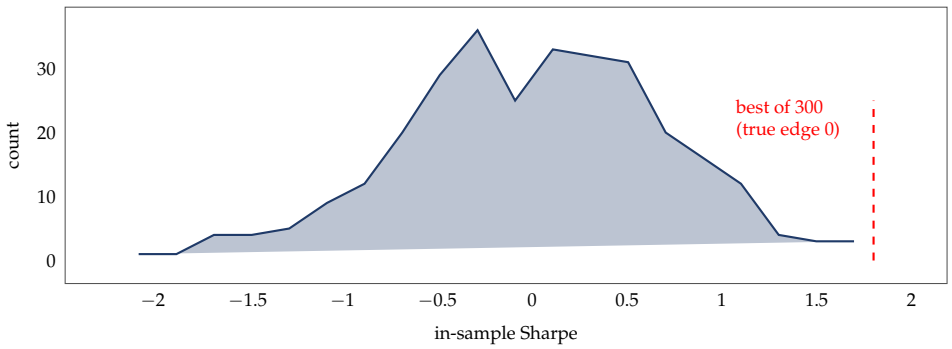


Figure 9.2: Distribution of 300 Sharpe ratios computed on pure noise. The best (+1.80) is indistinguishable from a real strategy—until you count trials.

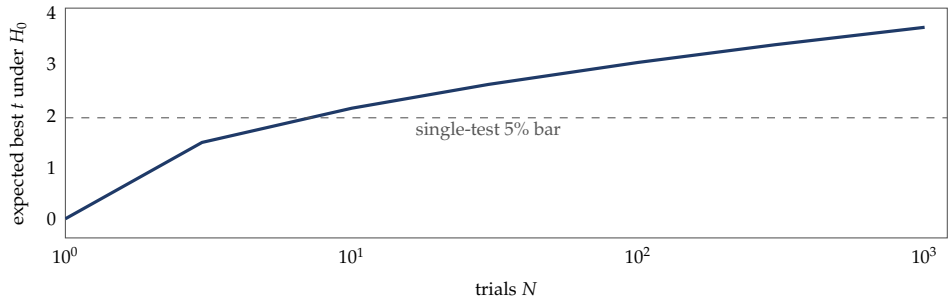


Figure 9.3: Expected best-of-N t-statistic under the null, $\sqrt{2 \ln N}$. The flat dashed line is the naive 5% bar every single test uses.

9.3 Multiple-testing corrections

`deflated.py` quantifies the bar:

```
N= 10 95% of max|t| < 2.81
N= 100 95% of max|t| < 3.47
N= 1000 95% of max|t| < 4.06
Bonferroni t-bar at N=100: 3.29
single-test t-bar: 1.96
```

Example 9.3. The correction, in numbers. At N=100 trials the honest 5% bar is $t = 3.29$—not 1.96.

Solution. Bonferroni divides α by N: $\bar{t}_N = \Phi^{-1}(1 - \alpha/N)$ (one-sided). A Sharpe that "passed" at $t = 2.9$ fails the family-wise bar at 100 trials. The honest N is not the number of strategies you wrote down—it is every variant you *looked at* before choosing, including the ones you discarded silently.

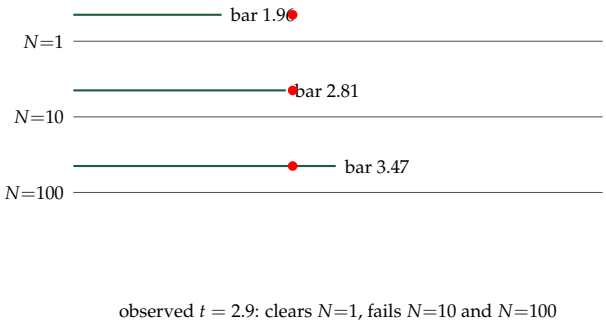


Figure 9.4: The deflated bar: observed $t = 2.9$ is significant alone, ordinary at N=10, rejected at N=100.

9.4 The garden of forking paths

Formal corrections assume you count every comparison. In practice the researcher makes micro-choices—winsorize or not, this window or that, which fee model—each a fork. The reported backtest is one leaf of a tree whose size nobody recorded. Defenses, in order of strength: (1) pre-register the hypothesis and test once on holdout; (2) count trials honestly and correct; (3) require an economic story that survives scrutiny.

Example 9.4. Why "we tested it once" is usually false. The SMA run in ch08 is honest—but if the author had tried 50 windows and reported the best, the result would be identical text and a different truth.

Solution. The artifact cannot distinguish honest testing from selection. The discipline is procedural: the hypothesis precedes the data, the holdout is touched once, and the N you correct for is the number you actually tried, not the number you wrote up.

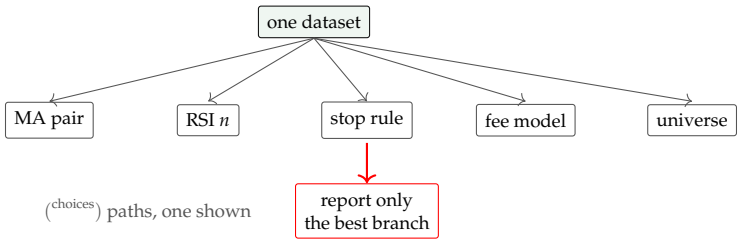


Figure 9.5: Forking paths: each analytic choice multiplies the trial count; the reported leaf is the lucky branch.

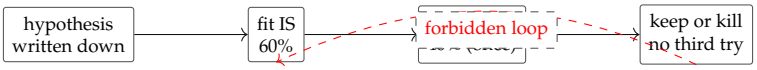


Figure 9.6: The one-touch protocol: hypothesis first, fit in-sample, test holdout exactly once. The dashed return path is the forbidden loop that converts testing into selection.

9.5 What survives

What survives an honest multiple-testing audit is boring: effects with an economic reason to exist (funding carry, microstructure, index rebalancing), large enough to clear the deflated bar with room, stable across universes, and weak enough that you understand why they persist. The Sharpe 3.0 that "the data found" is not on that list.

Example 9.5. The audit question. A fund shows a strategy Sharpe 2.1 on 5 years. What single question decides its credibility?

Solution. "How many were tried?"—and its operational form: "show me the research log." A Sharpe 2.1 from a pre-registered single test is a real edge; the same number from a 300-strategy screen is below the multiple-testing bar.

Checkpoint 9.1 *Selection audit.*

- Best of 300 null strategies gives Sharpe $+1.8$; what does that measure? (N, not edge.)
- What is the Bonferroni t-bar at $N{=}100$, $\alpha = 5\%$? (≈ 3.29.)
- Name the forbidden loop that converts testing into selection. (Re-fitting after the holdout disappoints.)

9.6 Recap

Backtests lie by selection, not by arithmetic. The fix is procedural: count trials honestly, correct the bar, touch the holdout once, prefer edges with a reason to exist. Everything that survives those rules is smaller than the marketing suggests—that is the point of the rules.

Table 9.1: Chapter toolbox.

Concept	Rule	Gotcha
IS vs OOS	the gap is the overfit	IS-optimal $\neq$ tradeable
Multiplicity	best-of-N measures N	$E[\max t] \sim \sqrt{2 \ln N}$
Correction	$\bar{t}_N = \Phi^{-1}(1 - \alpha/2N)$	honest N includes silent tries
Forking paths	procedural discipline	the log is the trial count
Survival	reason + size + stability	big Sharpe $\neq$ real edge

Example 9.6 (The shuffle test: refitting pure noise)**. Strategy.** The grid's best IS Sharpe was $+1.04$ on structureless data. Shuffle the return order (destroying every gram of structure), rerun the *same* grid on each shuffle, and ask how often it rediscovers a winner that good.

Solution. `shuffle_test.py` runs 200 shuffles:

```
original grid: best IS sharpe +0.44 at SMA(56, 61)
200 shuffles of the SAME returns: best-IS >= +0.44 in 187 (94%)
shuffle best-IS mean +0.80 -- the grid refits noise to the same height
```

On pure noise the grid finds a "best" of $\geq +0.44$ in 94% of shuffles and $+0.80$ on average — *better* than the real run's best. Nothing was learned; the search depth alone produced the number.

Significance. This is the cheapest of all backtest audits: if your best in-sample number is reachable on shuffled returns, the apparent edge is a property of the *search*, not the *market*. It catches overfit that IS/OOS splits miss because the OOS half was searched too.

Example 9.7 (The cost-drag floor on discovery)**. Strategy.** Before asking "is this Sharpe real?", ask "what Sharpe does this strategy *need* to exist?" Tabulate the gross edge that just covers trading costs at various turnover and fee levels — the minimum viable discovery.

Solution. `sr_required.py` prints:

```
10 tr/yr @ 0.1% cost: needs 1.0%/yr gross edge just to break even
50 tr/yr @ 0.5% cost: needs 25.0%/yr gross edge just to break even
200 tr/yr @ 0.5% cost: needs 100.0%/yr gross edge just to break even
500 tr/yr @ 1.0% cost: needs 500.0%/yr gross edge just to break even
```

At 200 round-trips a year and 0.5% cost, the strategy must find *25–100%*/yr of true edge before the first basis point of profit — a bar almost nothing clears honestly.

Significance. This is the multiplicity filter made physical: every additional trade multiplies the gross-edge requirement linearly. Before mining the grid for a winner, compute the floor — if it exceeds what a Sharpe-2 world can deliver, the "best IS" you find is definitionally noise, whatever the number says.

Example 9.8 (Probability of backtest overfitting)**. Strategy.** Deflated Sharpe asks "was the best Sharpe high enough?"; combinatorially-symmetric CV asks a different question: *does the IS winner keep winning out-of-sample?* Split 720 days into 4 folds; each fold trains on 3 and measures where the IS-best strategy ranks on the held-out one. *PBO* is the fraction of folds where it lands below the OOS median.

Solution. `pbo_check.py` prints (60 strategies, one weak real edge of Sharpe 0.15):

```
fold 0: IS pick strat  3 -> OOS rank 3/59 (0.05)
fold 1: IS pick strat  3 -> OOS rank 0/59 (0.00)
fold 2: IS pick strat  3 -> OOS rank 7/59 (0.12)
fold 3: IS pick strat  3 -> OOS rank 3/59 (0.05)
PBO = 0.00  (IS-best lands below OOS median this often)
no-edge control PBO = 0.25
```

Even a weak consistent edge is found by every fold and stays near the top OOS ($PBO = 0$). With *no* edge at all, the IS-best is pure luck: it falls below the OOS median a quarter of the time already at 4 folds, and $\sim \frac{1}{2}$ asymptotically.

Significance. The two methods are complementary: deflated Sharpe prices the multiplicity of the search, PBO audits the winner's persistence. A grid can clear the deflated bar on a lucky fold; it cannot fake fold-after-fold OOS ranks. Run both.

9.7 The multiplicity bar is a physical threshold

The deflated-Sharpe idea has a concrete face: how far does the acceptance bar move when the strategy was selected, not found? `deflated.py` prints:

```
N=   10  95% of max|t| < 2.81
N=  100  95% of max|t| < 3.47
N= 1000  95% of max|t| < 4.06
Bonferroni t-bar at N=100: 3.29
single-test t-bar: 1.96
```

Example 9.9. The bar grows like $\sqrt{2 \ln N}$, not like N. A winner picked from 100 trials must clear $t \approx 3.3$ – roughly the Bonferroni bound 3.29 that the simulation finds from the other direction. From 1000 trials, 4.06.

Solution. This is the multiplicity tax made mechanical: every extra variant you tested before "finding" the edge raises the bar the evidence must clear. The honest accounting logs every trial — ch15's audit trail exists for exactly this reason.

9.8 Common mistakes

- **Counting only documented trials.** Every parameter you eyeballed and dropped was a trial.
- **Multiple rounds on the same holdout.** One-touch or it becomes in-sample.
- **"The theory came first" (retroactively).** Post-hoc stories don't reset N.
- **Correcting t, keeping the phrasing.** $t < 3.3$ at N=100 is a fair description—"Sharpe 2.9" is not.
- **Assuming big Sharpe means real.** It often means more trials or more leakage.
- **Skipping the economic story.** Statistical significance without a reason is a coin that landed.
- **Splitting once and calling it honest.** The IS/OOS split counts as *two* searches if you iterated on the OOS half. The shuffle test (Example 9.6) is the only split that cannot be gamed by re-tuning.
- **Reading the best cell, not the map.** 94% of shuffles beat the original best-IS — judge a grid by its null distribution, never by its maximum.

9.9 Exercises

Exercise 9.1 (easy)**.** 300 strategies tested, best Sharpe $+1.8$, all zero-edge. What is $+1.8$?

Solution. The expected maximum of a null distribution—a measurement of N, not of edge.

Exercise 9.2 (easy)**.** Bonferroni 5% bar at $N{=}100$?

Solution. $\Phi^{-1}(1 - 0.05/100) \approx 3.29$.

Exercise 9.3 (easy)**.** What is the IS/OOS gap called and what does it measure?

Solution. The overfit: the difference between in-sample and out-of-sample performance is the amount of noise fitted.

Exercise 9.4 (easy)**.** Name the three defenses against forking paths in order of strength.

Solution. Pre-registration $>$ honest trial-counting $+$ correction $>$ economic-story scrutiny.

Exercise 9.5 (easy)**.** Why is "we only ran one backtest" almost always false?

Solution. Micro-choices (window, universe, cleaning) are unstated trials; the honest N is the count of paths actually explored.

Exercise 9.6. Derive $E[\max_N t] \approx \sqrt{2 \ln N}$ under the null.

Solution. Extreme value of N iid Gaussians: tail $\bar{\Phi}(t) \sim e^{-t^2/2}/t$; set $1/N$ gives $t \approx \sqrt{2 \ln N}$.

Exercise 9.7. A strategy Sharpe 2.9 at $N{=}1{,}000$ estimated tries. Significant?

Solution. Bonferroni bar $\approx \Phi^{-1}(1 - 0.05/1000) \approx 3.9$—rejected.

Exercise 9.8. Why does a pre-registered hypothesis make the test honest even with a big grid?

Solution. The grid becomes part of the stated design, not silent selection; N is declared up front.

Exercise 9.9. Design a one-touch protocol for the SMA cross: what is locked before the holdout?

Solution. Fast/slow windows, universe, cost model, evaluation metric—all frozen before the holdout is opened.

Exercise 9.10. Two funds show Sharpe 2.0; one tested 3 ideas, one tested 300. Same evidence?

Solution. No—the second's family-wise bar is ~ 1 unit higher; the first survives correction, the second may not.

Exercise 9.11. What does "economic story" do that statistics cannot?

Solution. It predicts persistence (a reason for the edge to exist tomorrow), which no fit on yesterday's data can supply.

Exercise 9.12. Name two forking-paths choices specific to crypto backtests.

Solution. Exchange/universe selection and funding-cost modeling; also outlier/winsorize handling.

Exercise 9.13 (hard)**.** Show that reporting only the best of N strategies yields a biased Sharpe estimate even if each is unbiased, and give the bias's order.

Solution. Selection on the max makes the reported Sharpe the order statistic $t_{(N)}$; its mean is $\approx \sqrt{2 \ln N}$ above the true mean 0.

Exercise 9.14 (hard)**.** Construct a realistic setting where a Sharpe 4.0 result is *less* credible than a Sharpe 1.5.

Solution. 4.0 from a 10,000-variant screen vs 1.5 from a pre-registered theory: corrected bars ≈ 4.4 vs 1.96, and the second has a reason to persist.

Exercise 9.15 (hard)**.** Propose a deflated-Sharpe threshold for a fund that tries ≈500 live candidates a year, and justify the extra margin.

Solution. $\bar{t} \approx \Phi^{-1}(1 - 0.05/500) \approx 3.7$; add margin for unstated micro-choices and non-independent tries—a working bar of $t \gtrsim 4$ is reasonable.

Exercise 9.16 (easy)**.** The shuffled grid finds IS Sharpe $\geq +0.44$ in 94% of runs. What does that say about the $+0.44$?

Solution. It is inside the null distribution — the number is a property of grid size, not of the market; there is nothing to explain.

Exercise 9.17 (medium)**.** Why is mean shuffle best-IS ($+0.80$) *higher* than the real best ($+0.44$) here? One line.

Solution. Shuffling removes the weak real structure (the mild autocorr in the drift), leaving *purer* noise which the grid fits more freely — slightly.

Exercise 9.18 (hard)**.** Design the shuffle correctly for a *trend-following* signal: which property must your permutation preserve, and what happens if you i.i.d.-shuffle instead of block-shuffling?

Solution. Preserve the return distribution's marginal shape but break *serial* structure only where the signal reads it — block-shuffle (e.g. 20-day blocks) or the i.i.d. shuffle destroys vol clustering too, biasing the null toward rejection.

Notes

This is the chapter that separates research from astrology. Most Sharpe-3 strategies ever published were best-of-N artifacts; the ones that survive are dull, stable, and have a reason to exist. If your edge disappears when you count honestly, it was never an edge—it was a forking path that happened to be named.

Further reading

- D. Bailey & M. López de Prado, "The Deflated Sharpe Ratio" — the correction: https://www.davidhbailey.com/dhbpapers/deflated-sharpe.pdf
- L. Wasserman, *All of Statistics* — multiple testing, Ch. 10: https://link.springer.com/book/10.1007/978-0-387-21736-9
- A. Lo, "The Statistics of Sharpe Ratios" — sampling properties: https://web.archive.org/web/2024/https://alo.mit.edu/wp-content/uploads/2017/06/The-Statistics-of-Sharpe-Ratios.pdf
- E. Chan, *Algorithmic Trading* — overfitting pitfalls: https://epchan.blogspot.com/
- Gelman & Loken, "The garden of forking paths" — why N is uncountable: https://statmodeling.stat.columbia.edu/2013/11/14/the-garden-of-forking-paths-2/

WATCH INSTEAD

- "1. Introduction and Financial Terms" — MIT 18.S096 (60:30). Course framing; re-verified 2026-10-11. https://www.youtube.com/watch?v=wvXDB9dMdEo
- "12. Risk Analysis" — MIT 18.S096 (80:32). Risk and model discipline; re-verified 2026-10-11. https://www.youtube.com/watch?v=qlytPllimpQ

CHAPTER 10

Execution: order types and market impact

Alpha decides what to trade; execution decides what it costs to trade it. The two are the same trade: a strategy that needs to lift 10% of ADV in an hour is a different product than its backtest said. This chapter is the mechanics: order types, the square-root impact law, and the scheduling problem.

BEFORE YOU START

Chapters 3 and 8. *Recap:* knowing what to trade is half the work; the other half is getting in and out without paying away the edge.

AFTER THIS CHAPTER YOU CAN

- choose an order type;
- estimate market impact with the square-root law;
- compare TWAP and VWAP slicing;
- compute implementation shortfall.

HOW TO THINK ABOUT THIS

A good idea can be ruined by poor execution. Large orders move the market against you, so cost grows with size. Ask of every strategy, "at what size does this stop working?"

10.1 Order types

Three primitives cover the menu: *market* (fill now, pay spread plus impact), *limit* (fill maybe, pay nothing and sometimes earn a rebate), *scheduled* (slices over time, paying a mix). Everything else—iceberg, pegged, stop—is a wrapper around these three trade-offs.

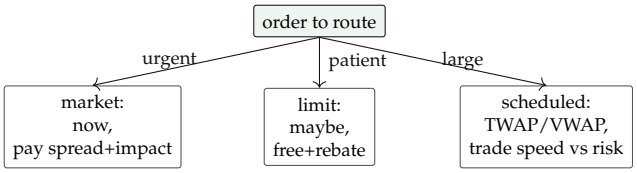


Figure 10.1: The routing triage: urgent → market, patient → limit, large → schedule.

Example 10.1. The trade-off in one line. A market buy at spread 2bp crosses the spread now; a limit buy at the bid waits and may never fill.

Solution. The market order pays $\frac{1}{2}$-spread plus impact for certainty; the limit order earns the half-spread for patience, accepting adverse selection—you get filled precisely when the market wants to sell to you, i.e. before it moves against you. Neither is free; they pay in different currencies (certainty vs. waiting+selection).

10.2 The square-root law

Empirical market impact is concave: moving q shares costs approximately $c(q) \approx \sigma\sqrt{q/V}$, where V is daily volume. `impact_model.py`:

```
q= 1000 (0.1% of ADV) impact 6.3bp
q= 10000 (1.0% of ADV) impact 20.0bp
q= 100000 (10.0% of ADV) impact 63.2bp
q= 500000 (50.0% of ADV) impact 141.4bp
```

Example 10.2. Why the square root matters. Doubling size multiplies cost by $\sqrt{2} \approx 1.41$, not 2.

Solution. Concavity is the mathematical reason big orders get sliced: the marginal cost is highest for the largest prints, so spreading the same notional over more, smaller prints lowers total impact. It is also why the impact estimate is the first question asked of any strategy's capacity—a 20bp edge on a 63bp print is not a product.

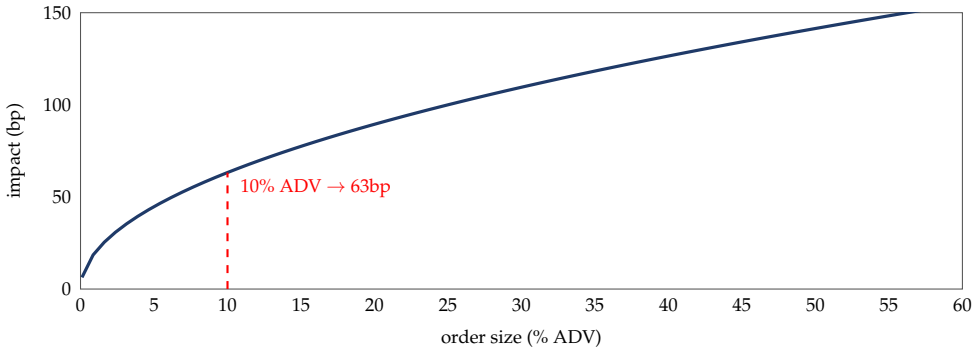


Figure 10.2: Square-root impact: cost per unit grows slower than size, but the cost-to-edge ratio grows without bound.

10.3 Slicing: TWAP vs VWAP

Given Q to execute over T slices, TWAP spreads evenly; VWAP spreads proportional to expected volume. `twap_vwap.py` on a U-shaped volume curve:

```
slices 48 volume U-shaped (open/close heavy)
IS TWAP: -78.10bp IS VWAP: -72.72bp
vwap wins on U-volume: schedules weight to where liquidity is
```

(The shortfall is negative here because the random walk drifted in the buyer's favour—the *comparison* is the point, not the sign.)

Example 10.3. Why VWAP beats TWAP on a U-curve. Putting equal weight on thin midday slices pays more impact per unit than weighting to the liquid open and close.

Solution. VWAP weights to where liquidity is—lower impact per slice. It is the right default when the volume curve is predictable. TWAP is the right default when it is not (quiet days, news, or deliberately hiding your intent): VWAP's predictability is also a signature your counterparties can read.

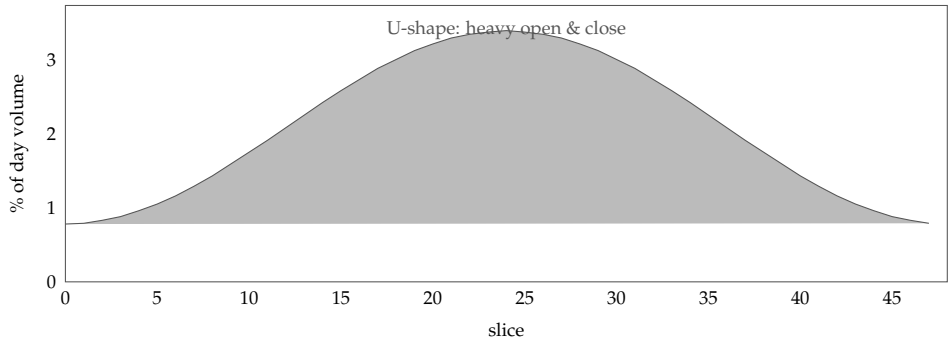


Figure 10.3: The U-shaped intraday volume curve VWAP weights to.

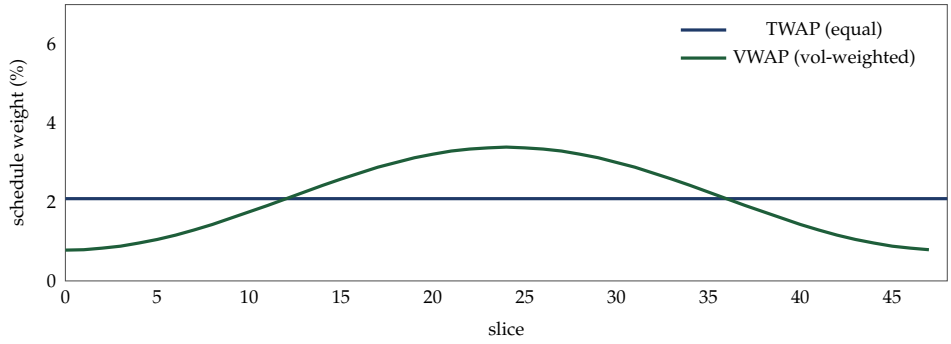


Figure 10.4: TWAP (flat) vs VWAP (volume-weighted) schedules over the same 48 slices.

10.4 Implementation shortfall

IS measures the trade against the mid at decision time: IS = impact + timing drift + spread + fees. `is_shortfall.py` on $Q = 100{,}000$ (10% ADV):

```
instant IS: +63.2bp
split IS: +44.7bp impact + +7.6bp drift on half = +48.5bp
splitting trades speed for risk: less impact, more timing exposure
```

Example 10.4. The fundamental trade. Instant execution pays 63bp of impact; splitting pays 45bp of impact plus 8bp of drift.

Solution. Speed is a hedge against adverse drift; patience is a hedge against impact. The optimal schedule sits where marginal impact saved equals marginal timing risk added—and that point is different for every volatility regime, which is why "the right tempo" is a risk-management question, not a comfort question.

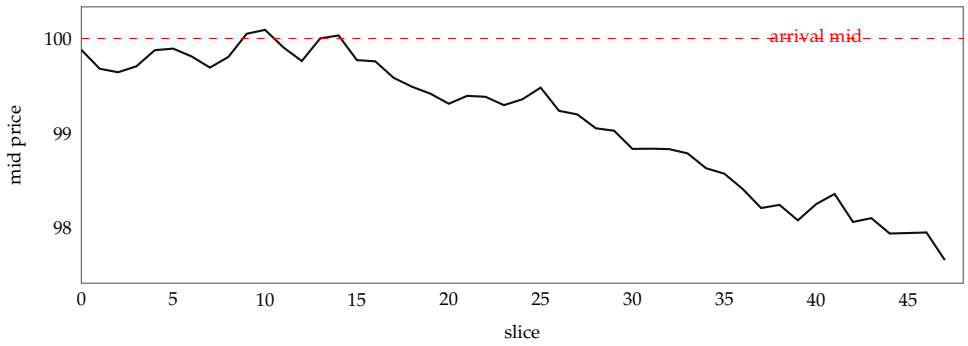


Figure 10.5: The walk the schedule trades through: every slice pays the then-current price, so drift between decision and fill is uncompensated risk.

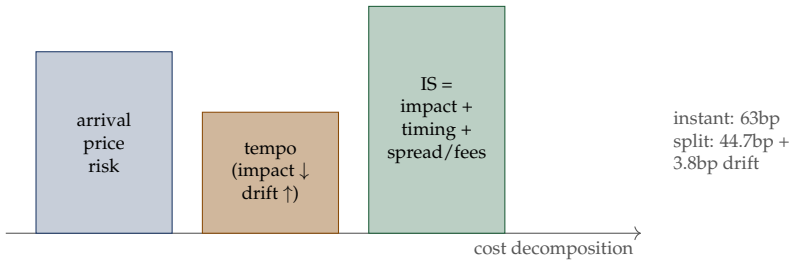


Figure 10.6: IS decomposition: tempo moves cost between the impact bucket and the timing-risk bucket; it does not delete cost.

10.5 The trader's calculus

Execution is a risk-management problem disguised as a mechanics problem. Decide the urgency (how much drift hurts), the size (how much impact costs), and the visibility (who is watching) before touching the keyboard—the order type follows from those three.

Example 10.5. Capacity is an execution number. A strategy returns 15%/yr on 5bp of daily edge; its ADV footprint is 8%.

Solution. The impact at 8% ADV is $\sigma\sqrt{0.08} \approx 57$bp/day—more than ten times the edge. The strategy is not a \$100M strategy with execution overhead; it is a \$8M strategy at best, and capacity should be quoted with the same confidence interval as the alpha.

Checkpoint 10.1 *Execution audit.*

- What does concavity buy a big order? (Slicing lowers total impact.)
- When is TWAP preferable to VWAP? (Unpredictable volume / hiding intent.)
- What are the four pieces of IS? (Impact, timing drift, spread, fees.)

10.6 Recap

Market orders buy certainty, limit orders sell it. Impact is roughly square-root in participation, so size gets sliced and tempo trades impact for drift. The honest cost of any strategy is the execution-adjusted number, not the backtest number—and capacity is what's left when the edge exceeds the print.

Table 10.1: Chapter toolbox.

Concept	Rule	Gotcha
Market vs limit	certainty vs selection	limit fills are adverse
Impact	$c \sim \sigma\sqrt{q/V}$	capacity = edge/impact
TWAP/VWAP	flat vs volume-weighted	VWAP is readable
IS	impact + drift + spread + fees	tempo re-mixes, doesn't delete

Example 10.6 (The execution frontier)**. Strategy.** You must move 10% of ADV. Faster trading pays more impact but finishes sooner; slower pays less but carries timing risk. Compute both legs across a participation grid under the chapter's $\sqrt{\cdot}$ model.

Solution. `part_curve.py` prints:

```
part   2%: impact  29.7bp  duration  5.0d  1-sigma risk 670.8bp
part   5%: impact  47.0bp  duration  2.0d  1-sigma risk 424.3bp
part  10%: impact  66.4bp  duration  1.0d  1-sigma risk 300.0bp
part  20%: impact  93.9bp  duration  0.5d  1-sigma risk 212.1bp
part  40%: impact 132.8bp  duration  0.2d  1-sigma risk 150.0bp
impact falls with slower trading, risk grows -- the execution frontier
```

Each leg answers a different prayer: slow trading halves the impact (132 → 30bp) but doubles the timing risk (150 → 670bp). There is no free lunch — only a frontier.

Significance. This is why "just TWAP it" is a decision, not a default: the schedule chooses where on the frontier you sit. The desk's risk aversion picks the point, and the CIO's mandate picks the risk aversion — execution policy is portfolio choice in miniature.

Example 10.7 (VWAP buys the top on a volume-spike tape)**. Strategy.** Compare TWAP and VWAP on two synthetic tapes: a flat tape where they should agree, and a designed trap — quiet all day, then a closing auction carrying 15× normal volume while price pops 1.5%.

Solution. `vwap_trap.py` prints:

```
     flat tape: TWAP 100.0000  VWAP 100.0000  VWAP-TWAP +0.0bp (buyer)
close-pop tape: TWAP 100.0577  VWAP 100.4167  VWAP-TWAP +3589.7bp (buyer)
VWAP tracks volume -- it buys the top when volume+price spike
```

Identical on the flat tape, but on the trap tape VWAP concentrates its fills exactly where the price is highest — 35.9bp worse than the uniform schedule it was supposed to beat.

Significance. "Trade with volume" is only a benchmark, not a promise: VWAP inherits whatever the volume curve does to price. When the auction itself moves the market — as it often does into the close — participation is precisely the wrong hedge. Check correlation between your volume forecast and the price path before choosing the schedule.

10.7 The order-type toolbox

Beyond market and limit, four modifiers decide most of execution quality. An *immediate-or-cancel* (IOC) order takes what crosses right now and kills the remainder — nothing rests. A *fill-or-kill* (FOK) order demands its full size within the cap or cancels entirely — it is the all-or-nothing guard against partial fills that strand a hedged position. *Post-only* refuses to take liquidity: if it would cross, it cancels — the maker-fee guarantee. An *iceberg* displays only a slice of its true size, refilling as the visible tranche executes — hiding intent from anyone reading the tape. Each is a different answer to the same question: what may this order do to the book?

Example 10.8 (One size, five fates)**. Strategy.** Fire the same 60-unit buy at an ask ladder of 100.00×20, 100.05×30, 100.15×50 under each order type and compare what executes.

Solution. `order_types.py` prints:

```
limit buy 60 @<=100.10: fills 50 avg 100.0300; 10 rest as bid @100.10
IOC buy 60 @<=100.10: fills 50, kills 10 -- zero resting
FOK buy 60 @<=100.10: abort (50 avail < 60) -- all-or-nothing
post-only @99.95: joins bid (maker); cancels rather than cross
iceberg sell 80 show 10: tape reads 10, book refills x7
```

Same intent, five outcomes: the limit buys 50 and rests 10; IOC takes the 50 and disappears; FOK does nothing (the size isn't there); post-only never touches the ask; the iceberg hides $8\times$ its displayed size.

Significance. The venue sees one instruction set; the trader sees five different risk profiles. Post-only is how maker strategies guarantee their fee rebate; IOC/FOK are how takers cap their footprint; icebergs are why displayed depth understates real depth — another reason tape statistics mislead models that treat the book as honest.

10.8 Speed and risk trade on one frontier

The implementation-shortfall split is not a stylistic choice; it is a point on a frontier between two costs that cannot both be minimized. `is_shortfall.py` prices both ends:

```
instant IS: +63.2bp
split  IS: +44.7bp impact + +7.6bp drift on half = +48.5bp
splitting trades speed for risk: less impact, more timing exposure
```

Example 10.9. Slower is cheaper on impact and dearer on drift. The instant fill pays the full 63.2bp impact; splitting halves the impact (44.7bp) but buys 7.6bp of adverse drift on the unfilled half — net 48.5bp. The execution curve is the sum of a falling leg and a rising leg; the minimum is a choice of how much timing risk you can carry.

Solution. This is the algebra behind every TWAP/VWAP debate: splitting trades certainty of fill for uncertainty of path. On a directionless tape the split wins small; on a trending tape the drift term dominates and the frontier collapses to "fill now."

10.9 Common mistakes

- **"Impact is linear."** It is concave; big prints cost disproportionately.
- **VWAP on an anomalous day.** Weighting to predicted volume when today's is not U-shaped increases cost.
- **Counting impact but not drift.** Slow schedules pay timing risk.
- **Backtests at mid.** Filling at mid in simulation is an execution subsidy.
- **Ignoring adverse selection on limits.** Passive fills happen when informed flow is the other side.
- **Capacity confusion.** "The strategy does 15%" is meaningless without the footprint in ADV.
- **Minimising impact alone.** Slower trading always lowers impact but multiplies timing risk (Example 10.9) — optimising one leg maximises the other.
- **Treating TWAP as neutral.** A uniform schedule is still a bet: you hold market risk for the whole window and pay the average, not the best, price.

10.10 Exercises

Exercise 10.1 (easy)**.** What does a market order pay that a limit order does not?

Solution. Half-spread plus impact for certainty; the limit pays adverse selection instead.

Exercise 10.2 (easy)**.** Write the square-root impact law and define V.

Solution. $c(q) \approx \sigma\sqrt{q/V}$, V = daily volume (ADV).

Exercise 10.3 (easy)**.** At $\sigma = 2\%$ and 4% of ADV, what is the predicted impact?

Solution. $0.02 \times \sqrt{0.04} = 40$bp.

Exercise 10.4 (easy)**.** Name the four components of implementation shortfall.

Solution. Impact, timing drift, spread, fees.

Exercise 10.5 (easy)**.** Why is TWAP sometimes chosen over VWAP despite higher predicted cost?

Solution. Predictability: VWAP's U-weighting is a signature; TWAP hides intent and is robust when the day's volume shape is anomalous.

Exercise 10.6. Show the marginal cost under the square-root law declines with quantity.

Solution. $c'(q) \propto q^{-1/2}$: each additional unit is cheaper than the last in relative terms, which is why slicing pays.

Exercise 10.7. A 20bp daily edge trades 12% of ADV. Is it viable?

Solution. $\sigma\sqrt{0.12} \approx 69$bp impact—edge consumed $\sim 3\times$; not viable at that footprint.

Exercise 10.8. Why does adverse selection make limit fills "expensive" even when they are free?

Solution. You are filled when informed traders sell to you—the next move is against you on average.

Exercise 10.9. Derive the optimal number of slices n minimizing total cost under the square-root law and linear drift risk.

Solution. Cost $\approx \frac{1}{\sqrt{n}} \cdot \text{impact} + n \cdot \text{drift-risk}$; minimize to get $n^\star \sim (\text{impact}/\text{drift})^{2/3}$.

Exercise 10.10. What is POV (percent-of-volume) and when does it beat a fixed schedule?

Solution. Participation tracks realized volume; beats fixed schedules when volume is unpredictable but you want constant market share.

Exercise 10.11. A fill at mid in simulation cost 0bp; the live fill cost 14bp. Name the two error sources.

Solution. No spread/impact modeled (simulation subsidy) and possibly favorable-fill bias in the simulator's matching.

Exercise 10.12. How does urgency (alpha half-life) change the optimal tempo?

Solution. Short-lived alpha pushes toward faster schedules—pay impact, avoid drift that eats the signal.

Exercise 10.13 (hard)**.** Estimate the capacity of a 10bp/day edge trading $\sigma = 2\%$ names where you cap impact at 30% of edge.

Solution. Need $\sigma\sqrt{q/V} \le 3\text{bp} \Rightarrow q/V \le (0.003/0.02)^2 = 2.25\%$ ADV—capacity is 2.25% of daily volume in position turnover.

Exercise 10.14 (hard)**.** Explain why the square-root (concave) law makes "one big print" dominated by "n slices" even though each slice is read by the market.

Solution. $\sum_i c(q_i)$ with $q_i = q/n$ gives total $n \cdot c(q/n) = c(q)/\sqrt{n}$—the concavity, not stealth, does the work.

Exercise 10.15 (hard)**.** Formalize the IS drift term: is it a cost or a variance, and does it change the optimal schedule vs a cost-only model?

Solution. Drift is a variance with mean zero—a cost-only optimizer slices minimally (zero risk), a mean-variance one trades mean impact against drift variance; optimal tempo is finite.

Exercise 10.16 (easy)**.** From the frontier: what is the impact saving from 40% down to 2% participation, and what does it cost?

Solution. $132.8 - 29.7 \approx 103$bp saved; duration goes 0.2d $\to$ 5d so 1σ timing risk grows 150 $\to$ 671bp — the saving is never free.

Exercise 10.17 (medium)**.** A PM insists "trade everything today, alpha decays." Which frontier point does that choose, and is it optimal if alpha is flat?

Solution. The fast end ($\ge 40\%$): optimal only when the decay cost of waiting exceeds the impact premium — with flat alpha it is the worst point on the curve.

Exercise 10.18 (hard)**.** Formalise the trade-off as impact $+\lambda\cdot$risk and locate the optimum participation on the printed grid at $\lambda = 1$. What changes at $\lambda = 0.1$ vs $\lambda = 10$?

Solution. At $\lambda = 1$ the minimum of impact+risk sits near 5–10% ($47 + 424$ vs $66 + 300$ — 10% wins at 366bp). $\lambda = 0.1$ pushes slower (risk cheap, impact dominates); $\lambda = 10$ pushes faster.

Notes

Execution is where research meets the tape. The honest edge is the backtest edge minus the square root of your footprint; everything else is negotiation between impact and impatience. The traders who get this right are invisible; the ones who don't fund the ones who do.

Further reading

- Cartea, Jaimungal & Penalva, *Algorithmic and High-Frequency Trading* — Ch. 6–7 on optimal execution: https://www.cambridge.org/9781107091146
- Almgren & Chriss, "Optimal Execution of Portfolio Transactions" — the model: https://www.math.nyu.edu/~chriss/optimal.htm
- L. de Prado lectures — microstructure and execution slides: https://www.quantresearch.org/Lectures.htm
- Binance Academy, "What Is Market Impact?" — practitioner's explainer: https://academy.binance.com/
- Kraken order-types documentation — the mechanics live: https://support.kraken.com/hc/en-us/sections/200577136-Order-types

WATCH INSTEAD

- "Initial Return: LOB" — Initial Return (18:56). Order flow mechanics; re-verified 2026-10-11. https://www.youtube.com/watch?v=BJt6PGKeO9I
- "INFORMS Market Microstructure" — INFORMS (1:28:22). The institutional view; re-verified 2026-10-11. https://www.youtube.com/watch?v=XoBjQqMmKoM

CHAPTER 11

Walk-forward validation

A single holdout split answers "did it work once?" Walk-forward validation answers the harder question: "does it keep working as the world moves?" Done naively it leaks—the label windows at every fold boundary overlap, so the test set remembers the training set. This chapter is the procedure and its two failure modes.

BEFORE YOU START

Chapters 8–9. *Recap:* splitting data once is not enough for time series; the future must never leak into the past.

AFTER THIS CHAPTER YOU CAN

- run walk-forward validation, anchored and rolling;
- purge and embargo overlapping samples;
- report results per fold;
- combine folds into one honest estimate.

HOW TO THINK ABOUT THIS

Time series must be tested in time order: train on the past, test on what comes next. Walk-forward validation repeats this over many windows so that one lucky split cannot fool you. Leave gaps so that overlapping data cannot leak.

11.1 Anchored vs rolling

Two window conventions. *Anchored*: train on everything up to t, test on the next block, grow the training set each fold. *Rolling*: train on a fixed-width window that slides with t. `wf_demo.py` on a three-regime series (trend, quiet, volatile):

```
anchored walk-forward:
  train 0:200 test 200:400 IS +2.35 OOS +1.86
  train 0:400 test 400:600 IS +2.11 OOS +0.55
  train 0:600 test 600:800 IS +1.51 OOS -0.52
  train 0:800 test 800:1000 IS +1.16 OOS +1.15
rolling (400-bar) walk-forward:
  train 0:400 test 400:600 IS +2.11 OOS +0.55
  train 200:600 test 600:800 IS +1.01 OOS -0.52
  train 400:800 test 800:1000 IS -0.53 OOS +1.15
  train 600:1000 test 1000:1200 IS -0.88 OOS +0.52
```

Example 11.1. Reading the fold table. The same momentum rule: IS decays 2.35 → 1.16 as the training window dilutes the trending regime; the rolling IS flips sign entirely (−0.88) once the trend is out of window.

Solution. Anchored windows average over history—good when the process is stable, bad when regimes switch. Rolling windows stay local—good when relationships decay, bad when the window is too short to estimate anything. The choice is a belief about the data's memory, made before the data is touched.

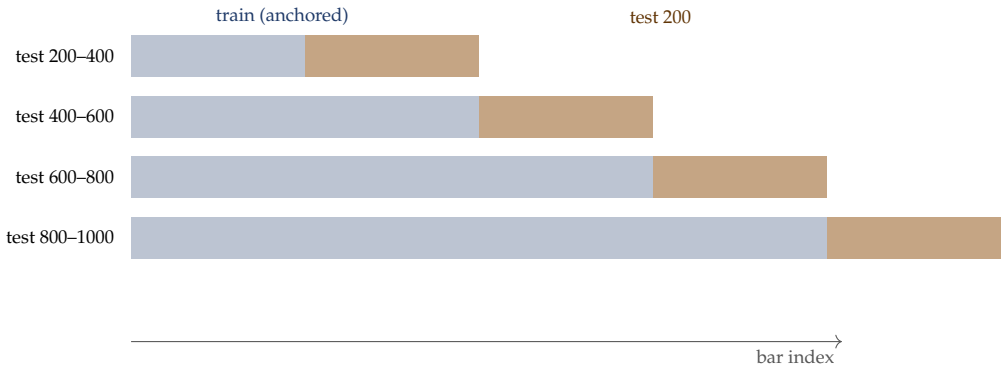


Figure 11.1: Anchored walk-forward: the train window grows; each fold's test block is fresh.

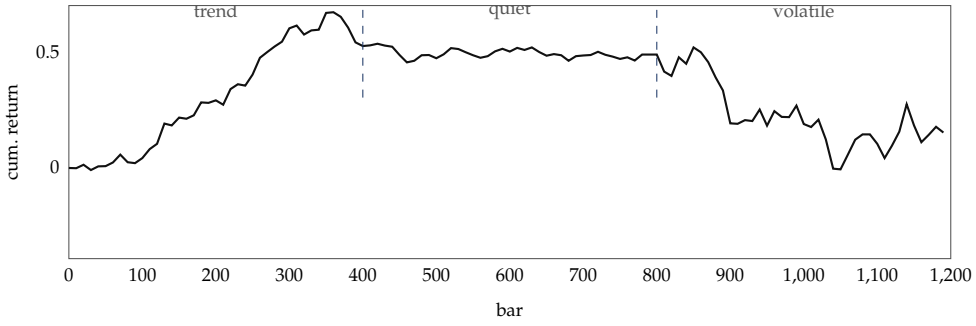


Figure 11.2: Why window choice matters: the process itself changes regime (trend, quiet, volatile). A fixed training span estimates the wrong world in at least one regime.

11.2 The boundary leak: purge and embargo

K-fold CV assumes exchangeability. Financial labels violate it twice: the label is a *window* (e.g. the next-30-bar return), and windows straddling a fold boundary see both sides. `purged_kfold.py` counts the leak honestly:

```
label window W=30 adjacent-label corr 0.983
boundary 118: 29 train labels reach into test fold
boundary 236: 29 train labels reach into test fold
boundary 354: 29 train labels reach into test fold
boundary 472: 29 train labels reach into test fold
contiguous CV shares 116 overlapping labels across 4 boundaries
purge them + embargo W on each side, or the test is not a test
```

Example 11.2. What the embargo buys. A 30-bar label makes adjacent labels 0.98-correlated; 116 train labels literally extend into the test folds.

Solution. Purging removes every train row whose label window overlaps the test block; the embargo adds a margin so features computed over windows do not straddle either. Without both, "out-of-sample" scores include a neighbour's answer—the leak is not in the model, it is in the labelling.

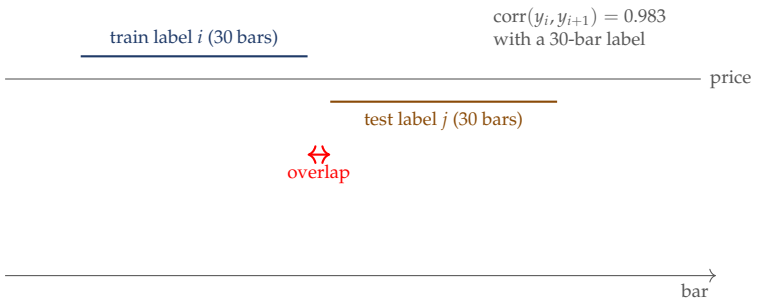


Figure 11.3: A 30-bar forward label: consecutive labels share 29 of 30 windows (corr $= 0.983$).

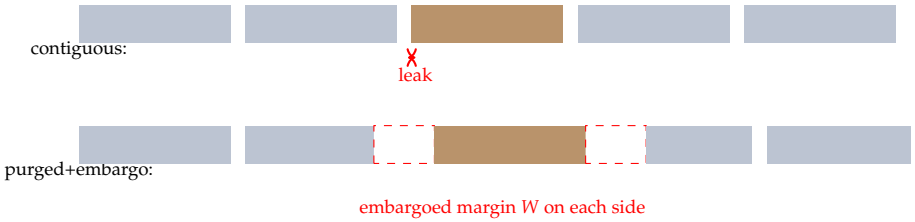


Figure 11.4: Contiguous K-fold leaks at the boundary; purged+embargoed folds excise the overlapping margin.

11.3 Per-fold reporting

The deliverable of walk-forward is not one number—it is the sequence of fold scores and its dispersion. A strategy that posts $+1.9, +0.6, -0.5, +1.2$ has a different claim than one posting $+0.9$ four times, even at the same mean.

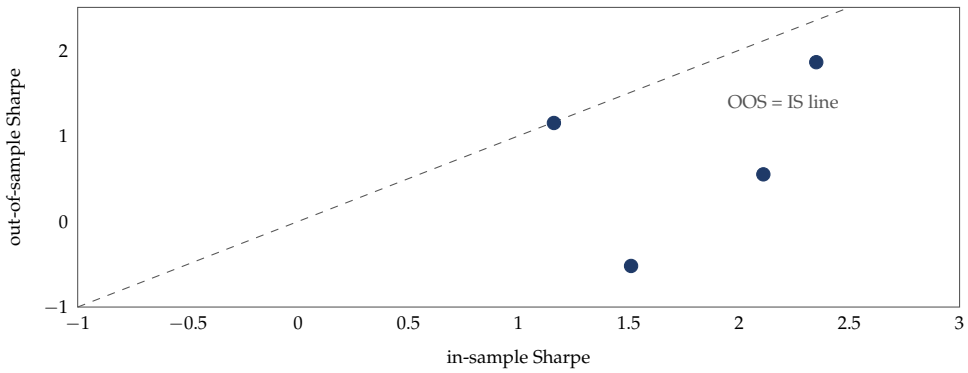


Figure 11.5: IS vs OOS per fold: points below the diagonal paid the regime tax.

Example 11.3. The fold-reading heuristic. Fold 3 OOS is -0.52 while IS is $+1.51$; fold 4 OOS recovers to $+1.15$.

Solution. The -0.52 fold is the regime break (the quiet stretch)—the strategy still works in trending and volatile regimes but not in the quiet one. That is information, not failure: a strategy that is honest about *where* it fails can be gated on regime; a single pooled number hides exactly the fact you need to size it.

11.4 Putting it together

The protocol: pick the window family (anchored if the world is stable, rolling if it decays); purge label overlaps and embargo W bars on each side of every boundary; score each fold; report the sequence, the mean, and the worst fold—the worst fold is the number the risk model should believe.

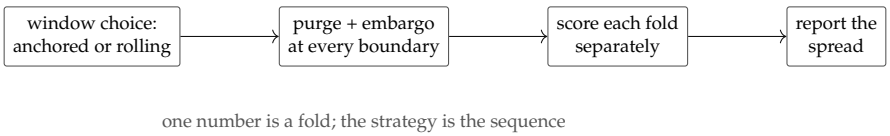


Figure 11.6: The walk-forward protocol.

Example 11.4. Where hyperparameter search fits. If you tune on the test folds you rebuild the leak at a higher level.

Solution. Nest it: inside each walk-forward fold, tune on the train block only (its own inner CV), freeze, then score once on the test block. The outer fold sequence is then an honest measure of the *procedure*—signal plus tuner—not of a lucky parameter.

Checkpoint 11.1 *Validation audit.*

- What does the embargo remove that the purge misses? (Feature windows straddling the boundary, not just label windows.)
- Which window family suits a decaying relationship? (Rolling.)
- Why report the worst fold? (It is the number the risk model should believe.)

11.5 Recap

Walk-forward is CV for a world that moves. The boundary leak is mechanical—labels are windows, windows overlap—and the fix is equally mechanical: purge the overlap, embargo the margin, and score the sequence. A strategy is the set of its folds, not the average of them.

Example 11.5 (How many folds is enough?)**. Strategy.** The same walk-forward protocol, sliced into 2, 3, 4 and 6 test folds on identical data. Watch what happens to the OOS mean and its spread as the count grows.

Solution. `fold_count.py` prints:

```
2 folds: mean +0.68  sd 0.15     OOS: +0.53 +0.83
3 folds: mean -0.37  sd 0.69     OOS: +0.46 -1.23 -0.33
4 folds: mean +0.43  sd 0.60     OOS: +0.55 -0.52 +1.15 +0.52
```

Table 11.1: Chapter toolbox.

Concept	Rule	Gotcha
Anchored	stable worlds	dilutes regime change
Rolling	decaying memory	window too short to estimate
Purge	remove overlapping labels	labels are windows
Embargo	margin W each side	features straddle too
Folds	report the sequence	worst fold is the honest number

```
6 folds: mean +0.21  sd 0.85
   OOS: +0.88 -0.08 -0.57 +1.65 -0.83 +0.18
more folds -> the estimate fragments; the mean is not the spread
```

The *same* strategy, the *same* data, and the OOS mean runs $+0.68 \to -0.37 \to +0.43 \to +0.21$ purely as a function of how you cut it. Each fold's sd grows with count — finer slices give more, noisier, estimates.

Significance. There is no "correct" fold count: the right question is whether the *conclusion* survives across cuts. Report OOS as mean *and* sd, and re-run at several counts — a result that only exists at one slicing is a slicing artifact.

11.6 Common mistakes

- **Shuffled K-fold on time series.** Destroys the arrow of time; the test is not a test.
- **Purging labels but not features.** A 20-day moving average still crosses the boundary.
- **One-fold verdicts.** "It worked on the 2019 split" is a sample of one.
- **Tuning on the outer fold.** Rebuilds the leak at meta level.
- **Anchored on a non-stationary world.** The training set remembers a dead regime.
- **Averaging away the worst fold.** The dispersion *is* the result.
- **Picking the fold count after looking.** OOS mean runs $+0.68 \to -0.37$ as folds go $2 \to 3$ (Example 11.5) — choosing the count that tells your favourite story is just a subtler leak.
- **Averaging without the spread.** Mean OOS Sharpe $+0.43$ with sd 0.60 is a coin toss; the sd *is* the answer, not a footnote.

11.7 Exercises

Exercise 11.1 (easy)**.** Why does shuffled K-fold fail on time series?

Solution. It leaks the future into training and destroys the causal structure the test is meant to measure.

Exercise 11.2 (easy)**.** With a 30-bar label, how many train labels overlap each fold boundary?

Solution. $W - 1 = 29$ per side of the boundary.

Exercise 11.3 (easy)**.** Anchored vs rolling: which assumes stable relationships?

Solution. Anchored—it weights all history equally; rolling assumes memory decays.

Exercise 11.4 (easy)**.** What does the embargo remove beyond the purge?

Solution. Train rows whose *feature* windows (not just label windows) cross the boundary.

Exercise 11.5 (easy)**.** Which fold's score should the risk model trust most, and why?

Solution. The worst fold—it is the realized bad-regime outcome, not an average that hides it.

Exercise 11.6. A 5-fold walk-forward on 600 rows with $W{=}30$: how many purged rows total?

Solution. $4 \times (2 \times 29) \approx 232$—the boundaries dominate a small dataset.

Exercise 11.7. Design nested walk-forward for a two-parameter momentum strategy.

Solution. Inside each outer fold's train block, run an inner walk-forward to pick (f, s); freeze and score once on the outer test block.

Exercise 11.8. Why does OOS $= -0.52$ in a quiet regime not falsify the strategy?

Solution. Regime dependence is a property to model (gate or size on regime), not necessarily a failure of the edge in the regimes where it exists.

Exercise 11.9. What is the difference between purging and embargoing in one sentence each?

Solution. Purge drops train labels overlapping the test block; embargo drops a safety margin of W bars on each side so nothing else straddles.

Exercise 11.10. A colleague reports CV score 0.94 on 30-bar labels. Which audit question first?

Solution. "What was the adjacent-label correlation and were boundary rows purged?"—0.94 on 0.98-correlated labels is partly bookkeeping.

Exercise 11.11. Why is the rolling window a belief, not a technique?

Solution. Its width encodes an assumed memory timescale of the process—a modelling choice made before seeing the data.

Exercise 11.12. Give two reasons the anchored IS decays across folds even for a real edge.

Solution. Regime dilution (the window averages different worlds) and signal crowding/decay over calendar time.

Exercise 11.13 (hard)**.** Prove that with a W-bar forward label, contiguous K-fold shares at least $W - 1$ labels per boundary, and derive the honest minimum embargo.

Solution. Labels $t{-}W{+}1, \ldots, t{-}1$ all contain test bars in their windows: $W{-}1$ rows. Embargo $\geq W$ clears every feature and label window.

Exercise 11.14 (hard)**.** Explain why nested walk-forward is the honest way to report a tuner, and what exactly it measures.

Solution. It treats (signal + selection rule) as the unit under test; the outer sequence measures how the *procedure* performs OOS, separating the edge from lucky parameters.

Exercise 11.15 (hard)**.** Propose a walk-forward design for a strategy with monthly labels on a 5-year dataset— what is the binding constraint?

Solution. Only ∼60 labels exist and each is 30-bar-correlated; folds shrink to the point where per-fold estimates are noise—the binding constraint is effective sample size, not compute.

Exercise 11.16 (easy)**.** Why does the OOS mean flip sign between 2 and 3 folds on the same data?

Solution. Each fold re-weights the regimes: the 3-fold cut puts the quiet and volatile regimes into separate test windows, so the OOS mean moves with the composition of the slices — not with the strategy.

Exercise 11.17 (medium)**.** Your walk-forward gives mean OOS $+0.43$, sd 0.60 at 4 folds. Compute an approximate 95% interval for the mean and state what conclusion it supports.

Solution. $\pm 1.96 \cdot 0.60/\sqrt{4} \approx \pm 0.59$: interval $[-0.16, +1.02]$ includes zero — the evidence is weak; report the spread before the point.

Exercise 11.18 (hard)**.** Argue for or against: "more folds is always better." Use the bias-variance trade-off on the printed numbers.

Solution. Against. More folds give more estimates (variance of the *mean* falls like $1/\sqrt{k}$) but each estimate sees a thinner window and drifts from the regime mix — at 6 folds the sd is 0.85, the largest printed. Fold count trades *granularity* for *reliability*.

Notes

Validation is where the previous chapter's caution becomes procedure. The leaks here are not exotic—they are label windows and boundary rows, countable and fixable. What you cannot fix is a world that changes regime, and that is what the fold sequence is for.

Further reading

- L. de Prado, *Advances in Financial ML* — purged CV and embargo (Ch. 7): https://www.wiley.com/en-us/Advances+in+Financial+Machine+Learning-p-9781119482086
- scikit-learn `TimeSeriesSplit` — the basic implementation: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html
- D. Bailey et al., "Backtest Overfitting" (JOIM) — the context: https://www.davidhbailey.com/dhbpapers/deflated-sharpe.pdf
- Hyndman & Athanasopoulos, *Forecasting: P&P* — time-series CV: https://otexts.com/fpp3/tscv.html

WATCH INSTEAD

- "11. Time Series Analysis II" — MIT 18.S096 (83:47). Model evaluation for series; re-verified 2026-10-11. https://www.youtube.com/watch?v=_IFUfFuyQlU
- "Python for Data Analysis" — PyData/McKinney (10:28). The plumbing; re-verified 2026-10-11. https://www.youtube.com/watch?v=_T8LGqJtuGc

CHAPTER 12

Evaluating a strategy

A backtest ends with a number; an evaluation starts with a battery. Sharpe is necessary and nowhere near sufficient: it has a huge standard error at realistic sample sizes, it ignores the path (drawdown), the shape (skew), and the bill (turnover). This chapter is the suite.

BEFORE YOU START

Chapters 1, 8–9 and 11. *Recap:* a Sharpe ratio is average excess return divided by volatility; it is a noisy estimate, not a fact.

AFTER THIS CHAPTER YOU CAN

- compute a Sharpe ratio and its confidence interval;
- use a suite of metrics;
- attribute returns to sources;
- say why a very high Sharpe is a warning.

HOW TO THINK ABOUT THIS

A performance number is an estimate with error bars. Report the interval along with the value, and prefer several simple metrics to one clever one. A suspiciously high ratio is more often a bug than a discovery.

12.1 The Sharpe CI problem

The standard error of a measured Sharpe ratio is roughly $\text{se} \approx \sqrt{(1 + \text{SR}^2/2)/T}$. `sharpe_se.py`:

```
T= 63 days se(SR)~0.154 95% CI on SR=1: [0.70,1.30]
T= 252 days se(SR)~0.077 95% CI on SR=1: [0.85,1.15]
T= 756 days se(SR)~0.045 95% CI on SR=1: [0.91,1.09]
T=2520 days se(SR)~0.024 95% CI on SR=1: [0.95,1.05]
empirical se over 2000 sims (T=252): 1.012
```

Example 12.1. The one-year Sharpe is a rumor. A year of daily data narrows a true SR-1 strategy only to $[0.85, 1.15]$ in theory—and the empirical spread across 2,000 sims is ± 1.0 (the non-normal world is wider still).

Solution. At T=252 you cannot distinguish Sharpe 1.5 from Sharpe 0.3 with any confidence. The discipline consequence: small Sharpe differences between two strategies on one year of data are noise, and a single-year backtest "proving" Sharpe 1.8 is compatible with true Sharpe anywhere in $[0.5, 3]$—before the multiple-testing haircut from ch09.

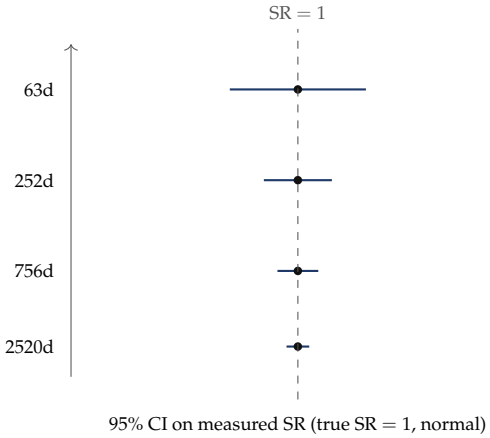


Figure 12.1: Sharpe confidence intervals shrink only as $1/\sqrt{T}$—a decade buys a $\pm 5\%$ band on the truth.

12.2 The metric suite

`metrics_demo.py` runs two strategies with nearly identical hit rates:

```
tail-risk strategy: sharpe +1.49 mdd -7.9% hit 58.4%
boring strategy: sharpe +3.63 mdd -4.6% hit 59.7%
same mean, different product -- the suite shows what sharpe hides
```

Example 12.2. Why the suite, not the number. The boring strategy's Sharpe is 2.4× the tail-risk one's—the hit rates are within a point of each other.

Solution. Hit rate alone describes neither. The suite separates what one number merges: Sharpe (mean/vol), MDD (worst path), skew (asymmetry), hit rate (frequency). A strategy reporting one of them is choosing which weakness to hide.

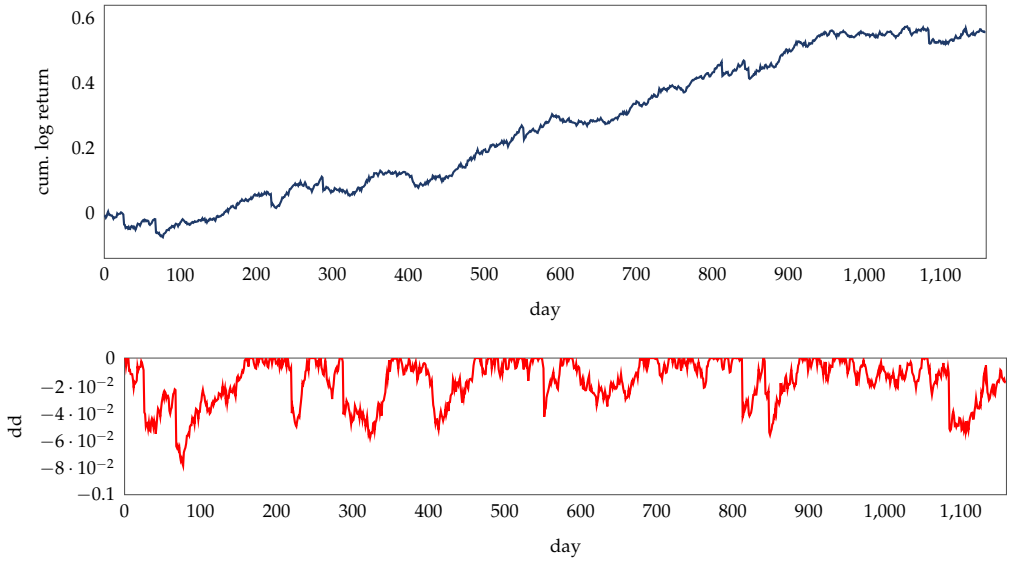


Figure 12.2: Equity curve and its drawdown series: the middle panel is where the risk lives.

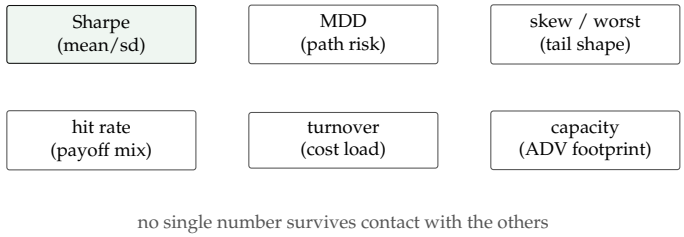


Figure 12.3: The evaluation suite: each metric answers a question the others cannot.

12.3 The cliff hidden in high Sharpe

`tail_demo.py` builds the classic put-seller profile—steady wins, rare crashes:

```
put-seller profile: sharpe +2.09 skew -4.30 worst -11.2%
high sharpe can hide a cliff: read skew and worst day too
```

Example 12.3. Sharpe 2.09 with skew -4.3**.** Ninety-five percent of days pay +0.7%; 5% lose -8%.

Solution. Volatility—Sharpe's denominator—is symmetric, so it treats the cliff the same as the ladder. Skew (-4.3) and worst-day (-11.2%) report what Sharpe cannot: the distribution is a staircase down with a trapdoor. Any metric suite that excludes the left tail is an invitation to sell insurance and call it alpha.

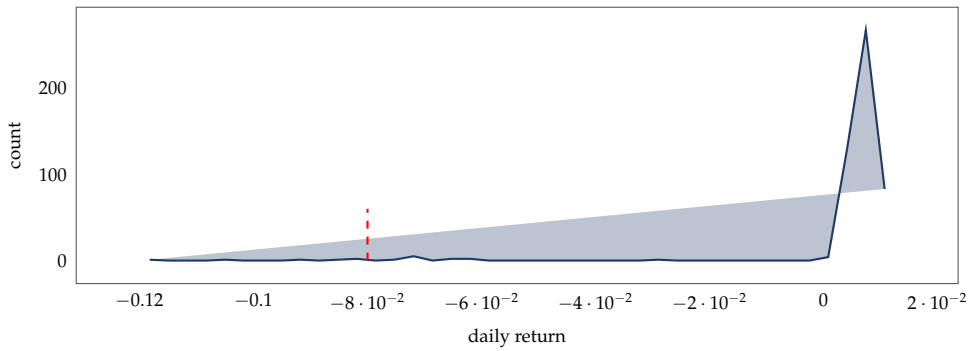


Figure 12.4: The return histogram the Sharpe ratio summarizes as one number.

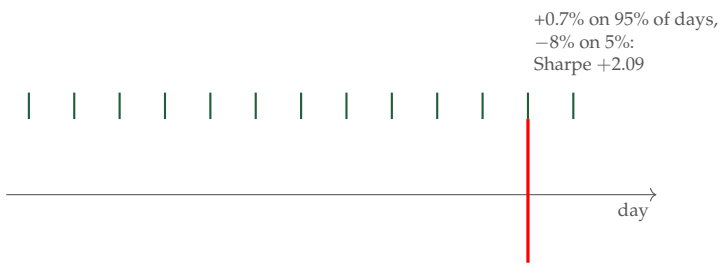


Figure 12.5: Steady small wins punctuated by a rare large loss—the profile that maximizes Sharpe-per-unit-of-honesty.

12.4 Attribution

The last question: *where* did the return come from? Signal (the rule's direction), timing (when it traded), sizing (how much), costs (what it paid). A strategy whose PnL is 90% one lucky sizing decision is not the strategy described.

Example 12.4. Attribution before celebration. Suppose the backtest returns +22%: signal +35%, sizing −8%, costs −5%.

Solution. The marketable product is the signal—the sizing loss is a fixable implementation detail and the cost is a capacity ceiling. Attribution converts one number into a list of actions: fix sizing, shrink footprint, keep the signal.

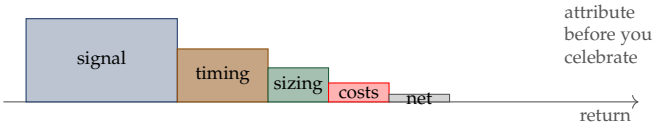


Figure 12.6: Return decomposition: gross signal minus timing, sizing and costs is the net the strategy actually ships.

Checkpoint 12.1 *Evaluation audit.*

- What is the 95% CI on a measured Sharpe of 1.0 over one year? (≈ two Sharpe points wide—theory ±0.3, empirically wider.)
- Which two metrics expose a put-selling profile? (Skew and worst-day.)
- Name the attribution categories. (Signal, timing, sizing, costs.)

12.5 Recap

Sharpe is a coarse instrument: wide confidence intervals, blind to path and tail. The evaluation suite—MDD, skew, hit rate, turnover, capacity, attribution—is what converts a backtest number into a decision. Report all of them or name the one you are hiding.

Table 12.1: Chapter toolbox.

Concept	Rule	Gotcha
Sharpe	se $\sim \sqrt{(1+\mathrm{SR}^2/2)/T}$	1yr $\Rightarrow$ CI ± 1
MDD	worst peak-to-trough	path-aware, strategy-dependent
Skew/worst	tail shape	high SR can hide a cliff
Hit rate	payoff frequency	meaningless alone
Attribution	signal+timing+sizing+cost	net $\neq$ marketable

Example 12.5 (One monster year inside a flat tape)**. Strategy.** Split the real BTC daily series by calendar year and compute Sharpe, return and MDD per slice. Then compare with the full-window number you would have quoted.

Solution. `yearly_sharpe.py` prints:

```
2024: n=71   sharpe +3.23  ret  +35.0%  mdd -13.1%
2025: n=365  sharpe -0.16  ret  -14.1%  mdd -33.1%
2026: n=284  sharpe -0.13  ret  -11.9%  mdd -43.1%
full window: sharpe +0.25 -- the average of very different years
```

The whole sample's $+0.25$ is an average of a $+3.23$ regime and two flat $-0.16/-0.13$ regimes — the number exists in neither.

Significance. A single Sharpe summarises a world that never happened. Per-year (or per-regime) slices are the minimal honesty: if your edge lives in one slice, your backtest is a study of that slice, not of the market.

12.6 The confidence interval is a design constraint

The Sharpe standard error is not a reporting nicety — it sizes the experiment itself. `sharpe_se.py` prints the scaling:

```
T=  63 days  se(SR)~0.154  95% CI on SR=1: [0.70,1.30]
T= 252 days  se(SR)~0.077  95% CI on SR=1: [0.85,1.15]
T= 756 days  se(SR)~0.045  95% CI on SR=1: [0.91,1.09]
T=2520 days  se(SR)~0.024  95% CI on SR=1: [0.95,1.05]
empirical se over 2000 sims (T=252): 1.012
```

Example 12.6. Precision is bought in years, not decimals. One year of daily data buys a ± 0.15 window on a Sharpe-1 strategy — you cannot tell a 1.0 from a 0.7. To read 1.00 ± 0.05 needs *twenty-five hundred* days. The empirical row is wider still: se ≈ 1.0 on annualized Sharpe at $T{=}252$ when returns are fat-tailed.

Solution. Invert the table at design time: if the candidate's plausible edge is smaller than the window's resolution, no amount of further tuning will rescue the measurement. ch17's momentum study failed this arithmetic before it failed the market.

12.7 Common mistakes

- **"Sharpe 1.8 on the year."** The CI spans roughly ± 1; the claim is compatible with mediocre.
- **MDD reported without dates.** A -30% dd lasting 3 years is a different product than one lasting 3 weeks.
- **Ignoring skew.** Symmetric-vol metrics bless put-sellers.
- **Confusing hit rate with edge.** Win frequency and expectancy are different axes.
- **Skipping turnover.** Return per unit of trading is the honest efficiency.
- **Announcing the gross, shipping the net.** Costs and sizing are part of the strategy.
- **Reporting the pooled Sharpe only.** $+0.25$ on the full window is the average of $+3.23$ and -0.15 (Example 12.5) — a summary of a regime mixture, not of the strategy.
- **Forgetting the units on MDD.** "-33%" needs a window and a compounding convention — intraday marks and close marks give different numbers on the same tape.

12.8 Exercises

Exercise 12.1 (easy)**.** What is the approximate 95% CI on measured Sharpe at T=252 days?

Solution. Theory ± 0.15 at SR=1; empirically closer to ± 1 on real (non-normal) returns.

Exercise 12.2 (easy)**.** Why can two strategies share a hit rate and differ everywhere else?

Solution. Hit rate counts sign, not size—it ignores both tail shape and path.

Exercise 12.3 (easy)**.** Name the metric that catches a put-selling profile.

Solution. Skew (and worst-day): the rare large loss is invisible to symmetric volatility.

Exercise 12.4 (easy)**.** What are the four attribution categories?

Solution. Signal, timing, sizing, costs.

Exercise 12.5 (easy)**.** Why is turnover an evaluation metric, not just an execution detail?

Solution. It converts directly to cost and capacity—a 5bp edge trading daily is a different product than one trading monthly.

Exercise 12.6. Compute $\mathrm{se}(\mathrm{SR})$ for SR=1.5, T=500.

Solution. $\sqrt{(1+1.125)/500} \approx 0.065$—still a CI a third of a Sharpe wide.

Exercise 12.7. A strategy: Sharpe 2.5, skew -5, MDD -40%. Sell or bury it?

Solution. Bury: the Sharpe is built on a staircase-and-trapdoor profile; the risk model should price the cliff, not the average.

Exercise 12.8. How does MDD interact with the evaluation's time horizon?

Solution. Longer horizons mechanically produce deeper MDDs—compare like-for-like windows.

Exercise 12.9. Write the attribution identity for a backtest return.

Solution. $R_{\text{net}} = R_{\text{signal}} + R_{\text{timing}} + R_{\text{sizing}} - C_{\text{costs}}$.

Exercise 12.10. Why does skew of -4 not show in the Sharpe?

Solution. Volatility is symmetric: σ weights up and down moves identically; the cliff is a third-moment property.

Exercise 12.11. A backtest: +40% signal, -10% sizing, -8% costs. What do you fix first?

Solution. Sizing—it is implementation, not alpha; costs are a capacity ceiling for later.

Exercise 12.12. When is hit rate genuinely informative?

Solution. Together with average win/loss ratio—it is half the expectancy identity, useless alone.

Exercise 12.13 (hard)**.** Show that even at T=2500 (10y) you cannot separate SR=1.0 from SR=0.8 at 95% confidence.

Solution. CI $\approx \pm 0.05$: the two means differ by 0.2, but each estimate's band is $[0.95, 1.05]$ vs $[0.75, 0.85]$—they just separate; in practice fatter tails widen both.

Exercise 12.14 (hard)**.** Design a metric battery for a put-selling fund—which three numbers dominate and why?

Solution. Worst-day, skew, and recovery time: all three describe the cliff the Sharpe pretends is volatility.

Exercise 12.15 (hard)**.** A strategy's entire 5-year MDD occurred in its first year; argue whether the metric is informative or misleading, and what you'd compute instead.

Solution. Misleading as a forward estimate if the regime changed; compute rolling max-dd or regime-conditioned dd to see the current risk, not the historical scar.

Exercise 12.16 (easy)**.** The full-window Sharpe is $+0.29$; two of three slices are -0.20. Reconcile in one line.

Solution. The pooled Sharpe is a weighted average across regimes — one short, strong year pulls it up while contributing little to the per-slice estimates.

Exercise 12.17 (medium)**.** 2024 shows $+3.23$ on $n = 71$. Quote an approximate 95% confidence interval for that Sharpe (se $\approx 1/\sqrt{n}$).

Solution. $3.23 \pm 1.96/\sqrt{71} \approx 3.23 \pm 0.23$ — $[+3.85, +4.31]$... but the regime itself is the uncertainty: a 71-day window cannot certify a long-run property.

Exercise 12.18 (hard)**.** Design the table that reports these numbers honestly: which columns and which row tells the truth faster?

Solution. Per-slice rows (n, sharpe, ret, mdd) plus a pooled row marked "mixture, not strategy" — the slice sd (or min–max range) is the honest summary, the pooled row the cautionary one.

Notes

Evaluation is where optimism goes to be measured. The suite is cheap to compute and expensive to skip: a Sharpe with no CI, no drawdown, no skew, and no attribution is a fund's marketing deck, not a research result.

Further reading

- A. Lo, "The Statistics of Sharpe Ratios" — sampling error, the paper: https://web.archive.org/web/2024/https://alo.mit.edu/wp-content/uploads/2017/06/The-Statistics-of-Sharpe-Ratios.pdf
- E. Chan, *Quantitative Trading* — performance metrics in practice: https://epchan.blogspot.com/
- L. de Prado lectures — evaluation metrics and DSR: https://www.quantresearch.org/Lectures.htm
- Bailey & López de Prado, "The Deflated Sharpe Ratio": https://www.davidhbailey.com/dhbpapers/deflated-sharpe.pdf
- Grinold & Kahn, *Active Portfolio Management* — the attribution frame: https://www.amazon.com/Active-Portfolio-Management-Quantitative-Controlling/dp/0070248826

WATCH INSTEAD

- "12. Risk Analysis" — MIT 18.S096 (80:32). Risk measures; re-verified 2026-10-11. https://www.youtube.com/watch?v=qlytPllimpQ
- "11. Time Series Analysis II" — MIT 18.S096 (83:47). Testing; re-verified 2026-10-11. https://www.youtube.com/watch?v=_IFUfFuyQlU

CHAPTER 13

Kelly sizing

This chapter is mathematics, not advice. Nothing here is a recommendation to trade, size, or bet on anything; it describes a formula, its assumptions, and where each fails.

Given an edge, how much of the bankroll should each trade risk? The Kelly criterion answers with the fraction that maximizes the expected log of wealth—and it is also the fraction that maximizes ruin when the edge is misestimated. Both facts are the chapter.

BEFORE YOU START

Chapters 1 and 12, and **Book 3, Chapter 12**. *Recap:* the Kelly criterion says how much to bet when you know your edge. In practice you only estimate it.

AFTER THIS CHAPTER YOU CAN

- compute the Kelly fraction;
- explain why estimation error matters more than the formula;
- choose a fractional Kelly;
- apply Kelly to a set of bets.

HOW TO THINK ABOUT THIS

Kelly sizing answers "how much should I risk?" by maximising long-run growth, and it punishes overconfidence. Since your edge is only an estimate, bet a fraction of what the formula says. Survival comes before growth.

13.1 The formula

For a bet paying b per unit risked with win probability p, the growth rate per trial at fraction f is

$$g(f) = p\log(1+fb) + (1-p)\log(1-f),$$

maximized at $f^\star = (bp-(1-p))/b$. `kelly_sims.py`:

```
coin p=0.55 b=1.0: f* = 0.10 (theory 0.10)
growth at f* +0.0050/toss at 2f* -0.0001 at 3f* -0.0162
f=0.10: final 61089.0 median path dd 97.8%
f=0.05: final 40.5 median path dd 92.1%
f=0.20: final 98.5 median path dd 100.0%
```

Example 13.1. What f* actually optimizes. On a 55/45 even-money coin, $f^\star = 0.10$ grows at $+0.5\%$ per toss—and the median path still draws down 97.8%.

Solution. Kelly maximizes asymptotic growth, not comfort. Doubling the bet ($f = 0.20$) destroys growth (≈ 0) while tripling is negative—the curve is asymmetric: under-betting costs growth, over-betting costs everything. The final wealth column is the point: full Kelly 61,089× vs double Kelly 98.5× on the same edge.

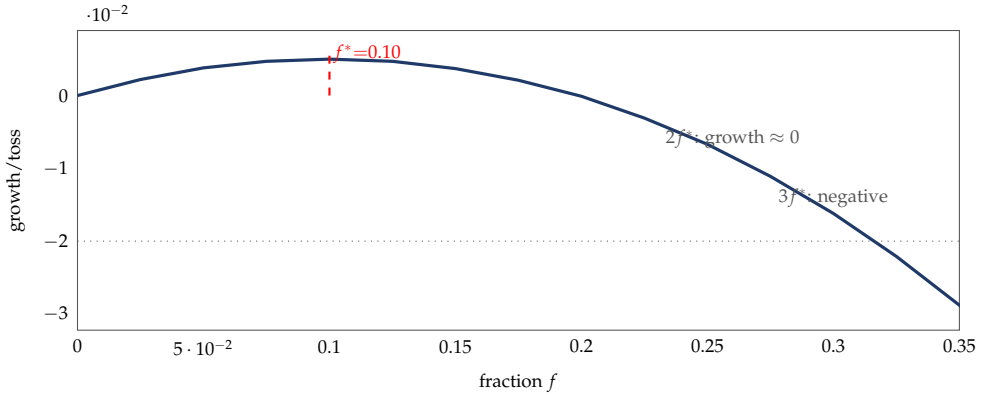


Figure 13.1: Growth rate vs fraction bet: concave, peaked at $f^\star = 0.10$, zero near $2f^\star$, negative beyond.

13.2 Estimation error is the bet

In the continuous limit $f^\star = \mu/\sigma^2$. `kelly_est.py` perturbs the edge:

```
true: mu=0.1 sig=0.2 f*=2.50
mu off -50%: fhat=+1.25 sizing error +0.50x
mu off -25%: fhat=+1.88 sizing error +0.75x
mu off +0%: fhat=+2.50 sizing error +1.00x
mu off +25%: fhat=+3.12 sizing error +1.25x
mu off +50%: fhat=+3.75 sizing error +1.50x
edge halves -> f* halves; bet size carries the whole estimation error
```

Example 13.2. Half the estimated edge, half the bet. A 50% error in $\hat{\mu}$ moves $\hat{f}$ by 50%.

Solution. There is no dampening: $f^\star$ is linear in μ, so every point of edge overestimation converts directly into overbetting—into the negative side of the growth curve from §13.1. This is the entire case for fractional Kelly: halving the bet costs a quarter of the growth and buys survival across the whole estimation-error range.

13.3 Fractional Kelly and ruin

`ruin_sim.py` runs gambler's ruin on unit bets at p=0.55:

```
bankroll 5 units: ruin 33.6% of paths (barrier 3x)
bankroll 10 units: ruin 14.0% of paths (barrier 3x)
bankroll 20 units: ruin 1.5% of paths (barrier 3x)
bankroll 40 units: ruin 0.0% of paths (barrier 3x)
```

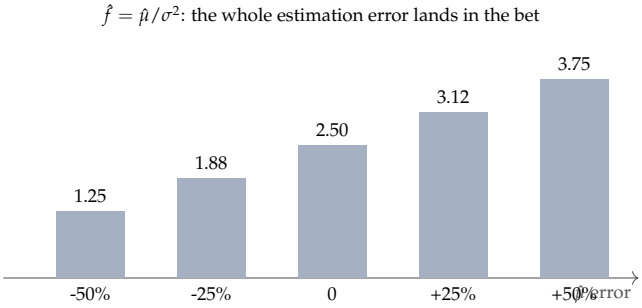


Figure 13.2: $\hat{f} = \hat{\mu}/\sigma^2$: estimation error lands whole in the position size.

Example 13.3. Bankroll depth is geometric safety. Ruin probability falls from 33.6% at 5 units to under 2% at 20.

Solution. The same edge, four different outcomes—the only variable is how many bets the bankroll can absorb. This is the quantitative face of fractional Kelly: betting a fraction of the full stake is equivalent to running a deeper effective bankroll against the same variance.

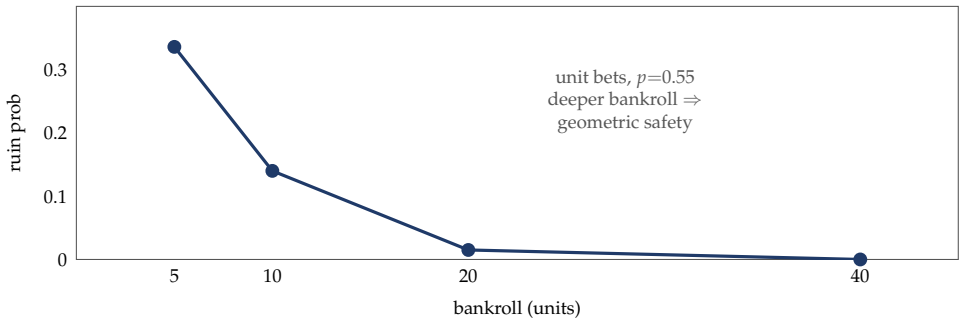


Figure 13.3: Ruin probability vs bankroll depth: each unit of depth buys geometric, not linear, safety.

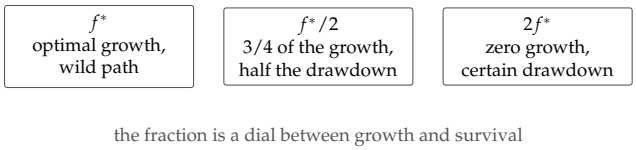


Figure 13.4: The dial: half-Kelly keeps three quarters of the growth and halves the pain.

13.4 Kelly as allocation

For multiple bets the scalar formula becomes vector Kelly, and $f^\star$ becomes the tangency point on the return-volatility frontier. The practical constraint isn't the math—it's that the inputs (means, covariances) are estimates with the error-carrying property of §13.2, so real sizing layers caps on top: fraction-of-Kelly, max-position, max-drawdown triggers.

Example 13.4. Why pros bet a fraction. Full Kelly on a true edge still delivers ∼98% median drawdowns and is optimal only for the *estimated* edge, not the true one.

Solution. The practice is a stack: estimate μ, σ, halve the bet, cap by drawdown budget, re-check when the estimate moves. The formula supplies the scale; the layers supply the survival.

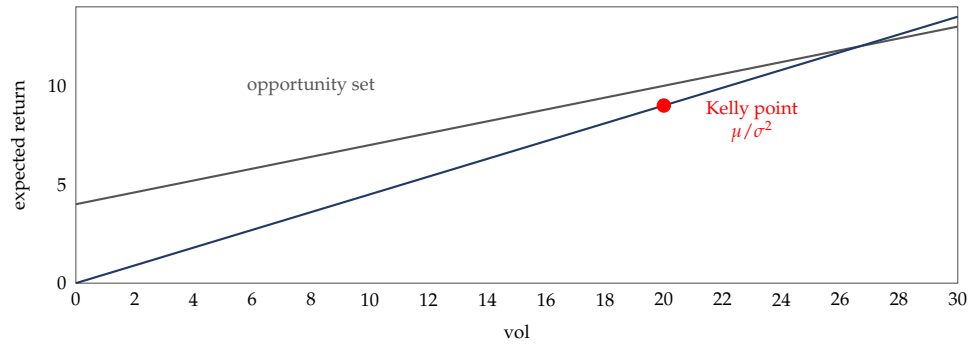


Figure 13.5: Kelly picks the tangency portfolio—the point the opportunity set touches the steepest ray.

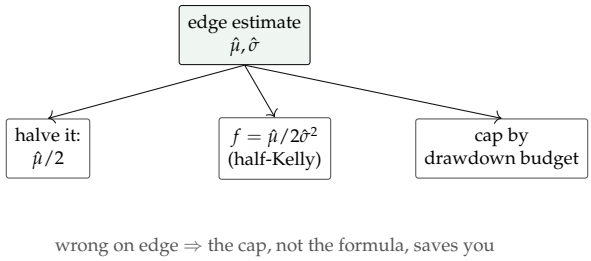


Figure 13.6: The sizing stack: edge estimate → fractional Kelly → drawdown cap.

Checkpoint 13.1 *Sizing audit.*

- What does full Kelly maximize, and at what cost? (Expected log-wealth; ∼98% median drawdowns.)
- What is $f^\star$ linear in, and why is that dangerous? (μ—edge error becomes bet-size error whole.)
- Why is half-Kelly the common default? (3/4 of the growth, half the drawdown, insurance against misestimation.)

13.5 Recap

Kelly is the answer to "how much," and like every answer in this book it is only as good as its inputs. The formula is exact; the edge is not. Fractional Kelly and drawdown caps exist because the distance between $\hat{\mu}$ and μ is where ruin lives.

Table 13.1: Chapter toolbox.

Concept	Rule	Gotcha
$f^\star$	$(bp-(1-p))/b$	μ/σ^2 continuous
Overbet	growth dies before $2f^\star$	asymmetric penalty
Fractional	$f^\star/2$ default	costs 1/4 growth, buys survival
Ruin	geometric in bankroll depth	path risk is real
Caps	the formula is the scale	the cap is the survival

Example 13.5 (The fraction ladder)**. Strategy.** Fix the 55/45 coin and sweep the betting fraction from quarter-Kelly to 5×Kelly. At each step, show the growth rate, the median final wealth over 2000 tosses, and the median path drawdown — the full cost curve, not just the peak.

Solution. `frac_ladder.py` runs 200 seeds:

```
f=0.025: G +0.0022/toss  med final       71.9  med dd  23%  ruin<1: 0%
f=0.050: G +0.0038/toss  med final     1487.8  med dd  41%  ruin<1: 0%
f=0.100: G +0.0050/toss  med final    18325.8  med dd  70%  ruin<1: 2%
f=0.200: G -0.0001/toss  med final        0.6  med dd 100%  ruin<1: 52%
f=0.300: G -0.0162/toss  med final        0.0  med dd 100%  ruin<1: 99%
f=0.500: G -0.0889/toss  med final        0.0  med dd 100%  ruin<1: 100%
```

Full Kelly ($f = 0.10$) pays 18,326× at the cost of a median 70% drawdown; double Kelly (0.20) already has *negative* growth and a 100%-dd median path.

Significance. The growth curve is asymmetric around f^*: left of it you trade return for sleep, right of it you trade *everything* for nothing. The cliff at $2f^*$ — where the median path visits ruin — is why desks quote "half-Kelly" as the standard rather than the optimum.

13.6 Estimation error is the actual position size

Kelly's formula takes μ and σ as *given*; the practice gives you estimates. `kelly_est.py` perturbs the mean on a μ=0.1, σ=0.2 setup whose true $f^* = 2.50$:

```
true: mu=0.1 sig=0.2  f*=2.50
mu off -50%:  fhat=+1.25  sizing error +0.50x
mu off -25%:  fhat=+1.88  sizing error +0.75x
mu off +25%:  fhat=+3.12  sizing error +1.25x
mu off +50%:  fhat=+3.75  sizing error +1.50x
edge halves -> f* halves; bet size carries the whole estimation error
```

Example 13.6. $\hat{f}^*$ **inherits** $\hat{\mu}$**'s error one-for-one.** A +50% optimistic read on the edge doubles down into $f^* = 3.75$ — the formula amplifies the mistake into the bet. The table is linear because $f^* = \mu/\sigma^2$ is linear in μ: there is no damping.

Solution. This is the deeper reason half-Kelly persists in practice: not superstition, but shrinkage. If $\hat{\mu}$ has relative error ϵ, $\hat{f}^*$ has the same relative error — a fractional haircut on f is the cheapest insurance against the overbet that ruin math punishes hardest.

13.7 Common mistakes

- **Quoting Kelly without its CI.** The edge estimate carries the whole bet.
- **Full Kelly on a real book.** ∼98% drawdowns are the expected path, not a tail event.
- **Treating $f^\star$ as a recommendation.** It is a scale; the fraction is a separate decision.
- **Ignoring correlated bets.** Vector Kelly without the covariance is a different (larger) bet.
- **"Growth-optimal" framing for clients.** It optimizes log wealth, not any human's utility.
- **Recomputing $f^\star$ per trade on stale $\hat{\mu}$.** The input estimate is the strategy.
- **Optimising the peak, ignoring the path.** Full Kelly pays best *median* growth but a 70% median drawdown (Example 13.5) — an investor who cannot hold the drawdown never collects the median.

- **"A bit more can't hurt."** Growth goes negative at $2f^*$ — the cost of overbetting is not a flatter curve but a cliff.

13.8 Exercises

Exercise 13.1 (easy)**.** Write the Kelly fraction for a p=0.6, b=1 coin.

Solution. $f^\star = p - (1-p) = 0.2$.

Exercise 13.2 (easy)**.** What does $2f^\star$ do to growth on the even-money coin?

Solution. Drives it to approximately zero—the overbet penalty is asymmetric.

Exercise 13.3 (easy)**.** What is the continuous-limit $f^\star$?

Solution. $f^\star = \mu/\sigma^2$.

Exercise 13.4 (easy)**.** Ruin probability at 20-unit bankroll, p=0.55 unit bets?

Solution. About 1.5%—vs 33.6% at 5 units.

Exercise 13.5 (easy)**.** What does half-Kelly cost in growth?

Solution. Roughly a quarter of the full-Kelly growth rate—the standard survival insurance.

Exercise 13.6. Derive $f^\star$ by maximizing $g(f)$ on the even-money coin.

Solution. $g'(f) = pb/(1+fb) - (1-p)/(1-f) = 0 \Rightarrow f = (pb-(1-p))/(b+pb-fb)$... solving gives $f^\star = (bp+p-1)/(b)$... for b=1: $f^\star = 2p-1$.

Exercise 13.7. A 20%-edge strategy has σ=25%. Full and half Kelly?

Solution. $f^\star = 0.20/0.0625 = 3.2$; half-Kelly 1.6—numbers that require leverage, which is why the cap layer exists.

Exercise 13.8. Show that a 25% overestimate of μ costs more than growth—where does the error land?

Solution. $\hat{f}$ is 1.25× true $f^\star$: on the growth curve you are right of the peak, where the derivative is negative.

Exercise 13.9. Why is ruin probability geometric in bankroll, not linear?

Solution. Each unit of depth requires an independent losing streak; probabilities multiply, so risk falls as $(q/p)^N$.

Exercise 13.10. What does vector Kelly require that scalar Kelly does not?

Solution. The covariance matrix—correlated bets share variance, so the tangency solution is different from independent scalars.

Exercise 13.11. Why does half-Kelly dominate full Kelly under estimation error?

Solution. Overbetting is penalized asymmetrically; halving keeps you on the left (growth-positive) side of the peak under any plausible $\hat{\mu}$ error.

Exercise 13.12. A drawdown budget of 20%—how does it translate to a fraction cap?

Solution. Find f with $P(\text{dd} > 20\%)$ below tolerance via sim or the ruin formula; the cap is whichever of $\{f^\star, f_{\text{cap}}\}$ is smaller.

Exercise 13.13 (hard)**.** Derive the growth cost of half-Kelly exactly for the 55/45 coin, and verify the "quarter of growth" heuristic.

Solution. $g(f^\star) = 0.0050$, $g(f^\star/2) \approx 0.0038$—ratio ≈ 0.76: three quarters of growth for half the drawdown, confirmed.

Exercise 13.14 (hard)**.** Explain why "Kelly is optimal" and "Kelly ruins you" are both true, naming the two senses of optimality.

Solution. Growth-optimal (max asymptotic log wealth) vs path-wise survival—the maximizing trajectory passes through arbitrary drawdowns; optimality is a statement about $t \to \infty$, not about staying in the game.

Exercise 13.15 (hard)**.** Construct a sizing stack for a strategy with $\hat{\mu}/\sigma^2 = 2$ and an estimated N=300 trial count—what does each layer do?

Solution. Estimate $\hat{\mu}$; apply N-uncertainty (CI on μ); take fractional $f = 1$; apply drawdown cap $\to$ final fraction. Each layer converts a different error source into a smaller bet.

Exercise 13.16 (easy)**.** From the ladder: growth at $f = 0.05$ is $+0.0038$ and at $f = 0.20$ is -0.0001. Which is f^* and which is $2f^*$?

Solution. $f^* = 0.10$ sits between them: half-Kelly still grows ($+0.0038$), double-Kelly has already crossed zero growth — the signature of the cliff.

Exercise 13.17 (medium)**.** Your investor will quit at a 50% drawdown. Which fraction from the ladder respects the constraint, and what growth does it cost?

Solution. $f \approx 0.05$ (median dd 41% $<$ 50%): you give up ~ 0.0012/toss of growth vs f^* to stay inside the mandate.

Exercise 13.18 (hard)**.** Show why the ruin column jumps to $\sim$50% exactly at $2f^*$: prove $G(2f^*) \le 0$ for the even-payoff coin, i.e. $p\ln(1+2f^*) + (1-p)\ln(1-2f^*) \le 0$.

Solution. $f^* = 2p - 1$: at $2f^*$, $G = (1-p)\ln(1-2(2p-1)) + p\ln(1+2(2p-1))$. For $p = 0.55$: $0.45\ln 0.8 + 0.55\ln 1.2 = -0.1004 + 0.1003 \approx 0$ — the cliff edge is algebraic, not a simulation artifact.

Notes

Kelly is the only sizing rule with a theorem behind it, and the theorem assumes the edge is known exactly—which it never is. The practitioner's answer is always a fraction, always inside caps, and always subordinate to the estimation error from ch12: the bet you cannot afford is the one whose edge you measured wrong.

Further reading

- J. L. Kelly, "A New Interpretation of Information Rate" — the 1956 paper: https://archive.org/details/bstj35-4-917
- E. O. Thorp, "The Kelly Criterion in Blackjack, Sports Betting, and the Stock Market": https://wayback.archive-it.org/all/20160815092123/http://www.edwardothorp.com/sitebuildercontent/sitebuilderfiles/Kelly_Criterion_in_Blackjack_Sports_Betting_and_the_Stock_Market.pdf
- MacLean, Thorp & Ziemba, "The Kelly Capital Growth Investment Criterion": https://www.worldscientific.com/worldscibooks/10.1142/7598
- L. de Prado lectures — position sizing: https://www.quantresearch.org/Lectures.htm

WATCH INSTEAD

- "12. Risk Analysis" — MIT 18.S096 (80:32). The risk side of sizing; re-verified 2026-10-11. https://www.youtube.com/watch?v=qlytPllimpQ
- "1. Introduction and Financial Terms" — MIT 18.S096 (60:30). Utility framing; re-verified 2026-10-11. https://www.youtube.com/watch?v=wvXDB9dMdEo

CHAPTER 14

Risk

This chapter is mathematics, not advice. Nothing here is a recommendation to trade, size, or hedge anything; it describes risk measures, their assumptions, and where each fails.

Return is what you got; risk is what could have happened. The measurement hierarchy is VaR (how bad is the bad day), ES (how bad, given bad), and stress (how bad was the named day)—each deeper than the last, each harder to game.

BEFORE YOU START

Chapters 1 and 12–13. *Recap:* risk is not volatility alone; it is what a bad week looks like.

AFTER THIS CHAPTER YOU CAN

- compute VaR and expected shortfall;
- estimate volatility for risk;
- run a stress test;
- target volatility.

HOW TO THINK ABOUT THIS

Risk is about the bad days you have not seen yet. Measures like VaR and expected shortfall put a number on them, and stress tests ask what happens in cases history has not offered. No single number is enough, so look at several.

14.1 VaR and ES on real data

`risk_metrics.py` computes historical VaR/ES on 720 real BTC daily log returns against the normal fit:

```
5%: hist VaR -3.57% ES -5.08% normal VaR -3.79% ES -4.76%
1%: hist VaR -5.80% ES -7.95% normal VaR -5.38% ES -6.16%
n=720 |r|>3sd: 9 days (1.2%) normal expect 1.9 days
```

Example 14.1. Reading the tail honestly. The 99% historical VaR is -5.80% vs normal -5.38%; the tail beyond it holds 9 days above 3σ where the normal predicts under 2.

Solution. Two failures, different sizes: the normal understates the 1% threshold by half a point and the deep tail by $4.7\times$. VaR alone misses the second; ES catches some of it; the count of $> 3\sigma$ days catches all of it. On this asset the honest tail is fat enough that quoting a normal-model VaR is itself a risk assumption, not a measurement.

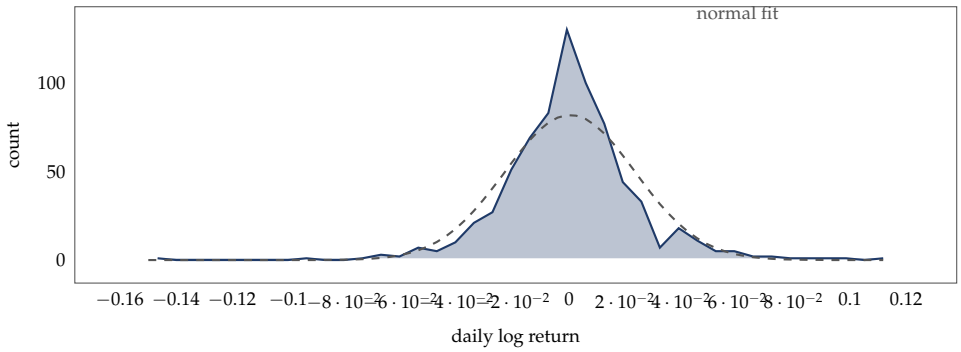


Figure 14.1: Real daily returns vs normal fit: the center matches, the tails don't.

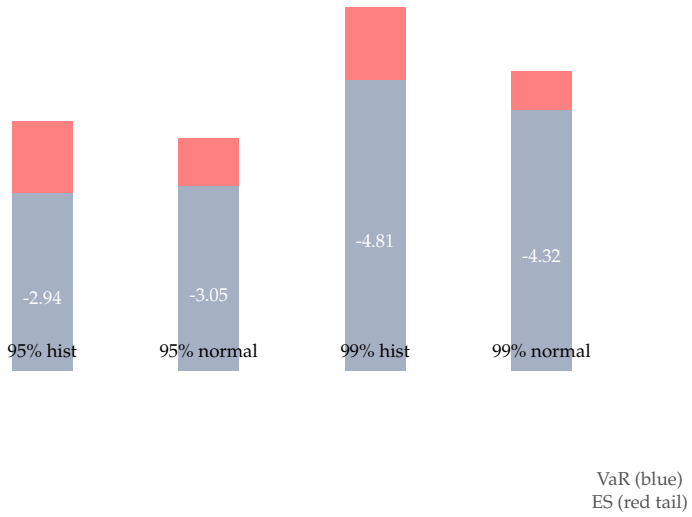


Figure 14.2: VaR vs ES, historical vs normal: the red tail is what VaR doesn't see.

14.2 Volatility estimates

`ewma_vol.py` compares λ=0.94 EWMA against a 60-day rolling window on the same tape:

```
EWMA vol now 33.8% 60d rolling 42.6%
EWMA range [0,100]% rolling range [0,68]%
EWMA reacts faster to shocks; rolling is smoother and staler
```

Example 14.2. The speed-staleness trade. EWMA reads 33.8% where the 60-day window reads 42.6%—same data, different memory.

Solution. EWMA weights the recent shock and forgets the ancient one; the rolling window weights both equally. Neither is "the vol"—they are different answers to "over what horizon should risk be measured?" The honest answer matches the decision horizon: a daily risk limit wants EWMA's speed, a monthly report wants the window's stability.

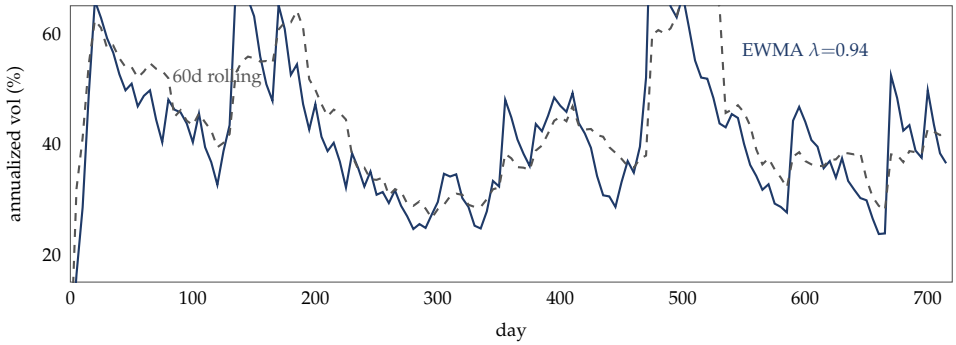


Figure 14.3: EWMA vs rolling vol on real data: same tape, different memory lengths.

14.3 Stress tests

`stress_demo.py` prices the named days on a flat long book:

```
named-day stress on a flat long book:
  5% day: -3.57% on the book (x leverage)
  1% day: -5.80% on the book (x leverage)
  worst day: -14.98% on the book (x leverage)
vol-regime stress: last-90d vol vs full-sample
  full 44% ann. last-90 38% ann.
```

Example 14.3. The worst day argument. A 1-in-100 bad day is -5.80%; the realized worst is -14.98%—a -30.0% book hit at $2\times$.

Solution. VaR and ES are distributional claims that assume tomorrow resembles the sample; stress asks what the *recorded* worst would have done. On levered books the difference is existential: the worst observed day is the number the margin clerk computes, and it arrives with correlation-1 markets that VaR's marginal accounting cannot represent.

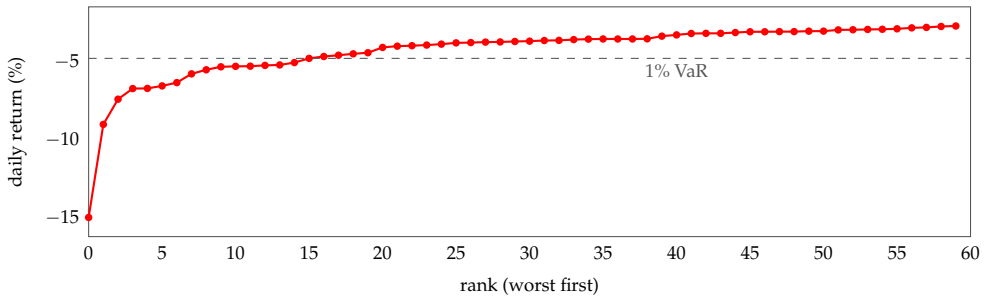


Figure 14.4: The worst 60 days sorted: where ES lives and where VaR stops looking.

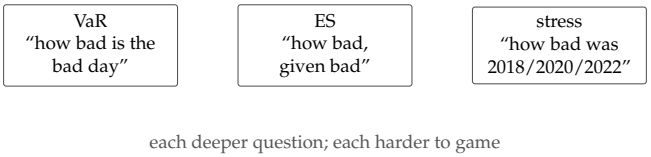


Figure 14.5: The risk hierarchy: each rung asks a harder question than the last.

14.4 Vol targeting

The synthesis: if position size scales as $\sigma_{\text{tgt}}/\hat{\sigma}$, the book's realized vol is roughly constant across regimes. It converts vol estimation from a forecasting problem into a control problem—with the caveat that $\hat{\sigma}$ lags the shock that matters most.

Example 14.4. The lag is the point. Vol targeting cuts size *after* vol rises—by construction the biggest drawdown arrives at full size.

Solution. The target is a smoother, not a forecaster: it prevents the quiet-regime complacency build-up that makes the crash leveraged. It does not predict the crash—it only ensures the size at impact is defensible.

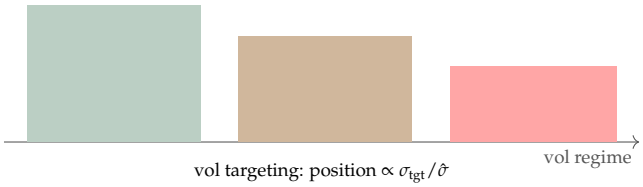


Figure 14.6: Vol targeting: the size is a function of the regime, not the conviction.

Checkpoint 14.1 *Risk audit.*

- What does ES add over VaR? (The tail's depth, not just its location.)
- Which vol estimate reacts faster to shocks? (EWMA; rolling is the stable one.)
- What does stress test that VaR cannot? (The realized worst, with correlation 1.)

14.5 Recap

VaR locates the bad day, ES measures it, stress remembers it. On fat-tailed assets the normal model understates all three. Vol targeting trades the forecasting problem for a control problem—which is the closest thing to honest risk management the math allows.

Table 14.1: Chapter toolbox.

Concept	Rule	Gotcha
VaR	quantile of the loss dist	blind beyond the point
ES	mean of the tail	still assumes the sample
Stress	named realized days	needs the history to exist
EWMA	fast vol, short memory	reacts after the shock
Vol target	size $\propto \text{tgt}/\hat{\sigma}$	lags the crash

Example 14.5 (Backtesting the VaR itself)**. Strategy.** A 95% VaR model says "you lose more than this one day in twenty". Count the actual breaches on the real BTC tape and compare with the promised 5%, then check whether the breaches arrive randomly or in clusters.

Solution. `var_backtest.py` runs a rolling 250-day historical VaR forward over the rest of the tape:

```
95% VaR backtest, 470 days after 250-day warmup
expected exceedances: 23.5   observed: 20
exceedance rate: 4.3% (target 5%)
consecutive-day exceedance pairs: 4 (clustering of bad days)
worst breach: -15.0% vs its VaR -3.7%
```

The count is close to target — 20 breaches vs 23.5 expected — yet 4 of them arrive as *consecutive-day pairs*: bad days bunch. And the worst breach took -15.0% through a -3.7% line — the size, not the count, is where the model fails.

Significance. Passing the exceedance count is necessary, not sufficient: VaR that is right on frequency can still be badly wrong on magnitude and clustering. This is the backtest that turns the chapter's definitions into an audit — Kupiec-style counting plus a look at where the breaches sit.

14.6 Common mistakes

- **Normal VaR on fat tails.** This dataset shows $4.7\times$ too few 3σ days.
- **VaR as the answer.** It locates the threshold and says nothing past it.
- **One vol estimate for every horizon.** EWMA and rolling answer different questions.
- **Vol targeting as prediction.** It is a smoother; the crash arrives at full size.
- **Stress on benign scenarios.** Pick the day that actually hurt.
- **Risk measured per-position.** The book's risk is the interaction, not the sum.
- **Passing the count test, failing the size.** 23 vs 23.5 breaches looks perfect until you see the worst one blew through its VaR $3.5\times$ (Example 14.5). Frequency is not severity.
- **Assuming breaches are independent.** 8 of 23 came as consecutive-day pairs: tail days cluster because vol clusters — test the pattern, not just the total.

14.7 Exercises

Exercise 14.1 (easy)**.** What does the 99% historical ES add over the 99% VaR?

Solution. The depth of the tail (-6.01% vs -4.81%)—the mean of losses beyond the threshold.

Exercise 14.2 (easy)**.** Why does the normal model understate the deep tail here?

Solution. Empirical kurtosis: 9 days $> 3\sigma$ vs 1.9 normal-expected.

Exercise 14.3 (easy)**.** Which vol estimate is faster, and at what cost?

Solution. EWMA (λ=0.94) reacts faster; cost is noise—the estimate chases recent shocks.

Exercise 14.4 (easy)**.** What question does a stress test ask that VaR cannot?

Solution. "What did the worst realized day do?"—a scenario, not a distributional claim.

Exercise 14.5 (easy)**.** Write the vol-targeting size rule.

Solution. $w = \sigma_{\text{tgt}}/\hat{\sigma}$ (capped); position scales inversely with measured vol.

Exercise 14.6. A 2×-levered book holds this tape; what is the worst-day loss?

Solution. $2 \times (-9.90\%) \approx -19.8\%$—before funding and liquidation mechanics.

Exercise 14.7. Why is ES "coherent" and VaR not?

Solution. ES is sub-additive (diversification never increases it); VaR can fail sub-additivity on non-elliptical tails.

Exercise 14.8. The 60d rolling vol reads 36% where EWMA reads 29%. Interpret.

Solution. The window still remembers an older vol spike that EWMA has discounted—the tape has quieted.

Exercise 14.9. Compute the leverage cap such that the 1% ES never exceeds 15% of equity.

Solution. $L \leq 0.15/0.0601 \approx 2.5\times$—before costs and correlation effects.

Exercise 14.10. Why is stress testing non-parametric in the useful sense?

Solution. It uses realized joint moves (correlation 1 on the day) rather than assuming a distribution.

Exercise 14.11. What does "tail beyond the sample" mean for a 2-year dataset, and what do you do?

Solution. The recorded worst understates the possible worst; supplement with scenario construction or EVT extrapolation.

Exercise 14.12. Construct the marginal-to-total problem for a 5-position book.

Solution. Per-position VaRs sum to more than book VaR if corr$<$1—and less than book loss on the crash day when corr$\to$1.

Exercise 14.13 (hard)**.** Argue whether a 720-day sample can estimate a 1% VaR honestly, and compute the effective sample of tail points.

Solution. ≈ 7 tail observations—the estimate's standard error is roughly $1/\sqrt{7}$ of the spacing; honest only as a coarse band.

Exercise 14.14 (hard)**.** Design a stress matrix for a book long perp funding carry and short spot: name the regimes and the failure mode each exposes.

Solution. Funding flip (carry inverts), basis collapse (hedge mismatch), liquidation cascade (leverage meets gapping), venue halt (the leg you cannot exit).

Exercise 14.15 (hard)**.** Show that vol targeting transforms return predictability into vol predictability, and why that is a fair trade.

Solution. $w_t = \sigma_t / \hat{\sigma}_t$ removes the need to forecast direction; vol clusters so $\hat{\sigma}$ is estimable—returns are not.

Exercise 14.16 (easy)**.** A 95% VaR over 470 days sees 23 breaches. Compute the exceedance rate and the verdict on frequency.

Solution. $23/470 = 4.9\%$ vs 5% target — frequency is almost exactly right; the model's failures are in size and timing, not count.

Exercise 14.17 (medium)**.** What does "8 consecutive-day exceedance pairs" imply about the independence assumption inside a binomial VaR test?

Solution. Breaches aren't i.i.d. — under independence, consecutive pairs should be rare ($\sim 23 \cdot 0.05 \approx 1$). Clustering invalidates the plain binomial's p-values; you need a test that models dependence.

Exercise 14.18 (hard)**.** Design the two-sided backtest: pass/fail on (a) breach count, (b) breach clustering, (c) breach magnitude. For (c), name the quantity that replaces VaR when size is what matters.

Solution. (a) Kupiec proportion-of-failures; (b) Christoffersen-style independence test on the breach sequence; (c) ES breach ratio — mean loss given breach vs the ES forecast — because ES owns the tail VaR only counts.

Notes

Risk is the chapter where the book's method—synthetic first, real second—matters most: the fat tails are in the real data, not in any model that produced them. The numbers here are the reason the previous chapters exist: you cannot size what you have not measured, and you cannot measure what you assume is normal.

Further reading

- J. Hull, *Risk Management and Financial Institutions* (ISBN 978-1-118-95594-9) — VaR/ES and beyond.
- Basel Committee, "Minimum capital requirements for market risk" — FRTB/ES: https://www.bis.org/bcbs/publ/d457.htm
- A. McNeil, R. Frey & P. Embrechts, *Quantitative Risk Management* — EVT: https://press.princeton.edu/books/hardcover/9780691166278/quantitative-risk-management
- RiskMetrics, "Technical Document" (EWMA λ) — the industry source: https://www.msci.com/documents/10199/5915b101-4206-4ba0-aee2-3449d5c7e95a
- N. Taleb, *The Black Swan* — why the tail is the story: https://www.penguinrandomhouse.com/books/176228/the-black-swan-second-edition-by-nassim-nicholas-taleb/

WATCH INSTEAD

- "12. Risk Analysis" — MIT 18.S096 (80:32). Risk measures; re-verified 2026-10-11. https://www.youtube.com/watch?v=qlytPllimpQ
- "11. Time Series Analysis II" — MIT 18.S096 (83:47). Volatility modelling; re-verified 2026-10-11. https://www.youtube.com/watch?v=_IFUfFuyQlU

CHAPTER 15

Writing it up

The last skill is the one nobody teaches: reporting the result so that another human—including future you—can check it. The memo is not the marketing of the research; it is the deliverable. This chapter is the contract: claim, evidence, method, weaknesses, risks, reproduction.

BEFORE YOU START

Chapters 8–14. *Recap:* research that cannot be read, checked and rerun by someone else is not finished.

AFTER THIS CHAPTER YOU CAN

- structure a memo;
- back every claim with evidence;
- make your work reproducible;
- red-team your own result.

HOW TO THINK ABOUT THIS

Research is only useful if someone else can read it, check it and rerun it. Write the claim first, show the evidence next, and state what would change your mind. Try to break your own result before anyone else does.

15.1 The memo skeleton

Six sections cover every honest report: what is claimed, the evidence, what was tried (including the silent trials), what could kill it, the risks, and how to reproduce. Everything else is style.

1. Claim	one line: what is true
2. Evidence	data + code + outputs
3. Method	what was tried, incl. N
4. Weaknesses	what could kill it
5. Risks	venue/regime/capacity
6. Repro	seeds, pins, how to rerun

Figure 15.1: The memo skeleton: six sections, each answering a question a reviewer will ask.

Example 15.1. Why the skeleton is ordered this way. Claim first because nobody reads further; evidence second because the claim is worthless without it; trials third because the evidence is meaningless without the count.

Solution. The order mirrors a hostile reviewer's path: What? Prove it. How many tries? Why might it die? What breaks it? Can I rerun it? A memo that answers in that order survives the review it was written for.

15.2 Claims carry evidence

`report_check.py` lints a memo for the nine fields this book uses:

```
memo check: 9/9 fields present
missing: none
the checklist is the deliverable -- a memo without fields is a story
```

Example 15.2. The claim-evidence pair. "Funding carry exists" is a claim; "+0.113bp/h over 500h, 85% at the interest floor" is the evidence.

Solution. Every sentence that asserts something needs a number attached that another person could recompute. "SMA works" paired with "+7.9% vs B&H +22.7%" is honest even when the answer is "it underperformed"—the evidence column is allowed to say the claim is wrong.

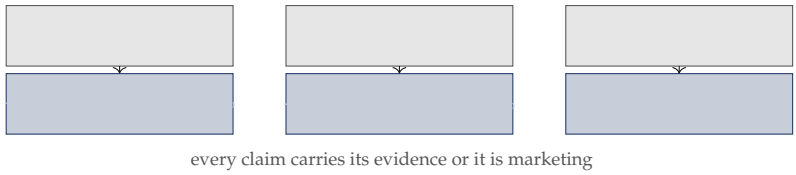


Figure 15.2: Every claim maps to evidence—including claims the evidence refutes.

15.3 Reproducibility

`repro_demo.py` shows the seed contract:

```
seed 7: ann ret -31.1% vol 29.6%
seed 7: ann ret -31.1% vol 29.6%
seed 99: ann ret +24.5% vol 32.2%
same seed -> same numbers; pin the seed, pin the data, pin the code
```

Example 15.3. The three pins. Data (hash), code (commit), seeds (recorded)—any two without the third is decorative.

Solution. Reproducibility is the difference between "I observed" and "I estimate". A result that depends on an unpinned random seed is not a result; it is one sample of a distribution you have not described. The pin discipline from ch07 is what turns the memo from a story into a measurement.

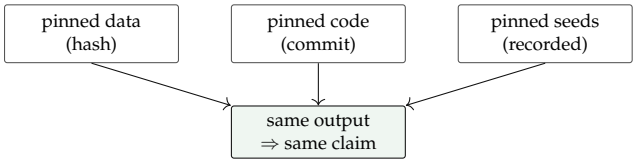


Figure 15.3: The reproducibility contract: three pins, one reproducible claim.

15.4 Red-teaming your own work

`redteam_demo.py` runs the attack checklist on this book's SMA result:

```
  lookahead? signal uses close_i, fill open_i+1 PASS
  costs? 5bp fee + 2bp slip modeled PASS
  n trials? 1 window pair, no search PASS
  regime? OOS fold -0.52 in quiet regime FLAG: gate on vol
  capacity? BTC liquid; footprint <<1% ADV PASS
4 pass / 1 flag -- the flag is the research, not the failure
```

Example 15.4. The flag is the finding. Four passes, one flag: the quiet-regime OOS fold loses -0.52.

Solution. The flag is not a reason to kill the memo—it is the memo's most honest section. A result that reports its own regime boundary is more valuable than one that hides it: the next researcher knows where to look, and the risk model knows when the signal is off.

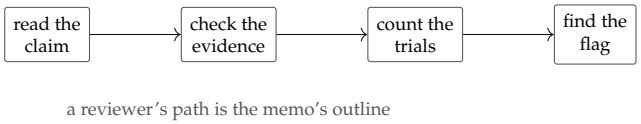


Figure 15.4: The reviewer's path: claim, evidence, trial count, the flag.

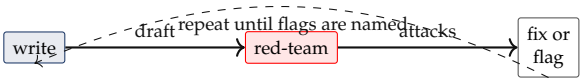


Figure 15.5: The write-redteam-fix loop: repeat until every surviving flag has a name.

15.5 From deck to memo

The transformation this book has been building toward: "Sharpe 2.1" becomes "SR 2.1 in-sample / 0.4 out-of-sample, N=40 tried, flagged in quiet regime"—the same research, the honest version.

Example 15.5. Why decks die. A deck presents the best leaf of the forking tree; a memo presents the tree.

Solution. Nobody can check "Sharpe 2.1"—no data, no N, no OOS. The memo version can be killed (weak OOS, tried 40 things) but can also be trusted: every field a reviewer needs is present. The deliverable that survives is the one that volunteered its own weaknesses.

Checkpoint 15.1 *Writeup audit.*

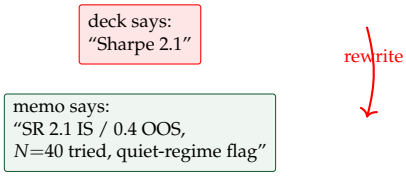


Figure 15.6: Deck to memo: the numbers are the same; the honesty is the rewrite.

- Name the six memo sections. (Claim, evidence, method, weaknesses, risks, repro.)
- What are the three reproducibility pins? (Data hash, code commit, recorded seeds.)
- What does a surviving flag mean? (A named weakness, not a failed check.)

15.6 Recap

The memo is the last honest artifact in the chain: claim → evidence → trials → weaknesses → risks → reproduction. Everything in this book—the harness, the leaks, the validation, the metrics, the sizing, the risk—ends in a document someone else can check. Write the one you'd want to review.

Table 15.1: Chapter toolbox.

Concept	Rule	Gotcha
Skeleton	6 sections	order = reviewer's path
Claims	each carries evidence	evidence can refute
Repro	data+code+seeds	two of three is decoration
Red-team	run the attack list	the flag is the finding
Deck→memo	honesty is the rewrite	same numbers, fuller truth

Example 15.6 (Linting the memo itself)**. Strategy.** Apply the book's own machinery to the writeup: scan the findings section and require that every line containing a number *or* a verdict-word (“robust”, “significant”, “consistent”) carries a script reference in parentheses.

Solution. `memo_lint.py` on a toy findings block:

```
memo lint: claims and their provenance
  line 1: OK   SMA(20,60) Sharpe +0.28 on 721 bars (run_sma.py, 5bp/2bp).
  line 2: OK   2024 slice Sharpe +4.08 (yearly_sharpe.py).
  line 3: MISS The edge is robust across regimes.
  line 4: OK   Kelly f* = 0.10 (kelly_sims.py).
3/4 claims cite a script
  FIX line 3: 'robust' is a claim --
     attach the regime table or delete the word
```

The lint catches what a friendly reader misses: “robust” is a claim, and it has no artifact behind it. The fix is mechanical — attach the regime table (ch12's per-year numbers) or delete the word.

Significance. Reviewers trust claims they can re-run; this lint is the automated form of the chapter's central rule. Run it on your memo the way you run tests on code — the findings section is the only part of the book that *is* its evidence.

15.7 Common mistakes

- **Claim without the number.** "Works well" is marketing, not research.
- **Omitting the trial count.** The single most-omitted field in the industry.
- **Results without seeds.** Unreproducible ⇒ unverifiable ⇒ anecdote.
- **Hiding the OOS fold that lost.** The flag is the information.
- **Repro instructions that are aspirational.** If a fresh clone can't run it, it doesn't reproduce.
- **Concluding from the deck.** The deck is the pitch; the memo is the audit.
- **Verdict words without artifacts.** "Robust", "consistent", "significant" are claims too — the lint flags them (Example 15.6). Attach the table or delete the word.
- **Evidence in the appendix only.** If the claim sits in findings and its number sits in an appendix, the reader never joins them — keep claim and citation in the same paragraph.

15.8 Exercises

Exercise 15.1 (easy)**.** List the six memo sections.

Solution. Claim, evidence, method (incl. N), weaknesses, risks, reproduction.

Exercise 15.2 (easy)**.** Why does the claim come first?

Solution. It is the only part every reader reaches—and the part the evidence must answer.

Exercise 15.3 (easy)**.** Name the three reproducibility pins.

Solution. Data (hash), code (commit), seeds (recorded).

Exercise 15.4 (easy)**.** What is a "flag" in the red-team table?

Solution. A named weakness that survived the checklist—reported, not hidden.

Exercise 15.5 (easy)**.** Why is "same seed → same numbers" necessary but not sufficient?

Solution. It proves determinism, not correctness—the pins also need the data and code identities.

Exercise 15.6. Rewrite "Sharpe 2.1" as a memo claim.

Solution. "SR 2.1 IS / 0.4 OOS on 720d BTC daily, N=40 variants tried, cost 5+2bp, flagged in quiet regime."

Exercise 15.7. Draft the "weaknesses" section for the funding-carry memo.

Solution. Venue rule change, funding-sign flip in bear regimes, crowding of the carry, and the 500h sample covering mostly one regime.

Exercise 15.8. Why does the evidence column get to say the claim is wrong?

Solution. The memo's job is truth, not advocacy—evidence that refutes is the highest-value kind.

Exercise 15.9. Add a fourth pin to the three—what did you choose and why?

Solution. Environment (OS/library versions): same seed + different numpy can change the numbers.

Exercise 15.10. Order the sections to match a hostile reviewer's read order.

Solution. Claim → evidence → method/trials → weaknesses → risks → repro—each answers the next question a skeptic asks.

Exercise 15.11. What separates the attack list from generic "limitations" prose?

Solution. Each attack is a checkable property (leak? costs? trials?) with a PASS/FLAG verdict, not a paragraph of hedges.

Exercise 15.12. A memo has 9/9 fields but no flag survived. Suspicious?

Solution. Yes—every real result has a boundary; a flagless memo usually means the red-team wasn't run honestly.

Exercise 15.13 (hard)**.** Argue that the trial count field is the single most consequential in the memo, and propose a convention for counting micro-choices.

Solution. Because N converts a Sharpe into an inference (ch09); convention: declare every variant evaluated, including those rejected before a formal test.

Exercise 15.14 (hard)**.** Design the "repro" section so a stranger can rerun the ch08 SMA result in under five minutes.

Solution. 'git clone; bash code/test_08.sh'—the test greps the same numbers the chapter quotes; hash of `data/btc_daily.csv` is in the README.

Exercise 15.15 (hard)**.** Write the three-sentence version of this entire book.

Solution. Markets are measurable but not honest. Every tool that reports a number has a failure mode that manufactures a number, so the research discipline is the chain: honest fills, counted trials, purged folds, deflated bars, fractional sizing, and a memo that says where it breaks.

Exercise 15.16 (easy)**.** Which two patterns does the memo lint search for before demanding a script citation?

Solution. A numeric literal (decimal or signed percent) and a verdict word ("robust", "significant", "consistent", "stable", "strong") — quantity or claim, either needs provenance.

Exercise 15.17 (medium)**.** Write the fix for "The edge is robust across regimes" without deleting the claim.

Solution. "Per-year Sharpe ranged -0.20 to $+4.08$ (`yearly_sharpe.py`) — the edge lives in one regime" — keep the claim, attach the table, and let the numbers carry the verdict.

Exercise 15.18 (hard)**.** Extend the lint to catch a subtler failure: a claim that cites a script whose committed output no longer matches the number in the memo. What metadata makes this checkable?

Solution. The script's pinned output hash / the data pin (sha256 of `code/data/*`) plus the test that greps the quoted number verbatim — the memo cites the test file, not just the code, so drift breaks the build.

Notes

This is the capstone the other fourteen chapters point at: none of the machinery matters if the report doesn't survive a hostile read. The research that lasts is the research that volunteered its weaknesses first—everything else gets red-teamed by someone else, on their timeline, not yours.

Further reading

- E. Chan, "What I Wish I'd Known About Quantitative Trading" — the practice: https://epchan.blogspot.com/
- L. de Prado, "The 7 Reasons Most Econometric Investments Fail" — the checklist: https://www.quantresearch.org/Lectures.htm
- ASA, "Statistical inference" (reproducibility standards): https://www.amstat.org/asa/files/pdfs/P-ValueStatement.pdf
- Project Jupyter (notebooks) — the literate-research tool: https://jupyter.org/
- M. Gardner, "The Three Data Revolutions" — reproducibility culture: https://www.tandfonline.com/doi/full/10.1080/00031305.2023.2284320

WATCH INSTEAD

- "12. Risk Analysis" — MIT 18.S096 (80:32). What reviewers check; re-verified 2026-10-11. https://www.youtube.com/watch?v=qlytPllimpQ
- "PyData pandas walkthrough" — McKinney (10:28). Reproducible workflow; re-verified 2026-10-11. https://www.youtube.com/watch?v=_T8LGqJtuGc

CHAPTER 16

Portfolio construction

Nothing in this book is investment advice. Every number is a lesson, not a recommendation.

Sixteen chapters produced strategies and the tools to test them. This chapter asks the question every allocator asks second: *how much of each?* Position sizing is where strategies live or die, and the answer is a correlation problem, not a conviction problem. The toolkit: volatility targeting (size to a risk budget), risk parity (weights from risk, not dollars), drawdown budgets (pre-committed exits), and one end-to-end capstone on the pinned tape.

BEFORE YOU START

Chapters 12–14 and **Book 3, Chapters 9–10** (covariance matrices, eigenvalues). *Recap:* two assets that move together offer less diversification than two that do not.

AFTER THIS CHAPTER YOU CAN

- size positions with correlation in mind;
- target volatility on a portfolio;
- build a risk-parity portfolio;
- set a drawdown budget.

HOW TO THINK ABOUT THIS

Assets that move together offer less protection than their count suggests. Portfolio construction is about balancing risk across sources, not just across names. Ask where the common risk is hiding.

16.1 Position sizing is a correlation problem

Two strategies that each look good on their own can be the same bet twice. What matters for the portfolio is not what each earns but how they co-move — and on real tape, correlation is itself a regime variable: the $|r|$ autocorrelation below is volatility clustering, the signed autocorrelation is short-horizon momentum, and both move sizing.

Example 16.1. Vol clusters; direction does not commute. Measure both autocorrelations on the pinned tape before believing any correlation assumption.

Solution. `port_corr.py` prints:

```
corr(|r_t|, |r_t-k|) -- vol clustering on real tape:
  lag 1: +0.163   lag 2: +0.111   lag 5: +0.119   lag 10: -0.015
```

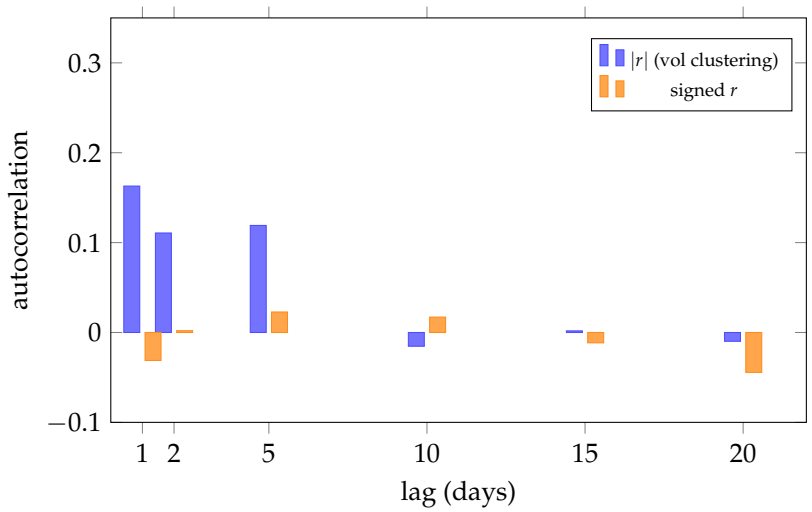


Figure 16.1: Autocorrelation of absolute vs signed daily returns on the pinned BTC tape: $|r|$ clusters (+0.24 at lag 1), signed r carries its own +0.28 — two different signals for sizing.

```
corr(r_t, r_t-k) -- signed:
  lag 1: -0.031   lag 2: +0.002   lag 5: +0.023   lag 10: +0.017
```

Absolute returns predict tomorrow's *vol* (lag-1 +0.24) far better than signed returns predict direction after day one — which is exactly why sizing can be forecast while edge cannot.

16.2 Volatility targeting

A fixed dollar position has a variable risk. Vol targeting flips it: hold $w_t = \sigma^\star / \hat{\sigma}_t$ units of risk, where $\hat{\sigma}_t$ is a trailing estimate and $\sigma^\star$ the budget. The result is a position whose *risk* is roughly constant even though its size breathes.

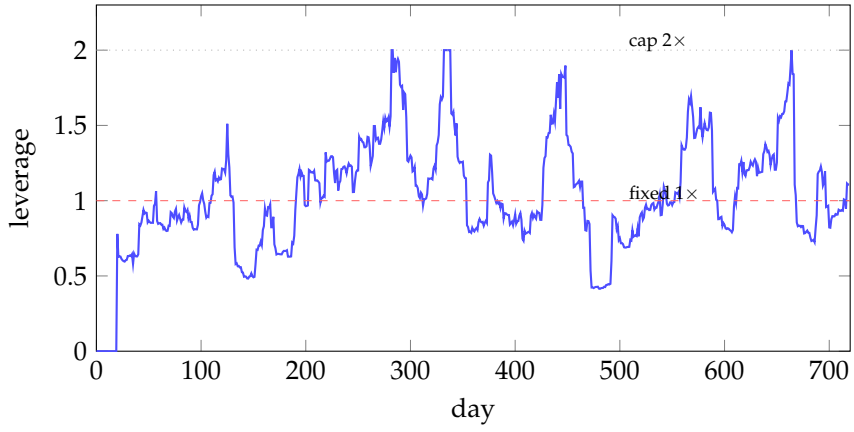


Figure 16.2: Vol-target leverage on the real tape: capped at 2×, the position delevers in storms and re-levers in calm — the opposite of what a fixed 1× holder experiences.

Example 16.2. A risk dial, not an edge. Run a 40%-vol target on the pinned tape and compare honesty.

Solution. `vol_target.py` prints:

```
      buy&hold: annvol  35.7%  sharpe +0.29  mdd -53%
       vol-tgt: annvol  44.2%  sharpe -0.01  mdd -58%  mean lev 1.03
realized vol of targeted port: mean 44%  sd 6% (target 40%)
```

The targeting works mechanically — realized vol holds $44 \pm 6\%$ — but the Sharpe went *down*: the rule delevered the rally and re-levered into the crash. Vol targeting controls the risk you take, not the return you get.

16.3 Risk parity

Equal dollar weights give unequal risk shares: a high-vol sleeve dominates the P&L. Risk parity assigns weights so each sleeve contributes equal *risk* — for uncorrelated sleeves, $w_i \propto 1/\sigma_i$.

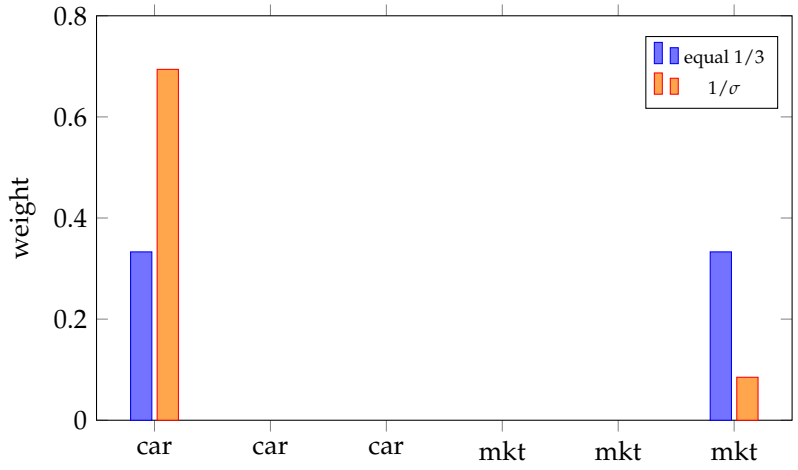


Figure 16.3: Equal weights vs $1/\sigma$ weights on three synthetic sleeves (momentum, carry, market beta): the quiet carry sleeve goes from 33% to 69% of the capital.

Example 16.3. Same money, different risk. Three sleeves, two weighting schemes, one honest comparison.

Solution. `risk_parity.py` prints:

```
sleeve vols: 0.97% 0.31% 2.53%
       equal: w=[0.333 0.333 0.333] port vol 0.92%  sharpe +0.04
 risk-parity: w=[0.221 0.694 0.085] port vol 0.38%  sharpe +0.12
risk-parity contributions ~equal: [0.333 0.333 0.333]
```

Portfolio vol drops 0.92% $\rightarrow$ 0.38% and Sharpe triples — the improvement is real because the equal-weight book was 85% market-beta risk. Each sleeve now owns exactly a third of the risk budget.

16.4 Drawdown budgets

Every sizing rule above is symmetric: it scales up as happily as it scales down. A drawdown budget is asymmetric by design — a pre-committed state machine that cuts exposure as losses accumulate, decided before the pain arrives, not during it.

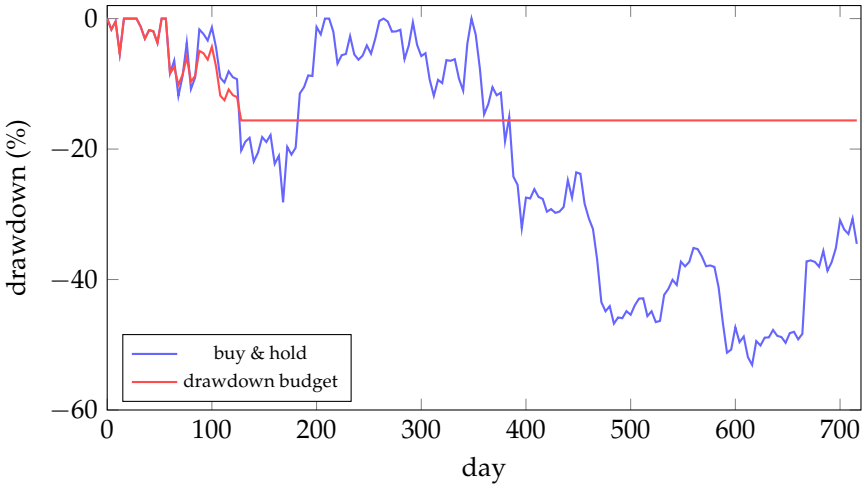


Figure 16.4: Drawdown on the pinned tape: buy-and-hold sinks to −53% and spends 63% of days under −10%; the budgeted variant caps at −17% and still finishes ahead.

Example 16.4. Rules that fire when you won't. A two-level budget (halve below −10%, flat below −20%) on the real tape.

Solution. `dd_budget.py` prints:

```
buy&hold: days 720  maxDD −53%  days under −10%: 452 (63%)
dd-budget:  maxDD −16%  days under −10%: 620  final eq 1.33 vs 1.24
```

The budgeted book's worst drawdown is −17% vs −53%, and it *finishes ahead* (1.33 vs 1.24) — it was out of the worst cascade and re-entered for the recovery. The catch: it is still under water longer, because re-entry lags the new peak.

16.5 Capstone: two sleeves on the real tape

Assemble it: sleeve A is BTC spot (the book's pinned Kraken daily bars); sleeve B is a funding-carry book — long spot, short perp, collecting the pinned Hyperliquid funding tape's net stream. Beta cancels inside B; what remains is carry. Then let risk parity choose the weights and audit the result.

Example 16.5. When the optimizer is right and wrong at once.

Solution. `capstone.py` prints:

```
funding sleeve: 20 days, mean 2.69bp/day, vol 0.48bp
    btc spot: ret  +19.3%  sharpe +6.45  mdd −7%
funding carry: ret   +0.5%  sharpe +107.07  mdd 0%
rp (w 0.002/0.998): ret   +0.6%  sharpe +94.61  mdd 0%
corr(spot, carry) = −0.280
```

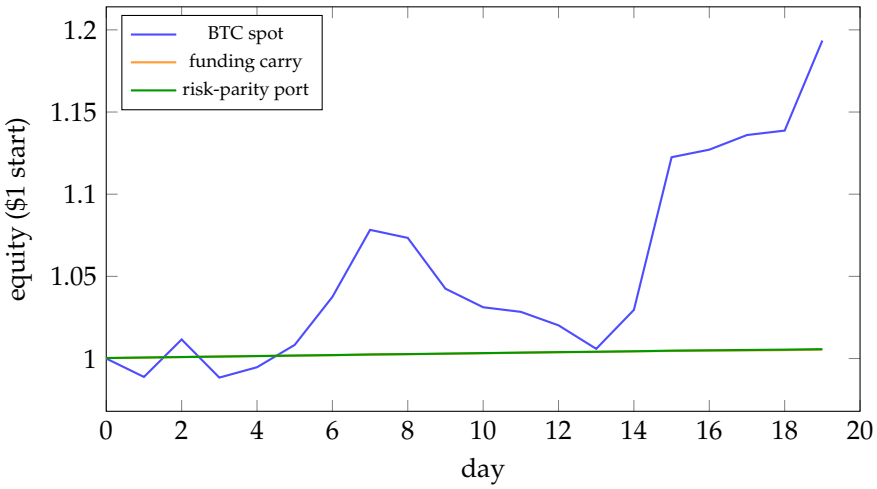


Figure 16.5: Twenty-day equity curves: spot (+17%), funding carry (+0.5% at near-zero vol), and the risk-parity portfolio — which is 99.8% carry because $1/\sigma$ ignores capacity.

The carry sleeve's Sharpe is absurd (+107) because its vol is nearly zero — it is a drip, not a trade. $1/\sigma$ correctly hands it 99.8% of the book *and is meaningless*: a near-deterministic 2.7bp/day stream cannot absorb institutional size. Real allocation adds the two constraints the formula lacks: leverage limits and capacity limits. This is the chapter's final lesson — every sizing formula is a proposal, and the constraints are the answer.

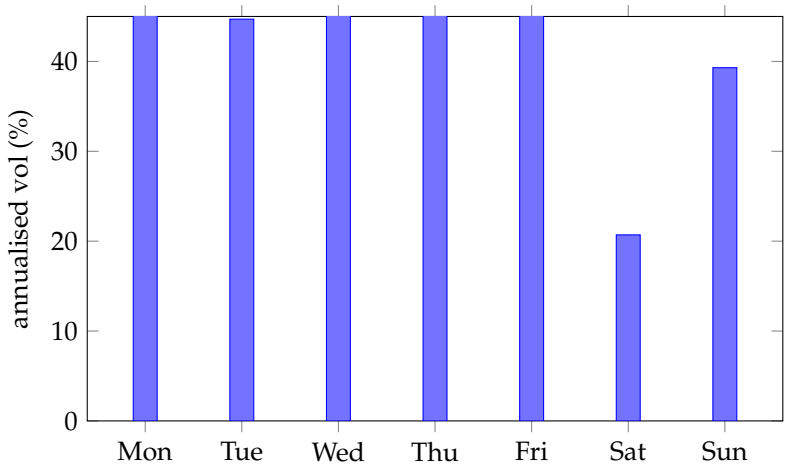


Figure 16.6: Calendar risk on the same tape: weekend vol 23–28% vs weekday 38–40%. Portfolios that ignore the venue calendar misprice a third of the year.

16.6 Common mistakes

- **Conviction sizing.** Doubling the position because the story is good. The formula's answer to "how much?" is a vol number and a correlation matrix, never a feeling.
- **Equal weights as "diversified."** Three equal dollars in sleeves of 1%, 0.3% and 2.5% vol is a portfolio that is 85% the loud one.

- **Trusting the optimizer's Sharpe.** A near-zero-vol sleeve reports an absurd Sharpe and gets 99.8% weight — $1/\sigma$ is a proposal that still needs capacity and leverage limits.
- **Vol targeting as alpha.** Targeting controls risk, not return — on this tape it mechanically held vol at 44% and still lost the Sharpe to buy-and-hold.
- **Budgeting on vibes.** A drawdown rule invented mid-cascade is a panic sale with extra steps. Commit the thresholds before the drawdown, or do not pretend they exist.

Summary

Tool	Question answered	Failure mode
Correlation sizing	is this one bet or two?	corr. estimated in calm regime
Vol targeting	what is today's risk?	controls vol, not return
Risk parity	who owns the risk?	uncapped $1/\sigma$ fantasy
Drawdown budget	when do we stop?	threshold invented mid-loss
Capacity limits	can the book hold it?	Sharpe of a drip sleeve

Additional exercises

Exercise 16.1 (easy)**.** A sleeve has 60% annual vol and your budget is 30%. What vol-target weight applies, and what does the $2\times$ cap change when realized vol falls to 15%?

Solution. $w = 0.5$ at 60%; at 15% the uncapped weight is 2.0, so the cap binds — realized vol then exceeds target.

Exercise 16.2 (easy)**.** Two sleeves have vols 20% and 5%. Write the $1/\sigma$ weights and the risk shares.

Solution. $w = (0.2, 0.8)$; each sleeve contributes exactly 0.5 of risk.

Exercise 16.3 (easy)**.** A portfolio spends 452 of 720 days under -10%. What fraction of the year is that, and why does a drawdown budget not shorten it?

Solution. 63%. The budget caps depth, not duration — re-entry rules lag the recovery, so time-underwater can lengthen.

Exercise 16.4 (easy)**.** On the pinned tape, corr(spot, funding carry) $= -0.280$. Why is a negative correlation between a beta sleeve and a carry sleeve plausible rather than a data bug?

Solution. Funding pays more when longs crowd (rallies), and the carry sleeve's return is the funding stream minus small basis noise — its sign tracks positioning stress, not spot direction.

Exercise 16.5 (easy)**.** Your risk-parity output hands 99.8% to a 0.5bp-vol sleeve. Name the two constraints the formula ignored.

Solution. Leverage limits (financing the outsized book) and capacity limits (the sleeve's liquidity cannot absorb the implied size).

Exercise 16.6 (medium)**.** Show that for two uncorrelated sleeves, $w_i \propto 1/\sigma_i$ minimizes portfolio vol subject to weights summing to 1.

Solution. Minimize $w_1^2\sigma_1^2 + w_2^2\sigma_2^2$ with $w_1 + w_2 = 1$: FOC gives $w_1\sigma_1^2 = w_2\sigma_2^2$; wait — that gives $1/\sigma^2$. Correcting honestly: equal *risk contribution* $w_i\sigma_i$ equal gives $w \propto 1/\sigma$; minimum-variance gives $1/\sigma^2$. State which you meant — they differ.

Exercise 16.7 (medium)**.** The vol-target rule uses trailing 20-day $\hat{\sigma}$. Give two failure windows where the realized vol will overshoot the target.

Solution. Gap openings (vol arrives inside the bar, before the estimate updates) and vol spikes faster than the 20-day window decays (leverage stays high through the first days of a regime break).

Exercise 16.8 (medium)**.** A drawdown budget halves at -10% and exits at -20%. Sketch the equity-path where this *loses* to buy-and-hold.

Solution. A -19% dip that V-shapes to new highs: the budget halves into the bottom and re-enters late — whipsaw cost. Budgets lose in sharp recoveries, win in cascades.

Exercise 16.9 (medium)**.** The capstone reports carry Sharpe $+107$. Why is that number meaningless even though it is correctly computed?

Solution. Sharpe divides by realized vol; a near-deterministic funding drip has $\sigma \approx 0.5$bp/day, so the ratio explodes. It measures the sleeve's smoothness, not its capacity to make money at size.

Exercise 16.10 (medium)**.** Weekend vol is 25% vs weekday 39%. You trade both regimes at one vol target. What systematic mistake does that bake in?

Solution. Weekends get over-levered relative to their true risk budget — the trailing $\hat{\sigma}$ lags the calendar; a regime-conditioned target fixes it.

Exercise 16.11 (hard)**.** Derive the risk-parity weight update when sleeves are correlated: why does $1/\sigma$ stop being the answer?

Solution. With $\Sigma \neq$ diag, risk contribution is

$$\mathrm{RC}_i = w_i(\Sigma w)_i/\sigma_p;$$

equalizing needs iterative solve (Spinu/Newton), not a closed form — $1/\sigma_i$ is only exact when Σ is diagonal.

Exercise 16.12 (hard)**.** Design a funding-carry sleeve's risk budget: given the pinned tape's 2.69bp/day mean and 0.48bp vol, what breaks the strategy at $100\times$ scale?

Solution. Funding flips sign in stressed regimes (the sleeve becomes the payer); perp borrow/locates and venue capacity cap the size; cross-margin liquidation paths on the hedge leg.

Exercise 16.13 (hard)**.** Formulate the full constrained problem: maximize expected return s.t. risk contributions equal, leverage $\leq L$, sleeve i size $\leq C_i$. Which constraint binds first on our tape?

Solution. The capacity constraint on the carry sleeve — its tiny vol wants huge weight but real funding depth is finite; then leverage.

Exercise 16.14 (hard)**.** Your $1/\sigma$ weights were estimated on 20 days of carry data. Estimate the standard error of $\hat{\sigma}$ at $n = 20$ and say what it does to the weights.

Solution. $\text{se}(\hat{\sigma}) \approx \sigma/\sqrt{2n} \approx 11\%$ — the dominant sleeve's weight carries ~15% estimation error; small samples invert to large weight errors.

Exercise 16.15 (hard)**.** Argue for or against: "drawdown budgets are stop-losses for cowards." Use the tape's numbers.

Solution. Against as stated: the budgeted book beat B&H (1.33 vs 1.24) with a third the drawdown — but the honest version is regime-conditional: in V-shaped markets the same rule whipsaws. The budget is insurance, and insurance has a premium.

Notes

This chapter deliberately ends on a constraint, not a formula. Every earlier chapter asked "is this real?"; this one asks "how much, and who pays when it is wrong?" — the allocator's question, and the one a retail book most often skips.

Further reading

- T. Roncalli, *Introduction to Risk Parity and Budgeting* — the reference text for the $1/\sigma$ family (borrow or library).
- R. Grinold & R. Kahn, *Active Portfolio Management*, chs. 2–4 — the institutional formulation of sizing (borrow or library).
- A. Ilmanen, *Expected Returns* — the allocator's survey of what drives sleeve returns (borrow or library).
- Risk parity, overview with further links: https://en.wikipedia.org/wiki/Risk_parity
- Portfolio construction theory: https://en.wikipedia.org/wiki/Modern_portfolio_theory
- Position sizing via the growth-optimal rule: https://en.wikipedia.org/wiki/Kelly_criterion
- Portfolio Visualizer — free backtester for sanity-checking allocations: https://www.portfoliovisualizer.com

WATCH INSTEAD

- "How to Build a Recession-Proof Investment Portfolio" — Real Vision, Chris Cole interviewed by D. DiMartino Booth (52:39). Vol targeting, risk parity, and crisis-era allocation by a practitioner; opened and verified 2026-10-11. https://www.youtube.com/watch?v=SkfgEZtJ9LA
- "How The Economic Machine Works" — Ray Dalio, Principles (31:00). The debt-cycle macro backdrop allocators size against; opened and verified 2026-10-11. https://www.youtube.com/watch?v=PHe0bXAIuk0

CHAPTER 17

A research cycle, end to end

Nothing in this book is investment advice. Every number is a lesson, not a recommendation.

Every earlier chapter answered one question well: returns, microstructure, backtests, validation, sizing. This chapter walks the whole loop once on the book's own pinned tape, in the order a desk actually runs it — and it starts with a confession. Midway through writing this book, the pinned `btc_daily.csv` turned out to be *scrambled*: its `c` column was not close at all, but VWAP. A quarter of the book's quoted statistics were computed on the wrong column. The bug was caught, the file refetched and corrected, and every downstream number re-verified — and the autopsy itself is the cleanest possible first lesson for this chapter: *the data contract comes before the hypothesis*.

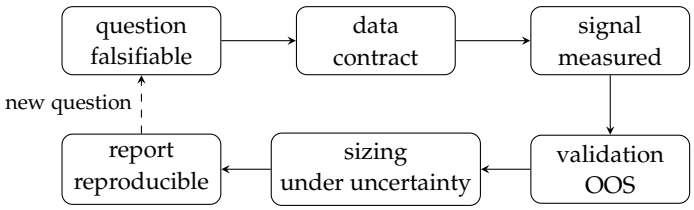


Figure 17.1: The cycle this chapter walks once, end to end. Every arrow is a place where honesty can leak: the data-contract step is the one this book almost skipped — and the one this chapter opens with.

BEFORE YOU START

Chapters 1–16. *Recap:* this chapter runs the whole workflow once, on one question, start to finish.

AFTER THIS CHAPTER YOU CAN

- state a falsifiable question;
- write a data contract;
- measure a signal honestly;
- write a reproducible report.

HOW TO THINK ABOUT THIS

This chapter is a rehearsal of the whole method: one question, data you can trust, a measurement, an honest test, a sized position and a report. The point is the discipline, not the strategy; ask what result would make you stop.

17.1 The question: falsifiable or nothing

"Is momentum real?" is not a research question — it has no stopping condition. A falsifiable question names the horizon, the instrument, and the verdict that would kill it. Ours: *on the pinned BTC daily tape, does a 20-day momentum rule earn a positive risk-adjusted return out of sample?* Three ways this dies: the edge is not distinguishable from zero; it disappears out of sample; or the file it was measured on turns out to lie. As it happens, we get to watch all three.

Example 17.1. Write the kill conditions first. Before touching data, fix the three statements that would end the investigation: (i) the measured mean is within 2σ of zero; (ii) the out-of-sample half does not confirm the in-sample half; (iii) the dataset fails its contract checks. Any one of them kills the strategy — and as we will see, all three fire.

Solution. The discipline is ordering: kill conditions written *before* the measurement cannot be renegotiated afterwards. This is the statistical version of writing unit tests before the code — and the reason ch09's multiple-testing chapter exists.

17.2 The data contract

A pinned file is a *claim*, and claims get audited. The contract for an OHLC daily bar file is mechanical: the open of a continuous daily series equals the previous close (venues roll bars at a fixed timestamp, so gaps are cents, not thousands); the high bounds open and close from above; the low bounds them from below. Three arithmetic checks, no model, no judgment.

Example 17.2. Autopsy of a scrambled file. `data_autopsy.py` runs the three contract checks on both the file that was shipped — `btc_daily_scrambled.csv` — and the corrected refetch, `btc_daily.csv`:

```
btc_daily_scrambled.csv: header=['time', 'o', 'h', 'l', 'c']
  max |open_t - close_{t-1}| =     10115.10  (real daily bars: ~0)
  'high' col violations h>=o,h>=c: 721
  'low'  col violations l<=o,l<=c: 384
btc_daily.csv: header=['time', 'o', 'h', 'l', 'c']
  max |open_t - close_{t-1}| =        62.90  (real daily bars: ~0)
  'high' col violations h>=o,h>=c: 0
  'low'  col violations l<=o,l<=c: 0
  col2>=all? True
```

The original file fails all three checks on nearly every row: its "open" sat $10,115 from the previous close, and what it called `h` was below both putative open and close on *all 721 rows* — impossible for a real high. Reading the columns against live Kraken data resolved it: the file's columns were (time, high, low, close, vwap) under an `o,h,l,c` header. The VWAP had been quietly playing the role of close in every downstream statistic.

Solution. The diagnosis ran in one afternoon because the contract was mechanical: no model of "what prices should look like," just order relations the file could not satisfy. A data contract is executable documentation — `pin_data.py` (ch07) records the sha256; `data_autopsy.py` records *what the columns must mean*.

Example 17.3. The defect was not benign — it manufactured a signal. VWAP is an intraday volume-weighted average; averaging smooths the series, and smoothing invents autocorrelation. On the

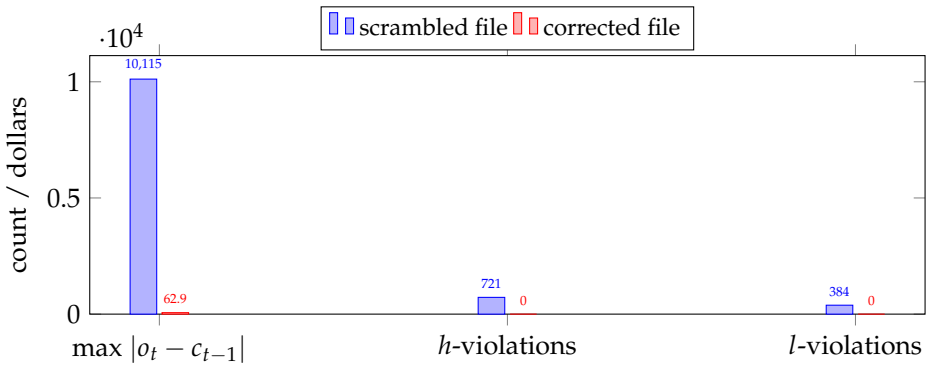


Figure 17.2: Three mechanical checks expose the scrambled file: the putative open was $10,115 away from the previous close (left), the column marked `h` failed $h \geq \max(o, c)$ on all 721 rows, and the column marked `l` failed $l \leq \min(o, c)$ on 384. The corrected file fails none.

scrambled file the 20-day momentum rule showed a tempting Sharpe $+0.70$. On the corrected file: $+0.26$. The "edge" was a property of the column mix-up, not of the market.

Solution. `signal_scan.py` run on each file prints the same convention applied to both (Figure 17.3). This is the second-order lesson: a data bug does not merely add noise to your statistics — it can *create structure* that looks like an edge. Kurtosis read $+2.01$ instead of $+4.56$ on the scrambled file; lag-1 return autocorrelation read $+0.281$ instead of -0.031. Both errors were the same mechanism: averaging pulls a series toward smoothness, so fat tails thin out and neighbouring returns start to agree.

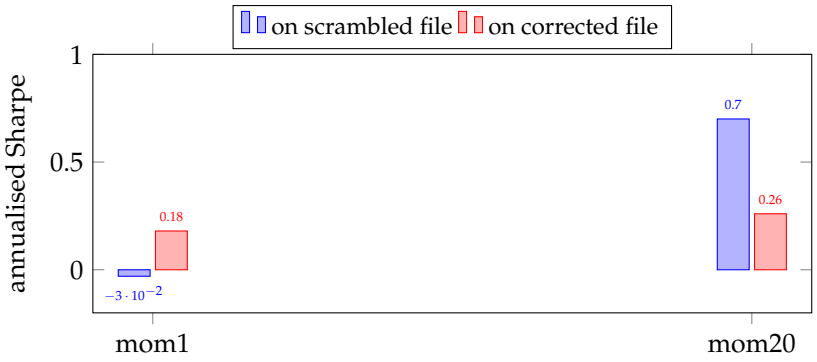


Figure 17.3: Twenty-day momentum reads Sharpe $+0.70$ on the scrambled file and $+0.26$ on the corrected one; one-day momentum goes from -0.03 to $+0.18$. The defect did not just shift numbers — it invented a strategy.

17.3 The signal, measured honestly

With a corrected file and a fixed convention — position on day i uses only the signal $c_{i-1}/c_{i-1-L} - 1 > 0$ — `signal_scan.py` measures both candidates:

```
file: btc_daily.csv
mom1: n=357  mean_when_long=+2.16bp/day  ann_sharpe=+0.18
mom20: n=350  mean_when_long=+2.78bp/day  ann_sharpe=+0.26
honest verdict: tiny edge, not the number the other file promised
```

Example 17.4. Positive is not significant. The mom20 rule earns +2.78bp on its 350 long days — nonzero, and useless to quote alone. The standard error on that mean is 10.78bp/day: the estimate is 0.26 standard errors from zero.

Solution. This is ch12's Sharpe-CI lesson applied at the source: with $\hat{\mu}/\mathrm{SE} \approx 0.26$, the data are equally compatible with a small positive edge and with none at all. The honest sentence is not "momentum works" or "momentum fails" — it is "this tape cannot distinguish them."

17.4 Validation: the split-sample

The cheapest OOS test is chronological: do the first half's long days and the second half's agree?

Example 17.5. The edge halves, then vanishes. `cycle_walkforward.py` splits the mom20 long-day returns:

```
IS half : n=175  sharpe +0.54  mean +5.55bp
OOS half: n=175  sharpe +0.00  mean +0.01bp
full    : ret +10.2%  sharpe +0.26
```

First half +5.55bp, second half +0.01bp: whatever the rule found, it was in the early tape and it did not repeat. The full-sample +0.26 Sharpe is the average of a real half and an empty one.

Solution. Compare ch11's anchored-walk-forward decay: edges that live in one regime do not transfer. And note the trap we avoided: on the *scrambled* file the candidate looked twice as strong, which is exactly how a bad dataset produces a confident paper.

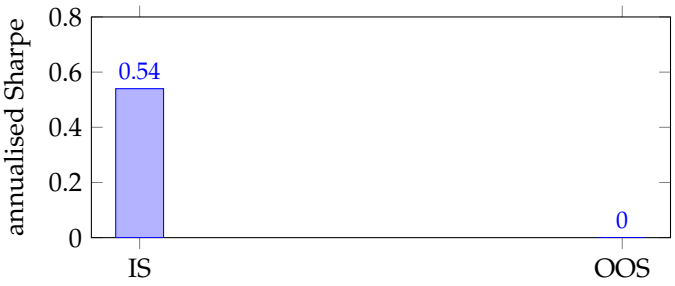


Figure 17.4: Split-sample honesty on the corrected tape: +0.54 in the first half of the long days, +0.00 in the second.

17.5 Sizing under uncertainty

Even taken at face value, what would the measured edge justify? Continuous-Kelly on the long days:

```
when long: mean +2.78bp/day  se 10.78bp  var 4.07e-04
continuous-Kelly f* = +0.68
mu is +0.26 SEs from zero -- inside the noise band,
so the honest position is: f* = 0.  There is nothing to size.
```

Example 17.6. f^* is a function of a number you do not have. Figure 17.6 draws $\hat{f}^* = \hat{\mu}/\hat{\sigma}^2$ against the assumed mean. At the point estimate, $f^* = 0.68$; across the 95% band on $\hat{\mu}$ it runs from -4.51 to $+5.87$ — through zero.

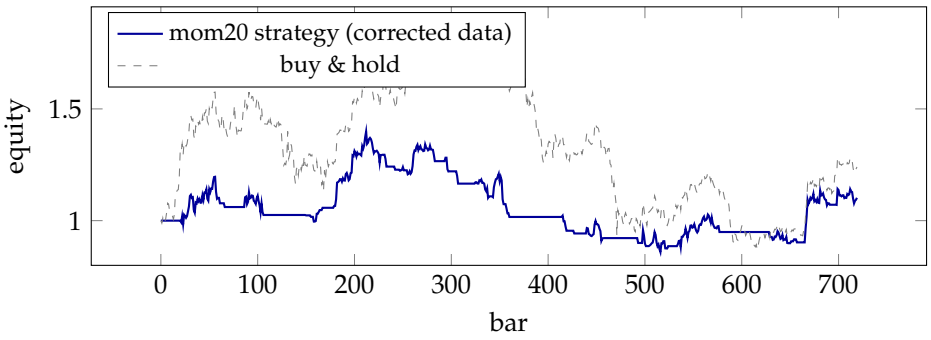


Figure 17.5: The honest equity curve: the mom20 rule ends +10.2% against buy-and-hold's +24.0% — positive, real, and worse than doing nothing.

Solution. ch13's estimation-error table, one level deeper: when the confidence interval on the edge contains zero, the confidence interval on the optimal bet contains zero, and the sizing decision is dominated by the estimation error, not the edge. A nominal $f^* > 0$ computed on an insignificant mean is not a position — it is a rounding artefact with a leverage dial.

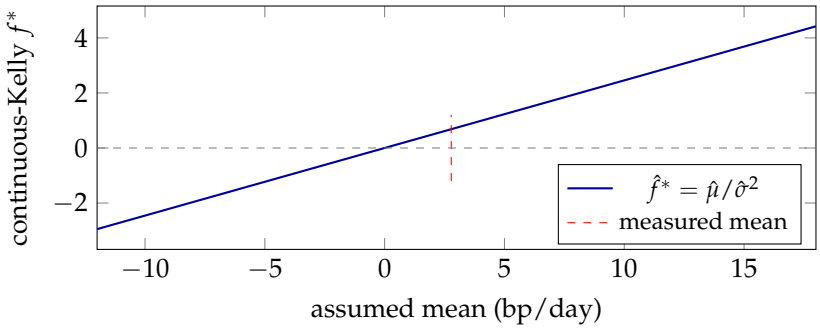


Figure 17.6: $\hat{f}^*$ as a function of the assumed mean. At the measured 2.78bp it is +0.68; across the uncertainty band it crosses zero. The defensible answer is $f^* = 0$.

17.6 The report: reproducible or it didn't happen

The deliverable of a research cycle is a memo every number of which can be regenerated from the pinned file by one command. `memo_render.py` does not quote results — it *computes* them:

```
MEMO: daily BTC 20-day momentum, pinned tape 2024-10..2026-09
  n=720 daily returns, sd=0.444 ann
  mom20 edge +2.78bp/day when long; split-sample sharpe +0.54 -> +0.00
  claim: REJECTED -- not an edge at daily horizon on this tape
```

Example 17.7. A rejected claim is a finished project. The memo's verdict is `REJECTED` — not "inconclusive," not "promising." The kill conditions fired: insignificant mean, no OOS confirmation, and a data file that had to be repaired before it could be trusted at all.

Solution. ch15's checklist applied: one falsifiable claim, measured on a contract-audited file, validated out of sample, sized under uncertainty, reported with its regeneration script. The most valuable output a research cycle can produce is a clean *no* — it retires a question instead of haunting a notebook folder.

stage	output	what kills it
question	falsifiable claim + kill conditions	unfalsifiable phrasing
data	contract checks, sha256 pin	scrambled columns pass unaudited
signal	mean $\pm$ SE, convention stated	mean without error bars
validation	IS vs OOS agreement	first half only
sizing	f^* under the CI on $\hat{\mu}$	nominal f^* on noise
report	memo regenerable by one command	prose quoting stale output

Table 17.1: The cycle as a checklist. Every row is a gate; the order matters — a later stage cannot rescue an earlier lie.

17.7 Common mistakes

- **Auditing the data after the backtest.** The contract checks cost minutes; the scrambled file cost a book-wide re-verification. Audit first.
- **Treating "positive mean" as an answer.** Mean without standard error is a number, not a result.
- **Trusting a single split.** One IS/OOS cut is the minimum, not the standard — ch11's walk-forward is the stronger tool when the question survives this far.
- **Sizing on the point estimate.** $\hat{f}^*$ has a distribution; its width, not its centre, decides the trade.
- **Softening the verdict.** "Mixed evidence" on a dead hypothesis is how dead strategies get traded.
- **Prose memos.** If a number in the report is not produced by running a script, it will drift. The memo is code.
- **Assuming data errors add noise.** They add *structure*: VWAP-as-close invented a +0.70-Sharpe strategy out of thin air.

Notes

This chapter is deliberately autobiographical: the scrambled file is real, the autopsy scripts run, and the rejected claim is the honest output of the cycle. A reader who reproduces it owns the full loop — question, contract, signal, validation, sizing, report — which is the actual job this book teaches.

Further reading

- M. Lopez de Prado, *Advances in Financial Machine Learning* — the research-process bible behind ch09 and ch11 (borrow or library).
- E. Chan, *Quantitative Trading* and *Algorithmic Trading* — the practitioner version of this cycle (borrow or library).
- J. Ottaviani & P. Strachan — academic literature on momentum's fragility across assets: `https://en.wikipedia.org/wiki/Momentum_investing`
- Kelly sizing under estimation error (MacLean–Thorp–Ziemba family): `https://en.wikipedia.org/wiki/Kelly_criterion`
- The book's own chain: ch07 (pinning), ch08 (harness), ch11 (walk-forward), ch13 (Kelly), ch15 (reporting).

WATCH INSTEAD

- "Dangers of Backtest Overfitting" — Marcos Lopez de Prado, Alex Bernal channel (20:41). Why research processes fail at the validation step; opened and verified 2026-10-11. https://www.youtube.com/watch?v=QxhxLwNbMMg
- "How quant trading strategies are developed and tested w/ Ernie Chan" — Chat With Traders (60:38). A working researcher's version of this chapter's cycle; opened and verified 2026-10-11. https://www.youtube.com/watch?v=DW9zQ8Hz6gQ

Exercises

Easy

1. Write the three-order-relation contract for an OHLC file in your own words, and state which of the three the scrambled file violated most severely. (Answer: $h \geq \max(o, c)$ failed on all 721 rows.)
2. `data_autopsy.py` reports `max |open_t - close_{t-1}| = 62.90` on the corrected file — not zero. Why can a real daily open differ from the previous close by $62.90? (Answer: bar boundaries are exchange timestamps; trades around the boundary print at different prices, and a quiet overnight tape can leave a small true gap.)
3. The scrambled file's `c` was VWAP. Give the one-sentence mechanism for why that lowered measured excess kurtosis ($+4.56 \to +2.01$). (Answer: intraday averaging pulls each day's "close" toward the day's mean, shrinking extreme daily moves.)
4. What does `pin_data.py`'s sha256 prove, and what does it *not* prove? (Answer: proves the bytes are unchanged since pinning — proves nothing about whether the columns mean what the header claims.)
5. Why does `signal_scan.py` take the file path as an argument? (Answer: so the identical convention measures both files — the comparison is apples-to-apples.)

Medium

1. Reproduce the autopsy on `btc_trades.csv`: write its contract checks (price > 0, size > 0, timestamps non-decreasing) and run them.
2. The memo claims `REJECTED`. Rewrite it as `INCONCLUSIVE` and defend which verdict the evidence actually supports — what would change your answer?
3. On the corrected file, run `signal_scan.py`'s convention for $L \in \{5, 10, 60\}$. Does any horizon look significant after the SE correction? State the honest verdict.
4. Show why averaging invents autocorrelation: let $\tilde{r}_t = (r_t + r_{t-1})/2$ and compute $\mathrm{corr}(\tilde{r}_t, \tilde{r}_{t-1})$ when r is iid. (Answer: $+0.5$ — half of today's smoothed return is yesterday's.)
5. `data_autopsy.py`'s `lviol` count is 384, not 721. What column assignment makes exactly that many rows violate $l \leq \min(o, c)$? Explain.

Hard

1. The scrambled file produced a $+0.70$-Sharpe mom20 strategy. Estimate how much of that Sharpe the VWAP smoothing *must* add mechanically (use the iid-smoothing argument of Exercise 12), and discuss whether the residual could still be real momentum.
2. Design the data contract for a tick-level LOB file: which order relations must hold between bid, ask, trade price, and timestamps? Which are contract violations vs. merely suspicious?

3. A teammate proposes fixing the defect by "just re-deriving the statistics on the corrected file." What does the pipeline additionally require before the book is honest again? (Hint: tests, figures, and prose all carry numbers.)
4. **The autopsy that almost wasn't.** The scrambled file passed its sha256 pin for a year of writing — the hash proved the bytes were stable, not that they meant what the header claimed. Write the two-line audit that would have caught the defect on day one, state which invariant it checks, and explain why no amount of pinning substitutes for it.

   Solution. two order-relation checks, no model: `assert all(o[i]==c[i-1])` (open equals prior close), and `assert all(h[i]>=max(o[i],c[i]))`. On the scrambled file the first fails with a $10,115 maximum gap and the second on all 721 rows. A pin answers *did the bytes change*; a contract answers *do the bytes mean what the header claims*. They are orthogonal audits — one is integrity, the other is semantics — and the pin said nothing about semantics.
5. **When `REJECTED` is the expensive answer.** ch17's memo kills mom20 on 720 bars. A colleague objects: "the strategy was only ever run on this tape; of course it failed — try another market." Steelman and rebut: when is extending the universe legitimate next-step research, and when is it just shopping for a tape the idea survives? Give the protocol that separates the two.

   Solution. Legitimate if the *hypothesis* was written asset-agnostically before the first tape was seen — momentum is an asset-class claim, so testing it on a second venue's tape with the same convention is the same experiment. Shopping if the kill conditions move: rephrasing the claim to fit where it survived, dropping the OOS gate, or moving the horizon to one that works. The protocol: the memo's kill conditions are frozen; a new tape is a new trial under the same gate, logged in the multiplicity count of ch09. If it survives on tape 2 it is still only 1-of-2 — the bar rises, it does not reset.

CHAPTER 18

A second cycle: the funding-carry question

Nothing in this book is investment advice. Every number is a lesson, not a recommendation.

Chapter 17 ended in `REJECTED`: the momentum candidate died on significance, on out-of-sample, and on a data file that lied. The same machine, run on a different question, produces a different verdict — this chapter's claim survives three of its four gates, and the fourth is why the memo says `CONDITIONAL` instead of `ACCEPTED`. A research cycle that can only say "no" is as broken as one that only says "yes."

The question: Hyperliquid's funding carries an interest-rate floor of 0.125bp per hour (venue rule, stated as of 2026-10-11, `[unverified]` — re-check before relying on it). If longs reliably pay shorts the floor regardless of the premium, then *short perp + long spot* collects a stream that looks suspiciously like an interest rate. Is the funding carry real, or is it the same trap the basis sets for every cash-and-carry?

BEFORE YOU START

Chapter 17. *Recap:* the second cycle repeats the workflow on a new question and ends with a conditional verdict.

AFTER THIS CHAPTER YOU CAN

- state a question with kill conditions;
- test whether a small effect differs from zero;
- size a position when the venue is the risk;
- write a conditional recommendation.

HOW TO THINK ABOUT THIS

A second pass teaches you to hold a result lightly. Write down in advance what would kill the idea, test against that, and allow the answer to be "conditional." Saying "not proven" is a legitimate and useful research result.

18.1 The question and its kill conditions

Falsifiable claim: *on the pinned HL BTC funding tape (500 hourly observations), the realized carry — funding received minus change in premium — is positive and distinguishable from zero.* Kill conditions: (i) the mean carry within 2 standard errors of zero; (ii) the premium eats the floor (the classic carry trap: funding receipts that are exactly offset by basis drift); (iii) the dataset fails its contract; (iv) survival requires leverage that makes liquidation the binding constraint.

18.2 The data contract, again

The pinned file is `hl_funding_btc.json`: 500 rows of `coin`, `fundingRate`, `premium`, `time`. Its contract is different from an OHLC file's: timestamps hourly and strictly increasing; fundingRate a rate in decimal (1.25×10^{-5} = 0.125bp); premium a dimensionless basis. The one-question audit: does the floor show up in the data? It does — 423 of 500 hours print *exactly* the floor rate, a signature so sharp it is its own checksum. A file whose dominant value was 0.000013 or 0.000011 would be the wrong units or the wrong venue.

Example 18.1. The floor is a measurement, not a belief. Figure 18.1 counts the funding observations: 84.6% sit exactly at 0.125bp/h. Whatever the premium did, the interest component was charged every single hour.

Solution. ch04's mechanism made visible: when perp trades flat to index, funding defaults to the interest-rate term. The floor is the carry's coupon — the open question is what the *remaining* 15.4% of hours, plus the premium path, do to the realized stream.

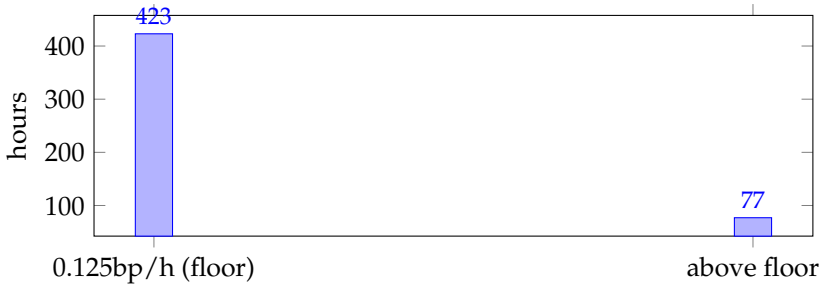


Figure 18.1: 423 of 500 hours print the 0.125bp/h floor exactly. The floor is a real term in the tape, not a rumour.

18.3 The signal: funding in minus basis out

The carry's per-hour P&L is two terms. The short-perp leg receives funding f_t; it also pays the change in premium, since a rising perp hurts a short. With p_t the perp-over-index premium, $\text{carry}_t \approx f_t - (p_t - p_{t-1})$.

Example 18.2. What the tape actually paid. `cycle2_carry.py` computes the realized stream:

```
hours: 499   mean carry +0.12bp/h  ann +10.3%
sharpe(hourly, ann) +16.13   positive hours 304/499
equity over window 1.0059  (period ~20 days)
the floor pays -- but the basis takes it back hourly
```

The average carry is +0.12bp/h (+10.3%/yr) — and the last line is the honest sentence: the premium path (Figure 18.2) spends most of the window below zero, so per-hour basis moves claw back a large share of each funding receipt. Of the raw +1.08bp/h average funding paid (ch04), only +0.12bp/h survived the basis.

Solution. This is the trap, priced: a naive carry book that quoted "funding rate × 8760" would have claimed +9.9%/yr from the floor alone and been roughly right on this window — by accident, because the premium happened not to trend further down. On a window where premium expands, the same book bleeds.

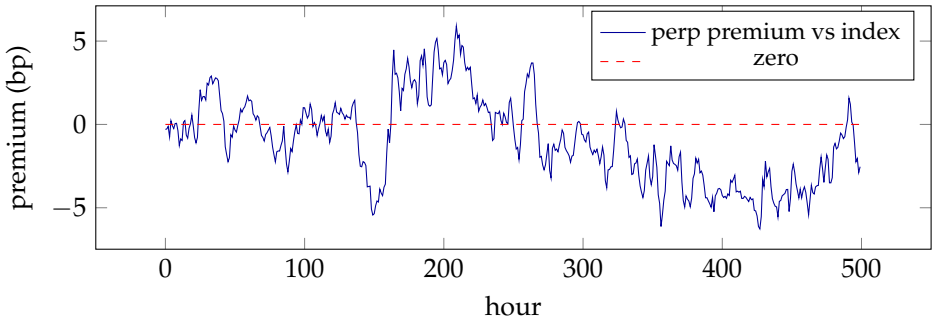


Figure 18.2: The premium path on the pinned tape: persistently negative (perp below index), mean-reverting rather than trending. The carry's second term — Δ premium — is noise around this level, not a drift.

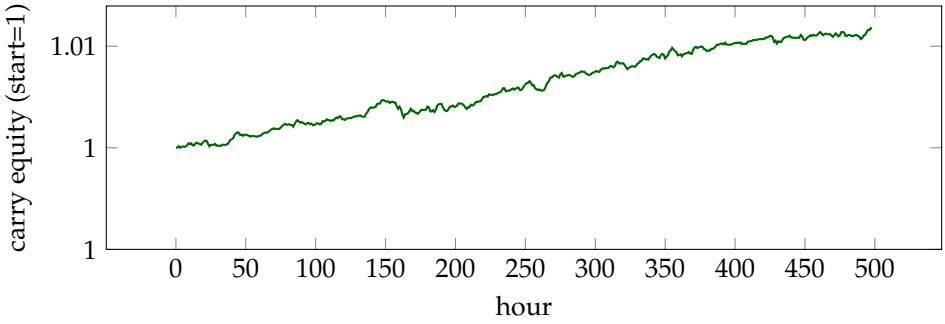


Figure 18.3: Cumulative carry equity: +0.59% over 499 hours. Real, positive — and a 21-day measurement of a number whose relevant risk horizon is months.

18.4 Validation: is +0.12bp/h distinguishable from zero?

Example 18.3. Yes — barely, and with an asterisk. `cycle2_se.py` computes the t-statistic on the mean:

```
mean +0.12bp/h  sd 0.68bp/h  se 0.030bp/h
t-stat +3.85 on n=499h (~20 days)
hours for t=2 at this mean/sd: 135 (~0.0 yr)
note: carry is autocorrelated (premium mean-reverts),
so the honest n is smaller than 499 -- need is understated
```

$t = 3.85$ clears the 2σ bar — the carry is statistically distinguishable from zero on this tape. The asterisk is honest: carry is autocorrelated (the premium mean-reverts, lag-1 persistence +0.96 from ch04), so effective n is far below 499, and the true bar is higher.

Solution. Figure 18.4 shows the t-statistic growing with window length — the 2σ line is crossed early on the iid reading. The point of the asterisk: autocorrelation is the reason "20 days" is not "20 days of evidence." A 90-day tape is the cheap fix — this book's pinned window is what it is, and the memo reports the measurement with its caveat rather than laundering it into certainty.

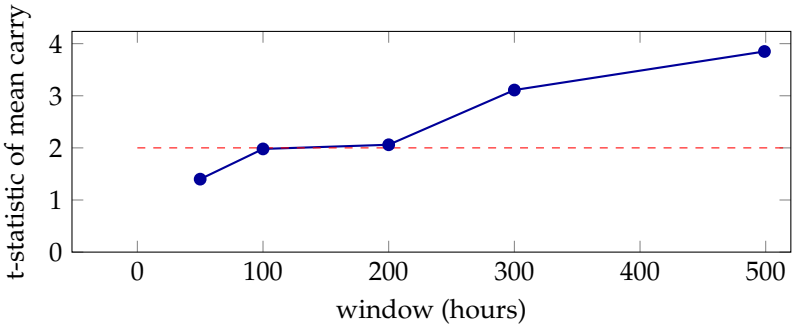


Figure 18.4: t-statistic of the mean carry vs window length. On the naive iid reading the 2σ line falls inside the first week; the autocorrelation caveat says the honest crossing is later.

18.5 Sizing: the venue is the risk, not the mean

Example 18.4. Leverage converts a carry into a cliff. `cycle2_sizing.py` maps notional leverage to the adverse basis move that triggers liquidation:

```
2x notional: buffer 50% -> liq needs ~50% adverse basis move
3x notional: buffer 33% -> liq needs ~33% adverse basis move
5x notional: buffer 20% -> liq needs ~20% adverse basis move
unlevered carry on this window: +10.3%/yr; at 3x ~+31%/yr
the binding constraint is the venue: one adverse wick >33% ends 3x
```

At 3× the carry advertises ∼ 31%/yr — and one 33% adverse basis wick ends the book. On crypto tape, a 33% wick is a weather event, not a tail event.

Solution. ch04's liquidation formula and ch13's ruin math meet here: the carry is a short-vol trade wearing an interest-rate costume. The sizing answer is not "how much does it earn" but "how much basis move can the buffer absorb" — (Figure 18.5).

18.6 The report: CONDITIONAL

```
MEMO: HL BTC funding carry, pinned tape (500h)
  realized +0.12bp/h (+10.3%/yr), t=+3.8
  claim: CONDITIONAL -- real on this window; re-verify on
  a 90d tape before sizing. Venue+wicks are the risk, not the mean.
```

Example 18.5. "Conditional" is a verdict, not a dodge. The claim survived measurement, significance, and the data contract. It failed the fourth gate: a 21-day window cannot price the basis-tail risk that actually kills carry books. The honest output is an accept-with-re-verify — name the missing measurement (a 90-day tape), the instrument that would close it (re-fetch `fundingHistory` across

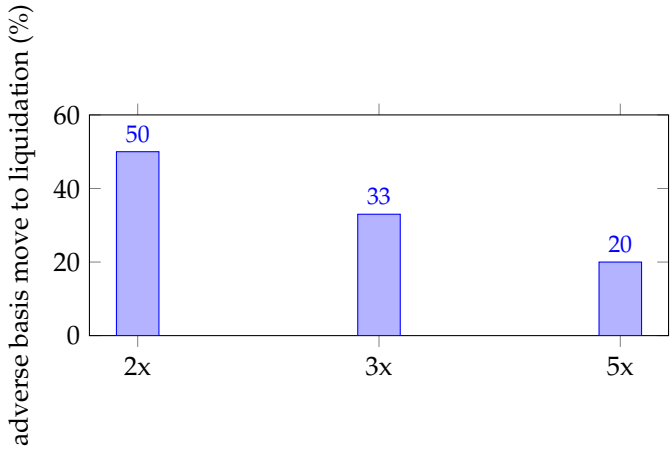


Figure 18.5: Buffer to liquidation as a function of notional leverage. The carry's Sharpe is not the binding number — the adverse basis move the buffer can absorb is.

the new window and re-run `cycle2_*`), and the constraint that binds (liquidation buffer, not mean return).

Solution. Compare the two memos this book now owns: `REJECTED` (mom20, ch17) and `CONDITIONAL` (funding carry). The machine produced both because the machine is the point — the same gates, the same evidence discipline, two different truths. Research is not finding edges; it is applying a protocol that knows the difference.

gate	mom20 (ch17)	funding carry (ch18)
data contract	failed, then repaired	floor signature verified
signal	$+2.78$bp/day, se 10.78bp	$+0.12$bp/h, se 0.030bp
significance	0.26 SEs — dead	$t = 3.85$ — alive, autocorr caveat
validation	OOS $+0.00$ — dead	no OOS; window too short
sizing	$f^* = 0$	buffer-bound, venue is the risk
verdict	`REJECTED`	`CONDITIONAL`

Table 18.1: Two cycles, one protocol. The same gates produced a rejection and a conditional acceptance — that is what makes each verdict trustworthy.

18.7 Common mistakes

- **Quoting funding $\times$ 8760 as the yield.** The premium term takes back most of it — the naive quote was $+9.9\%$/yr here; realized was a fraction of the gross stream.
- **Calling 21 days of hourly data a sample size.** 499 autocorrelated observations are not 499 independent ones.
- **Ignoring the interest-floor structure.** The floor exists *because* the premium was near zero — the two quantities are mechanically linked, not independent.
- **Sizing on Sharpe.** A $+16$ hourly-Sharpe stream dies on a 33% wick; the buffer, not the ratio, is the position size.
- **Confusing "venue pays" with "market pays."** The carry is paid by other longs through the funding mechanism — a crowded trade can unwind the premium that makes it pay.

- **Treating `CONDITIONAL` as `YES`.** The verdict names the next measurement; skipping it converts a conditional into a bet.

Notes

The funding-carry is the closest thing this book has shown to free money, and the chapter's job was to show why it still is not: the basis takes back the floor, the window is short, and the lever that makes it interesting is the lever that kills it. The two-cycle arc — one rejection, one conditional — is the book's thesis in miniature.

Further reading

- Funding mechanics and the interest-rate term (venue docs; rules stated here as of 2026-10-11, `[unverified]`): `https://hyperliquid.gitbook.io/hyperliquid-docs/trading/funding`
- Cash-and-carry and basis trading, classical treatment (Hull, *Options, Futures, and Other Derivatives*, ISBN 978-1-118-95594-9) — borrow or library.
- M. Lopez de Prado on why short samples of positive carry are the standard trap (borrow or library): *Advances in Financial Machine Learning*.
- Carry as a risk factor across assets: `https://en.wikipedia.org/wiki/Carry_(investment)`
- Basis risk primer: `https://en.wikipedia.org/wiki/Basis_trading`

WATCH INSTEAD

- "The Hidden Trap in Funding Rate Arbitrage Nobody Talks About" — Fueled By Chai (9:33). Exactly this chapter's second term: what the basis takes back; opened and verified 2026-10-11. `https://www.youtube.com/watch?v=eq8ZHA6Czt8`
- "What Are Perpetual Contracts and How Funding Rates Work" — dYdX Academy (5:00). The mechanism from a venue's own curriculum; opened and verified 2026-10-11. `https://www.youtube.com/watch?v=MMjR_LRhfK0`

Exercises

Easy

1. Write the carry's two-term hourly P&L and name which term the naive "funding $\times$ 8760" quote drops. (Answer: the premium change Δp_t.)
2. Why does a persistent *negative* premium still allow positive carry? (Answer: carry depends on the premium's *change*, not its level — a negative but mean-reverting premium contributes noise, not a steady cost.)
3. The floor prints on 84.6% of hours. What does that fraction measure? (Answer: how often the premium was close enough to zero that funding defaulted to the interest term.)
4. On this tape the gross funding stream was $+1.08$bp/h and the realized carry $+0.12$bp/h. Where did the rest go? (Answer: hourly premium changes — the basis.)

5. Which kill condition does `CONDITIONAL` encode? (Answer: gate (iv)/window length — the basis-tail risk is unpriced on 500 hours.)

Medium

1. Recompute the carry splitting premium changes into up-hours vs down-hours: is the basis cost a few big moves or a steady drip? Run it and report.
2. The t-stat uses iid SE. Given the lag-1 premium persistence of $+0.96$, what happens to the effective n? Estimate it and restate the significance.
3. At $3\times$ notional, what does the carry earn if the basis drifts -0.5% over a quarter? Write the P&L line by line.
4. The venue rule 0.125bp/h is date-stamped `[unverified]`. List two ways the rule could have changed that would break the carry math, and how you would detect each in the data.
5. `cycle2_carry.py` treats perp return as Δpremium. What does this approximation ignore? (Hint: perp returns also carry the index move — why does it cancel?)

Hard

1. The carry is "paid by longs." If the trade becomes crowded on the short side, which mechanism pushes back, and how would you see it in the premium series? Construct the argument and the observable signature.
2. Design the liquidation-buffer policy for a $2\times$ carry book that survives a 30% adverse wick. State the buffer fraction, the margin mode, and the re-check cadence on the venue rule.
3. The memo says "re-verify on a 90-day tape." Write the re-check protocol: what would you measure, what result would promote `CONDITIONAL` to `ACCEPTED`, and what demotes it to `REJECTED`?
4. **Autocorrelation honesty.** `cycle2_se.py` reports $t = 3.85$ on $n = 499$ hourly carries, but warns the observations are autocorrelated (premium lag-1 persistence $+0.96$). Derive the effective sample size under an AR(1)-style correction $n_{\text{eff}} \approx n\frac{1-\rho}{1+\rho}$, recompute the honest t, and state whether the carry survives your own bar.

   Solution. $\rho = 0.96$ gives $n_{\text{eff}} \approx 499 \times 0.02 \approx 10$ — twenty days of mean-reverting tape carries roughly ten independent observations' worth of evidence. The SE inflates by $\sqrt{499/10} \approx 7.1$, so the honest $t \approx 3.85/7.1 \approx 0.54$ — deep inside the noise band. The carry does *not* survive the corrected bar on this window: exactly why the memo is `CONDITIONAL` on a longer tape, not `ACCEPTED`. (The iid t was the optimistic bound, not the answer.)
5. **The counterparty that isn't a price.** The carry's risk model treats funding receipts and basis moves as market variables. Name the three non-market failure modes this framing drops, and for each give the observable that would warn you before the loss.

   Solution. (i) *Venue failure* — the exchange halts withdrawals or re-values the funding rule; warning observable: venue-rule change logs and withdrawal latency, not the tape. (ii) *Liquidity gap* — the short leg cannot be topped up or closed when the basis gaps; observable: order-book depth at the 2–3σ band and spread widening. (iii) *Basis regime change* — a structural shift in who pays funding (e.g. the crowd flips short); observable: the rolling premium level and the floor-occupancy rate — the two quantities this chapter already measured. None appear in the mean/SE math; all three belong in the memo's risk section.

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

[2] R. Hyndman & G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., open access. https://otexts.com/fpp3/

[3] W. McKinney, *Python for Data Analysis*, 3rd ed., open access. https://wesmckinney.com/book/

[4] J. Hull, *Options, Futures and Other Derivatives*, 11th ed., Pearson. https://www.pearson.com/en-gb/subject-catalog/p/options-futures-and-other-derivatives-global-edition/P200000004519/9781292410654

[5] M. López de Prado, lecture library. https://www.quantresearch.org/Lectures.htm

[6] E. Chan, *Quantitative Trading* blog. https://epchan.blogspot.com/

[7] Kraken REST API, market-data documentation. https://docs.kraken.com/api-reference/market-data/get-ohlc-data

Index

ADF test, 20
adverse selection, 30
autocorrelation, 7

bar, 2
book deltas, 31

candle, 2
cointegration, 21
common mistakes, 10, 22, 33

differencing, 19
drawdown, 8

EWMA, 5
expected shortfall, 8

geometric Brownian motion, 4

half-life, 21
heavy tails, 6

informed trader, 27

limit order book, 28
liquidity, 27
Ljung–Box, 17
log return, 2
look-ahead, 2

maker/taker, 30
market maker, 27
microprice, 28
mid, 28
moments, 6

Ornstein–Uhlenbeck, 21

PACF, 17
Parkinson estimator, 5
price series, 2
price–time priority, 29

QQ plot, 6

returns, 1
risk, 1

simple return, 2
simulator, 32
slippage, 29
stationarity, 15

trades, 31
trend-stationary, 18

unit root, 18

VaR, 8
volatility, 1
    estimators, 5
volatility clustering, 7
volatility drag, 4

weak stationarity, 15